

AMERICAN UNIVERSITY OF BEIRUT

Security Incident Detection and Response Management System

Sprint 1 Backend Implementation Report

Abdulrahman Kobeissi

Software Tools Lab
13 October 2026

This report describes a working backend with nine independently running services, role-based permissions, a relational SQLite database, documented APIs, and reproducible Postman tests. It explains the implementation and links each result to evidence captured from the local system.

Submission component	Included material
Source	Nine Flask services, SQL schema, launcher and backup utility
API documentation	60 business operations, OpenAPI reference and Postman collection
Verification	173 HTTP requests, 364 Newman assertions and 18 invariant tests
Evidence	Actual response captures, database records, diagrams and Git history

Practice seminar project: all nine services and report sections are assigned to Abdulrahman Kobeissi. This is a rehearsal exercise, not an assessed submission.

Repository: https://github.com/Abkob/Abdulrahman-Kobeissi-435L

Table of Contents

(Abdulrahman Kobeissi)

Every section is attributed to Abdulrahman Kobeissi. The evidence appendix is organized by the numbered Postman request, followed by database and GitHub captures.

Table of Contents (Abdulrahman Kobeissi) . . . 2
Introduction (Abdulrahman Kobeissi) . . . 3
System Architecture (Abdulrahman Kobeissi) . . . 4
Service Communication (Abdulrahman Kobeissi) . . . 5
Object Oriented Implementation (Abdulrahman Kobeissi) . . . 6
Users and Identity Management (Abdulrahman Kobeissi) . . . 7
Asset Management (Abdulrahman Kobeissi) . . . 8
Incident Management (Abdulrahman Kobeissi) . . . 9
Threat Intelligence (Abdulrahman Kobeissi) . . . 10
Security Audit Logging (Abdulrahman Kobeissi) . . . 11
Security Reports (Abdulrahman Kobeissi) . . . 12
Vulnerability Management (Abdulrahman Kobeissi) . . . 13
Notification Management (Abdulrahman Kobeissi) . . . 14
Security Playbooks and Response Actions (Abdulrahman Kobeissi) . . . 15
Database Design (Abdulrahman Kobeissi) . . . 16
Response and History Relationships (Abdulrahman Kobeissi) . . . 17
Database Dictionary 1 (Abdulrahman Kobeissi) . . . 18
Database Dictionary 2 (Abdulrahman Kobeissi) . . . 19
Database Dictionary 3 (Abdulrahman Kobeissi) . . . 20
Database Dictionary 4 (Abdulrahman Kobeissi) . . . 21
Database Dictionary 5 (Abdulrahman Kobeissi) . . . 22
Database Dictionary 6 (Abdulrahman Kobeissi) . . . 23
Database Dictionary 7 (Abdulrahman Kobeissi) . . . 24
Error Handling and Validation (Abdulrahman Kobeissi) . . . 25
API Documentation (Abdulrahman Kobeissi) . . . 26
API Walkthroughs (Abdulrahman Kobeissi) . . . 27
Authentication and Authorization (Abdulrahman Kobeissi) . . . 28
Verification and Recovery (Abdulrahman Kobeissi) . . . 29
GitHub and Version Control (Abdulrahman Kobeissi) . . . 30
References (Abdulrahman Kobeissi) . . . 31
Evidence Appendix (Abdulrahman Kobeissi) . . . 32

Use the PDF table of contents links to navigate. Page numbers are updated in the delivered document.

Introduction

(Abdulrahman Kobeissi)

Problem and scope

(Abdulrahman Kobeissi)

Security teams need to connect reported incidents with affected assets, analysts, evidence, known threats, and remediation work. Separate spreadsheets make it difficult to preserve ownership and status history. This backend centralizes those records while exposing a distinct HTTP service for each responsibility.

Sprint 1 covers account and role management, assets, incident investigation, threat intelligence, audit logs, reports, vulnerabilities, notifications, and response playbooks. A product frontend and public cloud deployment are outside this sprint. The read-only database evidence viewer is a development utility.

Objectives and results

(Abdulrahman Kobeissi)

The implementation provides the 54 required operations and six supporting business operations for logout, role assignment and removal, account status, permissions, and asset history. Four roles are enforced before protected operations. Validation rejects malformed data, unauthorized access and invalid workflow changes. Real HTTP tests exercise all nine processes.

The 173-request collection completed with 364 assertions and no failures. A separate 18-test suite checks security and persistence invariants. Live database checks confirm referential integrity, report reconciliation, empty results and verified backups. These results establish the demonstrated local behavior; they are not load-test or production-readiness claims.

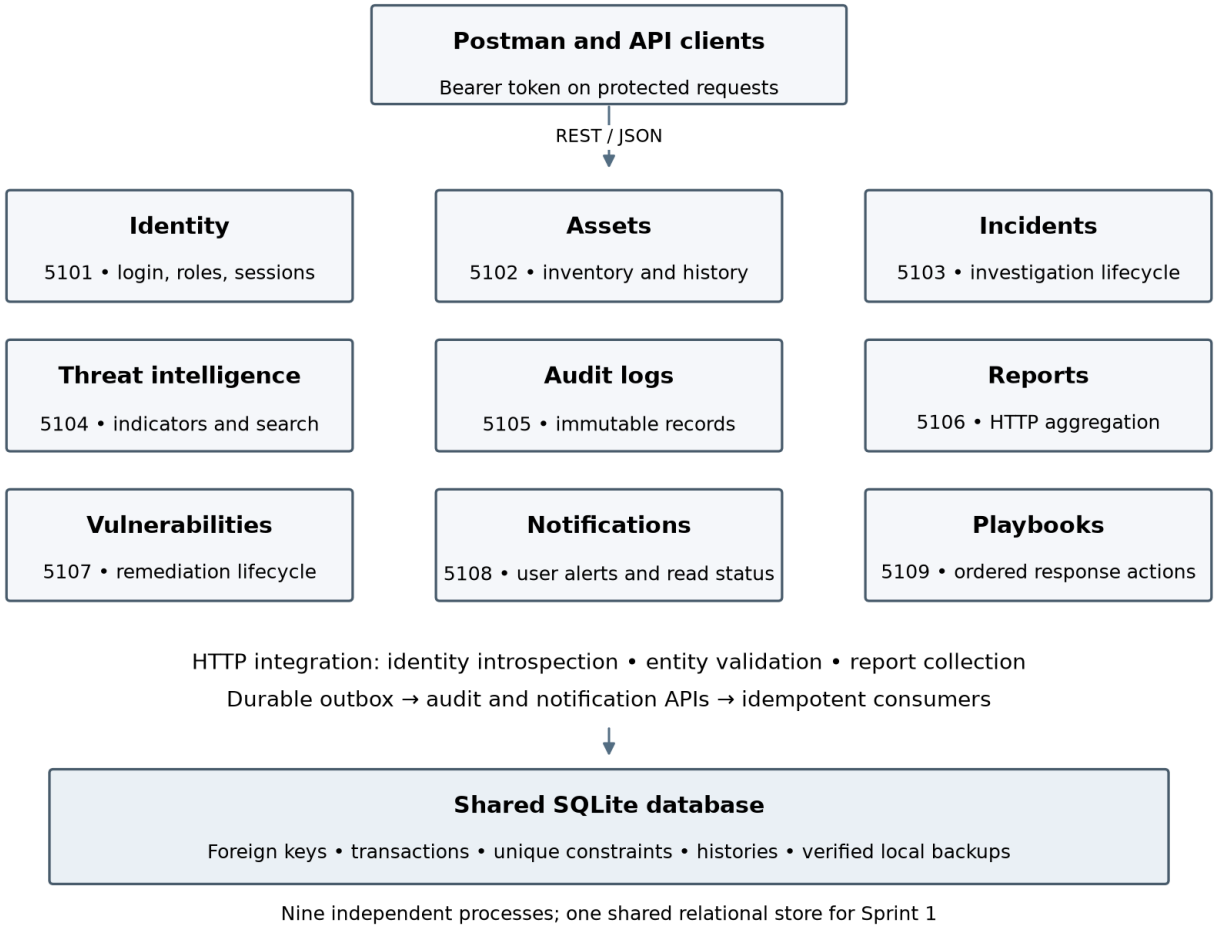
Practice seminar scope

(Abdulrahman Kobeissi)

This solo project is prepared for a practice theatrical seminar. The course handout provides the technical exercise requirements. The package supports a demonstration of the implementation and is not an actual course submission.

System Architecture

(Abdulrahman Kobeissi)



Abdulrahman Kobeissi — Nine service processes, HTTP integration and shared persistence.

Each service has its own Flask application and port. run.py supervises the nine processes. All public paths start with /Abdulrahman-Kobeissi. A shared SQLite database retains relational foreign keys across the sprint's entities. This is process-level service separation with shared persistence, rather than nine isolated databases.

Service Communication

(Abdulrahman Kobeissi)

Caller	HTTP dependency	Reason
Protected services	Identity introspection	Validate the bearer token and current user state
Assets, incidents, notifications, playbooks	Identity user lookup	Validate owners, assignees and recipients
Incidents and vulnerabilities	Assets lookup	Validate the affected asset before insertion
Reports	Users, assets, incidents, threats, vulnerabilities and logs	Aggregate actual service responses
Mutation event workers	Audit and notification internal APIs	Deliver durable activity records and alerts

The ServiceClient class makes HTTP calls with bounded timeouts. Internal endpoints require a local shared service key, which is generated into the private runtime folder. External requests use bearer sessions. If a required service is unavailable, the caller returns HTTP 503 instead of silently inventing a result.

A business transaction writes both its data and an outbox event. A worker in each service retries pending events through HTTP. Consumers identify events by a unique event_id, making retries idempotent. Pending and failed delivery counts are visible in health responses. This design preserves audit and notification intent when a downstream service is temporarily unavailable.

Example: assigning an incident validates the analyst through Identity, stores the assignment and event in one transaction, then creates an Incident Assignment notification through the Notifications service. Reports obtain source records through HTTP before grouping counts.

Operational boundary

(Abdulrahman Kobeissi)

The local teaching setup binds to 127.0.0.1. A production deployment would require production WSGI hosting, TLS, service-specific credentials, secret management, pagination and operational monitoring. No existing production data or schema was changed.

Object Oriented Implementation

(Abdulrahman Kobeissi)

Class	Responsibility	Use of the abstraction
Settings	Database location, key and service ports	One configuration object per app
Database	Connections and transactions	Commits successful operations and rolls back failures
Repository	Parameterized CRUD and approved filters	Shared persistence behavior without user-supplied SQL
Lifecycle	Allowed ordered status transitions	Reused for incidents, vulnerabilities and response actions
ServiceClient	HTTP communication	Encapsulates timeouts and service errors
EventBus	Durable audit and notification events	Queues and retries idempotent delivery
Context	Service dependencies and actor context	Composes persistence, client and events
CatalogService	Common catalog route and CRUD behavior	Base implementation for two catalogs
ThreatService and VulnerabilityService	Specialized field validation and update rules	Override base validation and lifecycle behavior

Encapsulation keeps SQL connection handling inside Database and common record operations inside Repository. Catalog inheritance removes duplicated route plumbing while preserving domain-specific validation. Lifecycle composition applies the same ordered-transition rule to three different workflows. Route functions coordinate these objects and return consistent JSON envelopes.

The source layout separates identity.py, assets.py, incidents.py, catalogs.py, audit_reports.py, notifications.py and playbooks.py. app.py creates the selected service and installs common error handling. The split keeps each service's authorization and validation close to its business behavior.

Users and Identity Management

(Abdulrahman Kobeissi)

Accounts, authentication, role assignment and account status. Usernames and email addresses are unique case-insensitively. Public registration creates active employees; only an authenticated administrator can create privileged accounts.

Method	Endpoint suffix	Required body fields
POST	/users/register	full_name, username, email, password, department, role
POST	/users/login	login, password
POST	/users/logout	None
GET	/users	None
GET	/users/{username}	Path or query only
PUT	/users/{user_id}	Optional update fields
DELETE	/users/{user_id}	Path or query only
DELETE	/users/{user_id}/role	Path or query only
PUT	/users/{user_id}/role	role
PUT	/users/{user_id}/status	account_status
GET	/users/permissions	None

All routes use the prefix /Abdulrahman-Kobeissi on port 5101.

Registration requires full_name, username, email, password, department and role. Login takes login and password. Responses omit hashes and expose only safe user fields. Profile updates accept full_name, email and department.

Persistence. users and sessions; login_attempts throttles repeated failures. Deleted users are hidden and suspended, while historical foreign keys remain valid.

Rules. A duplicate username returns 409. Invalid credentials return 401. Inactive and suspended accounts cannot authenticate. Role changes, deletion and account-status changes revoke sessions. The last active administrator cannot be removed.

Evidence: numbered collection requests 001–035. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Asset Management

(Abdulrahman Kobeissi)

Maintains inventory, ownership, location, criticality and current status. Administrators manage inventory; auditors read it. Analysts can inspect and search only assets related to their assigned incidents.

Method	Endpoint suffix	Required body fields
POST	/assets	name, asset_type, ip_address, department, owner_id, criticality
PUT	/assets/{asset_id}	Optional update fields
DELETE	/assets/{asset_id}	Path or query only
GET	/assets	None
GET	/assets/search	Path or query only
GET	/assets/{asset_id}	Path or query only
GET	/assets/{asset_id}/history	Path or query only

All routes use the prefix /Abdulrahman-Kobeissi on port 5102.

Creation requires name, asset_type, ip_address, department, owner_id and criticality. operating_system, location and status are optional. Single-asset responses also list related incident and vulnerability IDs.

Persistence. assets references its owner in users. asset_history stores before and after JSON and retains deletion history. Referenced assets cannot be deleted.

Rules. Criticality is a constrained text value: Low, Medium, High or Critical. Searches combine name, type, IP, department and criticality. SQL values are bound parameters. Invalid IPs and unknown owners are rejected.

Evidence: numbered collection requests 036–055; 069–071; 081. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Incident Management

(Abdulrahman Kobeissi)

Tracks reporting, assignment, notes and investigation status. Employees see their own reports; analysts see assigned incidents; administrators and auditors have broader read access.

Method	Endpoint suffix	Required body fields
POST	/incidents	title, description, category, severity, asset_id
GET	/incidents	None
GET	/incidents/{incident_id}	Path or query only
PUT	/incidents/{incident_id}/assign	analyst_id
PUT	/incidents/{incident_id}/status	status
POST	/incidents/{incident_id}/notes	content

All routes use the prefix /Abdulrahman-Kobeissi on port 5103.

Creation requires title, description, category, severity and asset_id; asset_id may be null. reporter_id is derived from authentication; an explicitly supplied reporter must match the caller. Notes take content.

Persistence. incidents references reporter, optional asset and assigned analyst. investigation_notes references incident and note author.

Rules. The sequence is Open → Under Investigation → Mitigated → Resolved → Closed. Assignment to an active analyst is required before advancing. Notes are visible to authorized investigators and auditors, but hidden from employee incident views. Resolution time is generated when resolved.

Evidence: numbered collection requests 056–080. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Threat Intelligence

(Abdulrahman Kobeissi)

Stores indicators and supports investigation searches. Administrators create, update and delete; analysts and auditors read and search.

Method	Endpoint suffix	Required body fields
POST	/threats	threat_type, value, description, severity, source
GET	/threats	None
GET	/threats/search	Path or query only
GET	/threats/{record_id}	Path or query only
PUT	/threats/{record_id}	Optional update fields
DELETE	/threats/{record_id}	Path or query only

All routes use the prefix /Abdulrahman-Kobeissi on port 5104.

Creation takes threat_type, value, description, severity and source. Updates accept description, severity and source. Responses include the record ID and creation timestamp.

Persistence. threats has a composite unique constraint on threat_type and value. High and Critical creations generate an alert to the acting administrator.

Rules. IP values are parsed, domains are normalized to lowercase and trailing-dot-free form, and hashes must be valid MD5, SHA1 or SHA256 hexadecimal strings. Duplicate normalized indicators return 409. Searches support type, value, severity and source. Test indicators use reserved laboratory addresses and example.test domains.

Evidence: numbered collection requests 082–099. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Security Audit Logging

(Abdulrahman Kobeissi)

Records security actions and permission violations. Administrators create and delete log records. Auditors inspect and search; analysts have the read permissions described in the service specification.

Method	Endpoint suffix	Required body fields
POST	/logs	user_id, action, description, severity, related_service
GET	/logs	None
GET	/logs/{record_id}	Path or query only
GET	/logs/search	Path or query only
GET	/logs/user/{user_id}	Path or query only
DELETE	/logs/{record_id}	Path or query only

All routes use the prefix /Abdulrahman-Kobeissi on port 5105.

Creation requires user_id, action, description, severity and related_service. A record includes a server UTC timestamp. Lists and searches return arrays; no matches return an empty array.

Persistence. logs references an optional user; unauthenticated denials may have no user. The database trigger rejects UPDATE operations. Unique event IDs deduplicate delivery.

Rules. Search filters cover user, date, action, severity and related service. Logs preserve the originally recorded values. The handout explicitly requires administrative deletion; a delete itself emits an audit event, while no modification endpoint exists.

Evidence: numbered collection requests 151–163. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Security Reports

(Abdulrahman Kobeissi)

Generates summary, incident, asset, threat, vulnerability and activity reports for administrators and auditors.

Method	Endpoint suffix	Required body fields
GET	/reports/summary	None
GET	/reports/incidents	None
GET	/reports/assets	None
GET	/reports/threats	None
GET	/reports/vulnerabilities	None
GET	/reports/activity	None

All routes use the prefix /Abdulrahman-Kobeissi on port 5106.

The six GET endpoints take no JSON body. Each returns a data object with generated_at, source and its report-specific totals or groups.

Persistence. The service does not create report tables. It reads source records over the other services' HTTP APIs and computes counts and groupings.

Rules. Incident reports group severity and status; asset reports group type and criticality; threat reports group severity/type and show recent indicators; vulnerability reports group severity/status; activity reports count actions and login attempts. Summary combines the sources. HTTP 503 surfaces a missing dependency.

Evidence: numbered collection requests 164–171. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Vulnerability Management

(Abdulrahman Kobeissi)

Tracks asset vulnerabilities through investigation and remediation. Administrators create and delete; analysts may update; auditors read and search.

Method	Endpoint suffix	Required body fields
POST	/vulnerabilities	title, description, cve, cvss, severity, asset_id, discovery_source
GET	/vulnerabilities	None
GET	/vulnerabilities/search	Path or query only
GET	/vulnerabilities/{record_id}	Path or query only
PUT	/vulnerabilities/{record_id}	Optional update fields
DELETE	/vulnerabilities/{record_id}	Path or query only

All routes use the prefix /Abdulrahman-Kobeissi on port 5107.

Creation requires title, description, cve, cvss, severity, asset_id and discovery_source. Updates accept description, severity, status, cvss and resolution_date.

Persistence. vulnerabilities references an affected asset. Resolution time is retained when Closed. Critical or High creation emits a notification.

Rules. CVE syntax is validated, without claiming a lookup in an external CVE database. CVSS must be numeric from 0 to 10. Status follows Open → Under Investigation → In Progress → Resolved → Closed. A supplied resolution time must be timezone-aware, not future and not before creation.

Evidence: numbered collection requests 100–119. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Notification Management

(Abdulrahman Kobeissi)

Supports manual and automatic alerts, recipient scoping and read tracking. Administrators create and delete; employees and analysts can read their own alerts; auditors have read-only access.

Method	Endpoint suffix	Required body fields
POST	/notifications	user_id, title, message, notification_type, severity
GET	/notifications/user/{user_id}	Path or query only
GET	/notifications/{record_id}	Path or query only
PUT	/notifications/{record_id}/read	Path or query only
GET	/notifications/search	Path or query only
DELETE	/notifications/{record_id}	Path or query only

All routes use the prefix /Abdulrahman-Kobeissi on port 5108.

Creation requires user_id, title, message, notification_type and severity. Marking read uses PUT with no body. It changes Unread to Read and records the first read timestamp.

Persistence. notifications references its recipient and stores status, created_at, read_date and an optional unique event ID.

Rules. Repeated marking as read preserves the original read time. Analysts cannot search another user's notifications. Searches combine recipient, notification type, severity and read status. A known user without notifications receives a successful empty array.

Evidence: numbered collection requests 120–135. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Security Playbooks and Response Actions

(Abdulrahman Kobeissi)

Organizes response procedures by incident category and severity, with ordered actions and execution history.

Method	Endpoint suffix	Required body fields
POST	/playbooks	name, description, incident_category, severity
GET	/playbooks	None
GET	/playbooks/{book_id}	Path or query only
POST	/playbooks/{book_id}/actions	description, step_number
PUT	/actions/{action_id}/status	status
DELETE	/playbooks/{book_id}	Path or query only

All routes use the prefix /Abdulrahman-Kobeissi on port 5109.

Create a playbook with name, description, incident_category and severity. Add actions with description and step_number; assigned_user_id is optional. Status updates require status and may include an assignee.

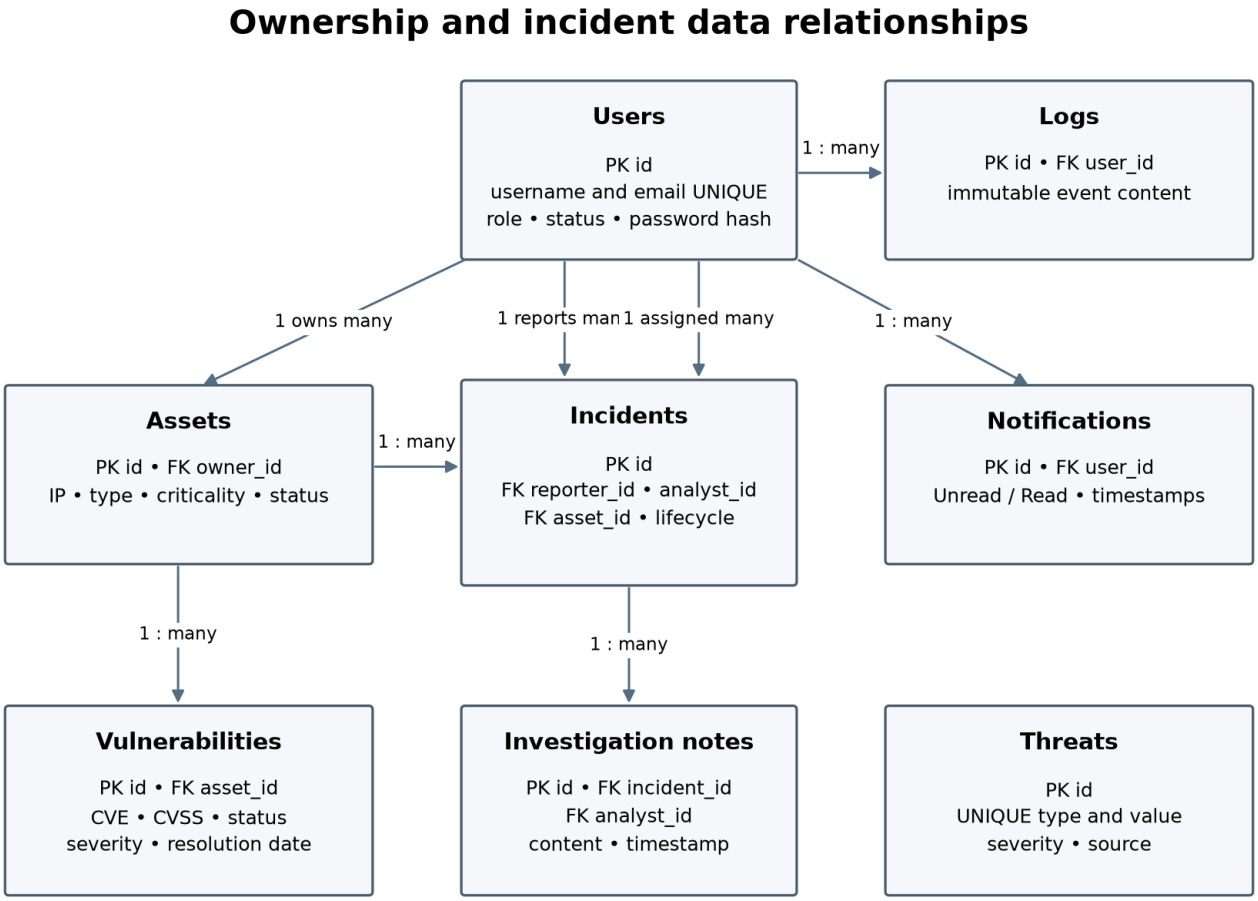
Persistence. playbooks references its creator. response_actions references its playbook and assignee. action_history retains every recorded progression. Soft deletion preserves execution history.

Rules. Steps must be consecutive within a playbook. Pending → In Progress → Completed is enforced, and earlier steps must be completed before later ones begin. Analysts may claim or update their own actions. Completion time is server-generated; completed actions cannot be reassigned.

Evidence: numbered collection requests 136–150. Screenshots appear in the evidence appendix; complete JSON responses and parameters are included in docs/API_REFERENCE.md and docs/openapi.json.

Database Design

(Abdulrahman Kobeissi)

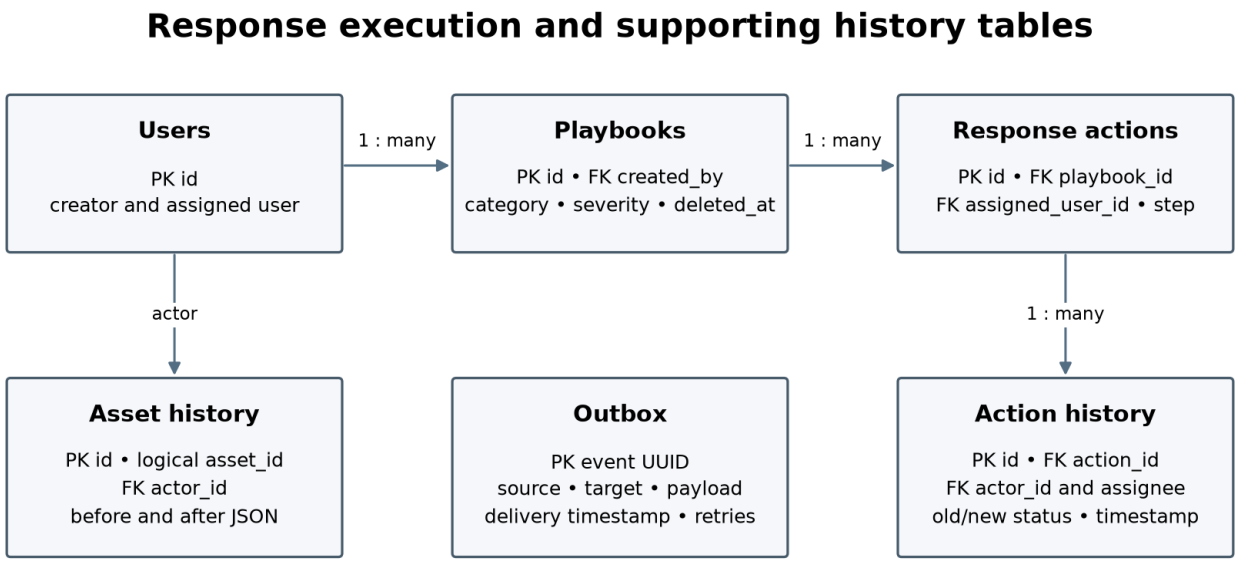


Abdulrahman Kobeissi — Core entity relationships and foreign-key direction.

SQLite enforces foreign keys on each connection. IDs are primary keys; session and outbox identifiers use text keys. Timestamps are stored in UTC ISO format. One user may own many assets, report many incidents and receive many notifications. Each note belongs to one incident. An asset may have many incidents and vulnerabilities.

Response and History Relationships

(Abdulrahman Kobeissi)



Sessions reference users; login attempts support rate limiting. Logical asset IDs retain history after deletion.

Abdulrahman Kobeissi — Playbook, action, notification and history relationships.

Playbooks contain ordered response actions. Users can be assigned many actions. Each action has many immutable historical status entries. Asset history retains a logical asset ID even after deletion, so asset_history.asset_id deliberately does not have a foreign key; actor_id remains linked to users.

User and playbook deletion is soft deletion to preserve relationships and evidence. Asset deletion is restricted if incidents or vulnerabilities reference it. Unique constraints protect usernames, emails, normalized indicator pairs, event IDs and step numbers within a playbook. Enum and range checks provide a second layer beneath API validation.

Database Dictionary 1

(Abdulrahman Kobeissi)

Users

(Abdulrahman Kobeissi)

Accounts and hashed credentials.

Field	Type	Key or nullability
id	INTEGER	PK
full_name	TEXT	NOT NULL
username	TEXT	NOT NULL
email	TEXT	NOT NULL
password_hash	TEXT	NOT NULL
department	TEXT	NOT NULL
role	TEXT	NOT NULL
account_status	TEXT	NOT NULL; default 'Active'
created_at	TEXT	NOT NULL
last_login_at	TEXT	Nullable
deleted_at	TEXT	Nullable

Sessions

(Abdulrahman Kobeissi)

Hashed bearer-token sessions.

Field	Type	Key or nullability
token_hash	TEXT	PK
user_id	INTEGER	NOT NULL; FK → users.id
expires_at	TEXT	NOT NULL
created_at	TEXT	NOT NULL

Login Attempts

(Abdulrahman Kobeissi)

Failed-login throttling.

Field	Type	Key or nullability
id	INTEGER	PK
login	TEXT	NOT NULL
address	TEXT	NOT NULL
created_at	TEXT	NOT NULL

Database Dictionary 2

(Abdulrahman Kobeissi)

Assets

(Abdulrahman Kobeissi)

Organizational inventory.

Field	Type	Key or nullability
id	INTEGER	PK
name	TEXT	NOT NULL
asset_type	TEXT	NOT NULL
ip_address	TEXT	NOT NULL
operating_system	TEXT	NOT NULL; default ”
department	TEXT	NOT NULL
owner_id	INTEGER	NOT NULL; FK → users.id
location	TEXT	NOT NULL; default ”
criticality	TEXT	NOT NULL
status	TEXT	NOT NULL
created_at	TEXT	NOT NULL

Asset History

(Abdulrahman Kobeissi)

Inventory changes before and after mutation.

Field	Type	Key or nullability
id	INTEGER	PK
asset_id	INTEGER	NOT NULL
actor_id	INTEGER	Nullable; FK → users.id
operation	TEXT	NOT NULL
before_json	TEXT	Nullable
after_json	TEXT	Nullable
created_at	TEXT	NOT NULL

Database Dictionary 3

(Abdulrahman Kobeissi)

Incidents

(Abdulrahman Kobeissi)

Reported security events.

Field	Type	Key or nullability
id	INTEGER	PK
title	TEXT	NOT NULL
description	TEXT	NOT NULL
category	TEXT	NOT NULL
severity	TEXT	NOT NULL
reporter_id	INTEGER	NOT NULL; FK → users.id
asset_id	INTEGER	Nullable; FK → assets.id
assigned_analyst_id	INTEGER	Nullable; FK → users.id
status	TEXT	NOT NULL
created_at	TEXT	NOT NULL
resolution_date	TEXT	Nullable

Investigation Notes

(Abdulrahman Kobeissi)

Analyst evidence and notes.

Field	Type	Key or nullability
id	INTEGER	PK
incident_id	INTEGER	NOT NULL; FK → incidents.id
analyst_id	INTEGER	NOT NULL; FK → users.id
content	TEXT	NOT NULL
created_at	TEXT	NOT NULL

Database Dictionary 4

(Abdulrahman Kobeissi)

Threats

(Abdulrahman Kobeissi)

Threat intelligence indicators.

Field	Type	Key or nullability
id	INTEGER	PK
threat_type	TEXT	NOT NULL
value	TEXT	NOT NULL
description	TEXT	NOT NULL
severity	TEXT	NOT NULL
source	TEXT	NOT NULL
created_at	TEXT	NOT NULL

Logs

(Abdulrahman Kobeissi)

Audit records.

Field	Type	Key or nullability
id	INTEGER	PK
user_id	INTEGER	Nullable; FK → users.id
action	TEXT	NOT NULL
description	TEXT	NOT NULL
severity	TEXT	NOT NULL
related_service	TEXT	NOT NULL
created_at	TEXT	NOT NULL
event_id	TEXT	Nullable

Database Dictionary 5

(Abdulrahman Kobeissi)

Vulnerabilities

(Abdulrahman Kobeissi)

Asset remediation records.

Field	Type	Key or nullability
id	INTEGER	PK
title	TEXT	NOT NULL
description	TEXT	NOT NULL
cve	TEXT	NOT NULL
cvss	REAL	NOT NULL
severity	TEXT	NOT NULL
asset_id	INTEGER	NOT NULL; FK → assets.id
discovery_source	TEXT	NOT NULL
status	TEXT	NOT NULL
created_at	TEXT	NOT NULL
resolution_date	TEXT	Nullable

Notifications

(Abdulrahman Kobeissi)

Recipient alerts and read tracking.

Field	Type	Key or nullability
id	INTEGER	PK
user_id	INTEGER	NOT NULL; FK → users.id
title	TEXT	NOT NULL
message	TEXT	NOT NULL
notification_type	TEXT	NOT NULL
severity	TEXT	NOT NULL
status	TEXT	NOT NULL
created_at	TEXT	NOT NULL
read_date	TEXT	Nullable
event_id	TEXT	Nullable

Database Dictionary 6

(Abdulrahman Kobeissi)

Playbooks

(Abdulrahman Kobeissi)

Response procedures.

Field	Type	Key or nullability
id	INTEGER	PK
name	TEXT	NOT NULL
description	TEXT	NOT NULL
incident_category	TEXT	NOT NULL
severity	TEXT	NOT NULL
created_by	INTEGER	NOT NULL; FK → users.id
created_at	TEXT	NOT NULL
deleted_at	TEXT	Nullable

Response Actions

(Abdulrahman Kobeissi)

Ordered response steps.

Field	Type	Key or nullability
id	INTEGER	PK
playbook_id	INTEGER	NOT NULL; FK → playbooks.id
description	TEXT	NOT NULL
step_number	INTEGER	NOT NULL
status	TEXT	NOT NULL
assigned_user_id	INTEGER	Nullable; FK → users.id
completion_date	TEXT	Nullable

Database Dictionary 7

(Abdulrahman Kobeissi)

Action History

(Abdulrahman Kobeissi)

Response execution history.

Field	Type	Key or nullability
id	INTEGER	PK
action_id	INTEGER	NOT NULL; FK → response_actions.id
actor_id	INTEGER	NOT NULL; FK → users.id
old_status	TEXT	Nullable
new_status	TEXT	NOT NULL
assigned_user_id	INTEGER	Nullable; FK → users.id
created_at	TEXT	NOT NULL

Outbox

(Abdulrahman Kobeissi)

Durable HTTP delivery queue.

Field	Type	Key or nullability
id	TEXT	PK
source	TEXT	NOT NULL
target	TEXT	NOT NULL
payload	TEXT	NOT NULL
created_at	TEXT	NOT NULL
delivered_at	TEXT	Nullable
attempts	INTEGER	NOT NULL; default 0
last_error	TEXT	Nullable

Error Handling and Validation

(Abdulrahman Kobeissi)

Every service uses a common JSON error envelope with status=”error”, a meaningful message and request_id. Error handlers map expected validation, authentication, authorization, lookup and relationship failures to consistent HTTP statuses. Unhandled failures return a generic server error rather than exposing internals.

HTTP	Meaning	Demonstrated case
400	Invalid input	Invalid email, IP, severity, CVSS or unsupported field
401	Authentication failure	Missing token, incorrect login or revoked session
403	Permission or ownership failure	Employee changes incident status; analyst deletes a threat
404	Missing record	Unknown user, asset, incident, catalog entry or playbook
409	Conflict	Duplicate username/indicator or skipped workflow state
415	Unsupported content type	JSON-required operation sent non-JSON content
429	Rate limit	Repeated failed login attempts
503	Dependency unavailable	A report source cannot be contacted

Input is parsed as JSON objects. Required fields, data types, length bounds, allowed values and unknown keys are checked before persistence. The API rejects empty or malformed requests. Search fields come from a fixed allowlist, and SQL values use placeholders; user input is never inserted into SQL syntax. HTML is not rendered by the backend, so data is stored as text rather than altered with lossy HTML stripping.

Negative requests are intentional test successes when the expected error is returned. Screenshots 013, 015, 025, 051, 065, 072, 085–087, 106, 117 and other appendix cases show these failures. The invariant suite also verifies that SQL-like text remains literal and that a direct log UPDATE fails at the database layer.

API Documentation

(Abdulrahman Kobeissi)

docs/openapi.json describes all 60 business operations. docs/API_REFERENCE.md lists purpose, method, complete route, role requirements, path/query parameters, accepted request fields, response examples and possible errors. postman/Sprint1.postman_collection.json contains every numbered test with descriptions, scripts and real response examples.

Protected calls send Authorization: Bearer <token>. Successful reads and updates use HTTP 200; creation uses 201. Data objects and arrays are wrapped in {"status":"success","data":...}. Empty lists remain successful arrays. Errors use the common envelope described in the previous section. Passwords and token values in published examples are redacted.

Registration walkthrough

(Abdulrahman Kobeissi)

POST /Abdulrahman-Kobeissi/users/register on port 5101 validates the six required fields and uniqueness, hashes the password, then inserts the account and durable events within a transaction. Public callers may register employees; privileged roles require an administrator token. A duplicate is a 409 conflict rather than a second row.


```
{
  "full_name": "Demo Analyst",
  "username": "r17918970950626932_analyst",
  "email": "r17918970950626932_analyst@example.test",
  "password": "[REDACTED]",
  "department": "Security",
  "role": "Security Analyst"
}
```

```
{
  "data": {
    "account_status": "Active",
    "created_at": "2026-10-13T13:11:36.964620+00:00",
    "department": "Security",
    "email": "r17918970950626932_analyst@example.test",
    "full_name": "Demo Analyst",
    "id": 7,
    "role": "Security Analyst",
    "username": "r17918970950626932_analyst"
  },
  "status": "success"
}
```


Actual request 003 creates an analyst under administrator authorization. The displayed password is redacted. Figure 003 contains the original Postman view; the collection retains the full safe response.

API Walkthroughs

(Abdulrahman Kobeissi)

Incident assignment

(Abdulrahman Kobeissi)

PUT /Abdulrahman-Kobeissi/incidents/{incident_id}/assign on port 5103 accepts analyst_id. It requires an administrator, checks the incident, calls Identity to verify an active Security Analyst, and stores the assignment. The same transaction queues an audit event and an Incident Assignment notification. Missing incident returns 404, invalid assignee 400, and insufficient permission 403.


```
{
  "analyst_id": 7
}
```



```
{
  "status": "success",
  "data": {
    "id": 5,
    "assigned_analyst_id": 7,
    "status": "Open"
  }
}
```


The response above is an explicitly shortened extract of request 067; the complete incident fields are in the saved response. Request 065 shows an unauthorized caller rejected from the same operation.

Incident report

(Abdulrahman Kobeissi)

GET /Abdulrahman-Kobeissi/reports/incidents on port 5106 requires an administrator or auditor and has no body. Reports requests the incident list through HTTP, counts severity and status, and adds generated_at plus the source label. The live reconciliation utility compares those groups with SQLite, not merely with a hardcoded expectation.


```
{
  "data": {
    "by_severity": {
      "Critical": 2,
      "High": 2,
      "Low": 2,
      "Medium": 2
    },
    "by_status": {
      "Closed": 2,
      "Open": 6
    },
    "generated_at": "2026-10-13T13:12:21.991662+00:00",
    "source": "Service HTTP APIs",
    "total": 8
  },
  "status": "success"
}
```


This is the saved execution response for request 165. Counts reflect the database at that run. Later reruns create additional uniquely named test records, so current totals may increase without invalidating the dated evidence.

Authentication and Authorization

(Abdulrahman Kobeissi)

Werkzeug scrypt hashes passwords with a salt. The database never stores plaintext passwords. Login verifies the hash and issues an opaque random token valid for one hour. Only a SHA-256 digest of that token is stored in sessions. Other services ask Identity to introspect the bearer token and check the current user role and active status.

Role	Allowed scope	Denied examples
Security Administrator	Manage users and records, assignments, reports and roles	Cannot bypass data validation or ordered lifecycle rules
Security Analyst	Assigned incidents, related assets, threat reads, vulnerability updates, own notifications and response work	User administration and destructive inventory/catalog operations
Regular Employee/User	Register, maintain own profile, submit and track own incidents, read own notifications	Organizational assets, threats, vulnerabilities, audit logs, reports and playbooks
Auditor	Read users, assets, incidents, catalogs, logs, reports, notifications and playbooks	Any creation, update, deletion or role modification

The protected decorator rejects missing/invalid authentication before calling the operation and then checks roles. Resource-specific checks constrain employee ownership and analyst assignments. Denials enqueue an audit record. Removing a privileged role demotes the account to Regular Employee/User. Role and account-status changes revoke existing sessions immediately.

Where the handout's broad role list and endpoint list differ, the README records the choice: employee and analyst incident lists are scoped to their own/assigned records; analyst asset reads are limited to related incidents; auditor access stays read-only. Login and self-service registration are public entry points, not administrative permissions.

Evidence pairs include login success/failure (001–002), authorized/unauthorized assignment (067/065), deletion pairs across services, inactive/suspended account tests, and token revocation. The test suite checks token storage, session expiry, role changes and protection of the last active administrator.

Verification and Recovery

(Abdulrahman Kobeissi)

Check	Observed result	Evidence file
Real HTTP scenarios	173 requests; 364 checks; no failures	evidence/http-summary.json
Newman collection execution	173 requests; 364 assertions; no failures	evidence/newman-results.json
Fresh GitHub checkout	173 requests; 364 assertions; no failures on a separate database	evidence/clean-checkout.json
Security and persistence invariants	18 tests pass	evidence/unit-tests.txt
SQLite integrity	ok; zero foreign-key violations	evidence/integration-checks.json
Report reconciliation	All six report types match stored data	evidence/integration-checks.json
Empty responses	Logs and notifications return []	Supplemental-evidence collection
Durable event delivery	Zero pending events at verification	evidence/integration-checks.json
Backup validation	Integrity, foreign keys, counts and SHA-256 checked	Verified local backup manifest

The invariant suite uses disposable databases and in-process service clients for focused failure tests. The 173-request HTTP and Newman runs use the actual nine local processes. These are complementary checks; in-process tests are not presented as network evidence.

The supervisor creates a SQLite backup on startup and hourly. The backup utility refuses to overwrite an existing destination, uses the SQLite backup API, validates integrity and foreign keys, and records counts plus a SHA-256 manifest. runtime/backup-status.json makes success or failure visible. Recovery tests open the copied database and verify its data.

Runtime databases, passwords, tokens and backups stay outside Git and the submission archive. The package includes the schema and reproducible sample-data/test procedures. GitHub protects source and sanitized evidence; it does not provide a backup of live application data. These are verified private local recovery copies, not a claimed cloud backup.

GitHub and Version Control

(Abdulrahman Kobeissi)

Repository: https://github.com/Abkob/Abdulrahman-Kobeissi-435L

The repository is private under Abkob. The project uses the 435L naming suffix requested by the student. Commits record the implementation, tests, documentation and evidence in meaningful stages. No retroactive dates, fabricated contribution history or teammate authorship are used.

```
aba6755 2026-10-13 Abdulrahman Kobeissi
Document all APIs and add verified backups and report reconciliation
790d457 2026-10-13 Abdulrahman Kobeissi
Add 173 live API scenarios, Postman automation and security invariant tests
87c8d7c 2026-10-13 Abdulrahman Kobeissi
Implement nine security services with RBAC and durable event delivery
```

All nine service areas, SQL design, testing and report sections are assigned to Abdulrahman Kobeissi's solo practice project. Commit attribution identifies the configured repository author.

The GitHub evidence appendix shows the actual repository commit page. The final packaging commit may follow that dated screenshot. Reviewers need permission to the private repository; no unrequested collaborators or public access were added.

Reproduction

(Abdulrahman Kobeissi)

Create a Python virtual environment, install requirements-lock.txt, and run python run.py. The first start creates runtime/bootstrap.json with a random administrator password. Import the collection and clean environment into Postman, select the local environment, set that password locally, and use a local Desktop Agent. Run all requests in order. The README includes exact commands and port assignments.

Reruns use unique names and do not reset an existing database. To start an isolated demonstration, use a separate SECURITY_RUNTIME and SECURITY_DB path. Keep the current database and verified backup before any restoration.

References

(Abdulrahman Kobeissi)

Course specification. Final Project Fall 2026-2027-Sprint 1.docx, supplied course handout.

Prior course work. Local Lab 1, Lab 3, Lab 4 and Lab 5 project materials and earlier project chats for Flask, OOP, SQLite, Postman, Git workflow and report conventions.

Flask. Official Quickstart, https://flask.palletsprojects.com/en/stable/quickstart/

Python SQLite. Official sqlite3 documentation, https://docs.python.org/3/library/sqlite3.html

Werkzeug. Official security helper documentation, https://werkzeug.palletsprojects.com/en/stable/utils/

Postman. Official Collection Runner documentation, https://learning.postman.com/docs/tests-and-scripts/running-collections/intro-to-collection-runs/

Online documentation was consulted on 13 October 2026. Sample incident descriptions, indicators and account data are local test fixtures, not claims about real attacks or affected people. CVE-formatted sample values demonstrate validation and do not establish real vulnerability findings.

Seminar use

(Abdulrahman Kobeissi)

The supplied course handout is used as a technical exercise brief for this practice seminar. This package is intended for rehearsal and demonstration.

Evidence Appendix

(Abdulrahman Kobeissi)

The following screenshots show real response examples captured by Newman from the running local services and opened in Postman. The saved example name identifies the request number and observed status. This is recorded execution evidence, not a mock server or a claim that each screenshot was taken during a fresh Send action.

Crops retain the request title, method, full student-name route, request panel and response/status panel. Long JSON bodies extend beyond the visible editor; the collection and evidence/newman-results.json preserve their complete content. Secrets are redacted. Database captures are fresh read-only queries against runtime/security.db through the included evidence utility.

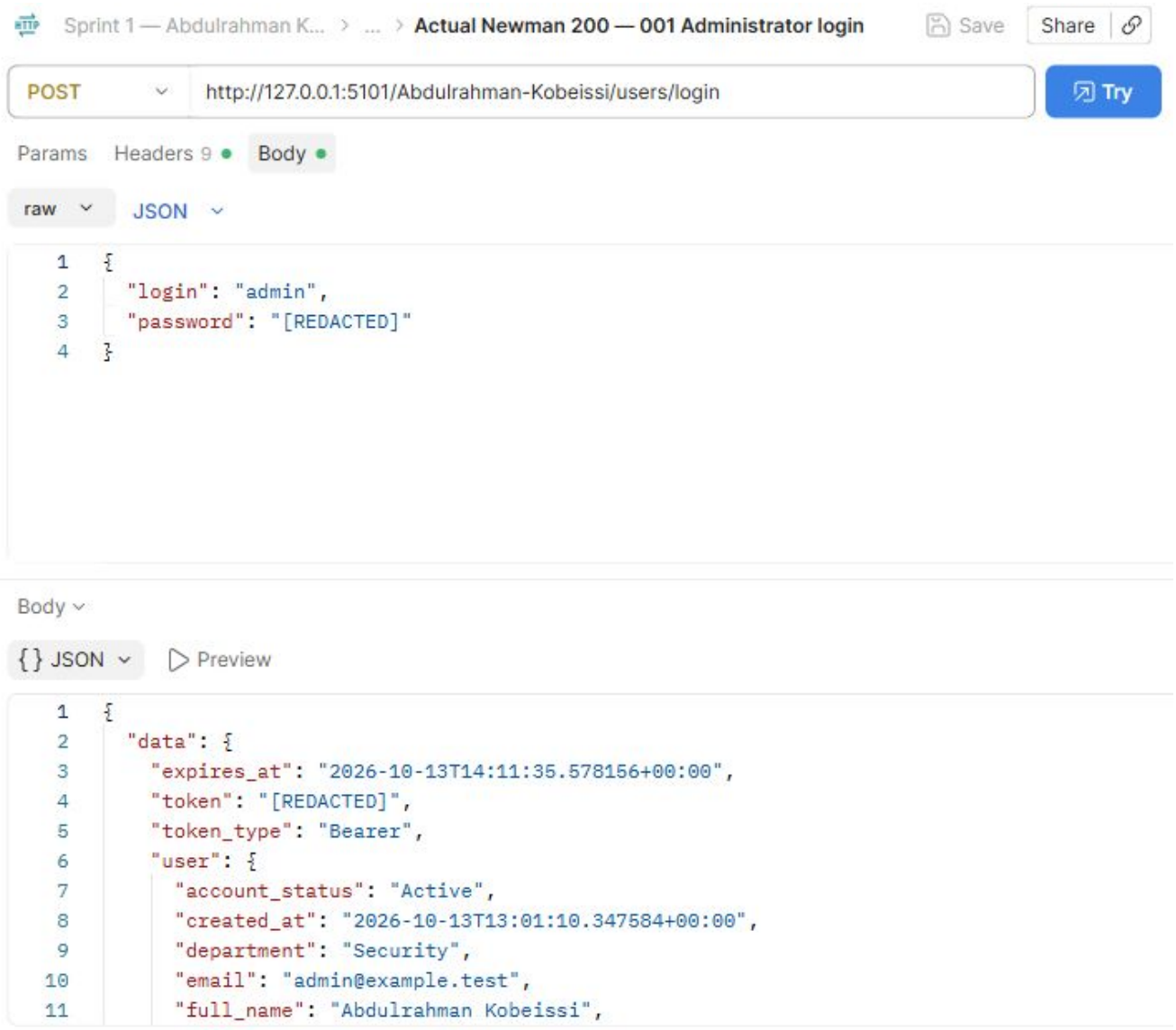
The request numbers follow the collection's execution order. Each caption names Abdulrahman Kobeissi and explains the operation. Successful rejection tests are labeled with the actual error status. The table below indexes the groups; individual screenshot captions identify the exact test.

Request numbers	Evidence group
001–035	Users and Identity Management
036–055; 069–071; 081	Asset Management
056–080	Incident Management
082–099	Threat Intelligence
151–163	Security Audit Logging
164–171	Security Reports
100–119	Vulnerability Management
120–135	Notification Management
136–150	Security Playbooks and Response Actions

Postman Evidence 1

(Abdulrahman Kobeissi)

Administrator login · POST · observed HTTP 200

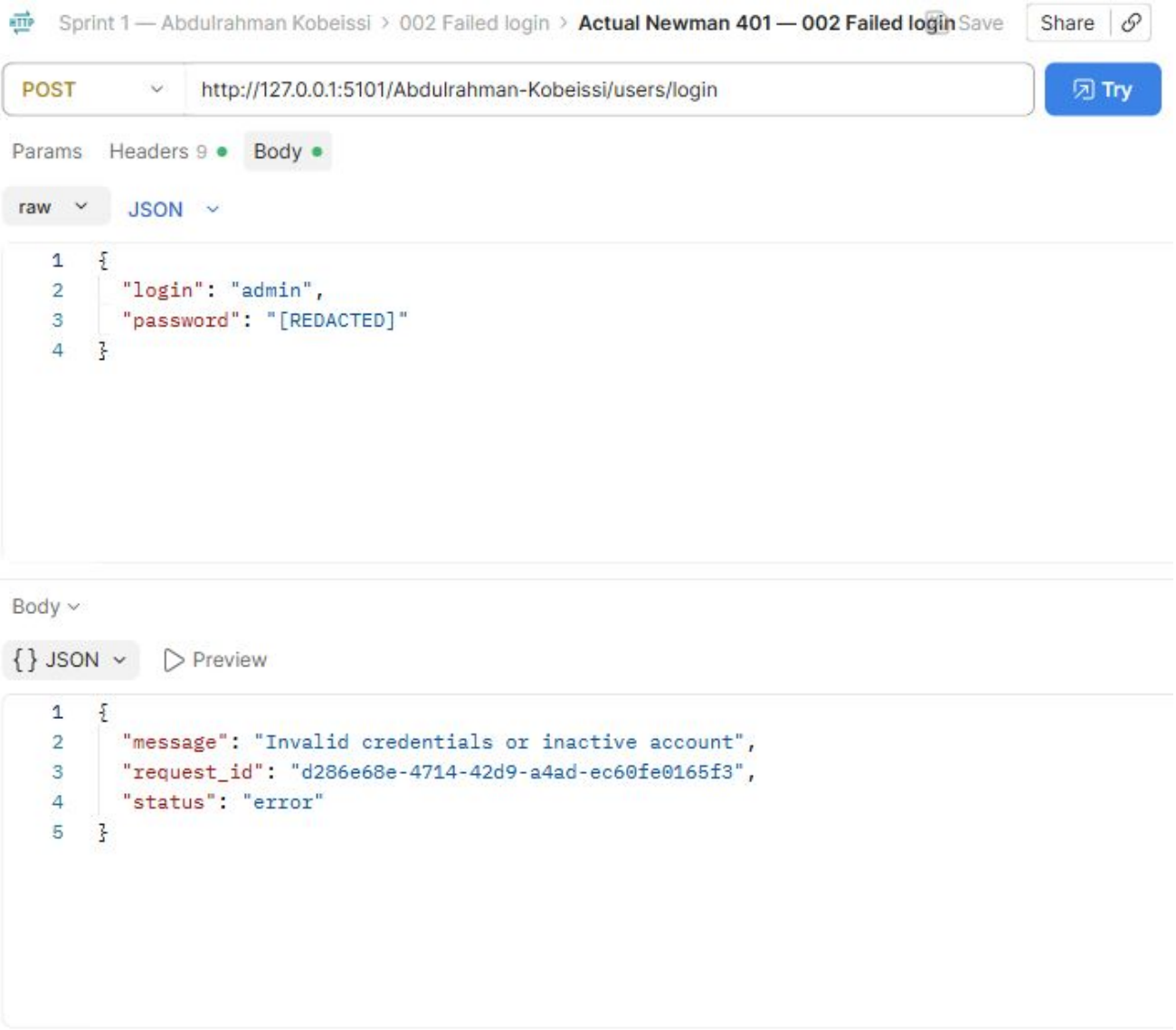


Abdulrahman Kobeissi — 001 Administrator login. The request and saved actual response demonstrate this operation.

Postman Evidence 2

(Abdulrahman Kobeissi)

Failed login · POST · observed HTTP 401

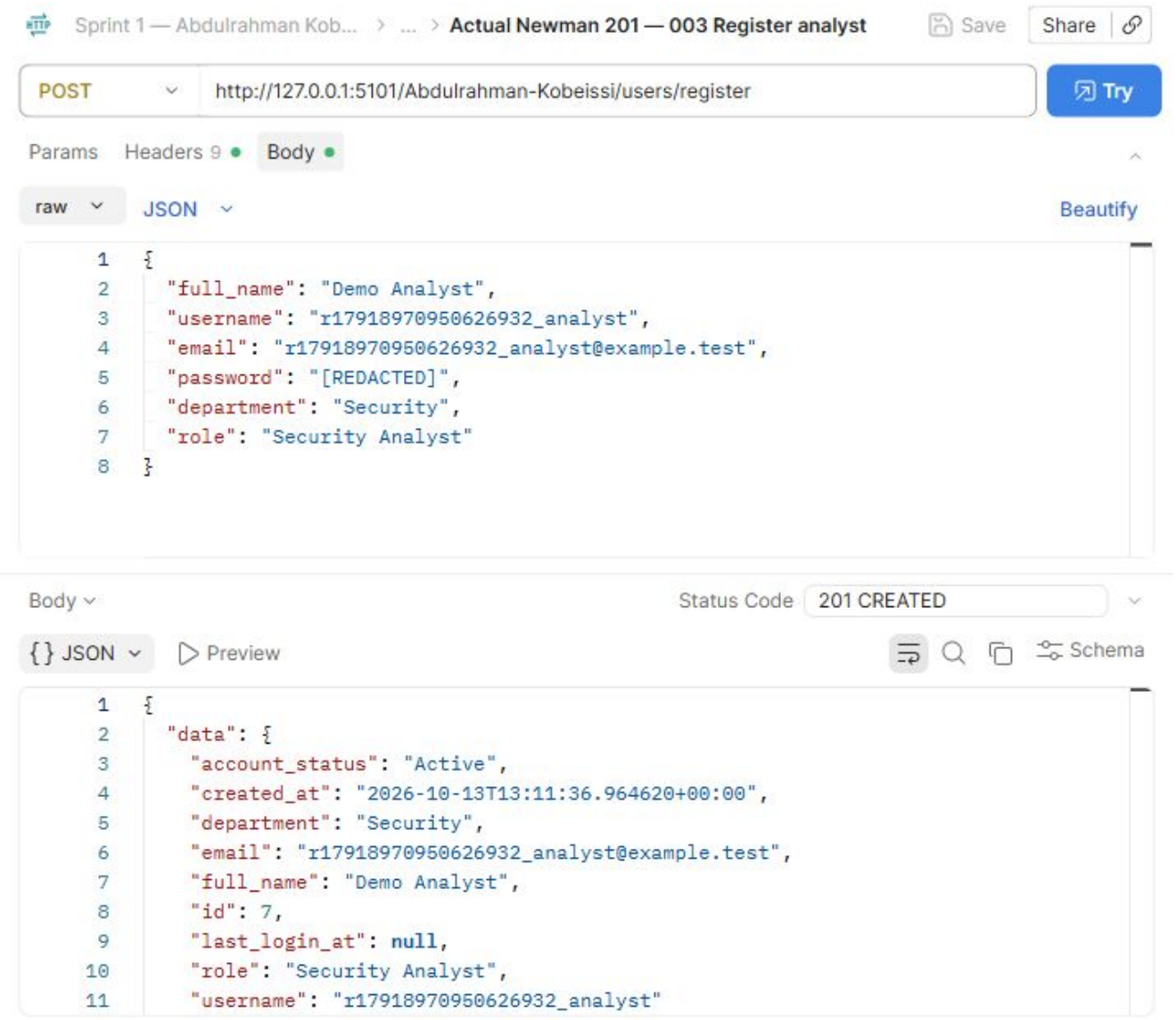


Abdulrahman Kobeissi — 002 Failed login. The request and saved actual response demonstrate this operation.

Postman Evidence 3

(Abdulrahman Kobeissi)

Register analyst · POST · observed HTTP 201

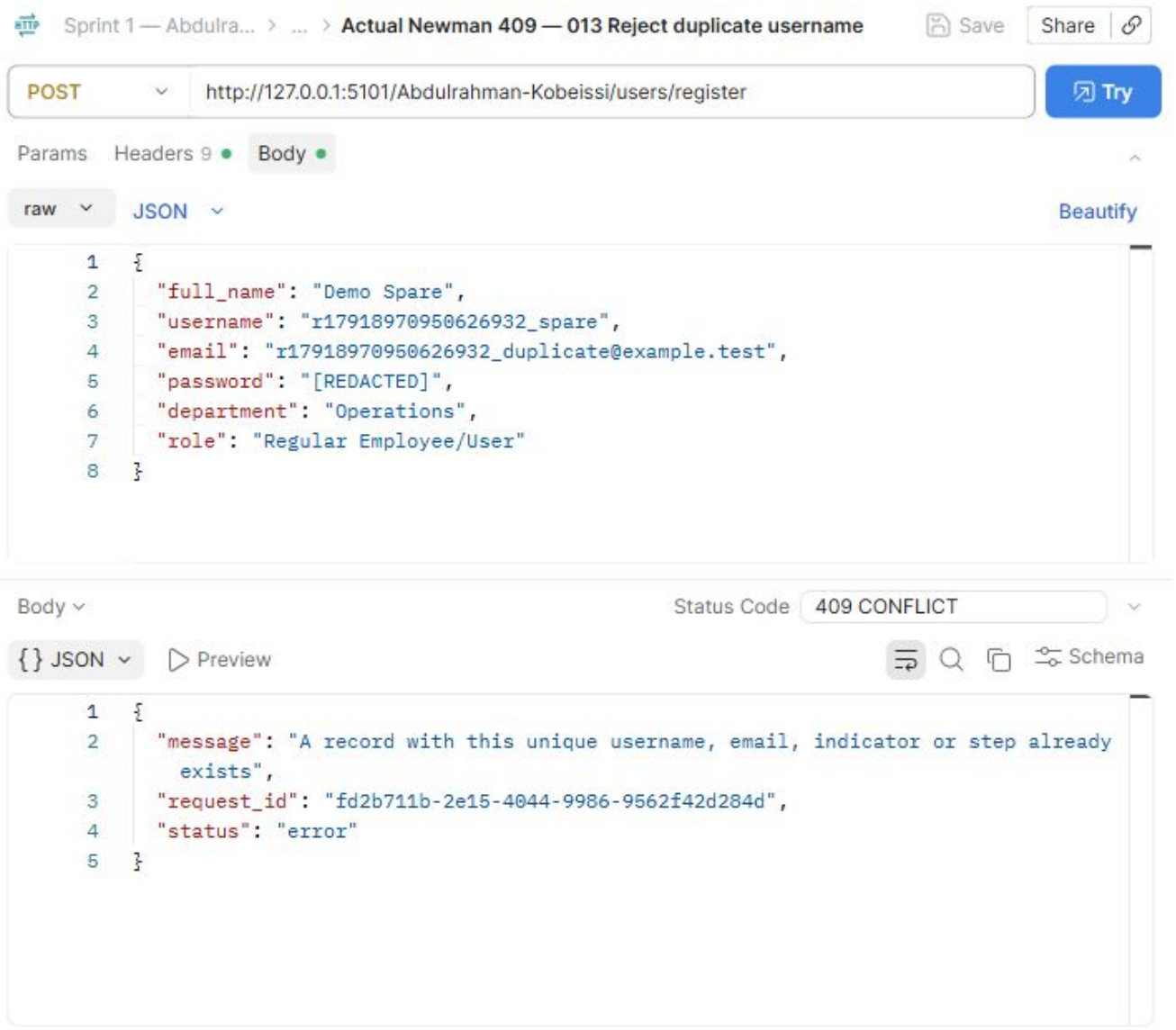


Abdulrahman Kobeissi — 003 Register analyst. The request and saved actual response demonstrate this operation.

Postman Evidence 13

(Abdulrahman Kobeissi)

Reject duplicate username · POST · observed HTTP 409

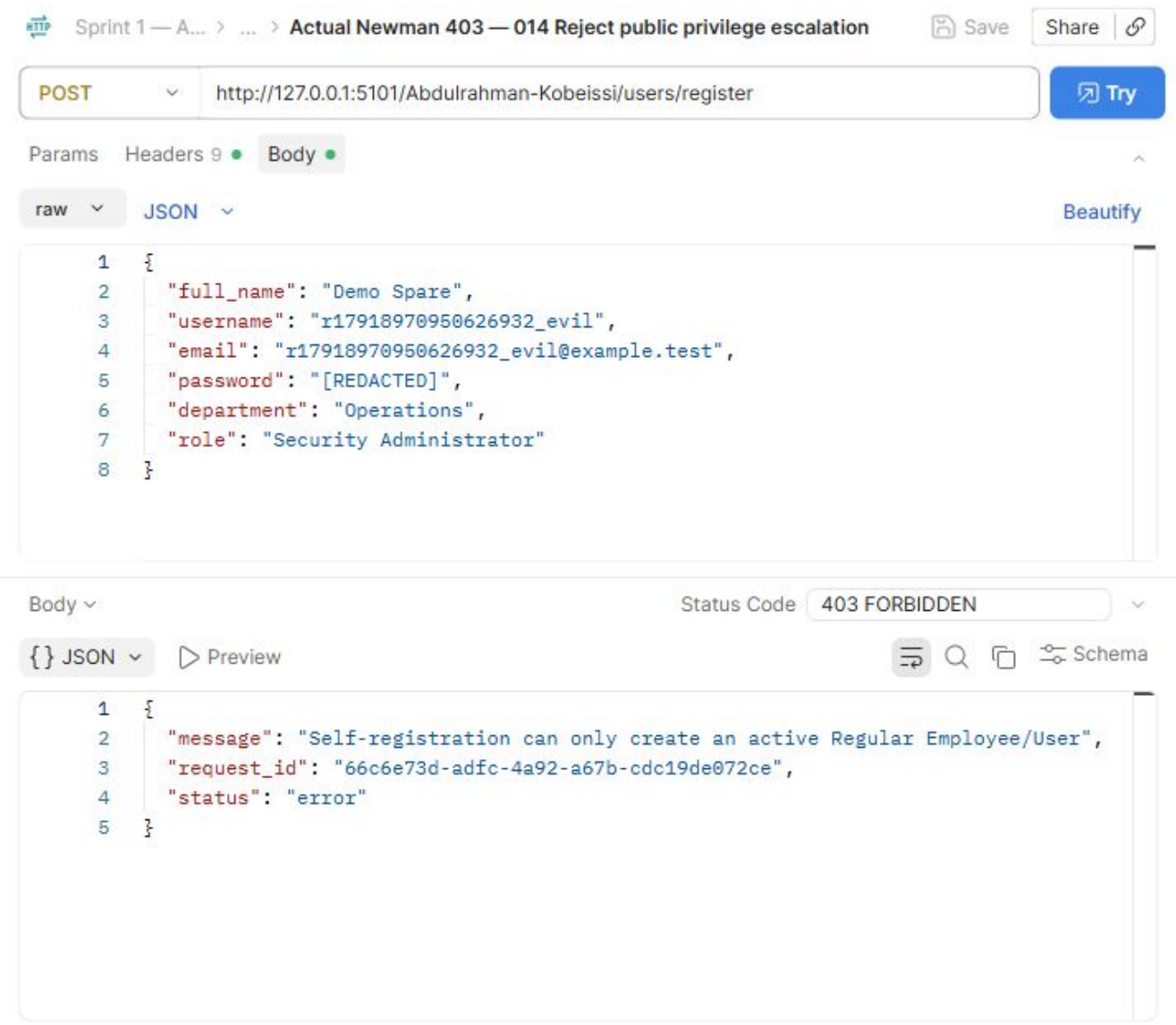


Abdulrahman Kobeissi — 013 Reject duplicate username. The request and saved actual response demonstrate this operation.

Postman Evidence 14

(Abdulrahman Kobeissi)

Reject public privilege escalation · POST · observed HTTP 403

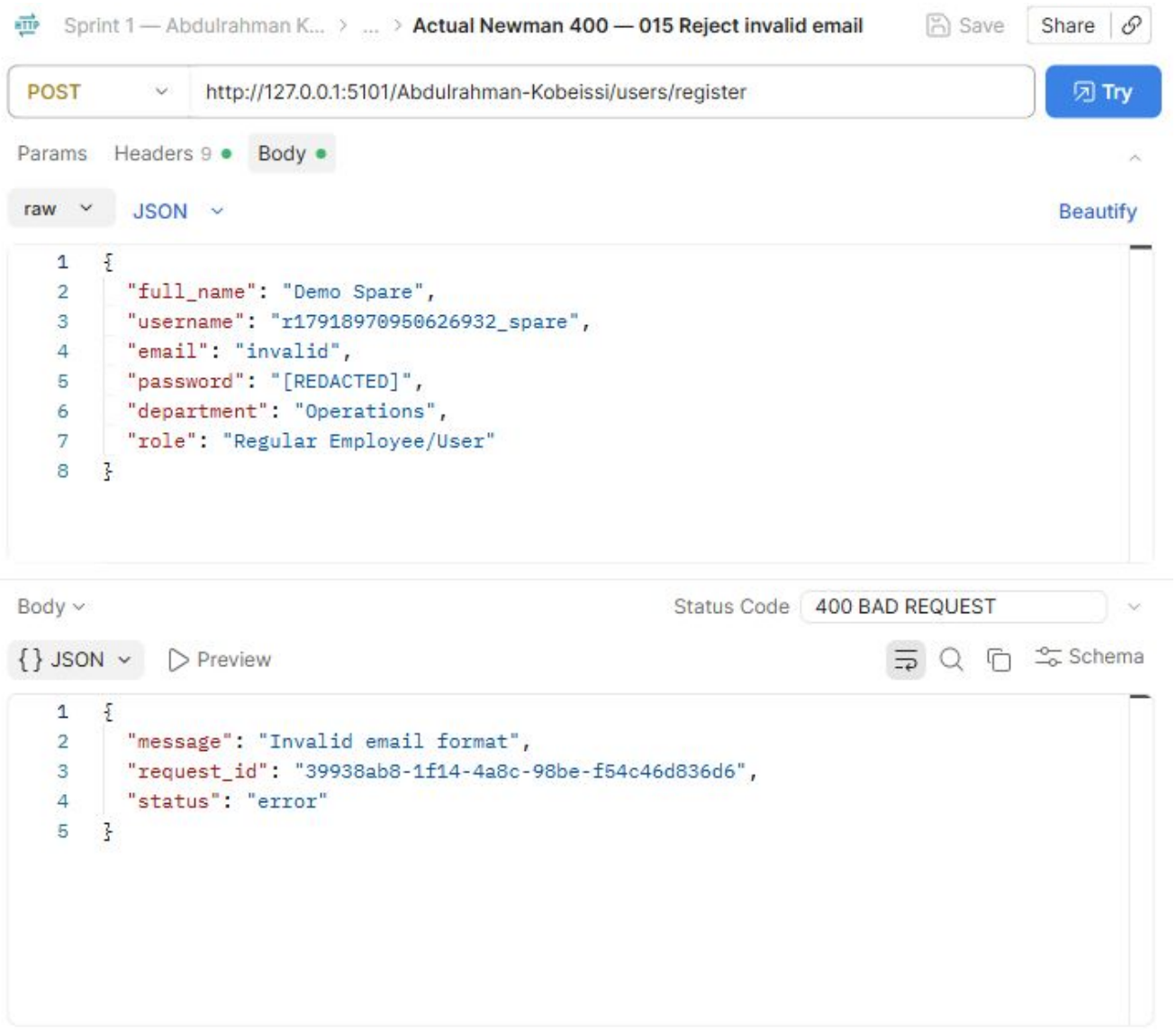


Abdulrahman Kobeissi — 014 Reject public privilege escalation. The request and saved actual response demonstrate this operation.

Postman Evidence 15

(Abdulrahman Kobeissi)

Reject invalid email · POST · observed HTTP 400

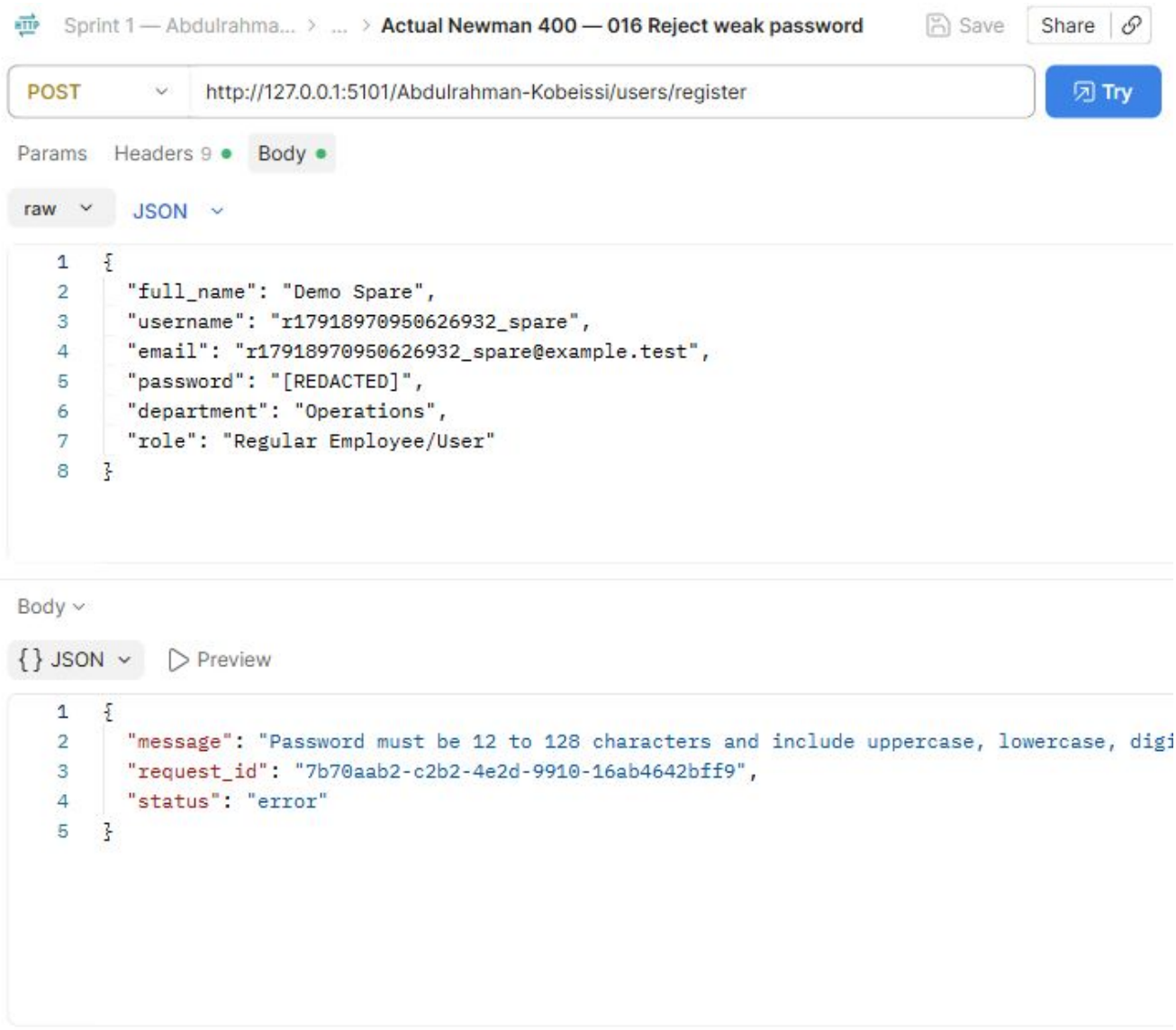


Abdulrahman Kobeissi — 015 Reject invalid email. The request and saved actual response demonstrate this operation.

Postman Evidence 16

(Abdulrahman Kobeissi)

Reject weak password · POST · observed HTTP 400

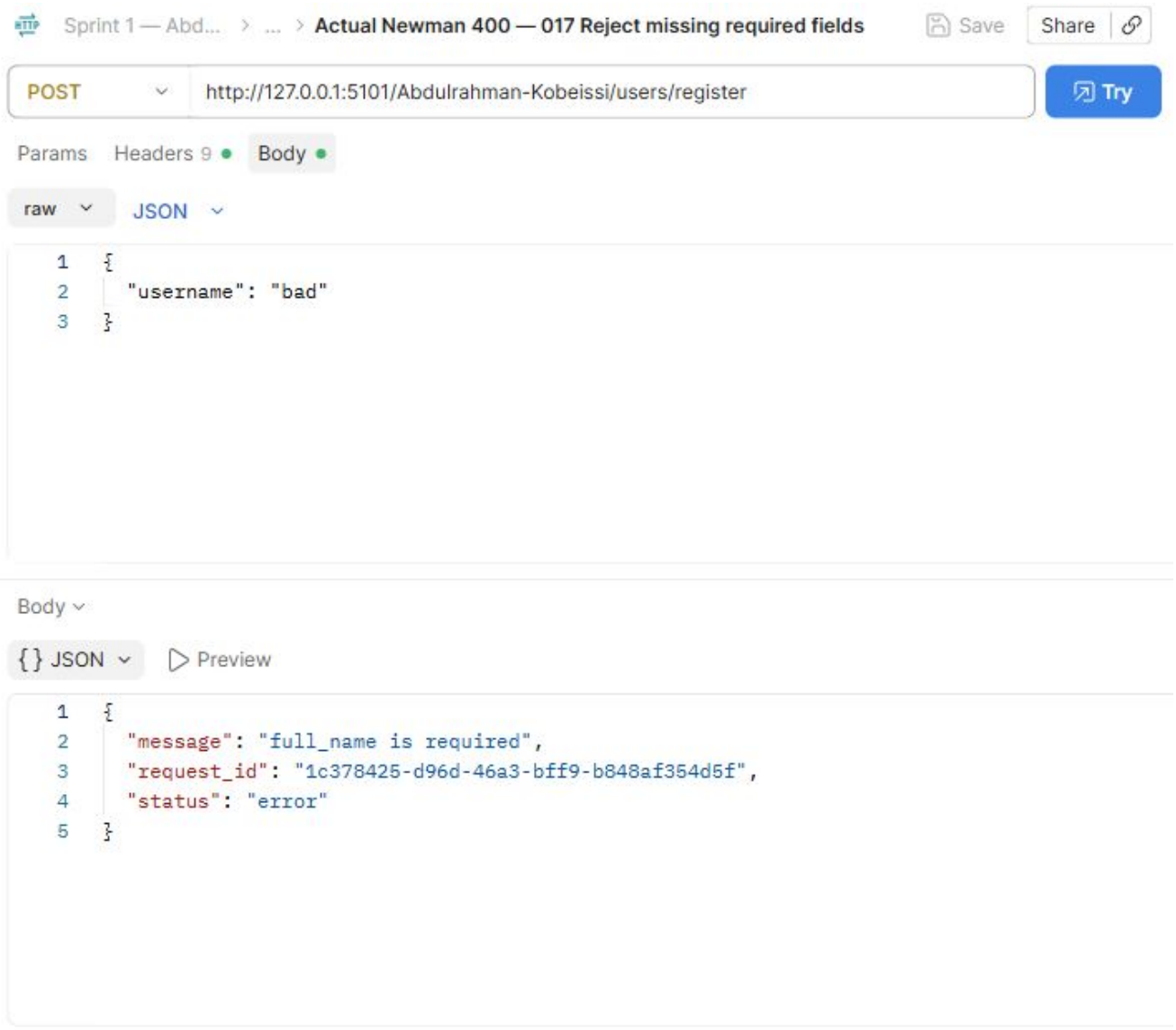


Abdulrahman Kobeissi — 016 Reject weak password. The request and saved actual response demonstrate this operation.

Postman Evidence 17

(Abdulrahman Kobeissi)

Reject missing required fields · POST · observed HTTP 400

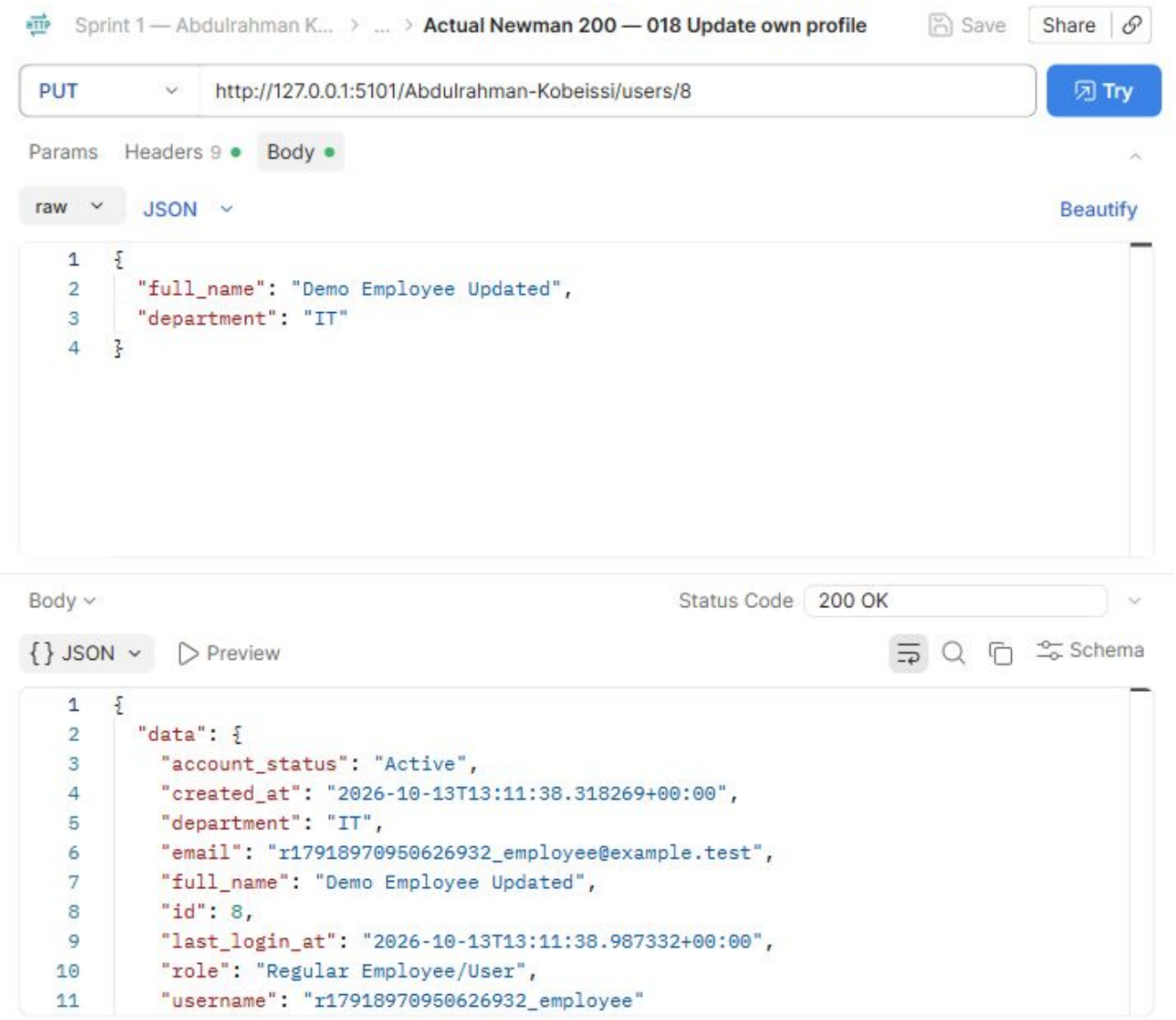


Abdulrahman Kobeissi — 017 Reject missing required fields. The request and saved actual response demonstrate this operation.

Postman Evidence 18

(Abdulrahman Kobeissi)

Update own profile · PUT · observed HTTP 200

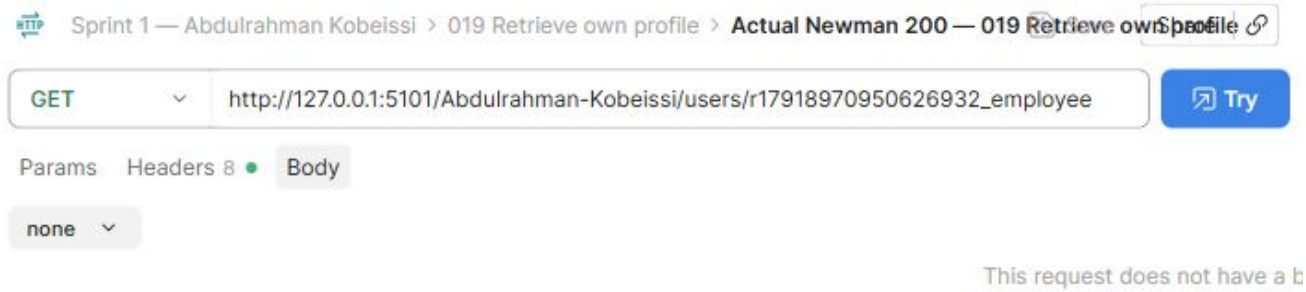


Abdulrahman Kobeissi — 018 Update own profile. The request and saved actual response demonstrate this operation.

Postman Evidence 19

(Abdulrahman Kobeissi)

Retrieve own profile · GET · observed HTTP 200



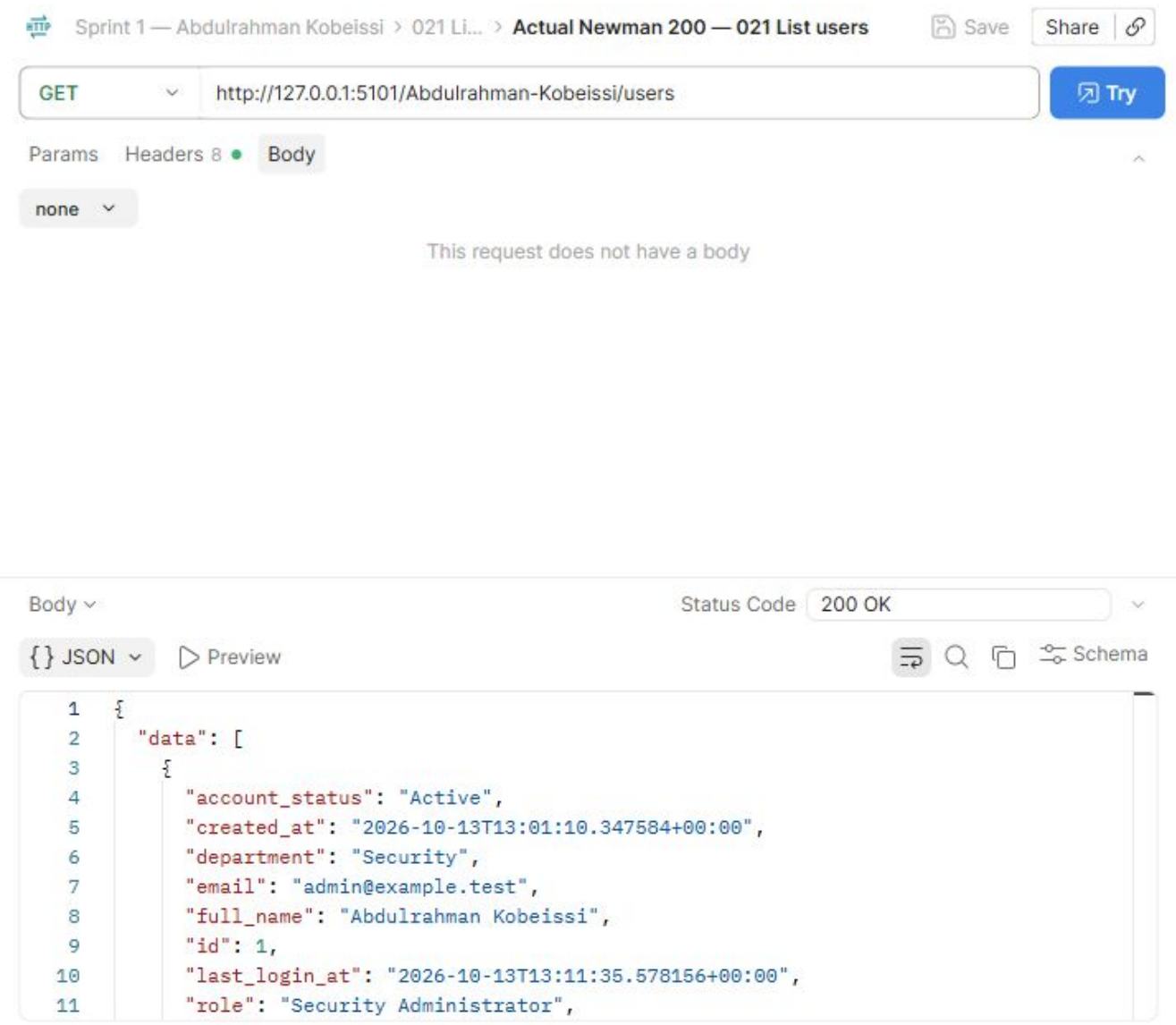


Abdulrahman Kobeissi — 019 Retrieve own profile. The request and saved actual response demonstrate this operation.

Postman Evidence 21

(Abdulrahman Kobeissi)

List users · GET · observed HTTP 200

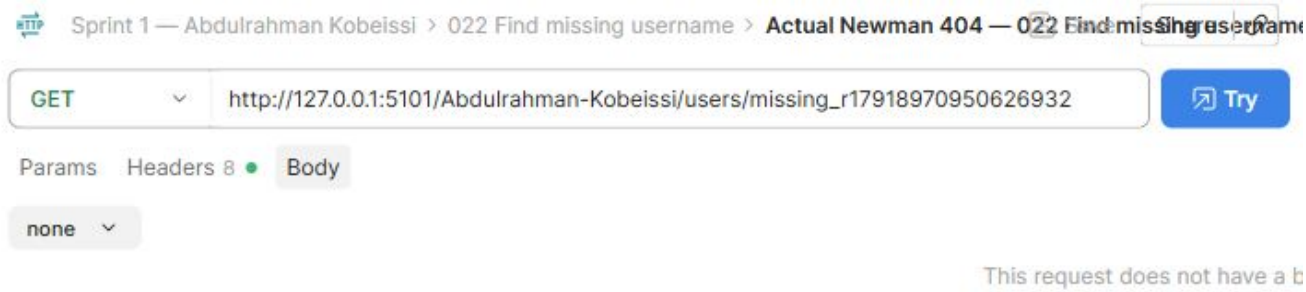


Abdulrahman Kobeissi — 021 List users. The request and saved actual response demonstrate this operation.

Postman Evidence 22

(Abdulrahman Kobeissi)

Find missing username · GET · observed HTTP 404



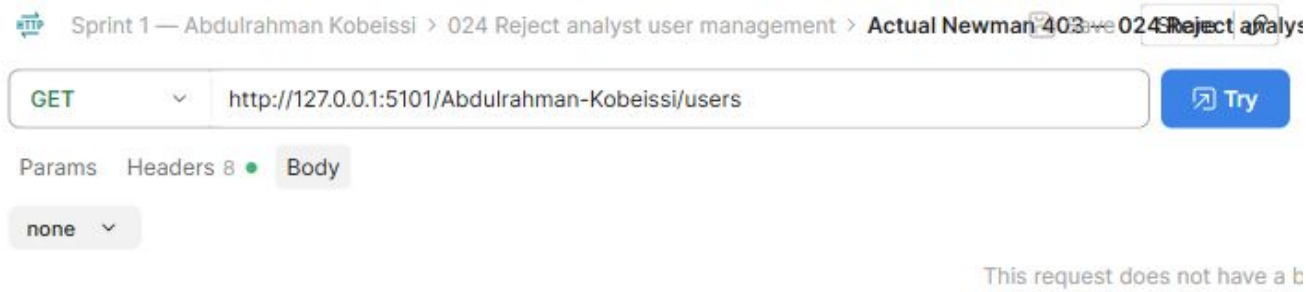


Abdulrahman Kobeissi — 022 Find missing username. The request and saved actual response demonstrate this operation.

Postman Evidence 24

(Abdulrahman Kobeissi)

Reject analyst user management · GET · observed HTTP 403



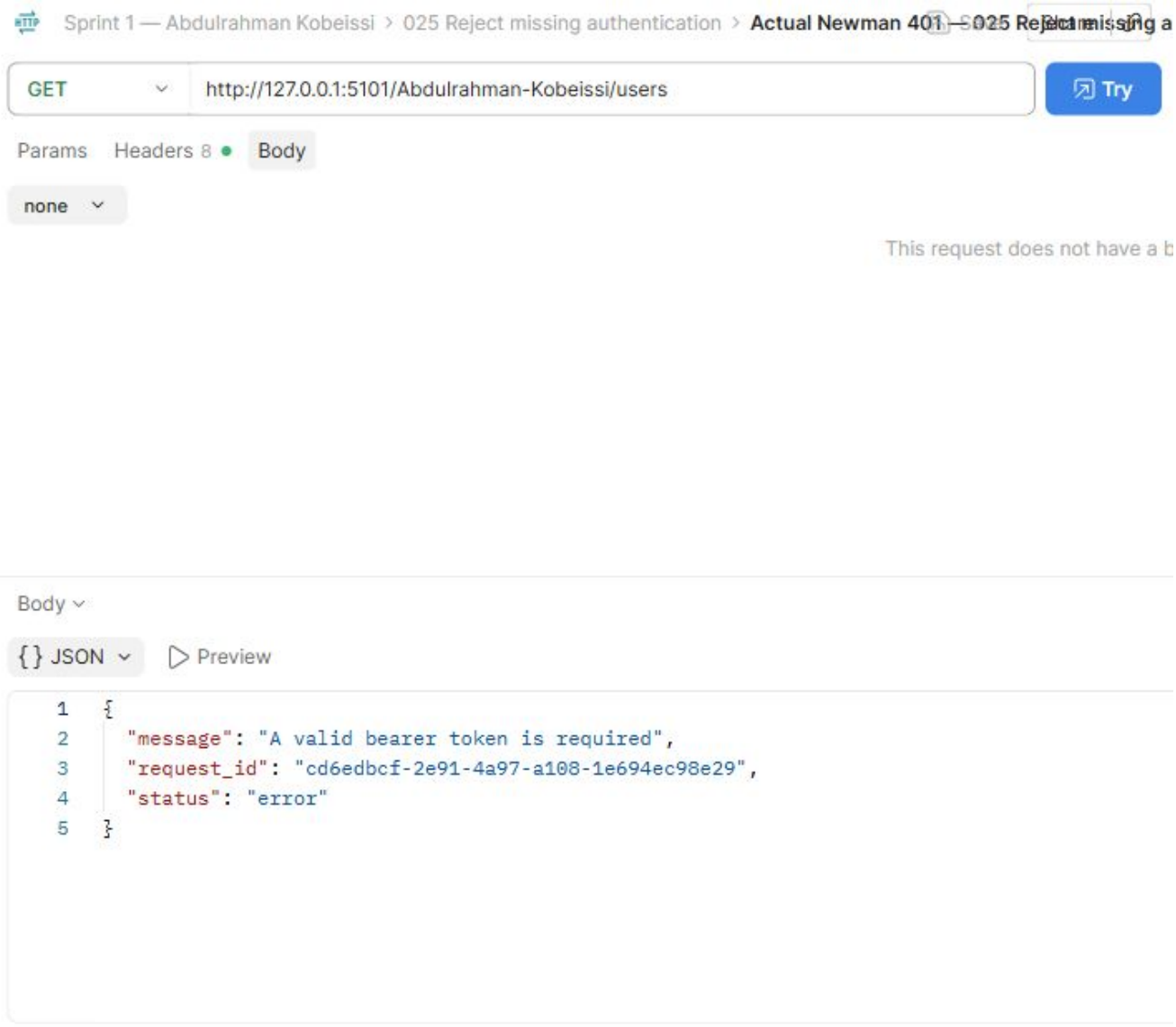


Abdulrahman Kobeissi — 024 Reject analyst user management. The request and saved actual response demonstrate this operation.

Postman Evidence 25

(Abdulrahman Kobeissi)

Reject missing authentication · GET · observed HTTP 401

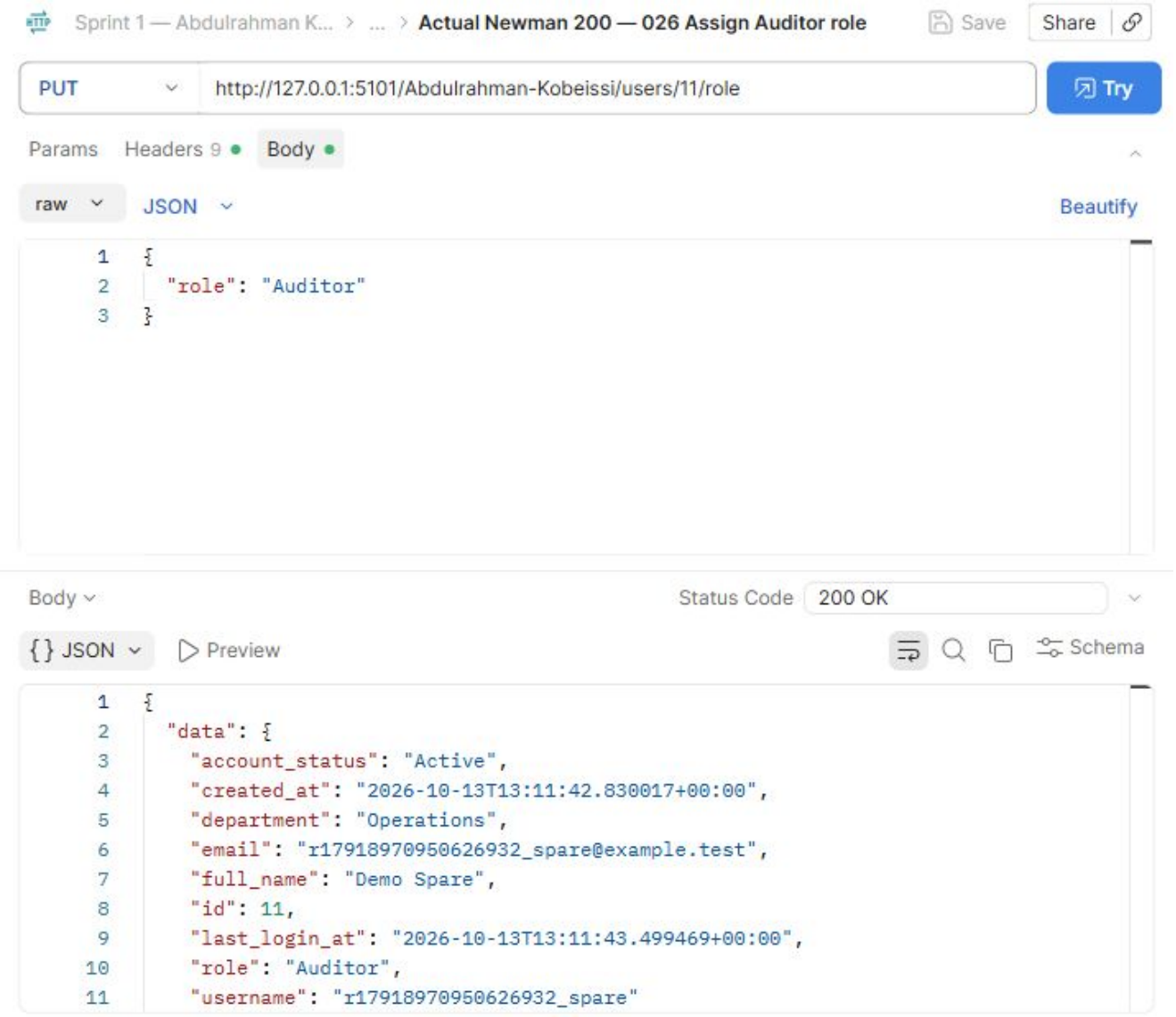


Abdulrahman Kobeissi — 025 Reject missing authentication. The request and saved actual response demonstrate this operation.

Postman Evidence 26

(Abdulrahman Kobeissi)

Assign Auditor role · PUT · observed HTTP 200

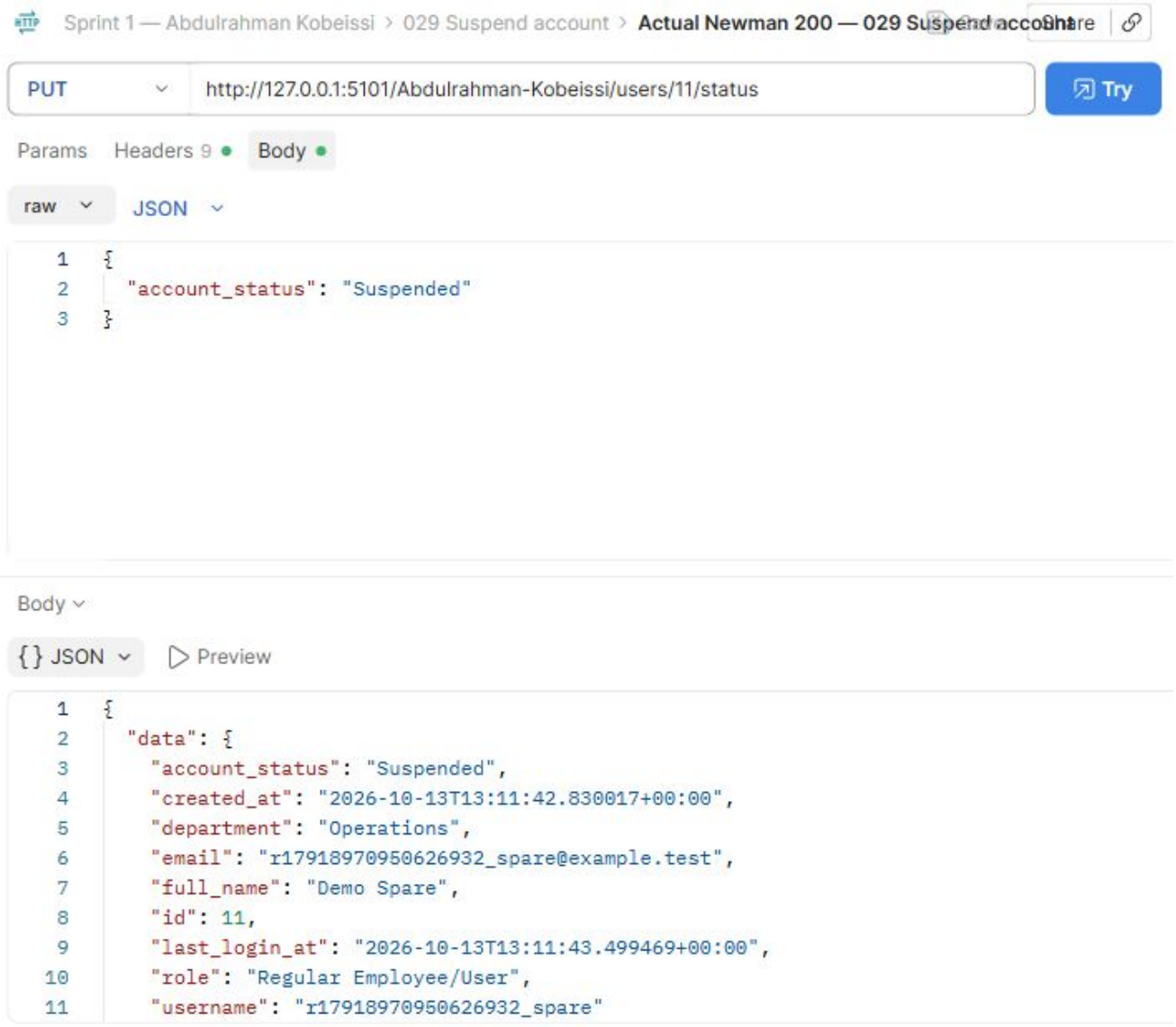


Abdulrahman Kobeissi — 026 Assign Auditor role. The request and saved actual response demonstrate this operation.

Postman Evidence 29

(Abdulrahman Kobeissi)

Suspend account · PUT · observed HTTP 200

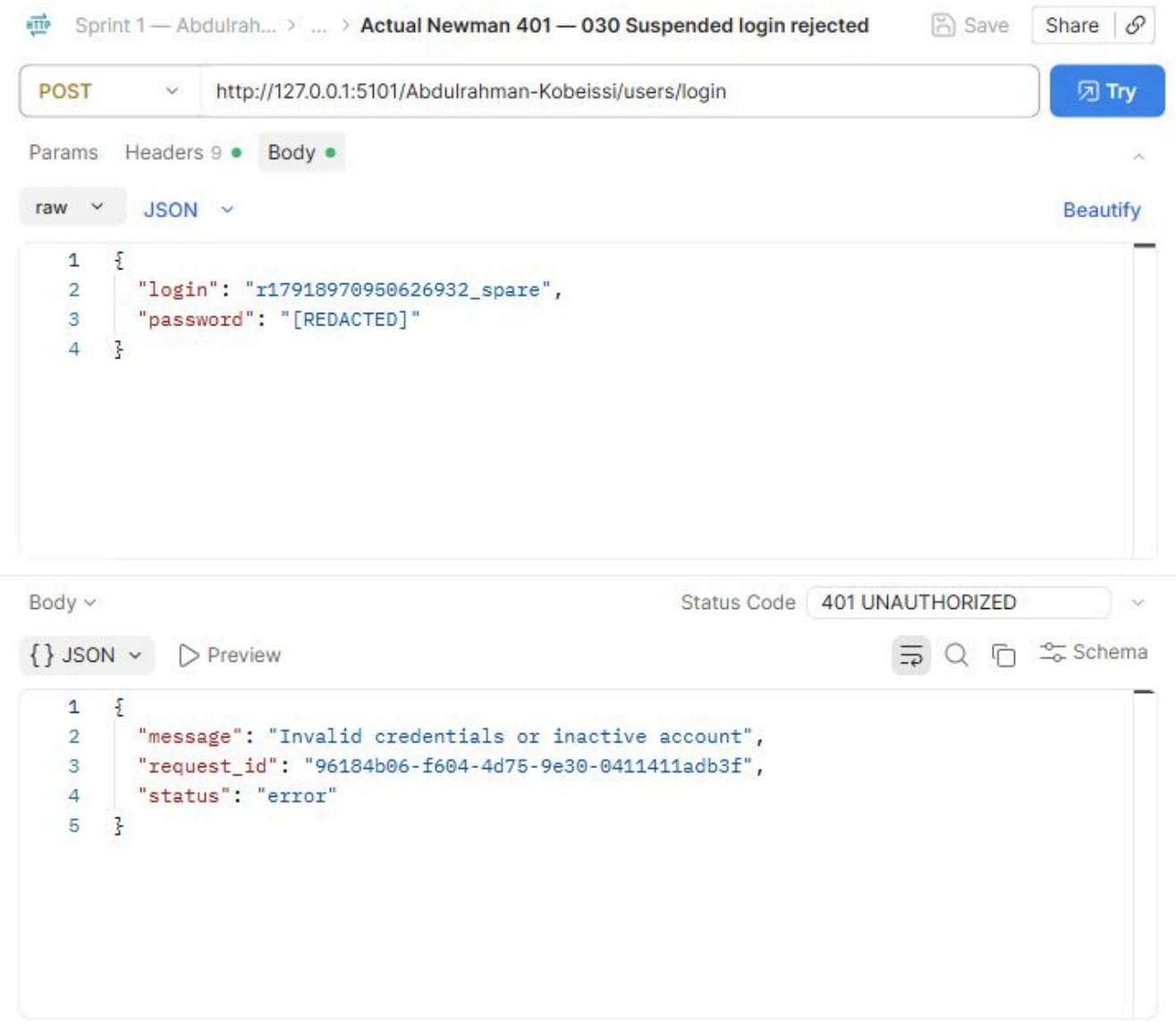


Abdulrahman Kobeissi — 029 Suspend account. The request and saved actual response demonstrate this operation.

Postman Evidence 30

(Abdulrahman Kobeissi)

Suspended login rejected · POST · observed HTTP 401

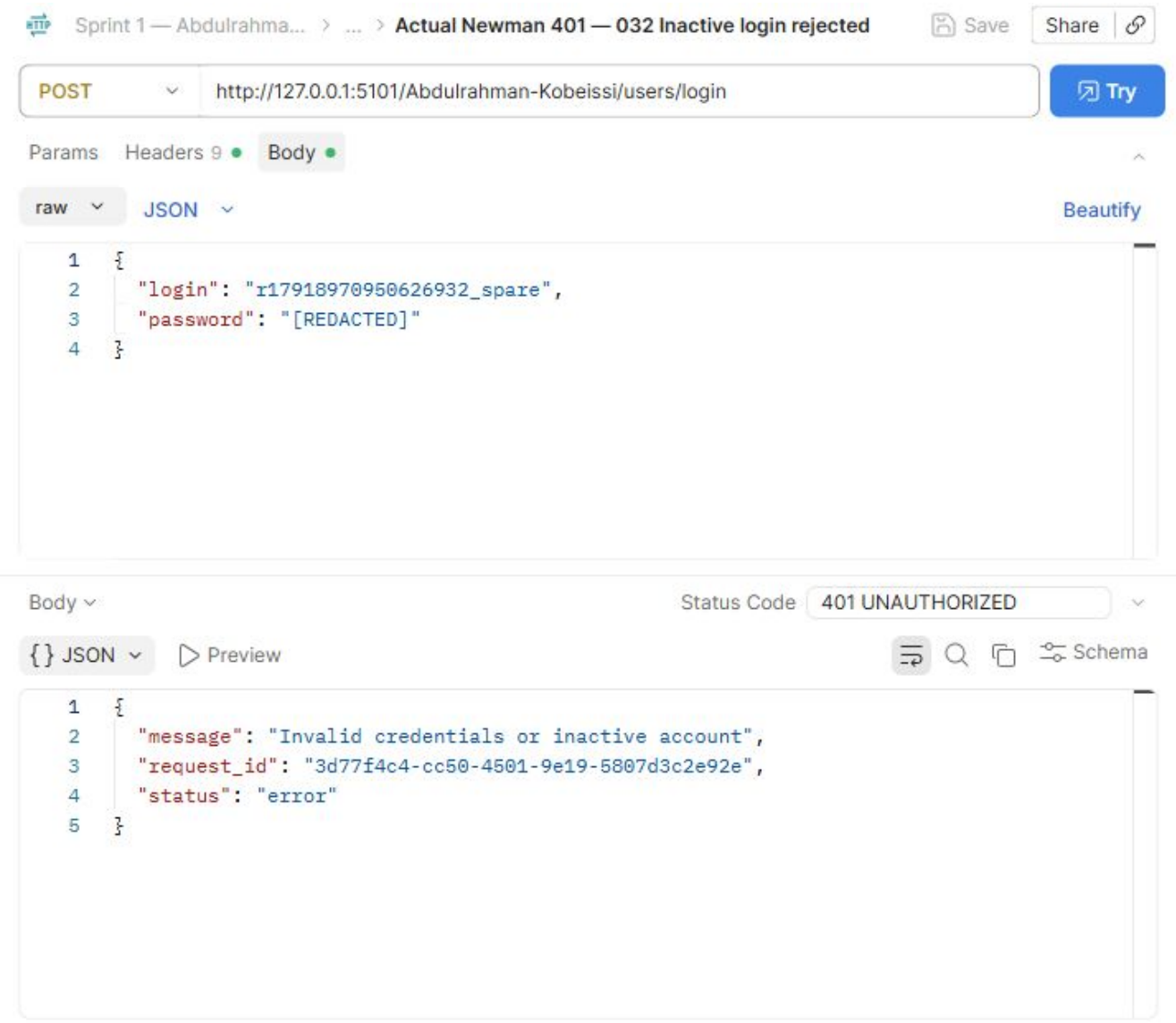


Abdulrahman Kobeissi — 030 Suspended login rejected. The request and saved actual response demonstrate this operation.

Postman Evidence 32

(Abdulrahman Kobeissi)

Inactive login rejected · POST · observed HTTP 401

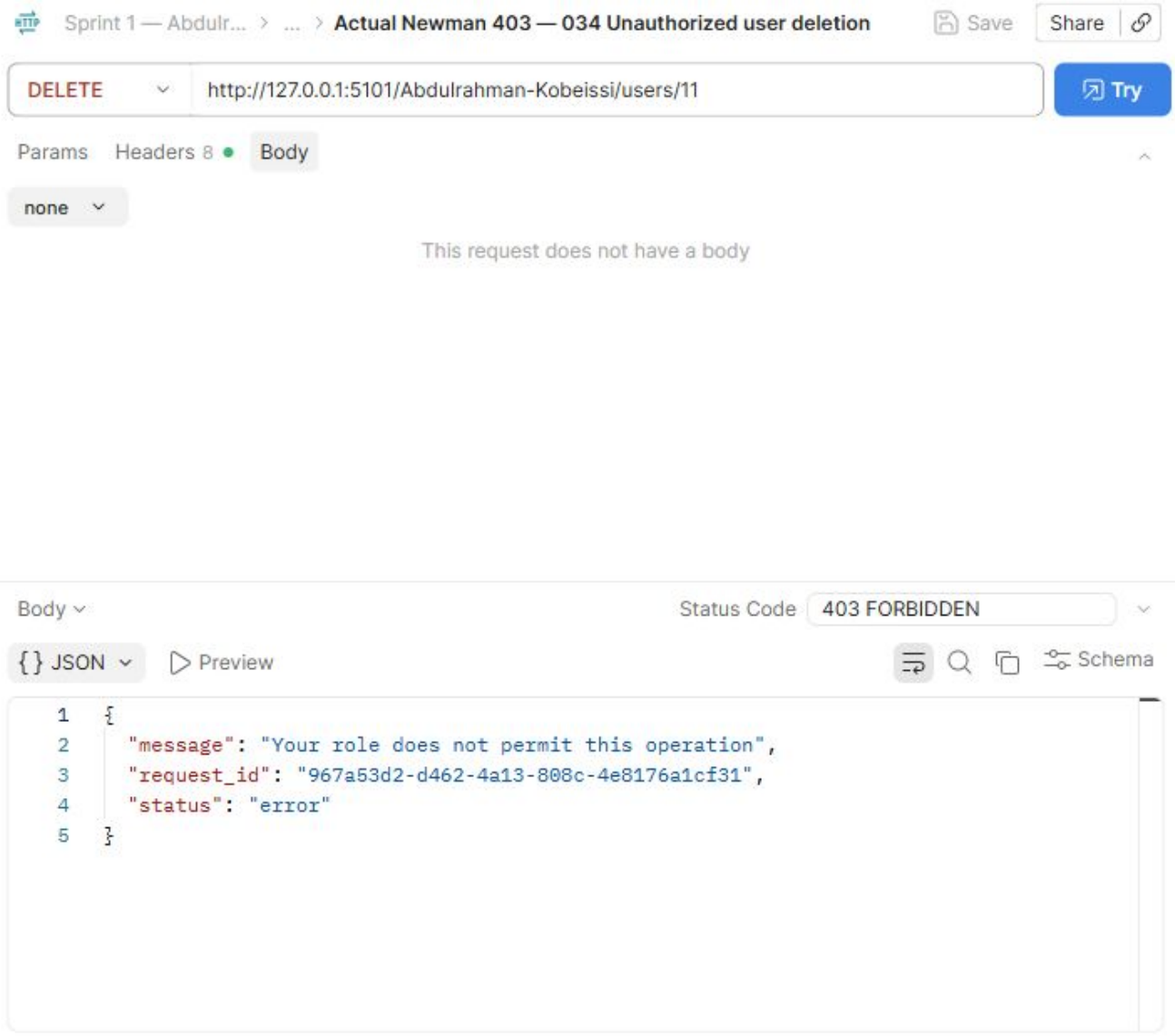


Abdulrahman Kobeissi — 032 Inactive login rejected. The request and saved actual response demonstrate this operation.

Postman Evidence 34

(Abdulrahman Kobeissi)

Unauthorized user deletion · DELETE · observed HTTP 403

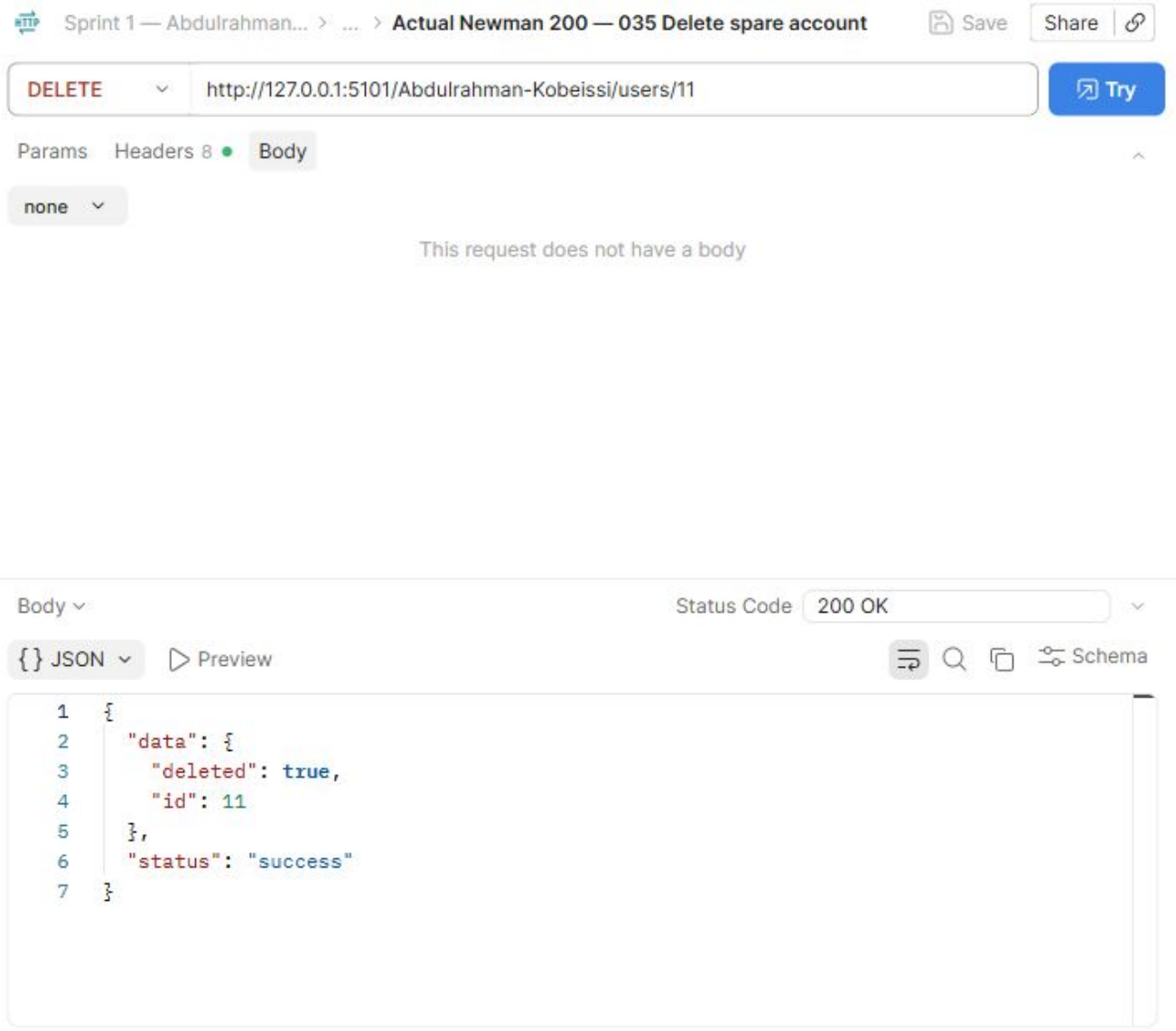


Abdulrahman Kobeissi — 034 Unauthorized user deletion. The request and saved actual response demonstrate this operation.

Postman Evidence 35

(Abdulrahman Kobeissi)

Delete spare account · DELETE · observed HTTP 200

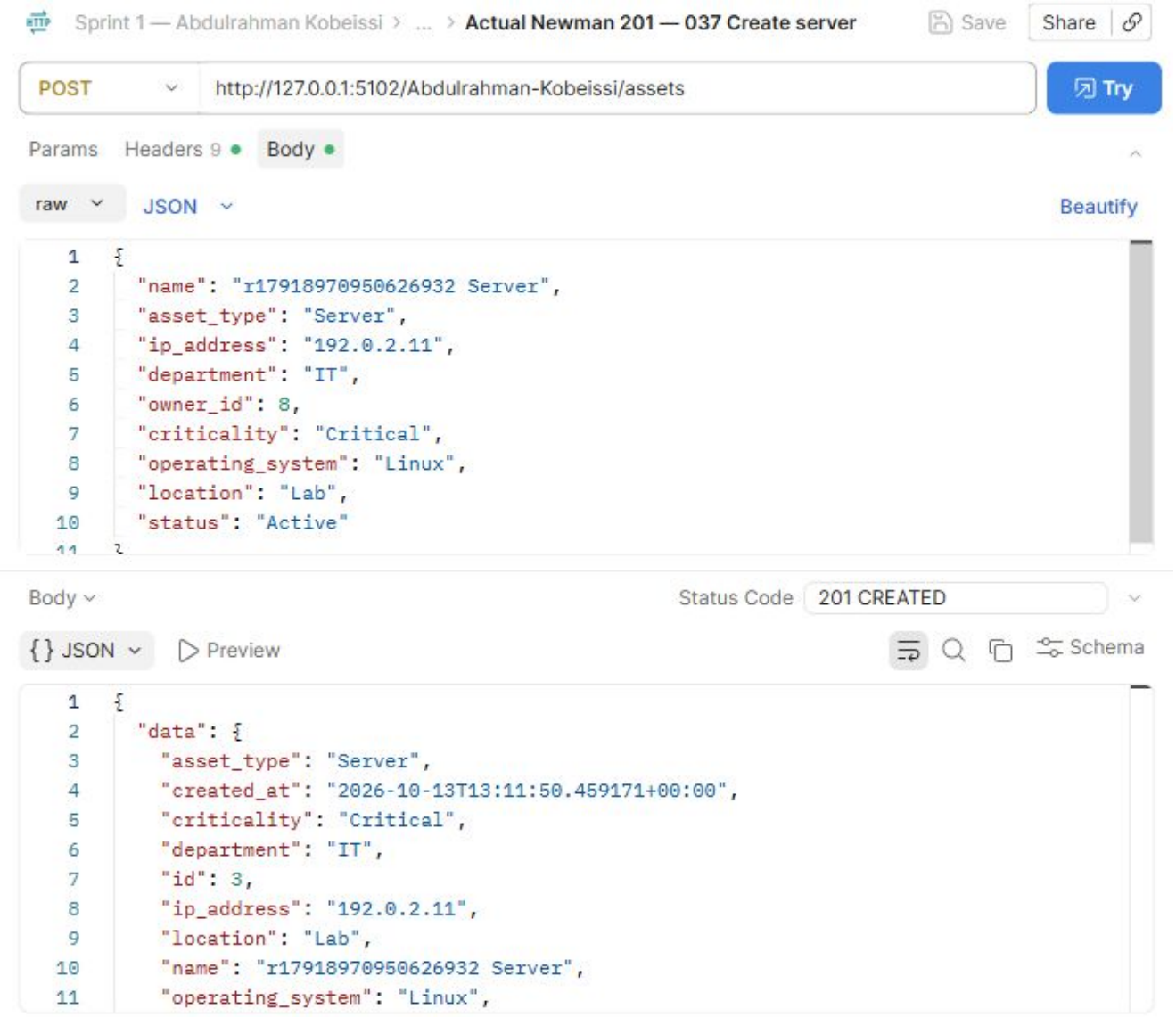


Abdulrahman Kobeissi — 035 Delete spare account. The request and saved actual response demonstrate this operation.

Postman Evidence 37

(Abdulrahman Kobeissi)

Create server · POST · observed HTTP 201

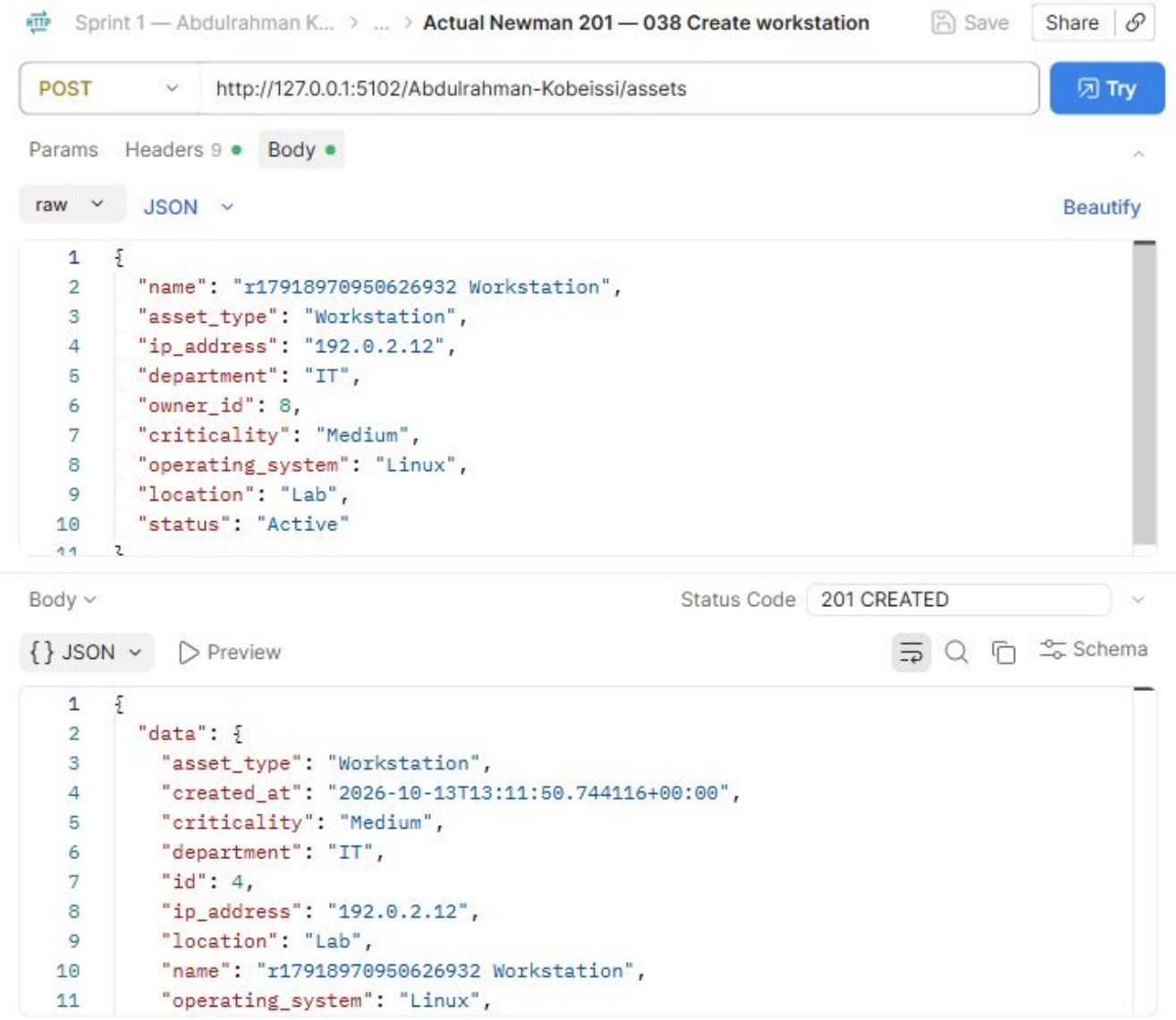


Abdulrahman Kobeissi — 037 Create server. The request and saved actual response demonstrate this operation.

Postman Evidence 38

(Abdulrahman Kobeissi)

Create workstation · POST · observed HTTP 201

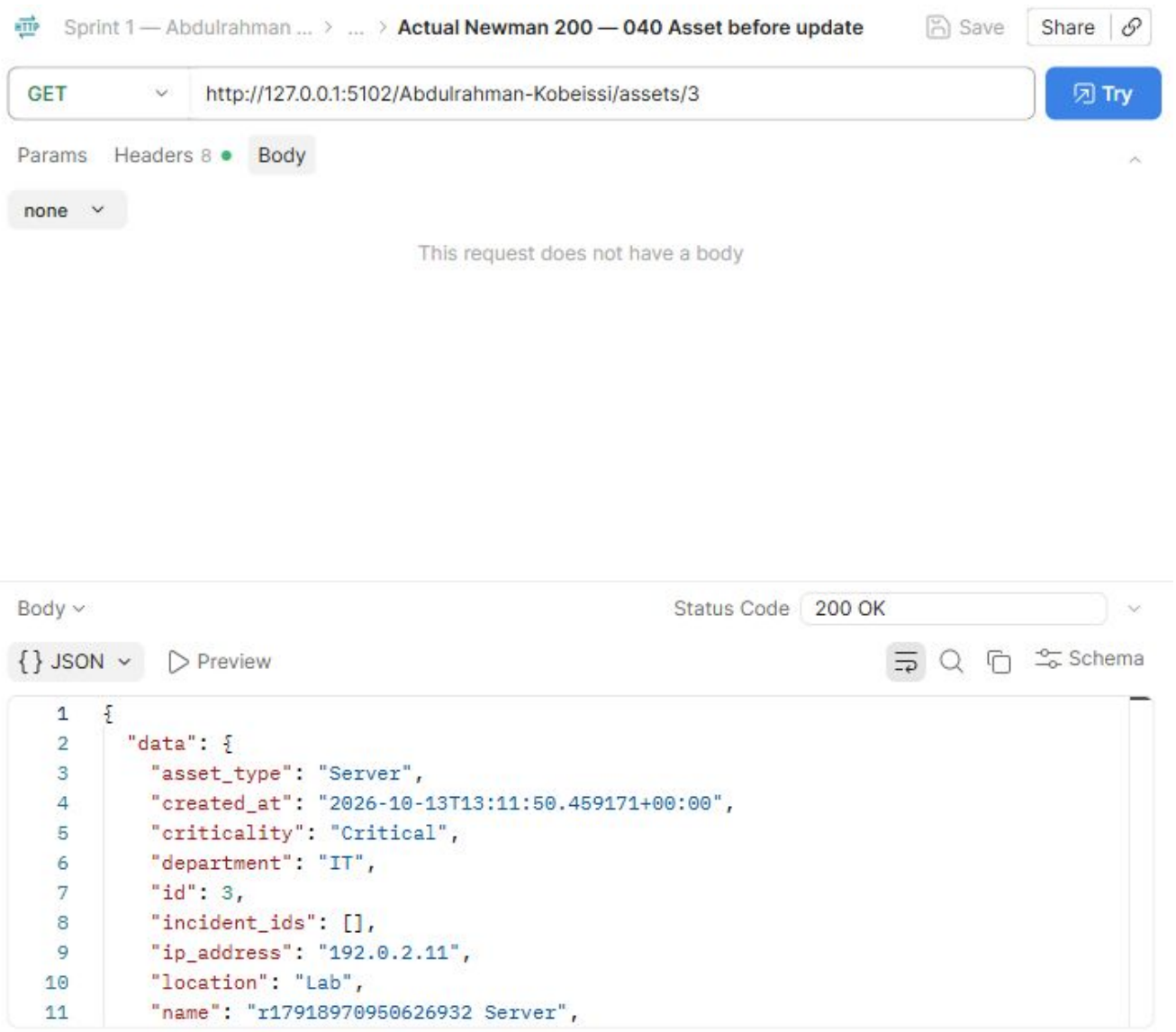


Abdulrahman Kobeissi — 038 Create workstation. The request and saved actual response demonstrate this operation.

Postman Evidence 40

(Abdulrahman Kobeissi)

Asset before update · GET · observed HTTP 200



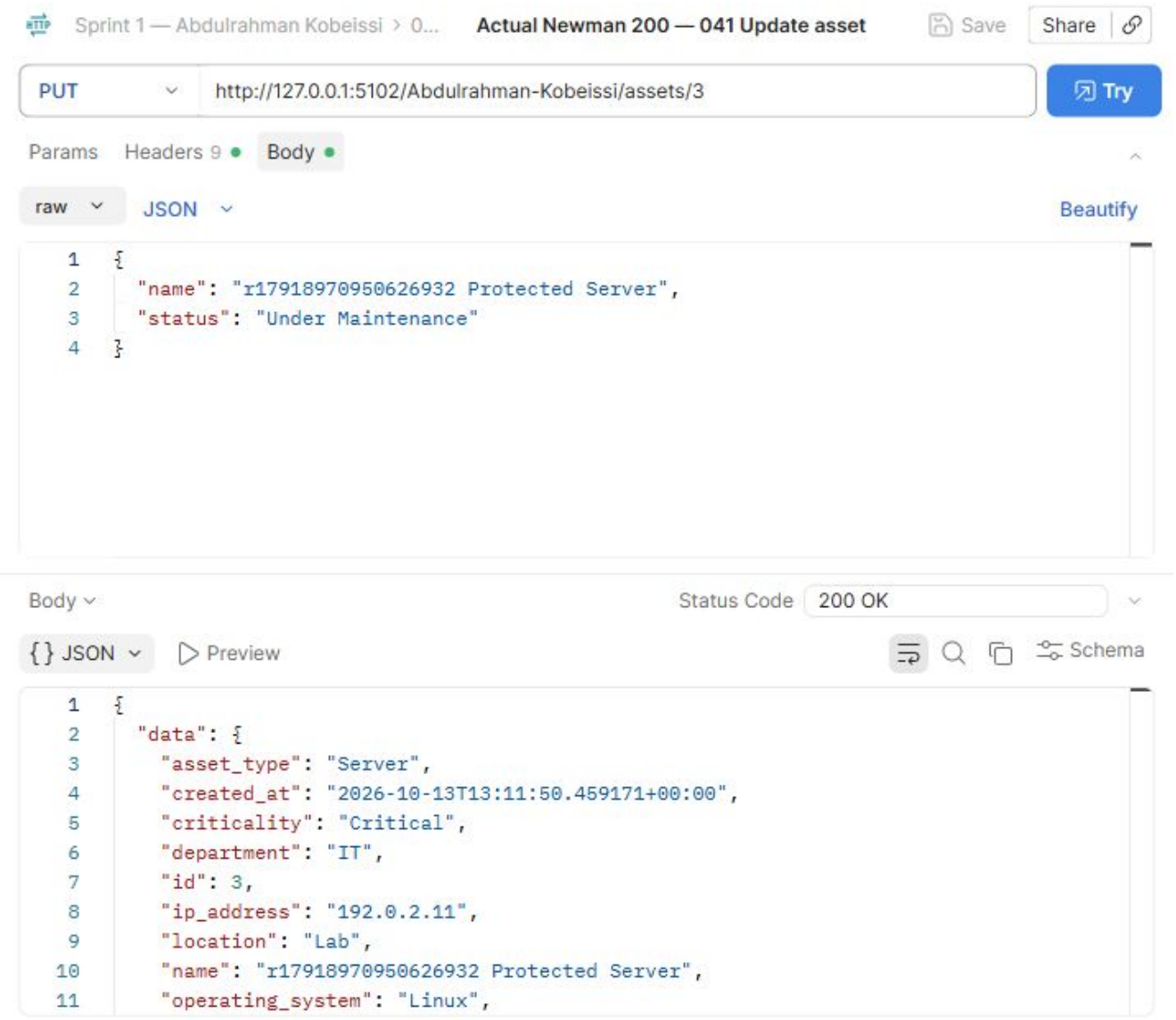
Abdulrahman Kobeissi — 040 Asset before update. The request and saved actual response demonstrate this operation.

Compare this view with the adjacent before/update/after requests. The full response remains available in the collection.

Postman Evidence 41

(Abdulrahman Kobeissi)

Update asset · PUT · observed HTTP 200

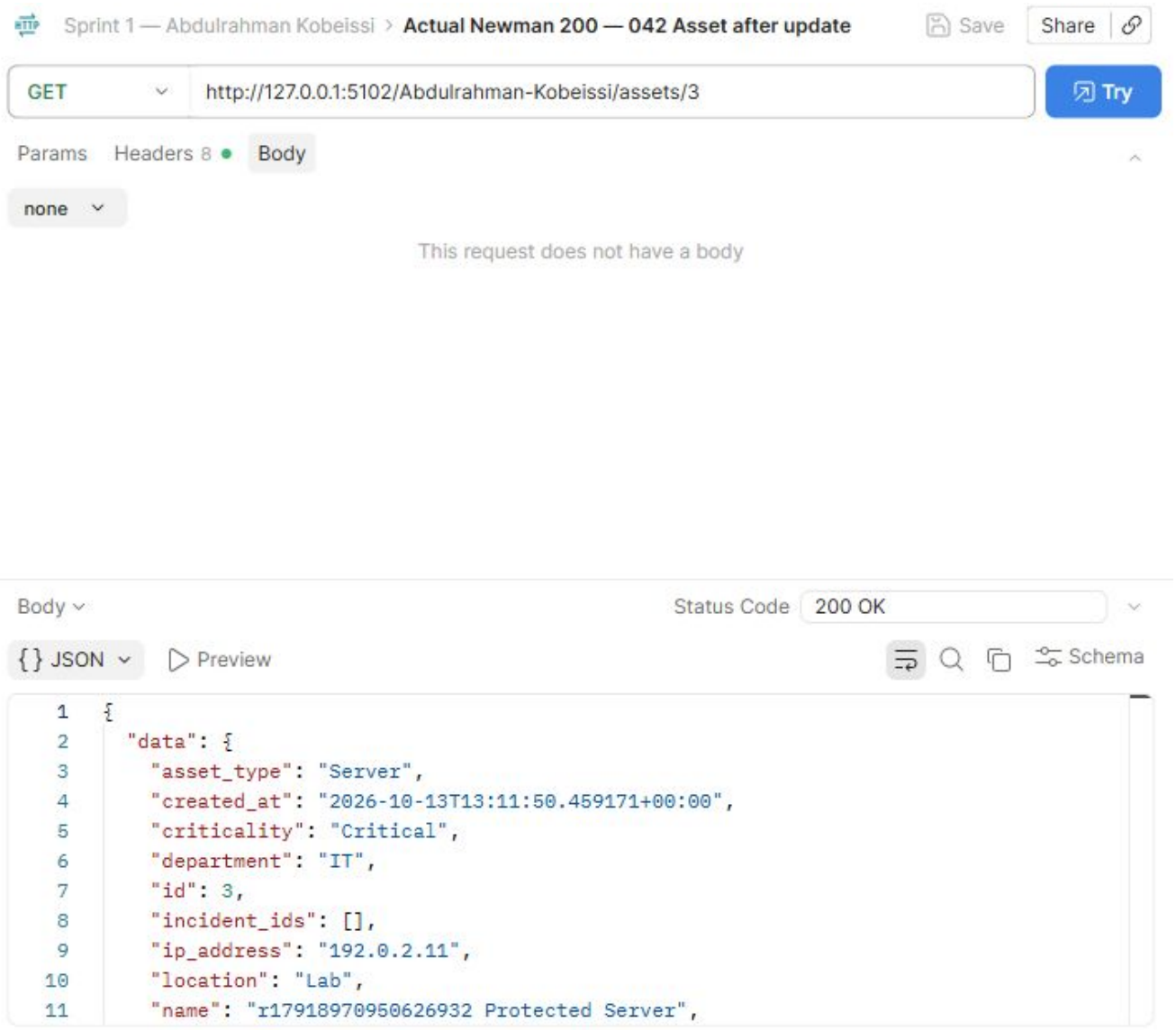


Abdulrahman Kobeissi — 041 Update asset. The request and saved actual response demonstrate this operation.

Postman Evidence 42

(Abdulrahman Kobeissi)

Asset after update · GET · observed HTTP 200



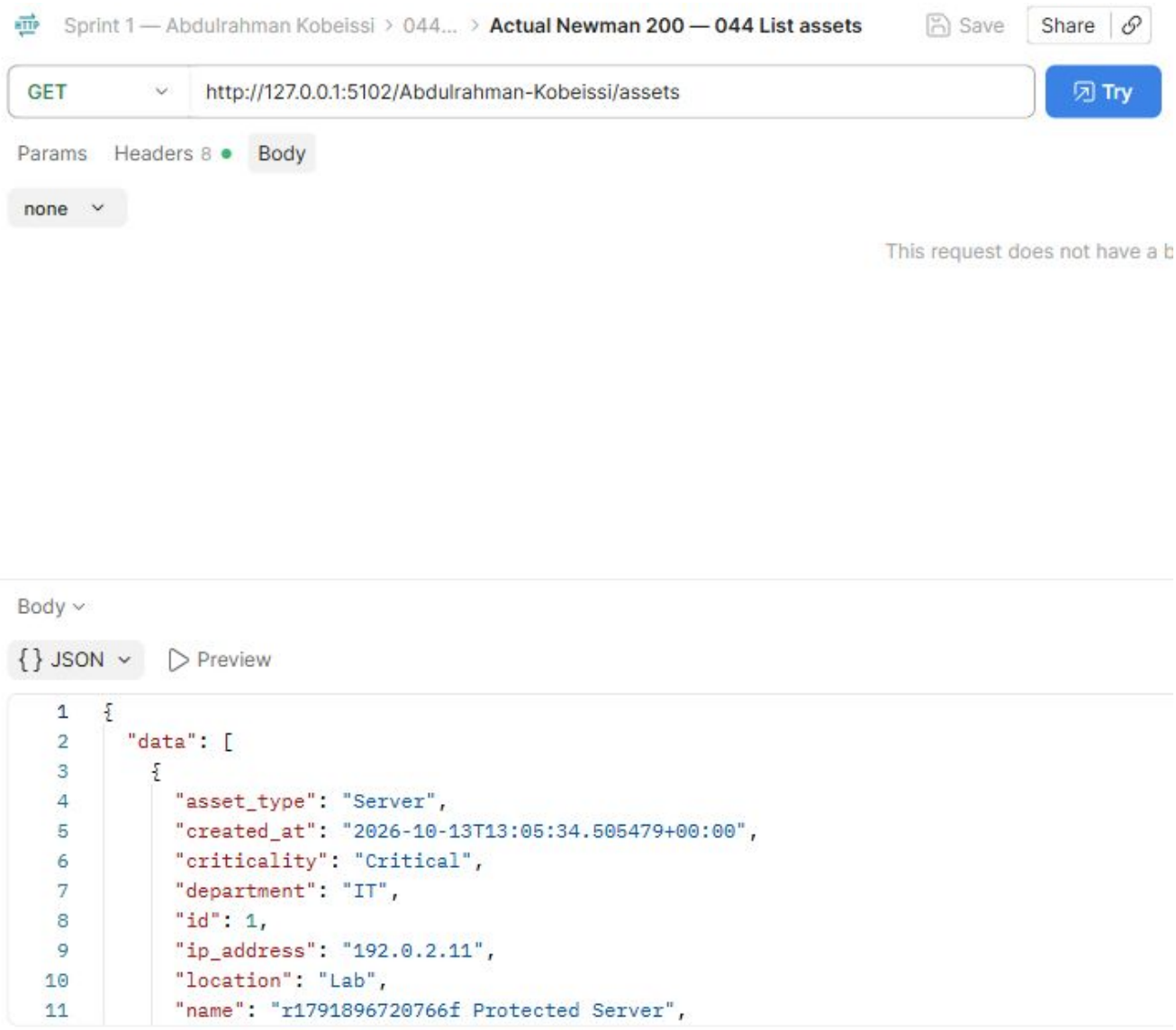
Abdulrahman Kobeissi — 042 Asset after update. The request and saved actual response demonstrate this operation.

Compare this view with the adjacent before/update/after requests. The full response remains available in the collection.

Postman Evidence 44

(Abdulrahman Kobeissi)

List assets · GET · observed HTTP 200

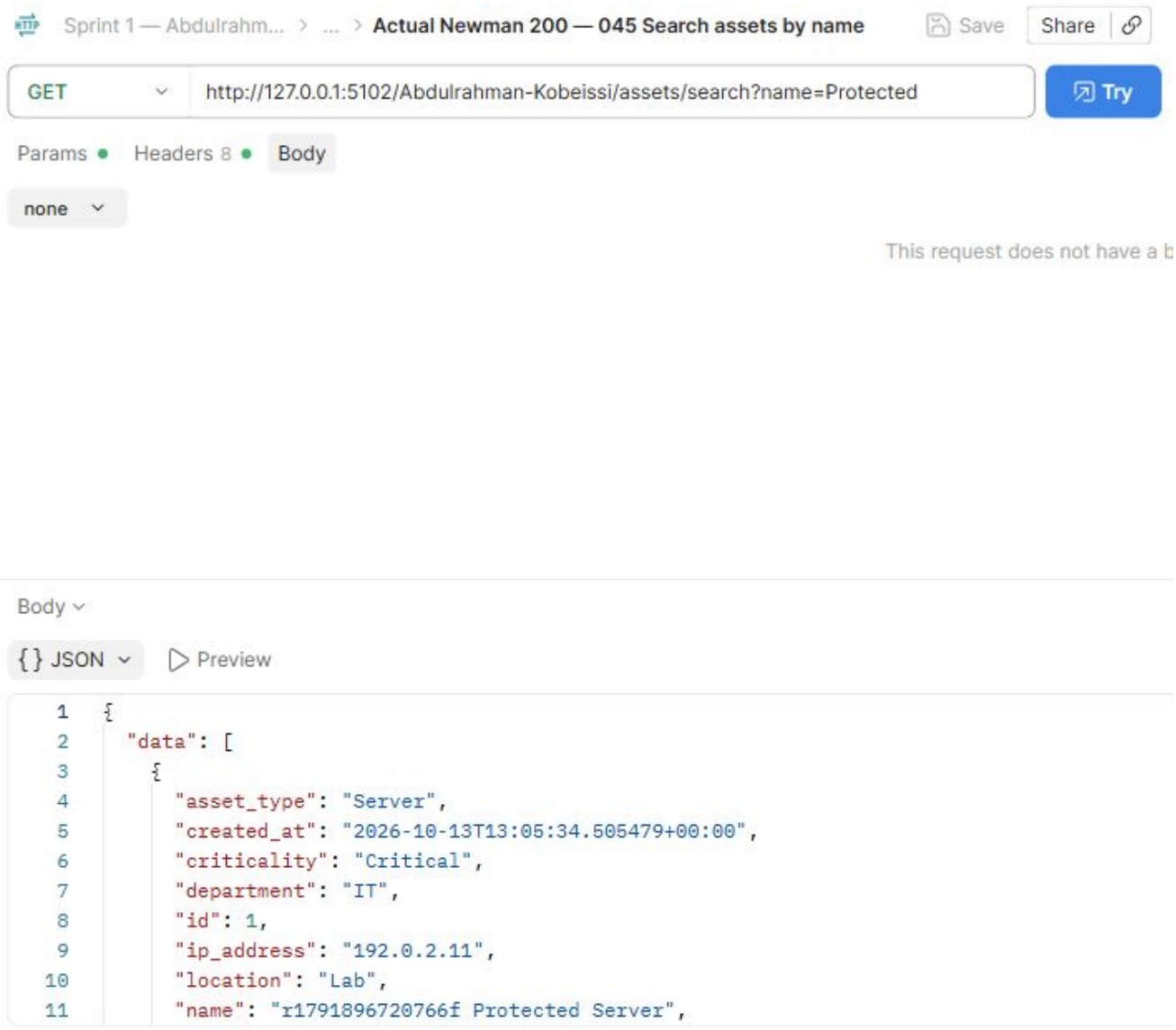


Abdulrahman Kobeissi — 044 List assets. The request and saved actual response demonstrate this operation.

Postman Evidence 45

(Abdulrahman Kobeissi)

Search assets by name · GET · observed HTTP 200

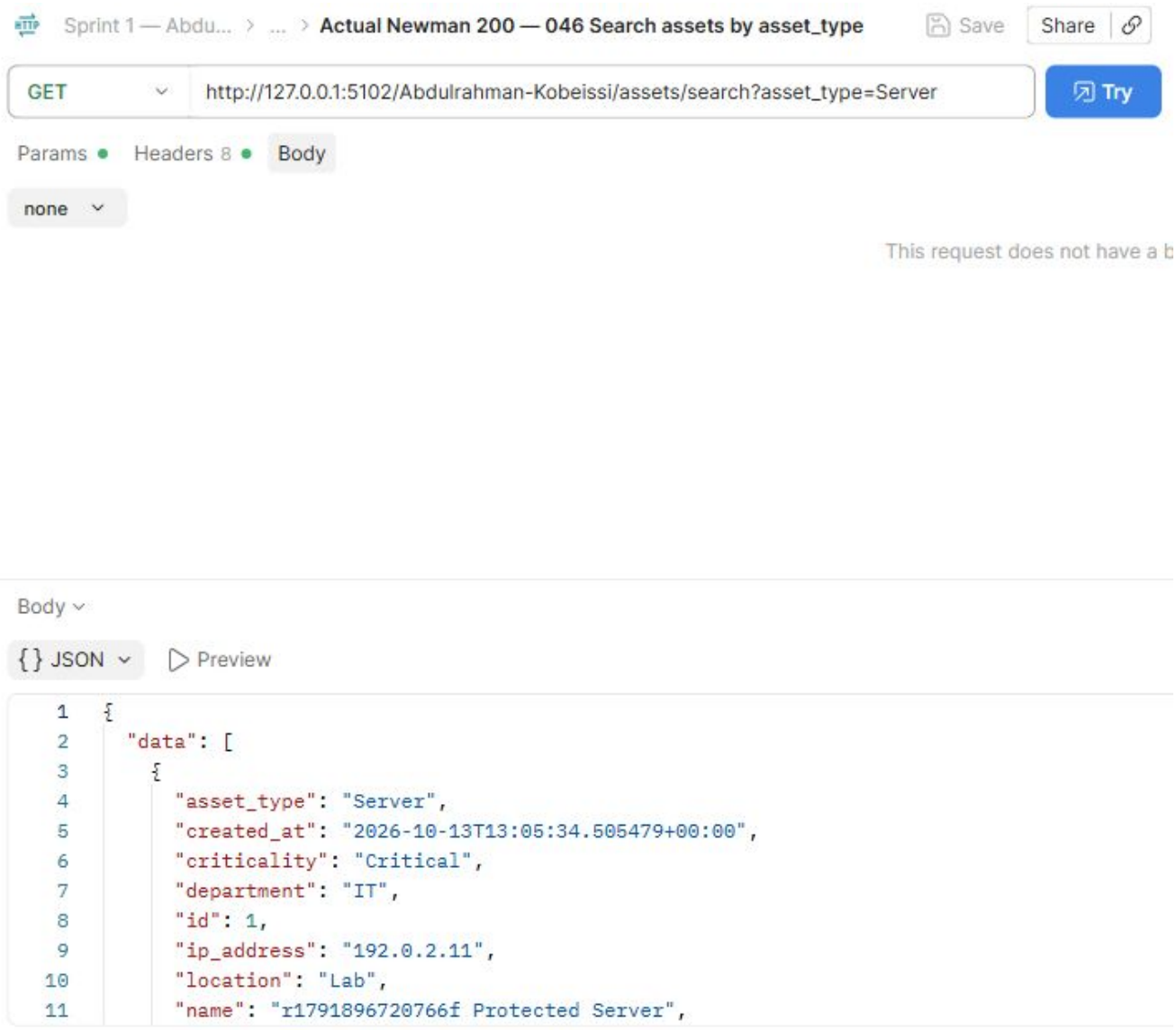


Abdulrahman Kobeissi — 045 Search assets by name. The request and saved actual response demonstrate this operation.

Postman Evidence 46

(Abdulrahman Kobeissi)

Search assets by asset_type · GET · observed HTTP 200

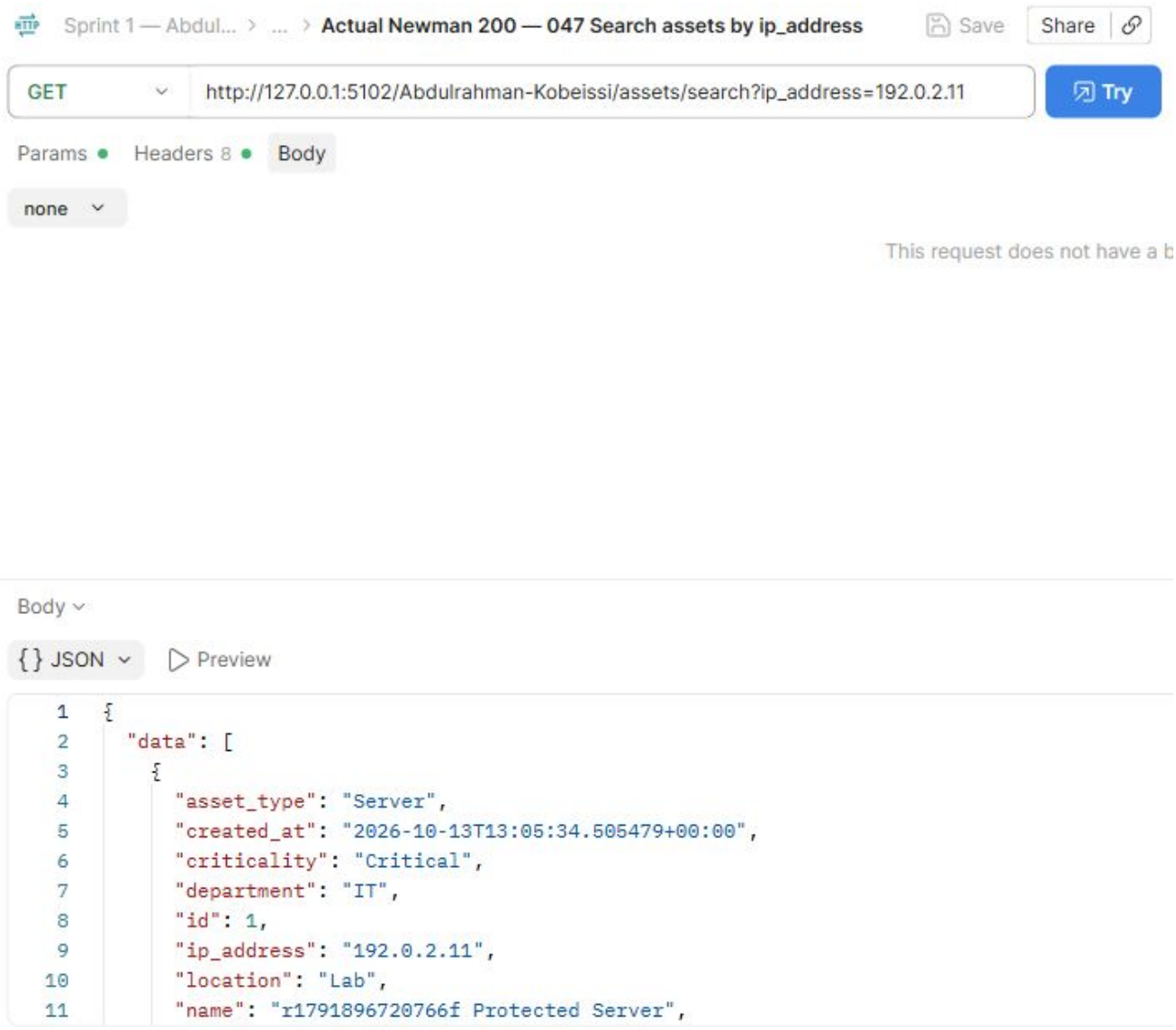


Abdulrahman Kobeissi — 046 Search assets by asset_type. The request and saved actual response demonstrate this operation.

Postman Evidence 47

(Abdulrahman Kobeissi)

Search assets by ip_address · GET · observed HTTP 200

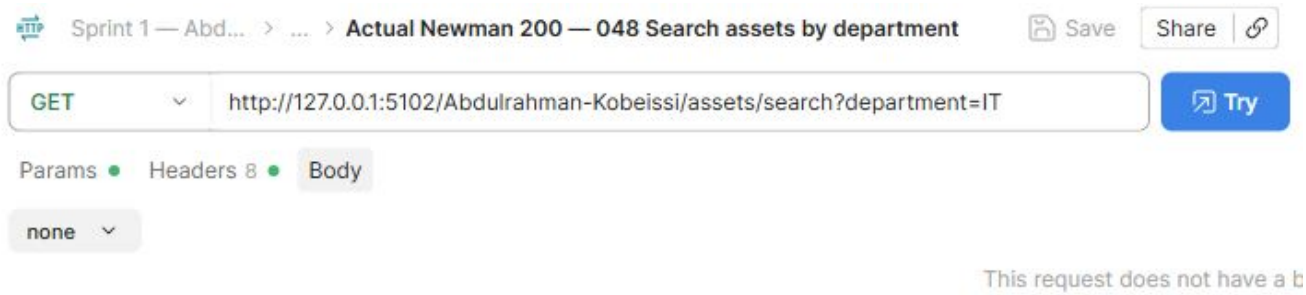


Abdulrahman Kobeissi — 047 Search assets by ip_address. The request and saved actual response demonstrate this operation.

Postman Evidence 48

(Abdulrahman Kobeissi)

Search assets by department · GET · observed HTTP 200



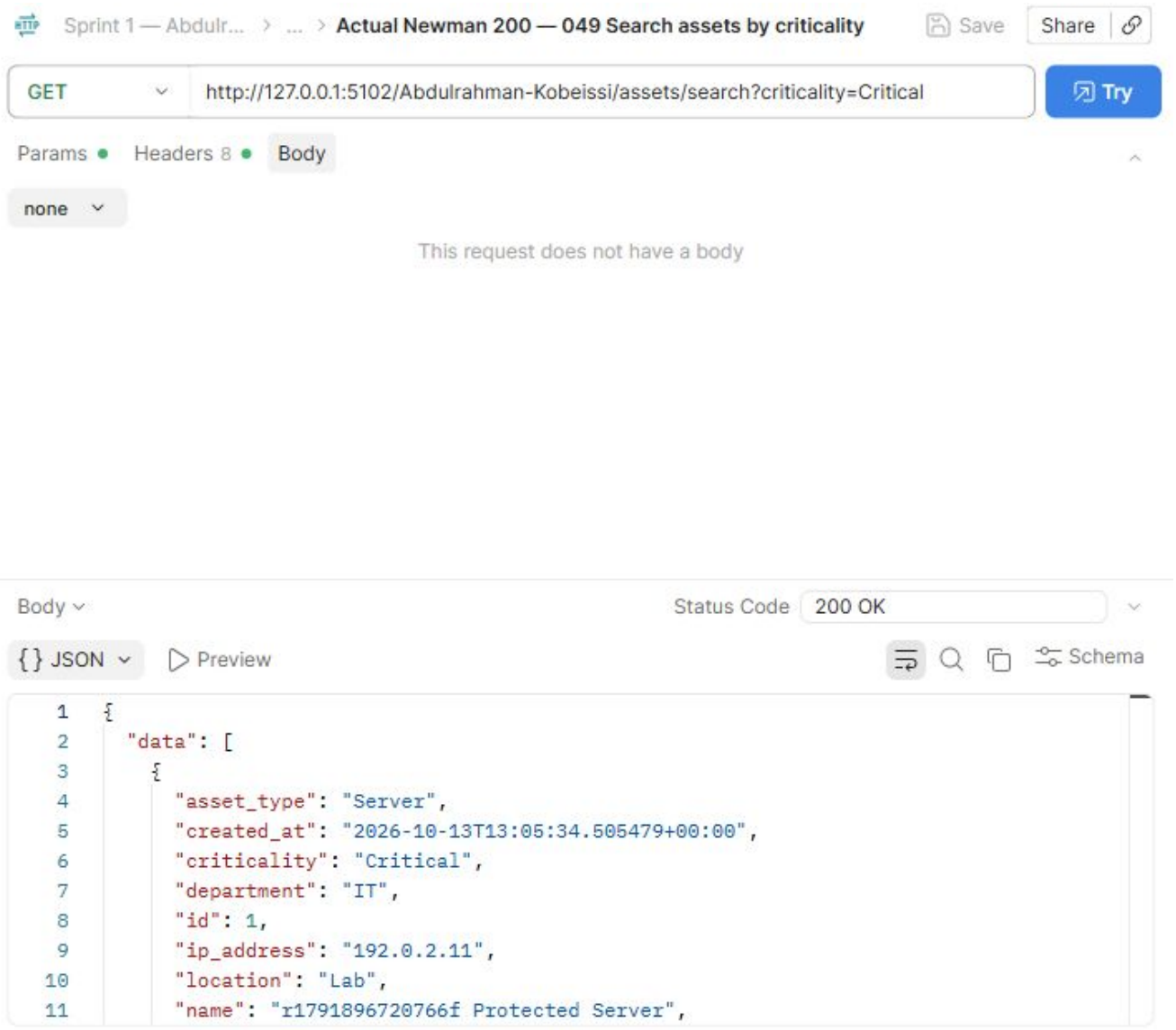


Abdulrahman Kobeissi — 048 Search assets by department. The request and saved actual response demonstrate this operation.

Postman Evidence 49

(Abdulrahman Kobeissi)

Search assets by criticality · GET · observed HTTP 200

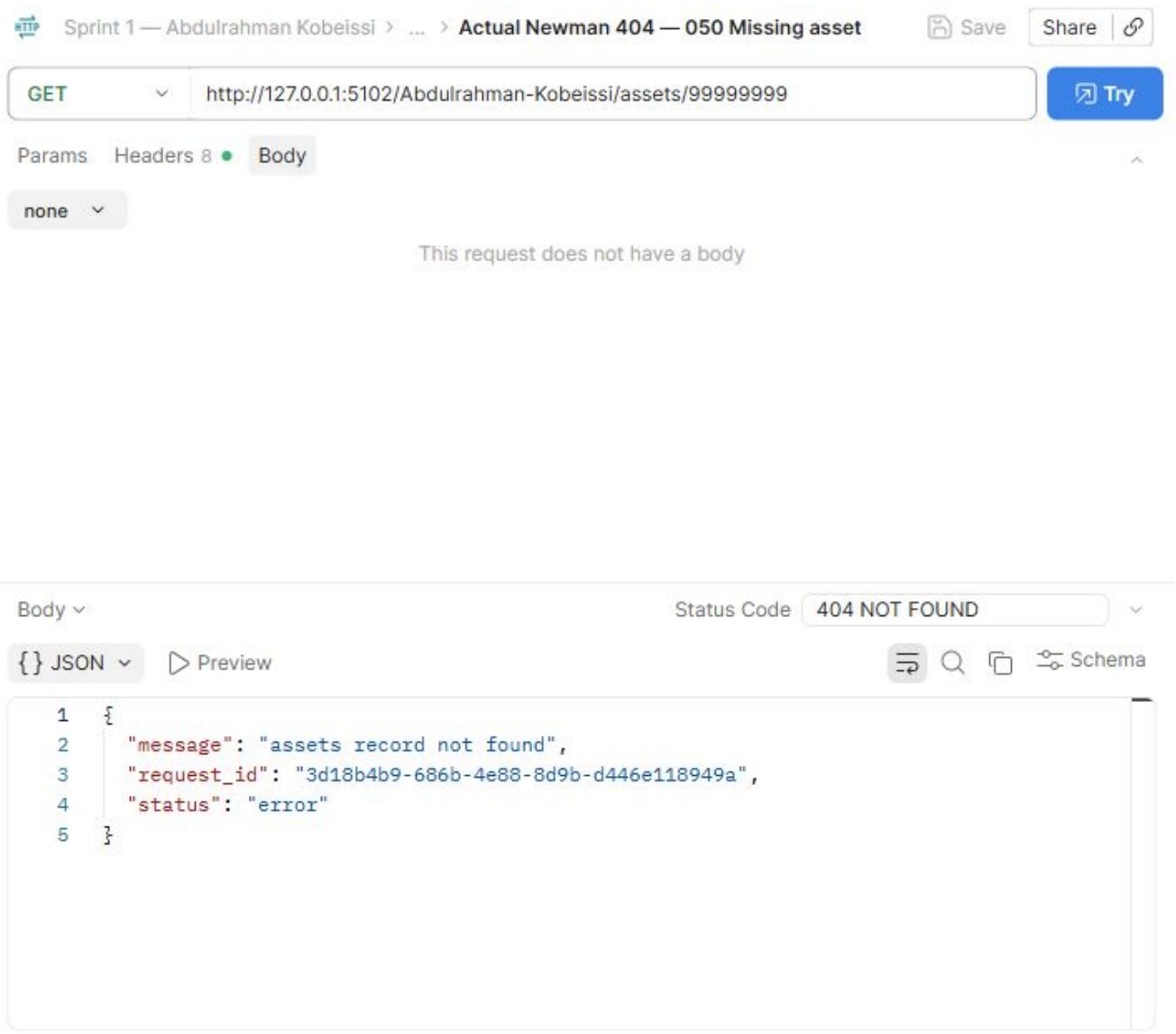


Abdulrahman Kobeissi — 049 Search assets by criticality. The request and saved actual response demonstrate this operation.

Postman Evidence 50

(Abdulrahman Kobeissi)

Missing asset · GET · observed HTTP 404

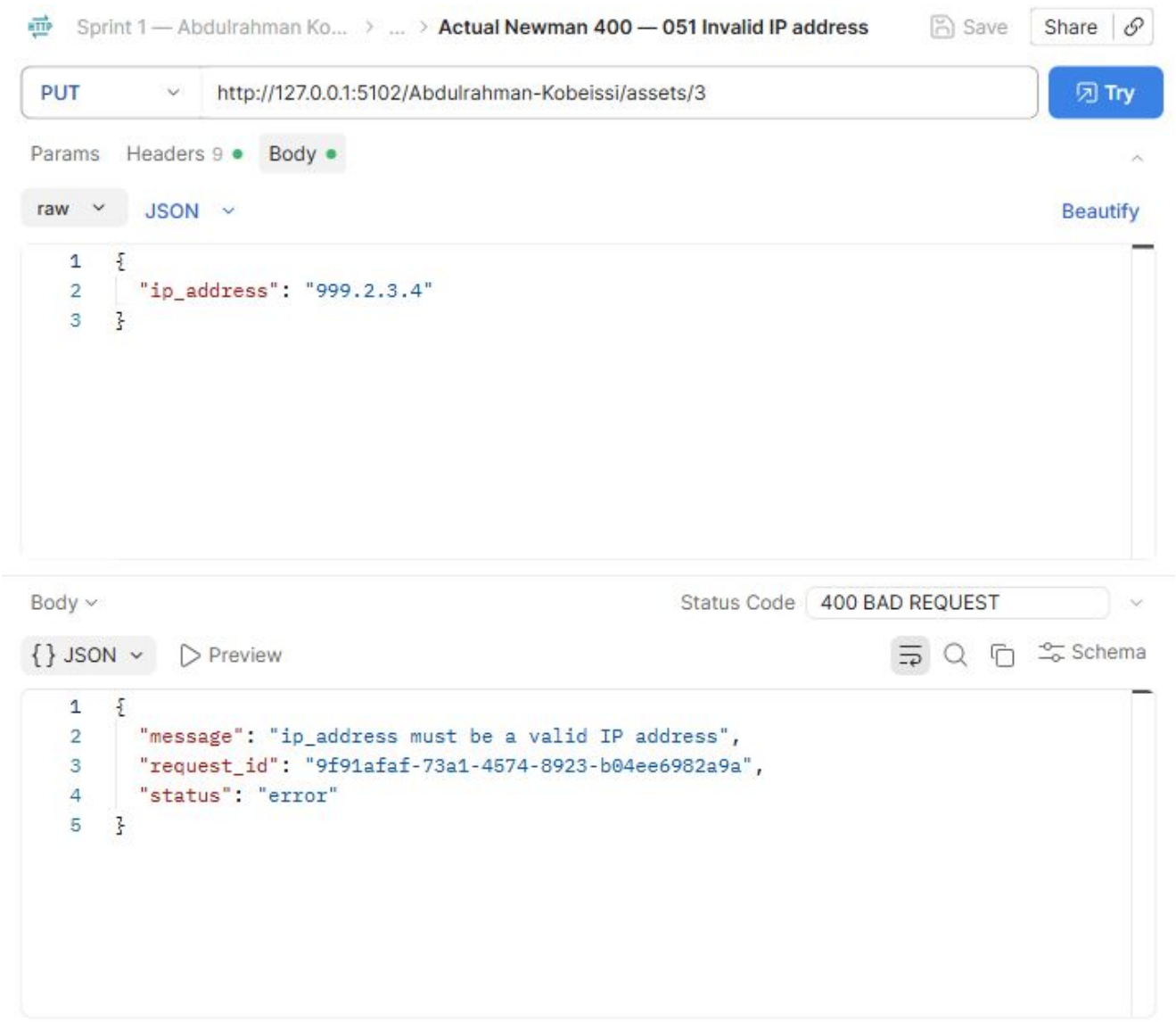


Abdulrahman Kobeissi — 050 Missing asset. The request and saved actual response demonstrate this operation.

Postman Evidence 51

(Abdulrahman Kobeissi)

Invalid IP address · PUT · observed HTTP 400

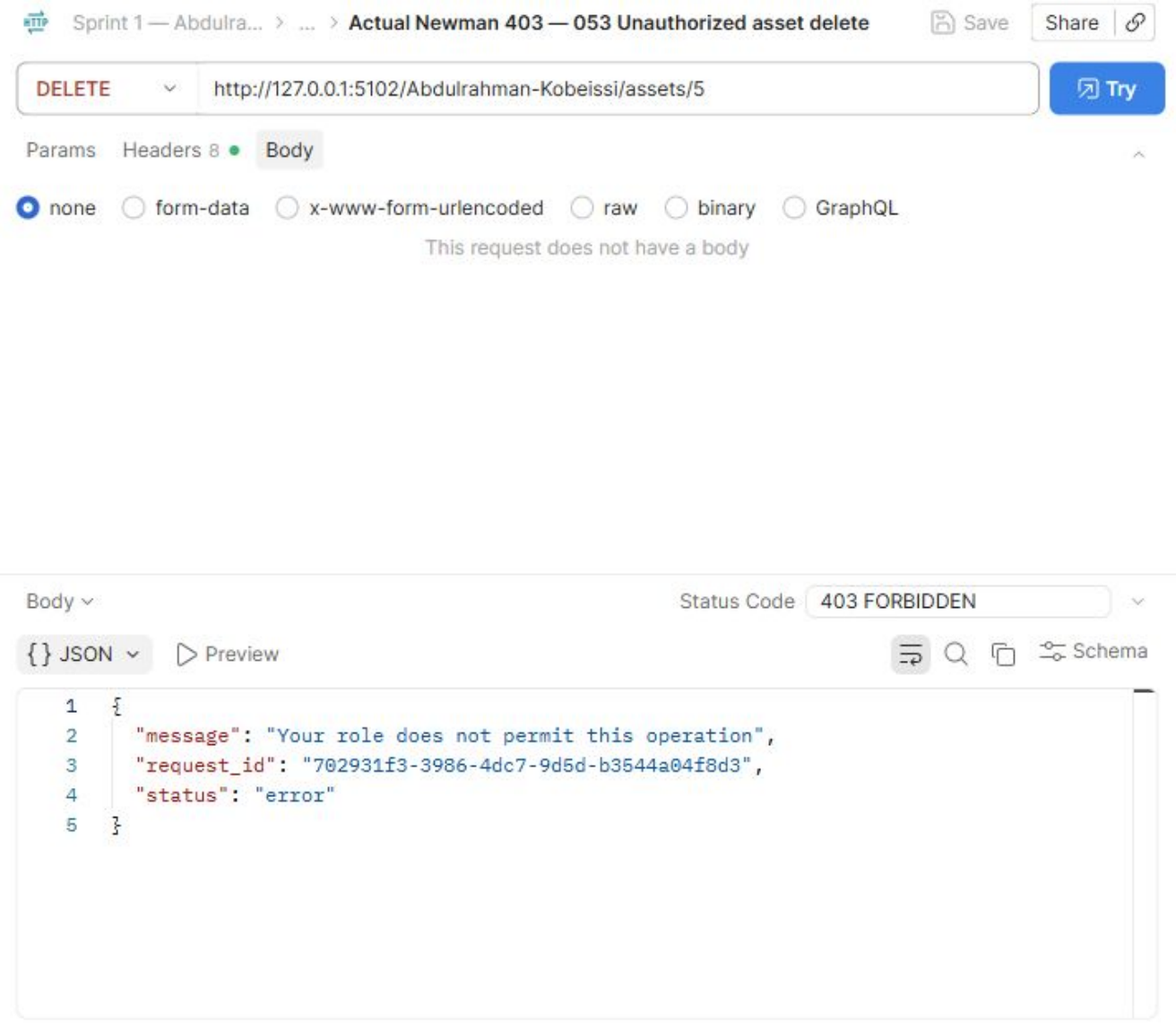


Abdulrahman Kobeissi — 051 Invalid IP address. The request and saved actual response demonstrate this operation.

Postman Evidence 53

(Abdulrahman Kobeissi)

Unauthorized asset delete · DELETE · observed HTTP 403

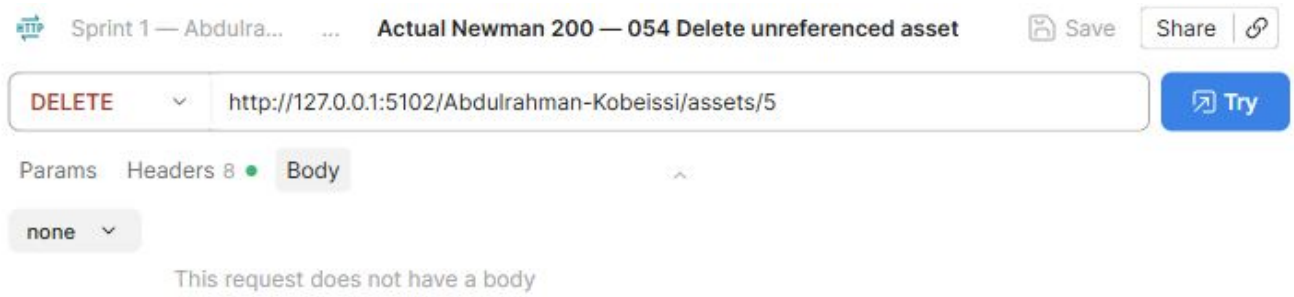


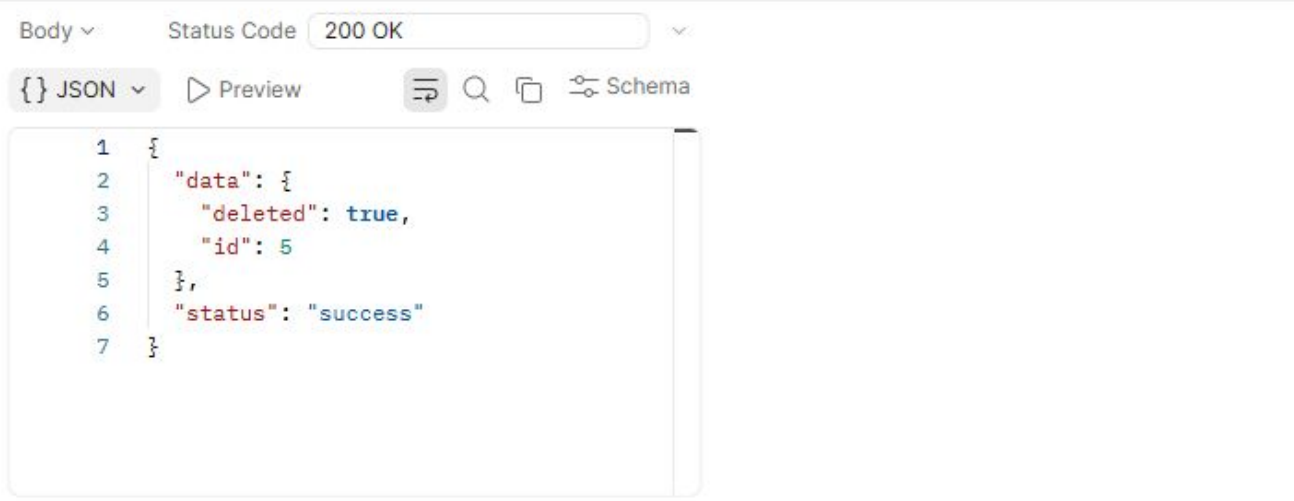
Abdulrahman Kobeissi — 053 Unauthorized asset delete. The request and saved actual response demonstrate this operation.

Postman Evidence 54

(Abdulrahman Kobeissi)

Delete unreferenced asset · DELETE · observed HTTP 200



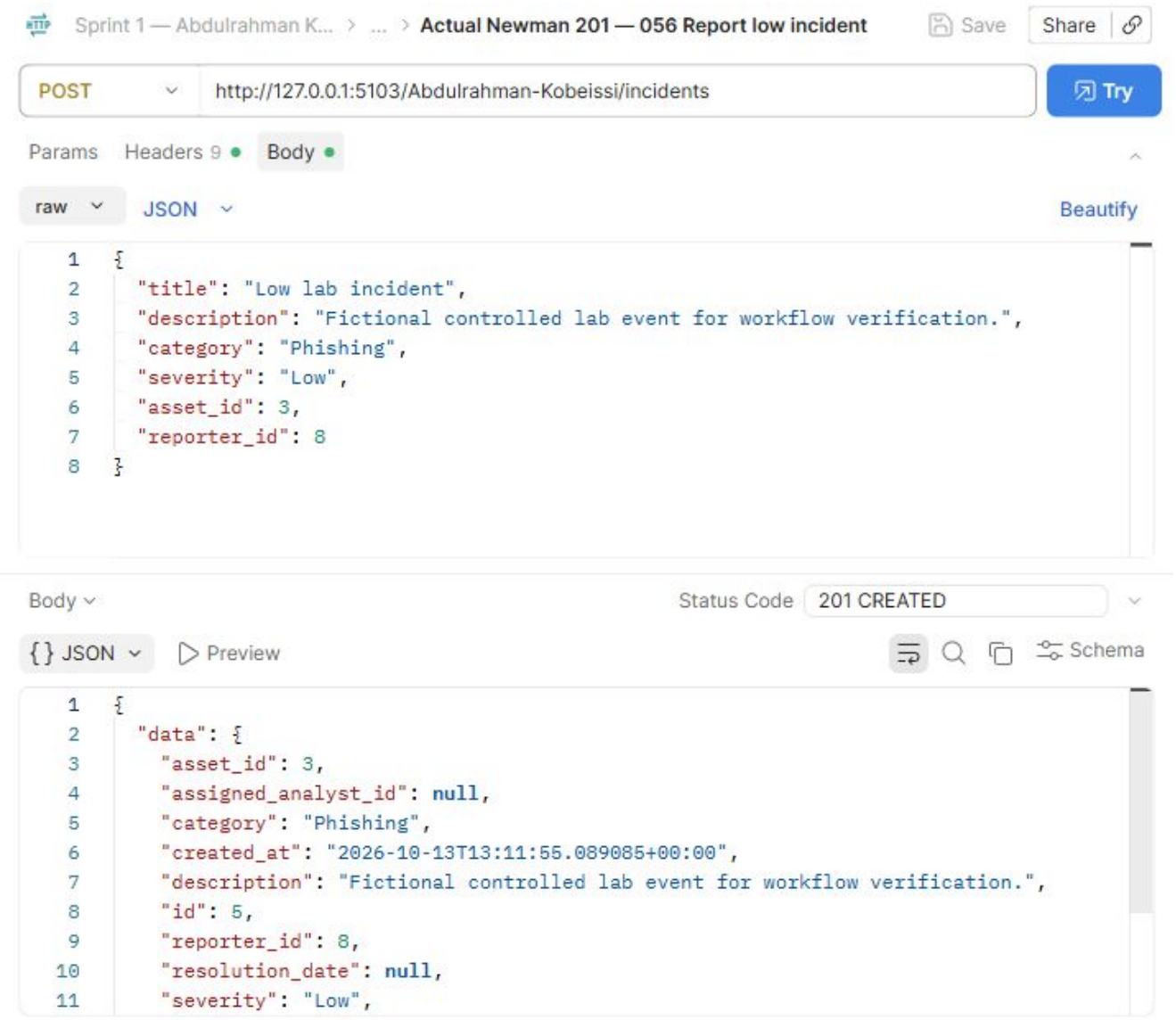


Abdulrahman Kobeissi — 054 Delete unreferenced asset. The request and saved actual response demonstrate this operation.

Postman Evidence 56

(Abdulrahman Kobeissi)

Report low incident · POST · observed HTTP 201



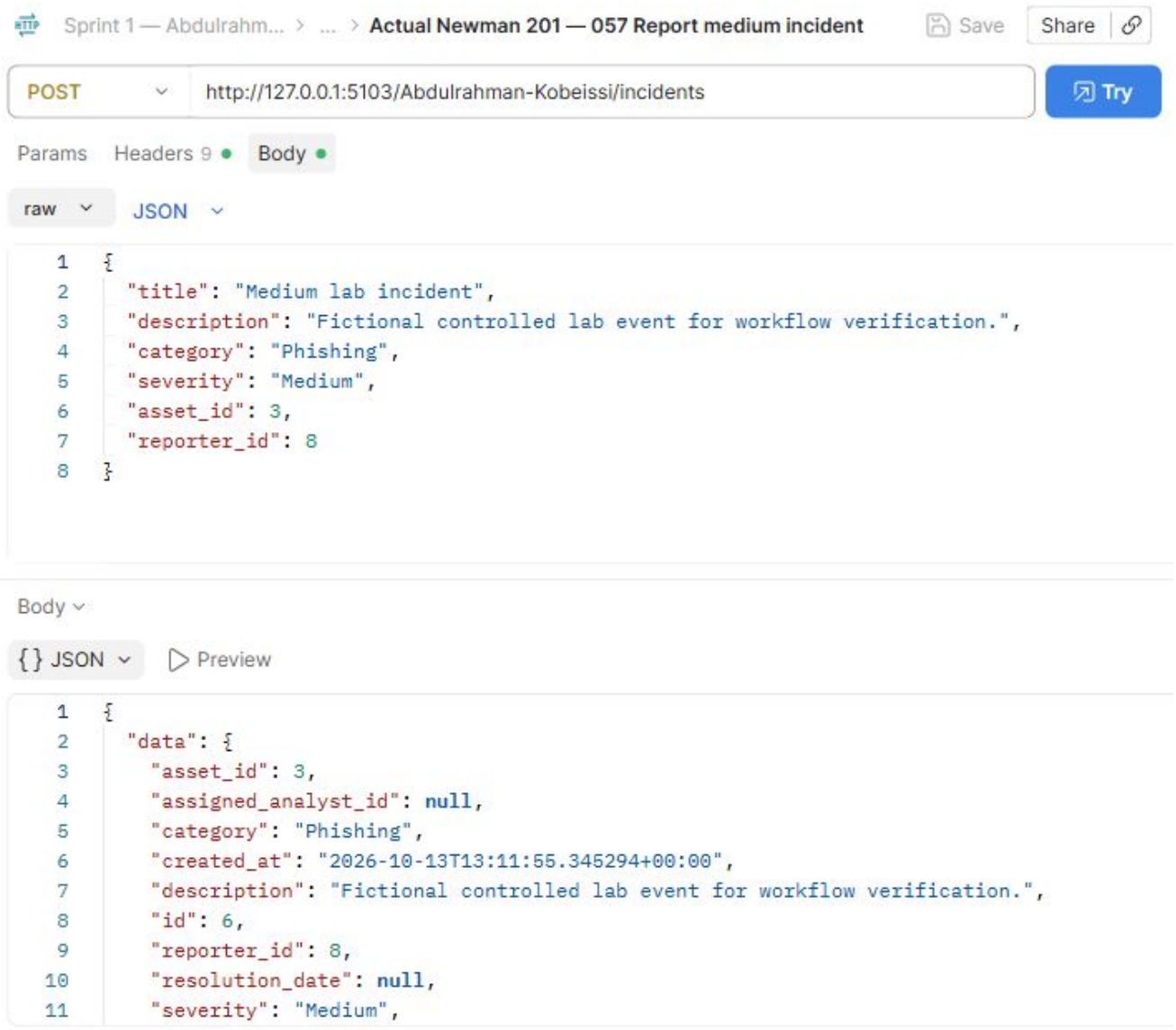
Abdulrahman Kobeissi — 056 Report low incident. The request and saved actual response demonstrate this operation.

Severity rationale: Low represents a limited-impact event. This fictional Phishing fixture verifies that the system stores that category correctly.

Postman Evidence 57

(Abdulrahman Kobeissi)

Report medium incident · POST · observed HTTP 201



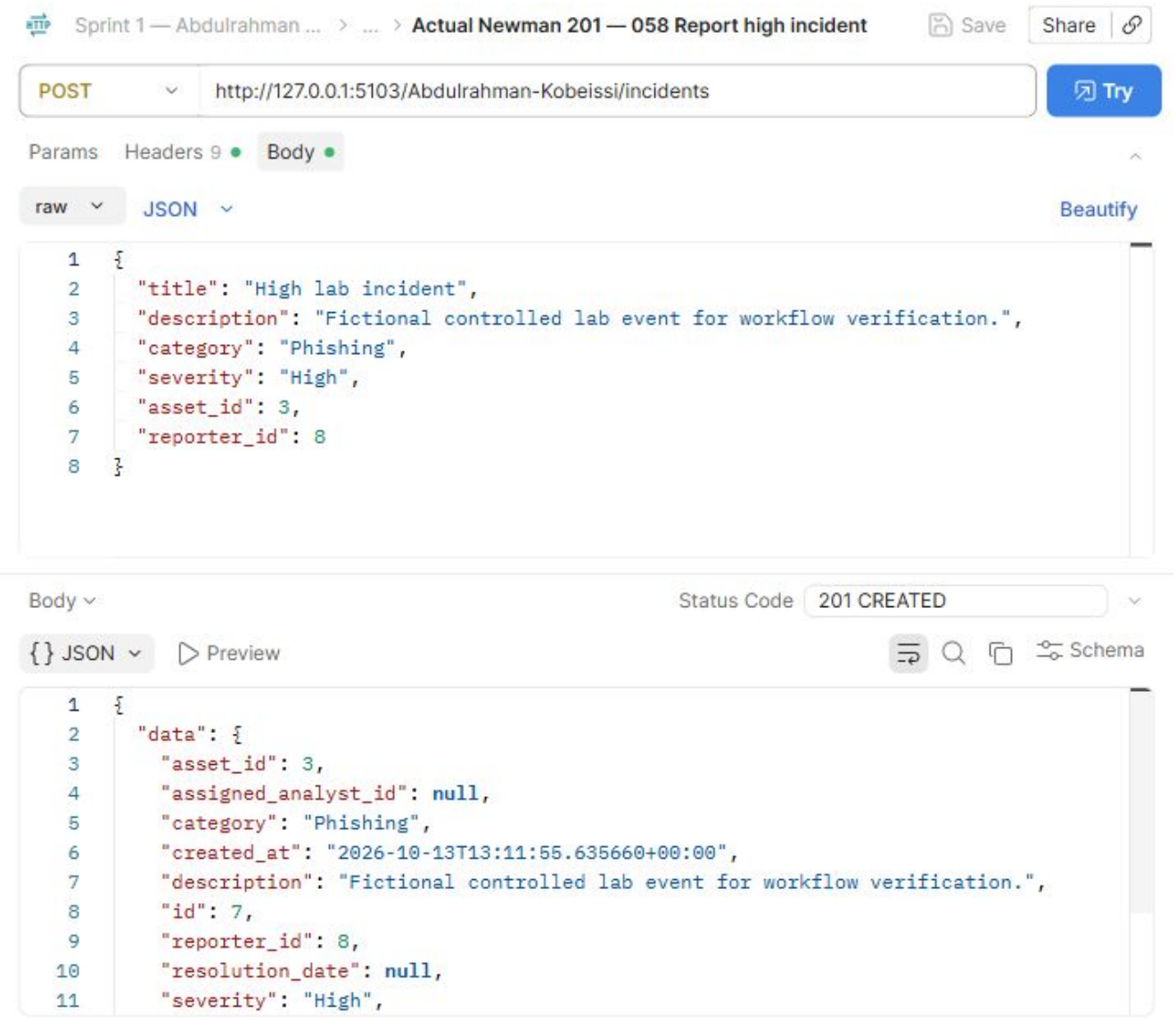
Abdulrahman Kobeissi — 057 Report medium incident. The request and saved actual response demonstrate this operation.

Severity rationale: Medium represents an event requiring investigation. This fictional Phishing fixture tests that level without asserting a real attack.

Postman Evidence 58

(Abdulrahman Kobeissi)

Report high incident · POST · observed HTTP 201



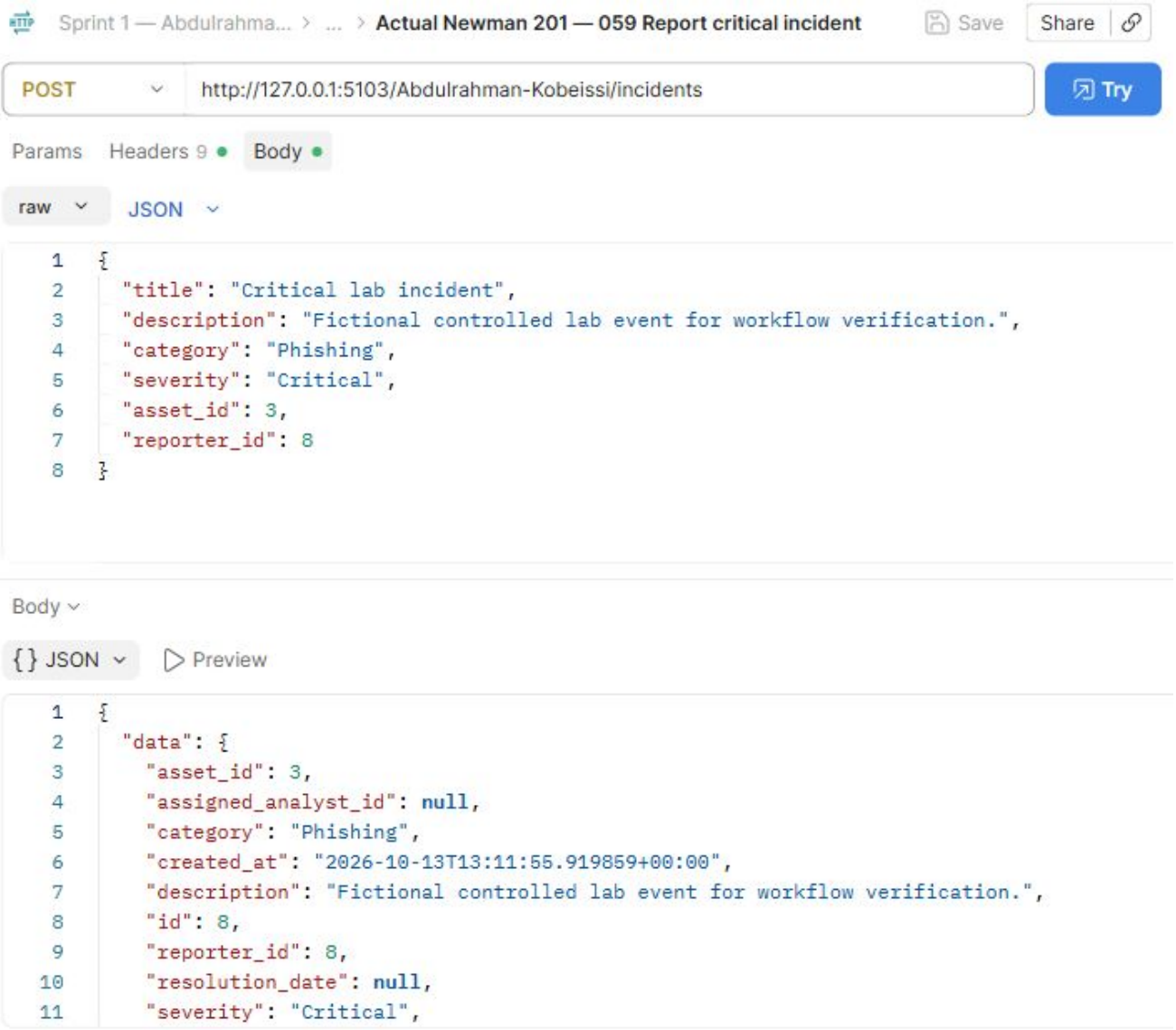
Abdulrahman Kobeissi — 058 Report high incident. The request and saved actual response demonstrate this operation.

Severity rationale: High represents an event affecting important systems or users. This fictional fixture exercises escalation data; no real compromise is claimed.

Postman Evidence 59

(Abdulrahman Kobeissi)

Report critical incident · POST · observed HTTP 201



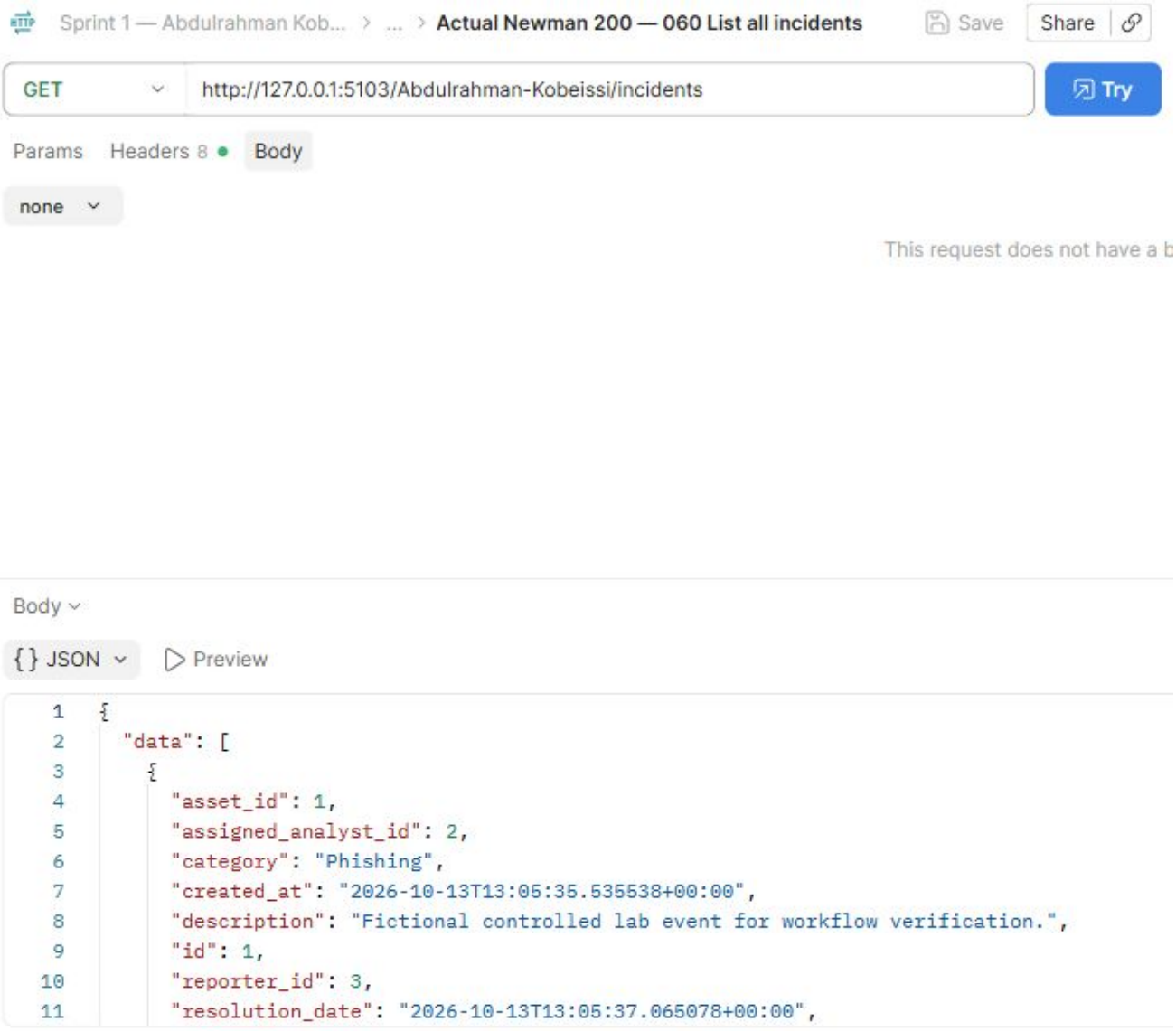
Abdulrahman Kobeissi — 059 Report critical incident. The request and saved actual response demonstrate this operation.

Severity rationale: Critical represents an event needing immediate response. This fictional fixture tests urgent classification; it is not a measured real-world impact assessment.

Postman Evidence 60

(Abdulrahman Kobeissi)

List all incidents · GET · observed HTTP 200

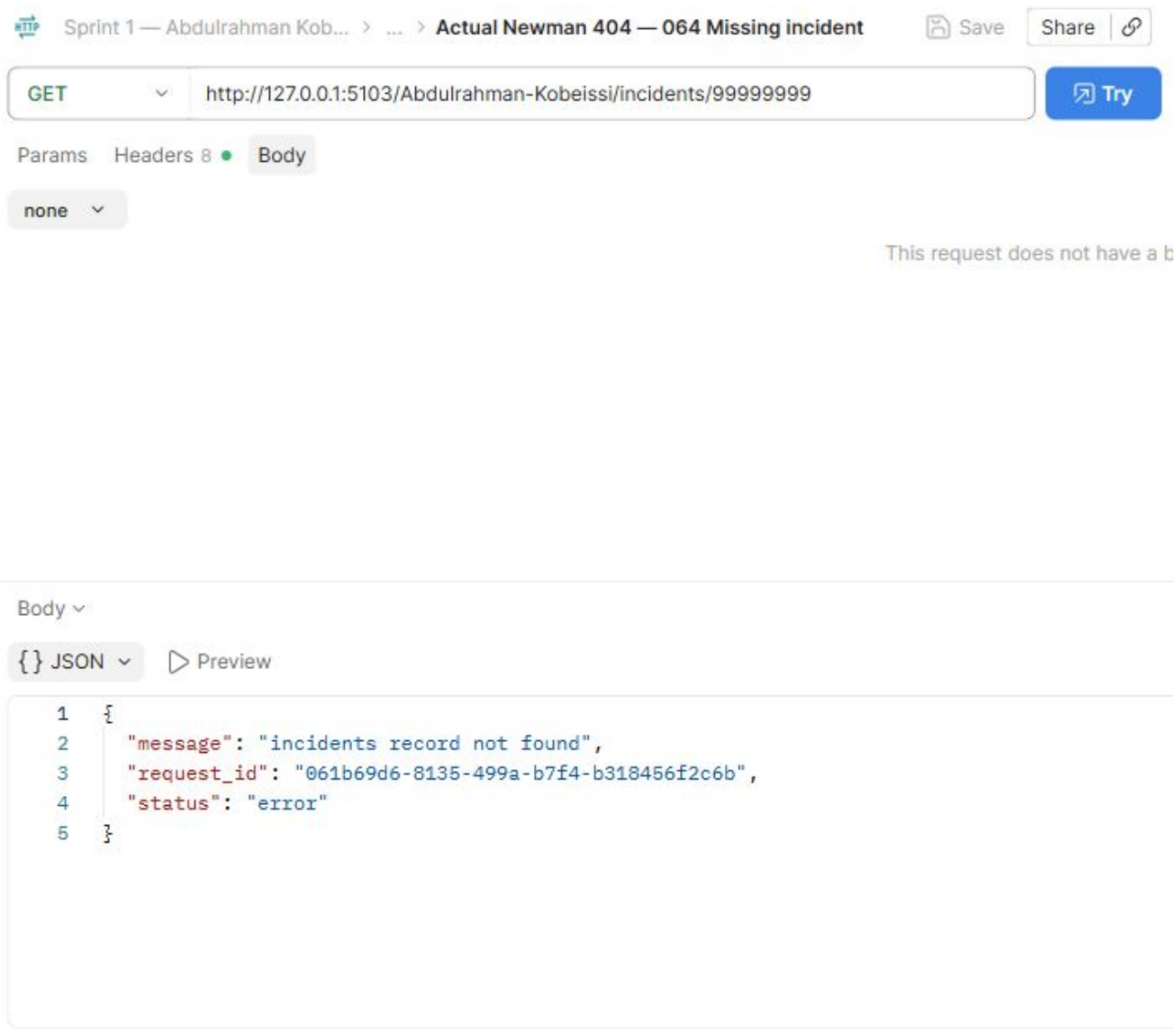


Abdulrahman Kobeissi — 060 List all incidents. The request and saved actual response demonstrate this operation.

Postman Evidence 64

(Abdulrahman Kobeissi)

Missing incident · GET · observed HTTP 404

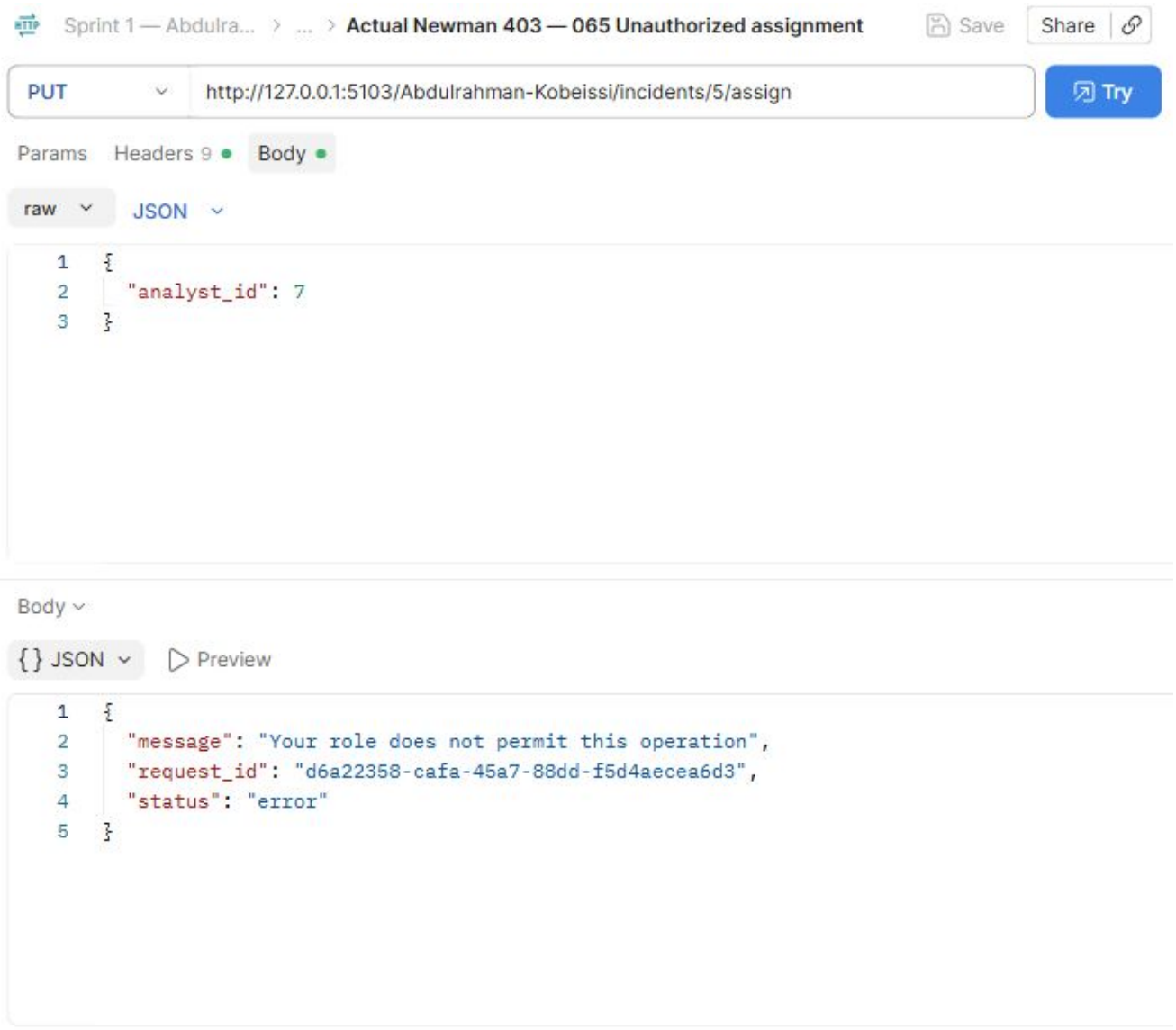


Abdulrahman Kobeissi — 064 Missing incident. The request and saved actual response demonstrate this operation.

Postman Evidence 65

(Abdulrahman Kobeissi)

Unauthorized assignment · PUT · observed HTTP 403

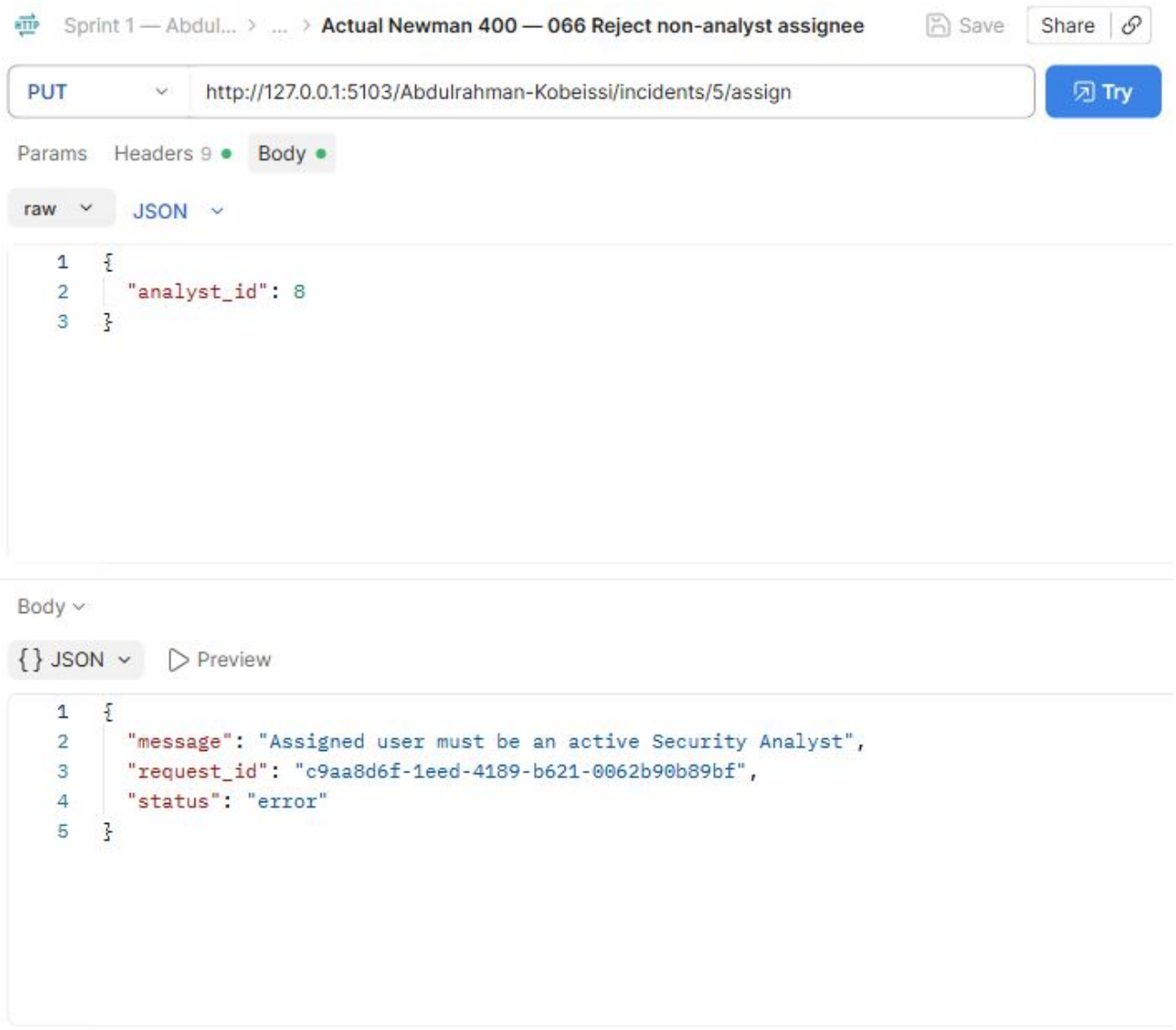


Abdulrahman Kobeissi — 065 Unauthorized assignment. The request and saved actual response demonstrate this operation.

Postman Evidence 66

(Abdulrahman Kobeissi)

Reject non-analyst assignee · PUT · observed HTTP 400

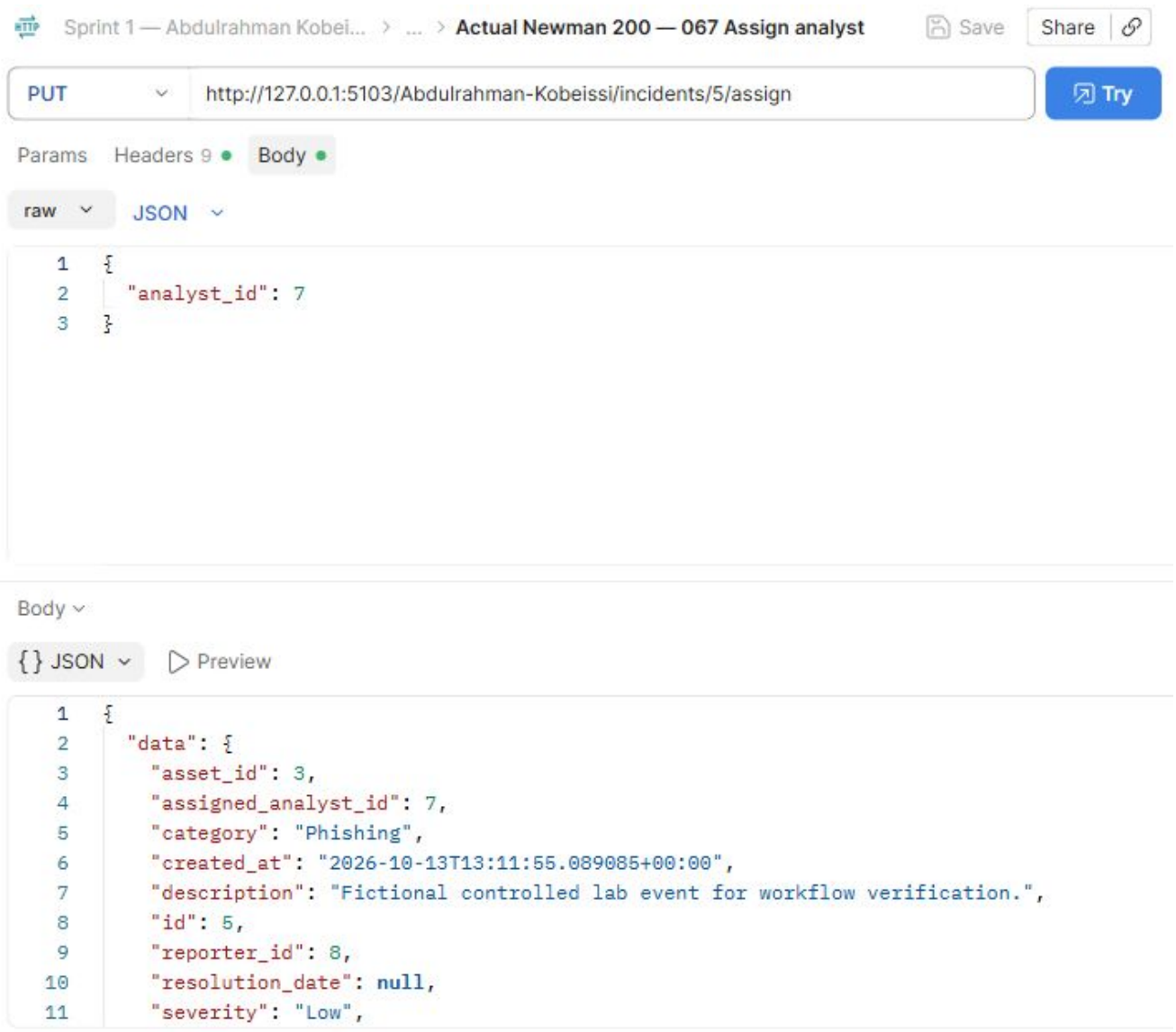


Abdulrahman Kobeissi — 066 Reject non-analyst assignee. The request and saved actual response demonstrate this operation.

Postman Evidence 67

(Abdulrahman Kobeissi)

Assign analyst · PUT · observed HTTP 200

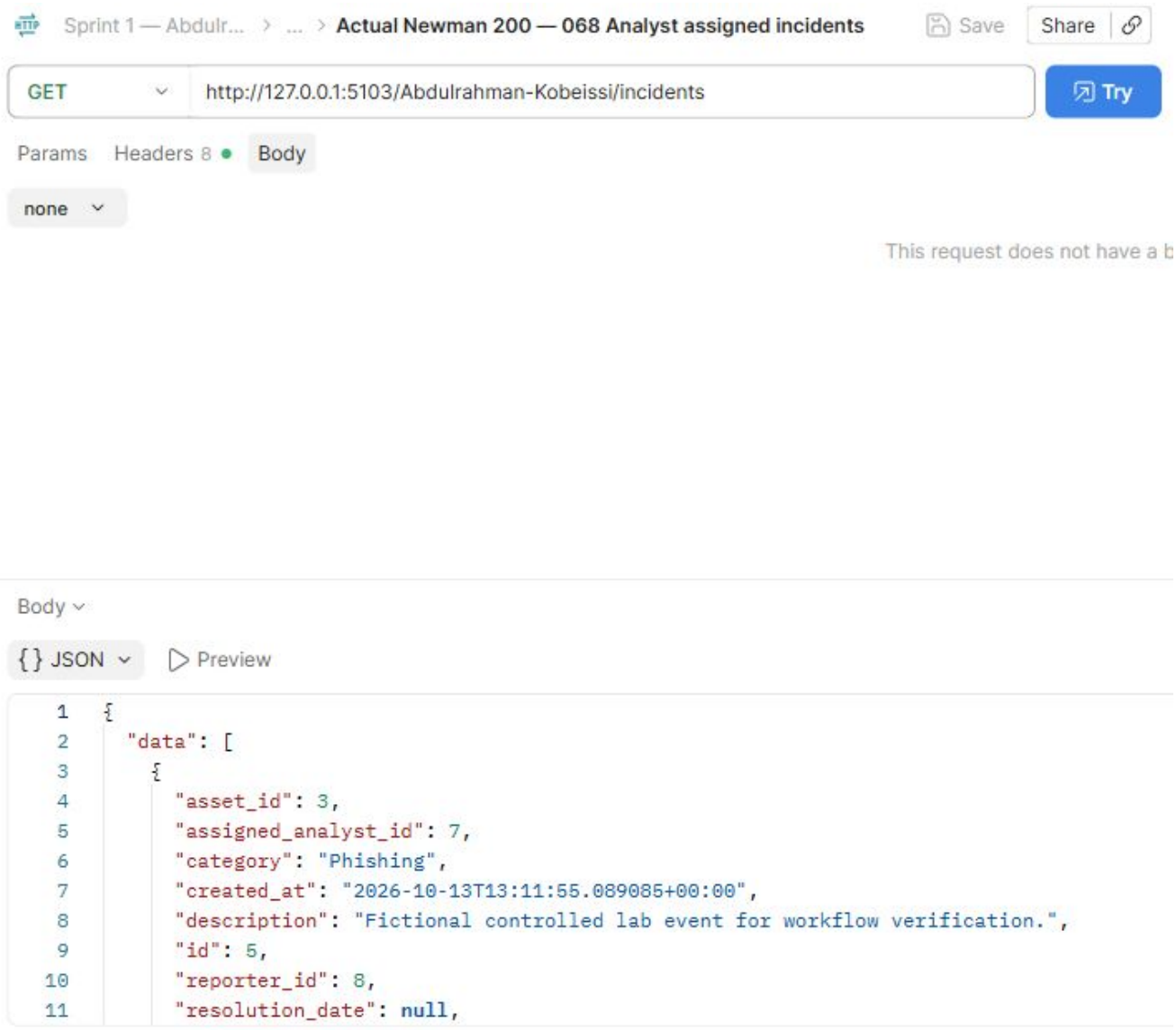


Abdulrahman Kobeissi — 067 Assign analyst. The request and saved actual response demonstrate this operation.

Postman Evidence 68

(Abdulrahman Kobeissi)

Analyst assigned incidents · GET · observed HTTP 200

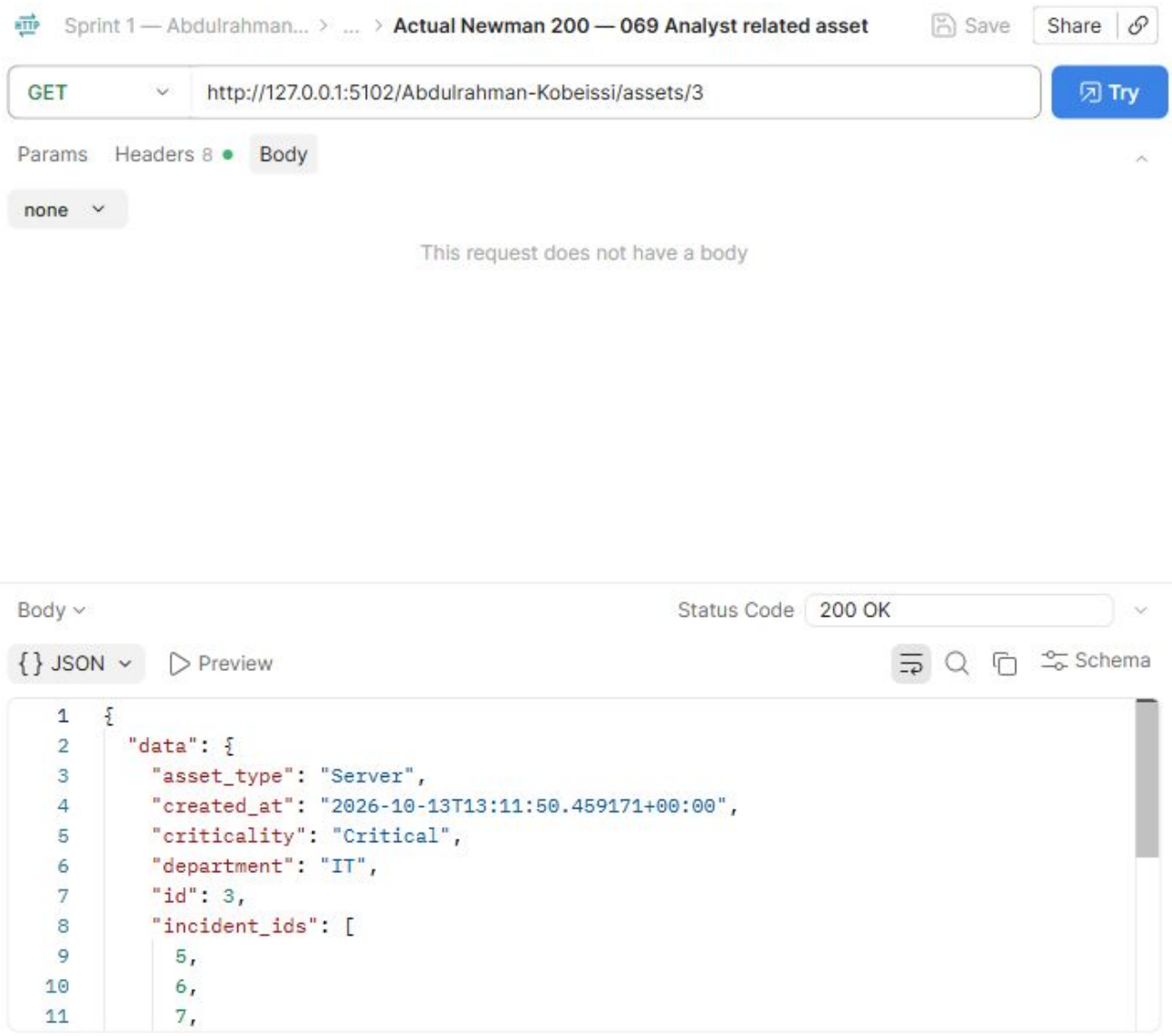


Abdulrahman Kobeissi — 068 Analyst assigned incidents. The request and saved actual response demonstrate this operation.

Postman Evidence 69

(Abdulrahman Kobeissi)

Analyst related asset · GET · observed HTTP 200

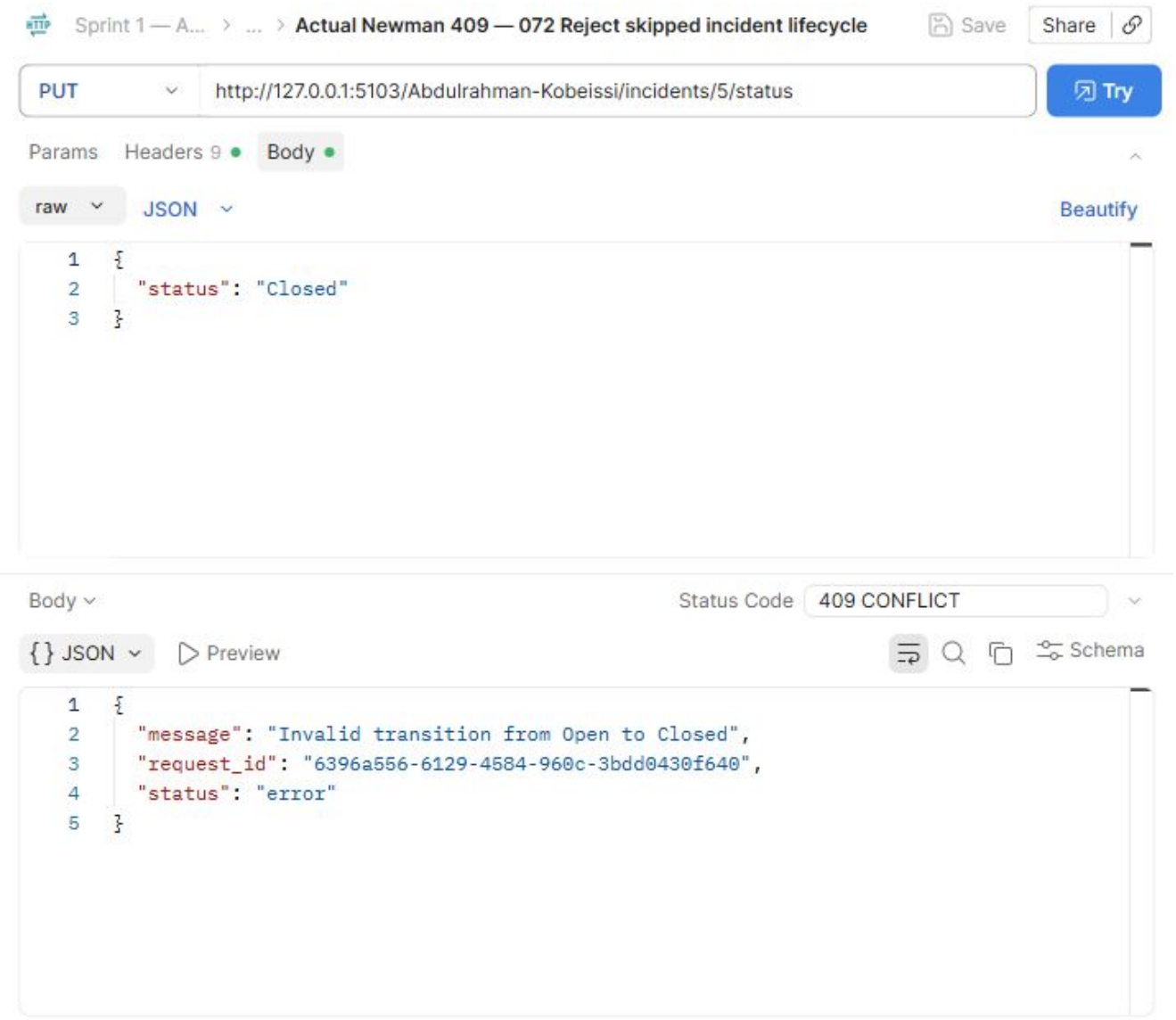


Abdulrahman Kobeissi — 069 Analyst related asset. The request and saved actual response demonstrate this operation.

Postman Evidence 72

(Abdulrahman Kobeissi)

Reject skipped incident lifecycle · PUT · observed HTTP 409

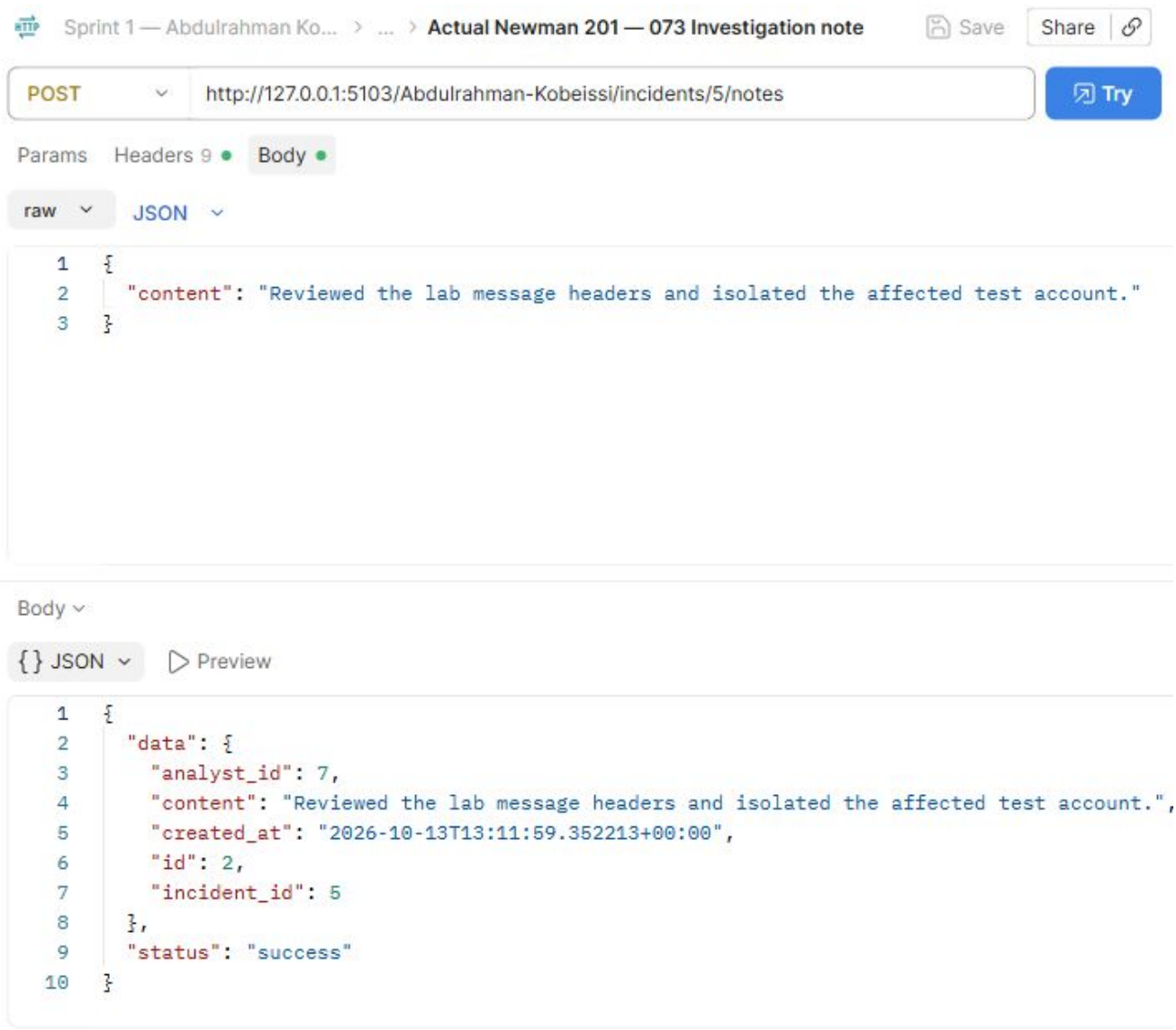


Abdulrahman Kobeissi — 072 Reject skipped incident lifecycle. The request and saved actual response demonstrate this operation.

Postman Evidence 73

(Abdulrahman Kobeissi)

Investigation note · POST · observed HTTP 201

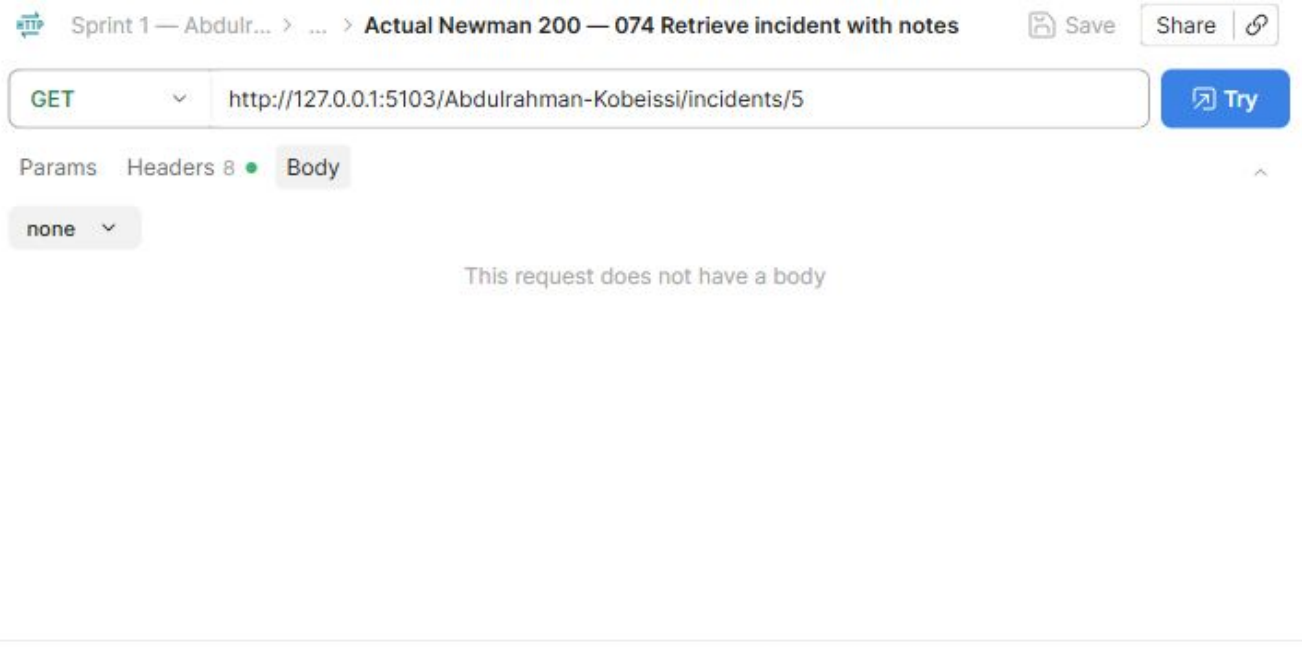


Abdulrahman Kobeissi — 073 Investigation note. The request and saved actual response demonstrate this operation.

Postman Evidence 74

(Abdulrahman Kobeissi)

Retrieve incident with notes · GET · observed HTTP 200



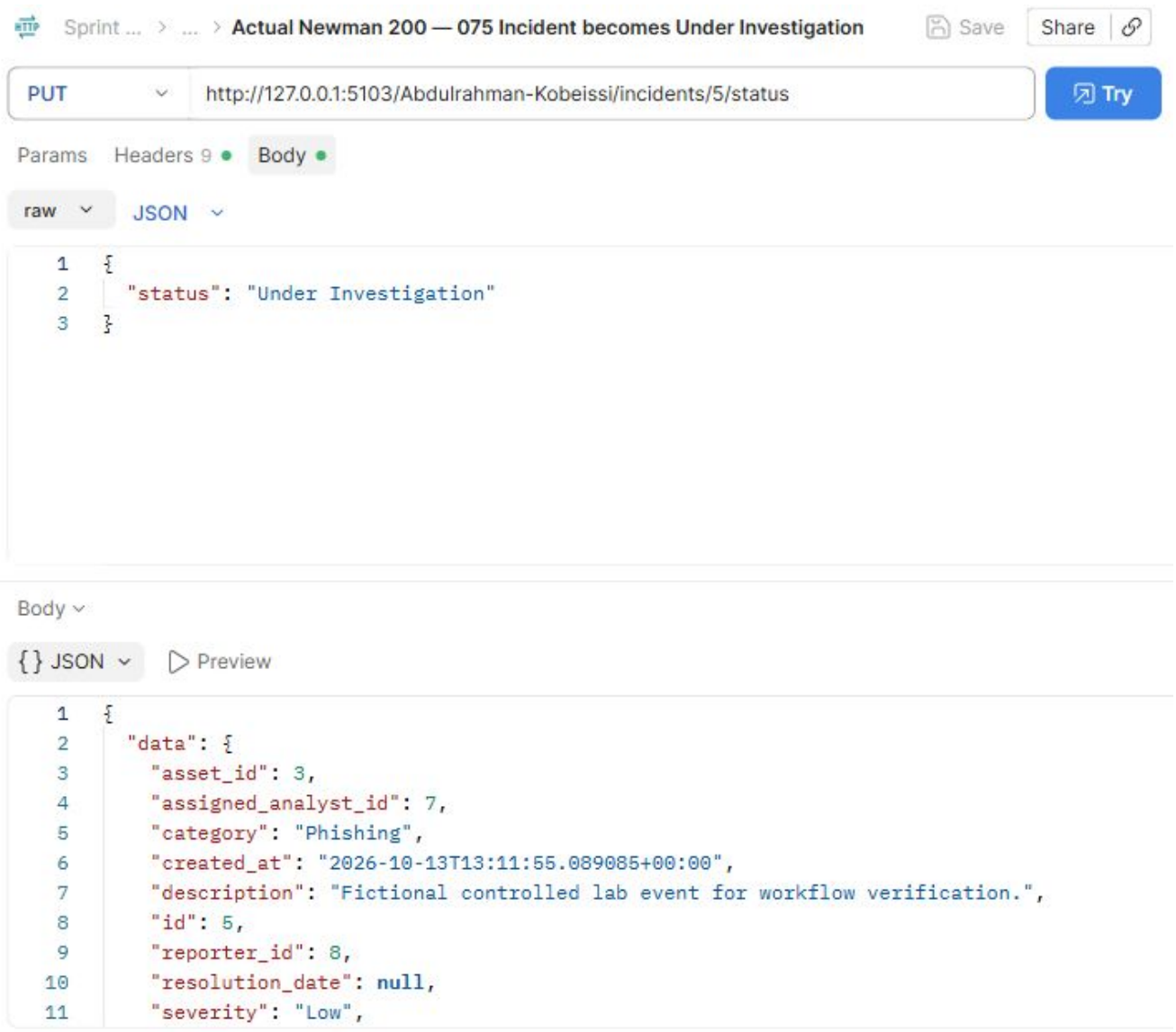


Abdulrahman Kobeissi — 074 Retrieve incident with notes. The request and saved actual response demonstrate this operation.

Postman Evidence 75

(Abdulrahman Kobeissi)

Incident becomes Under Investigation · PUT · observed HTTP 200

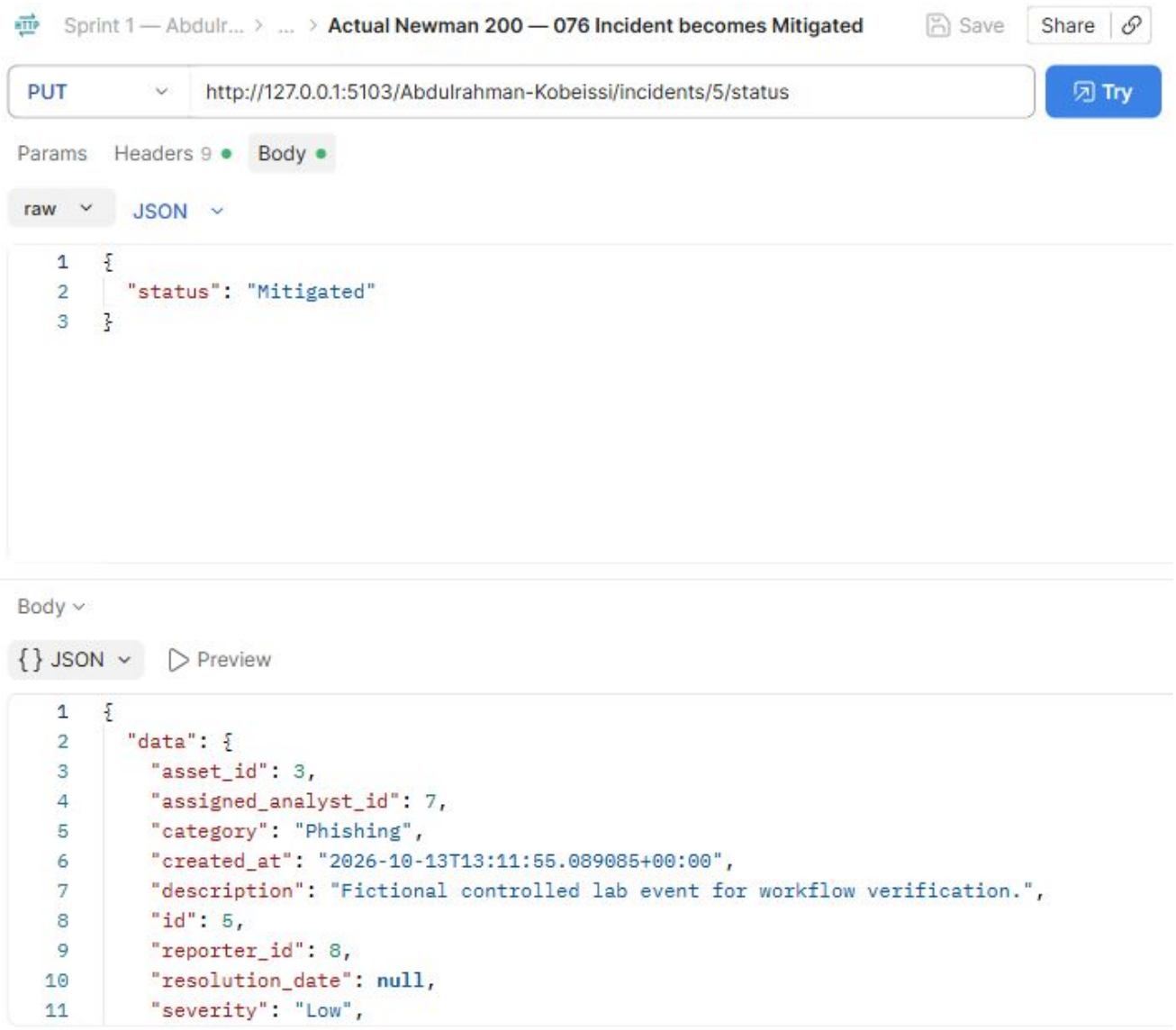


Abdulrahman Kobeissi — 075 Incident becomes Under Investigation. The request and saved actual response demonstrate this operation.

Postman Evidence 76

(Abdulrahman Kobeissi)

Incident becomes Mitigated · PUT · observed HTTP 200

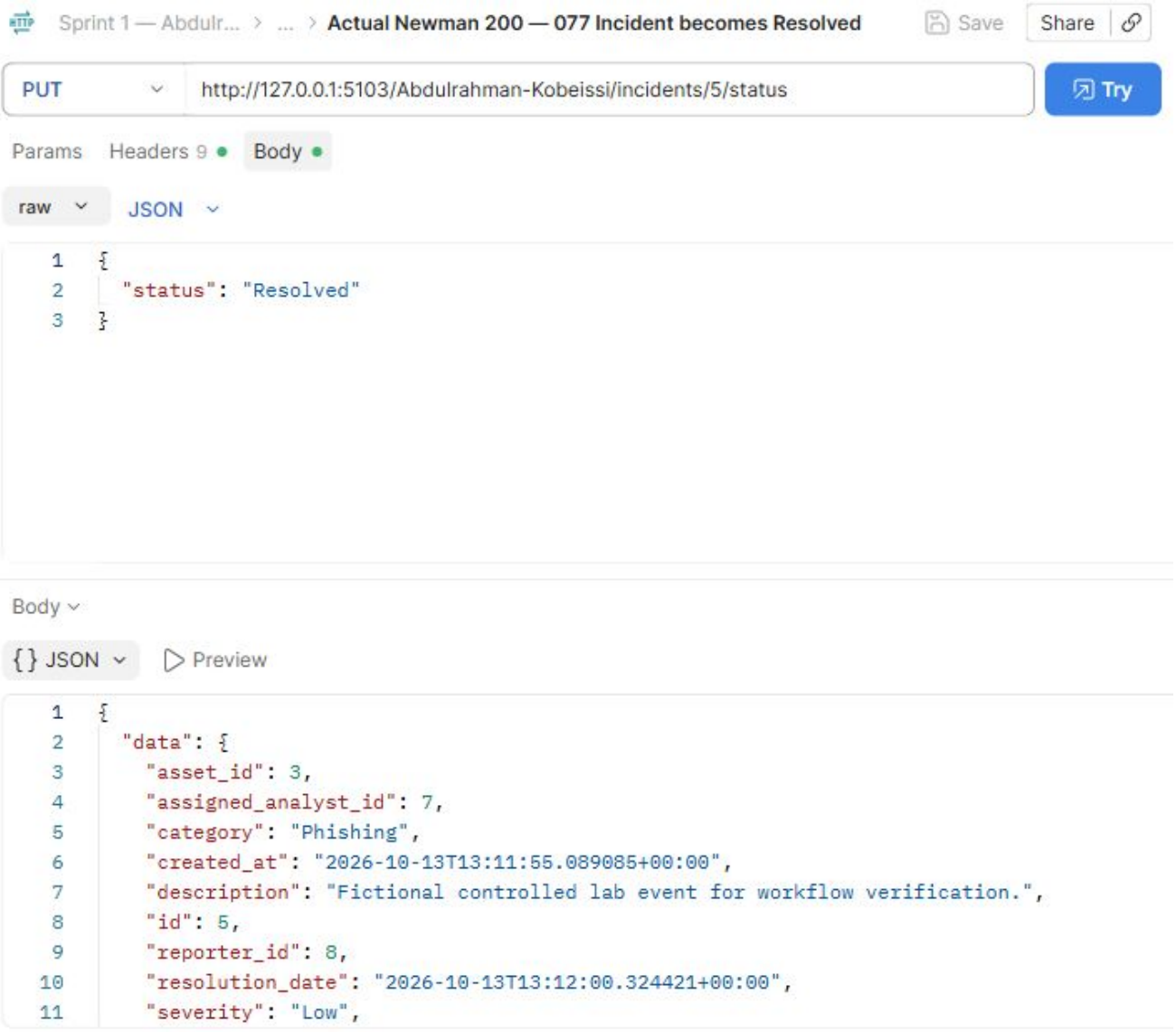


Abdulrahman Kobeissi — 076 Incident becomes Mitigated. The request and saved actual response demonstrate this operation.

Postman Evidence 77

(Abdulrahman Kobeissi)

Incident becomes Resolved · PUT · observed HTTP 200

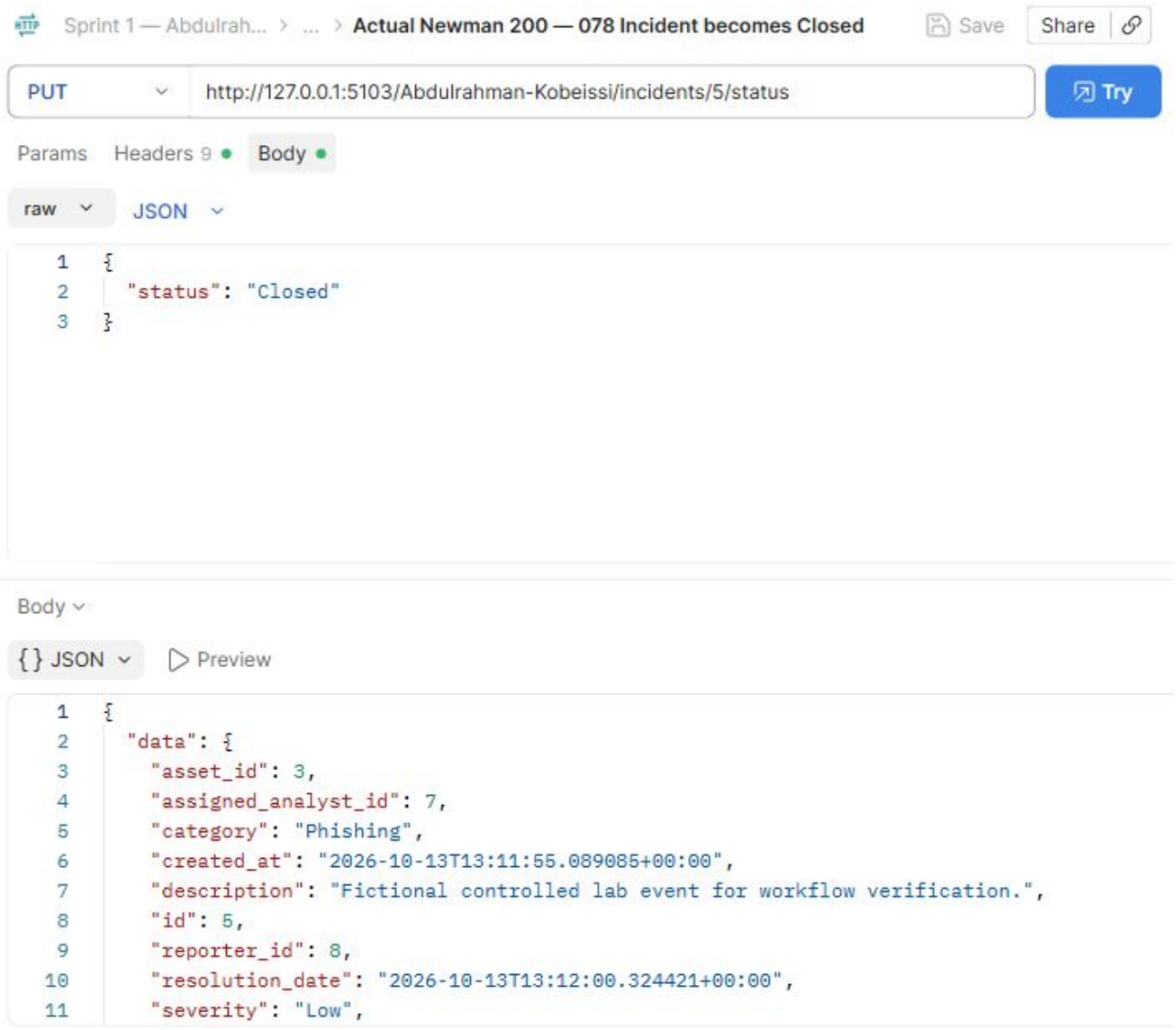


Abdulrahman Kobeissi — 077 Incident becomes Resolved. The request and saved actual response demonstrate this operation.

Postman Evidence 78

(Abdulrahman Kobeissi)

Incident becomes Closed · PUT · observed HTTP 200

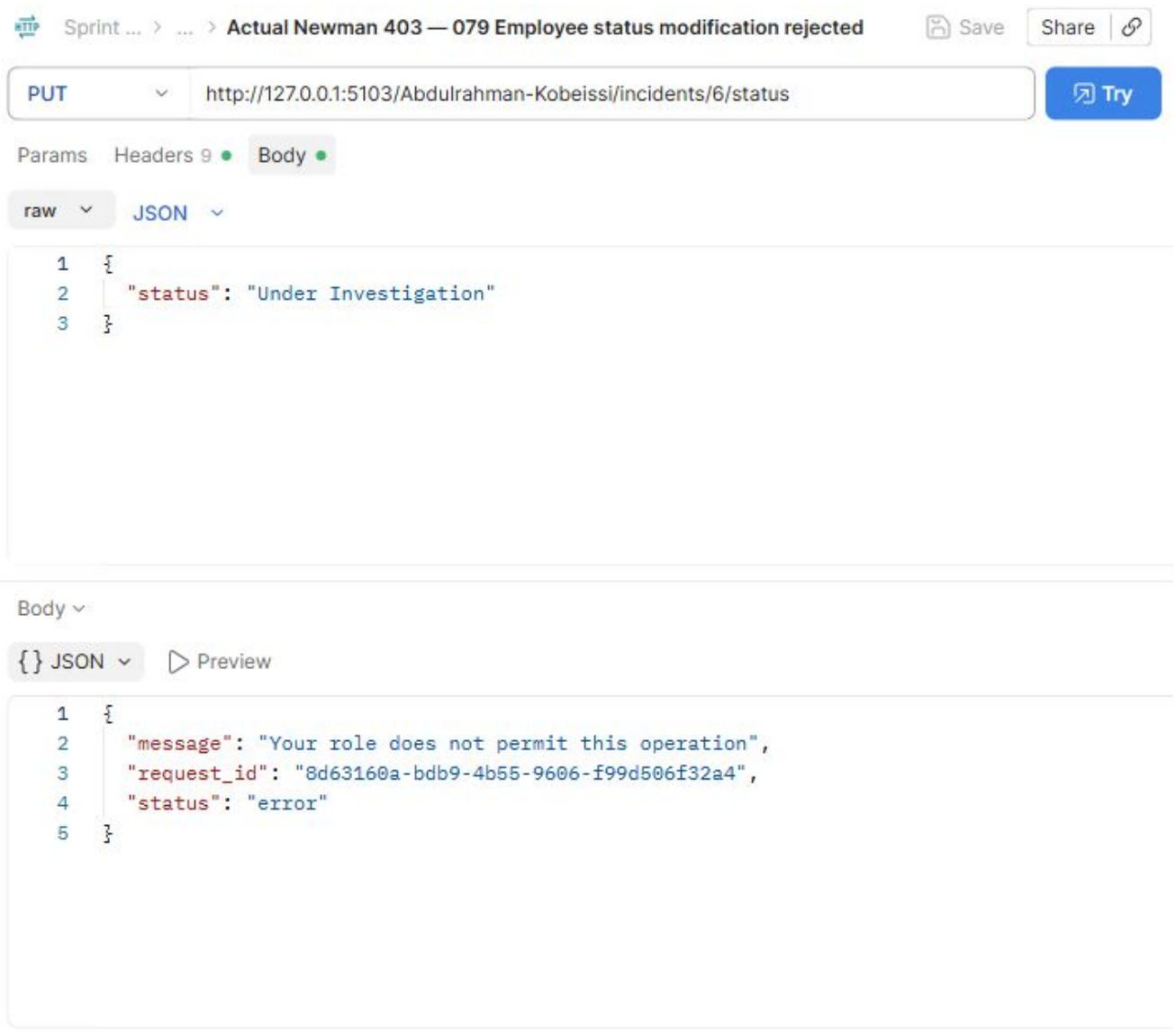


Abdulrahman Kobeissi — 078 Incident becomes Closed. The request and saved actual response demonstrate this operation.

Postman Evidence 79

(Abdulrahman Kobeissi)

Employee status modification rejected · PUT · observed HTTP 403

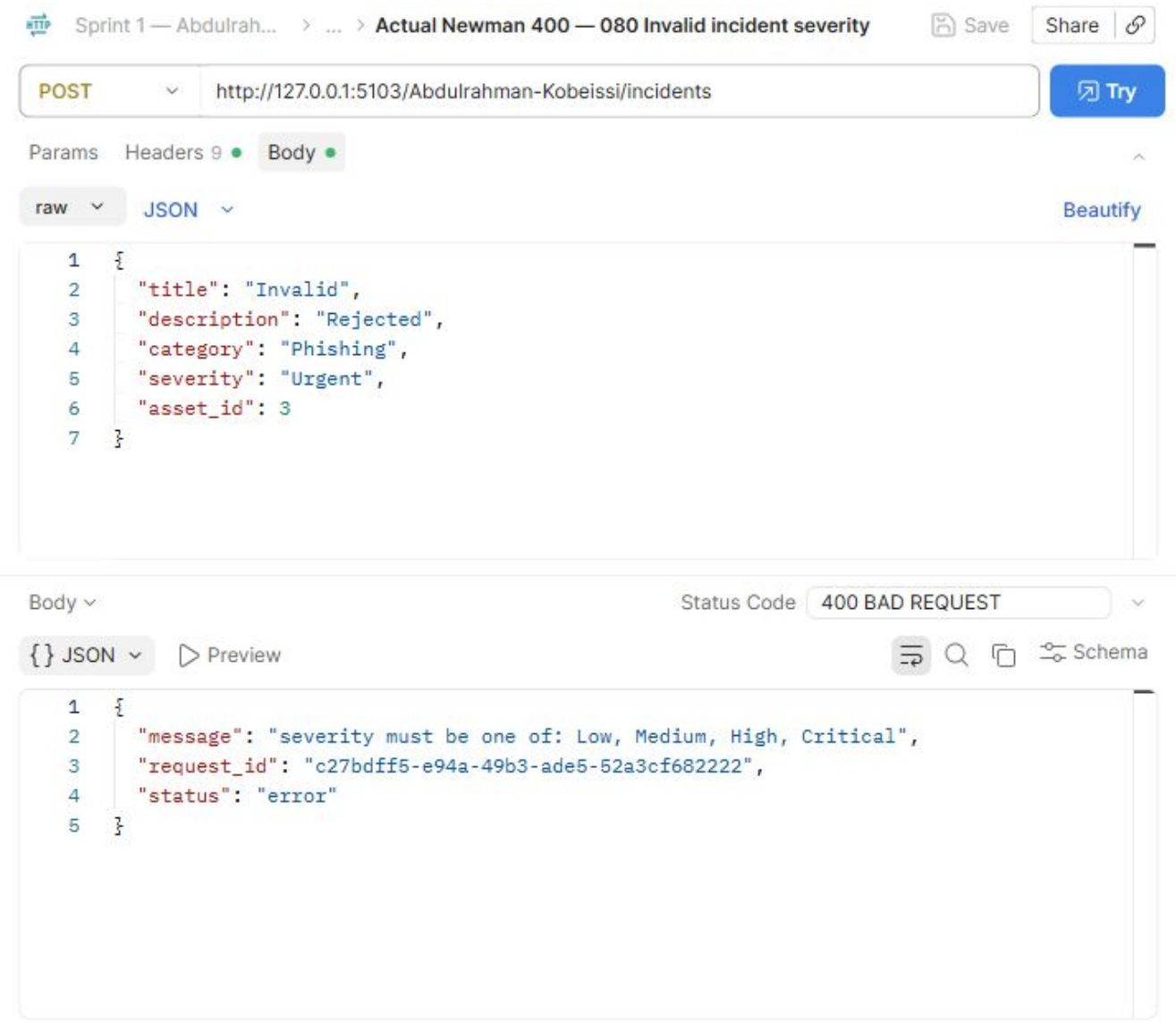


Abdulrahman Kobeissi — 079 Employee status modification rejected. The request and saved actual response demonstrate this operation.

Postman Evidence 80

(Abdulrahman Kobeissi)

Invalid incident severity · POST · observed HTTP 400

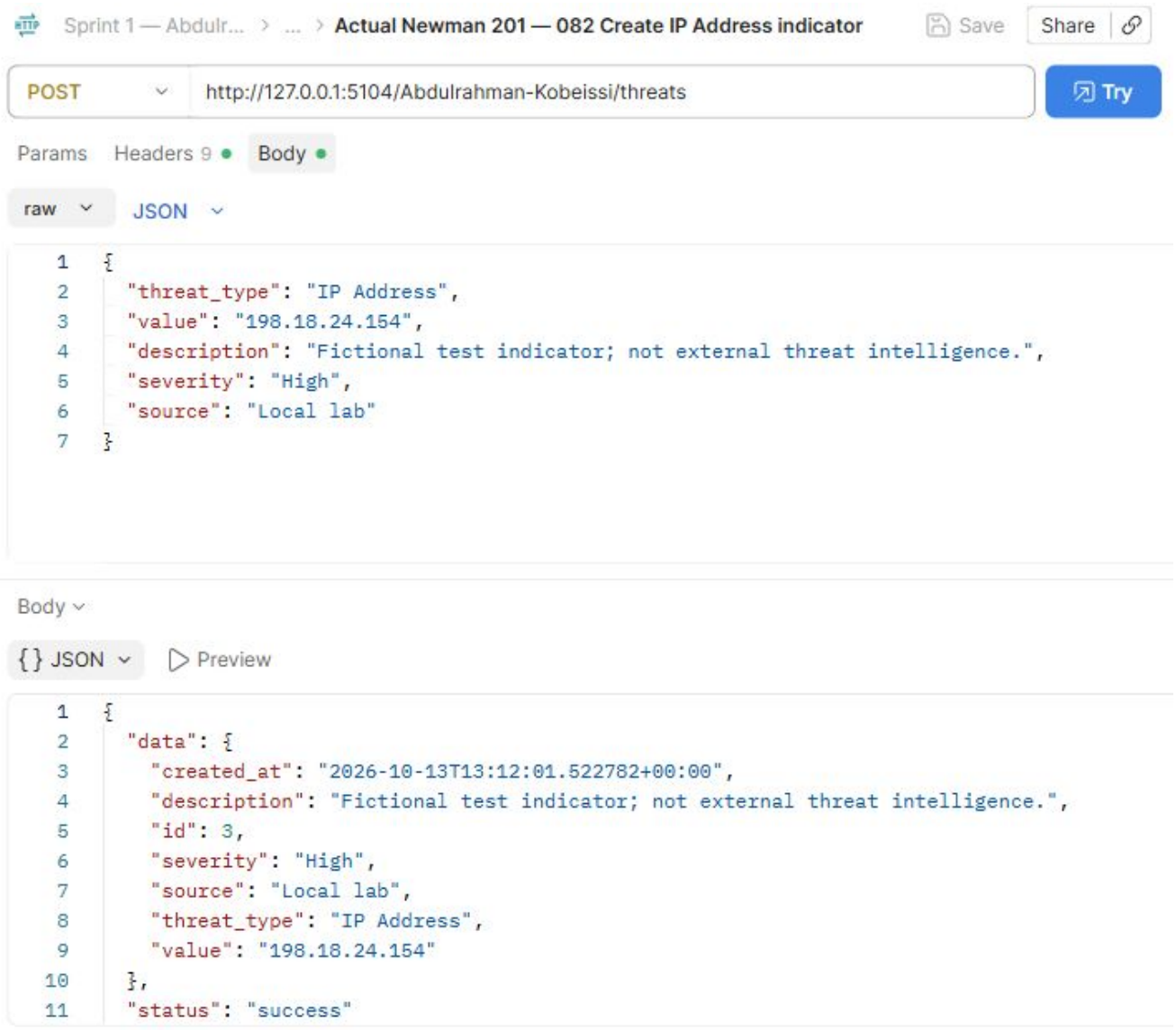


Abdulrahman Kobeissi — 080 Invalid incident severity. The request and saved actual response demonstrate this operation.

Postman Evidence 82

(Abdulrahman Kobeissi)

Create IP Address indicator · POST · observed HTTP 201

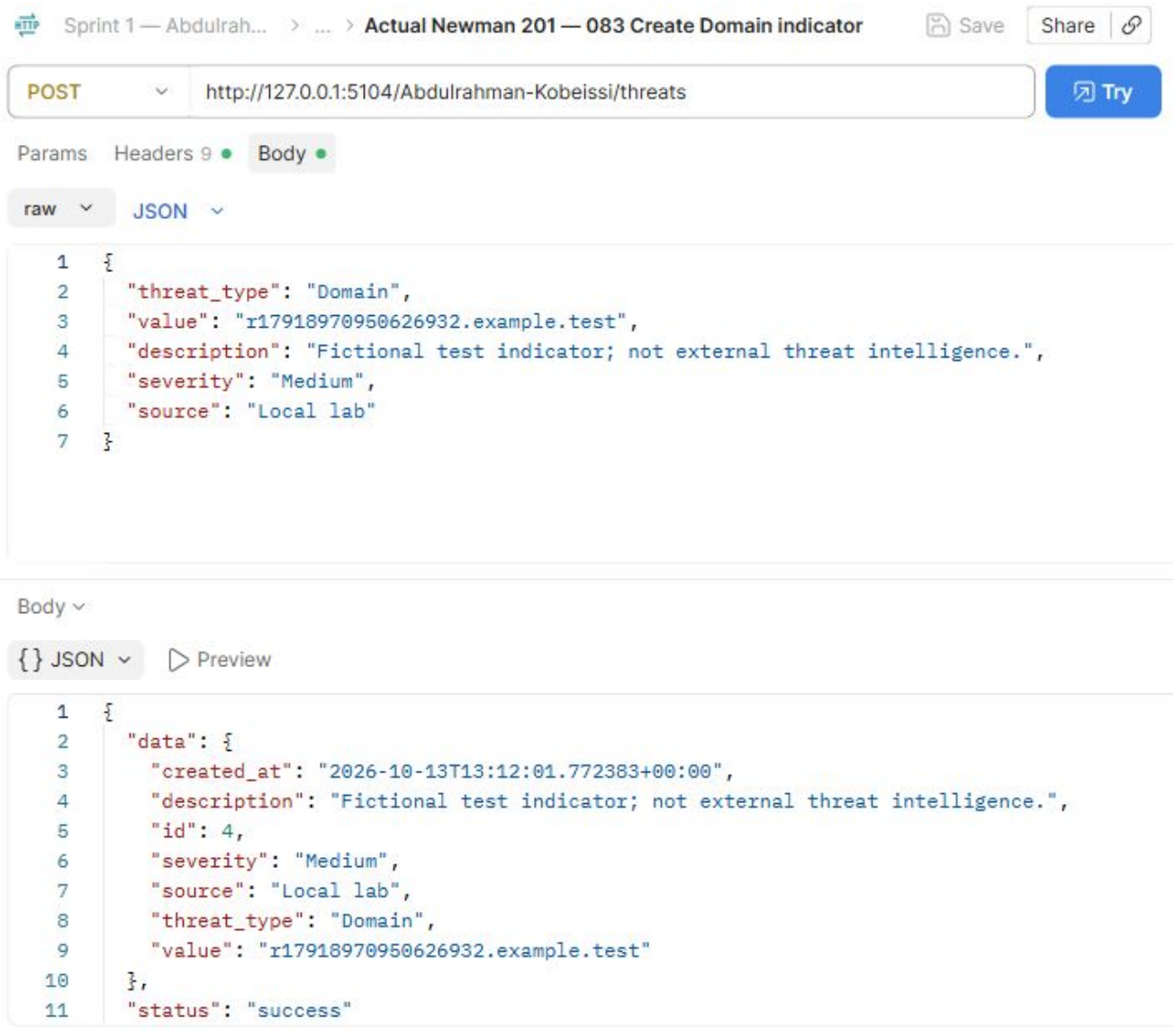


Abdulrahman Kobeissi — 082 Create IP Address indicator. The request and saved actual response demonstrate this operation.

Postman Evidence 83

(Abdulrahman Kobeissi)

Create Domain indicator · POST · observed HTTP 201

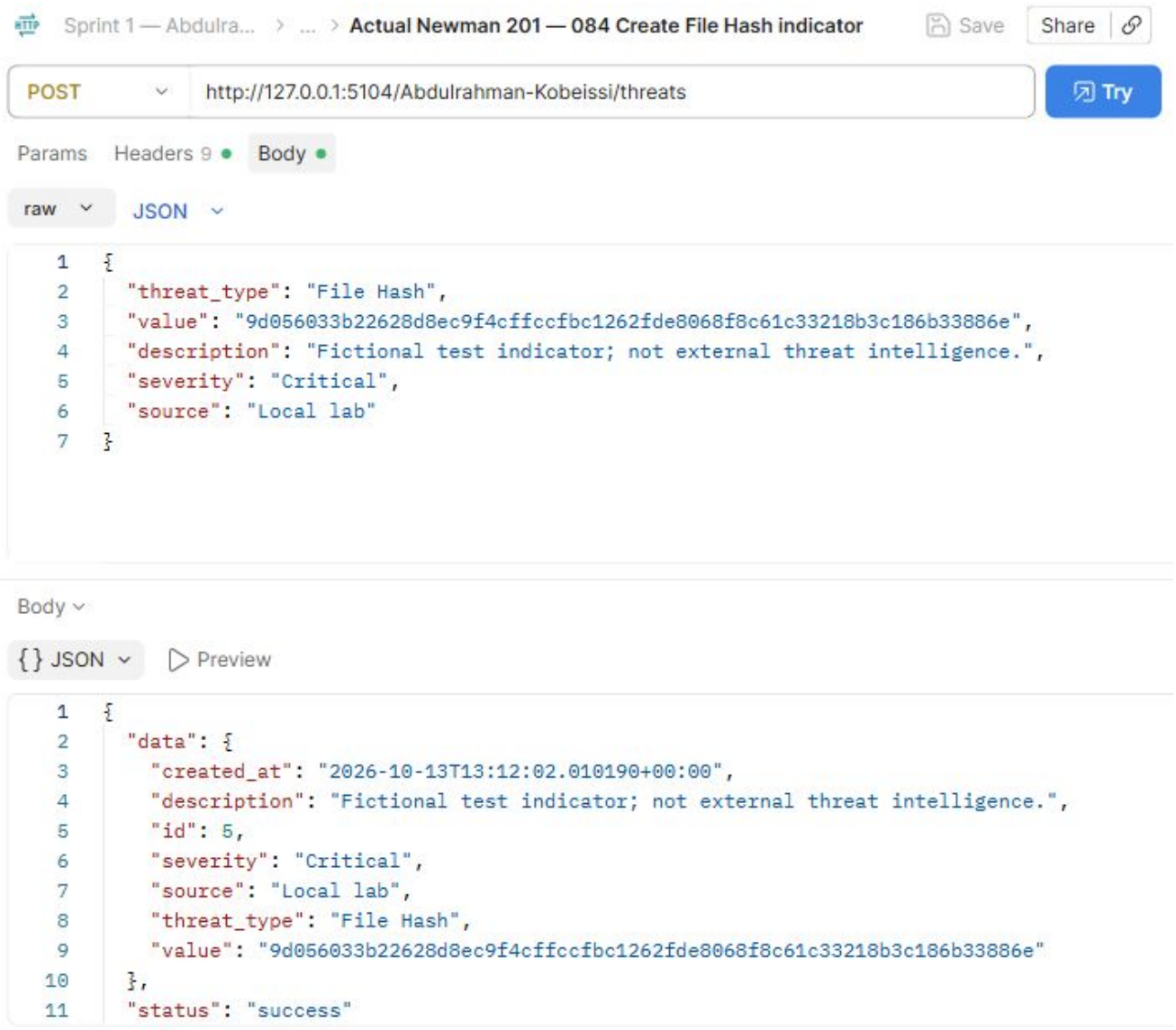


Abdulrahman Kobeissi — 083 Create Domain indicator. The request and saved actual response demonstrate this operation.

Postman Evidence 84

(Abdulrahman Kobeissi)

Create File Hash indicator · POST · observed HTTP 201

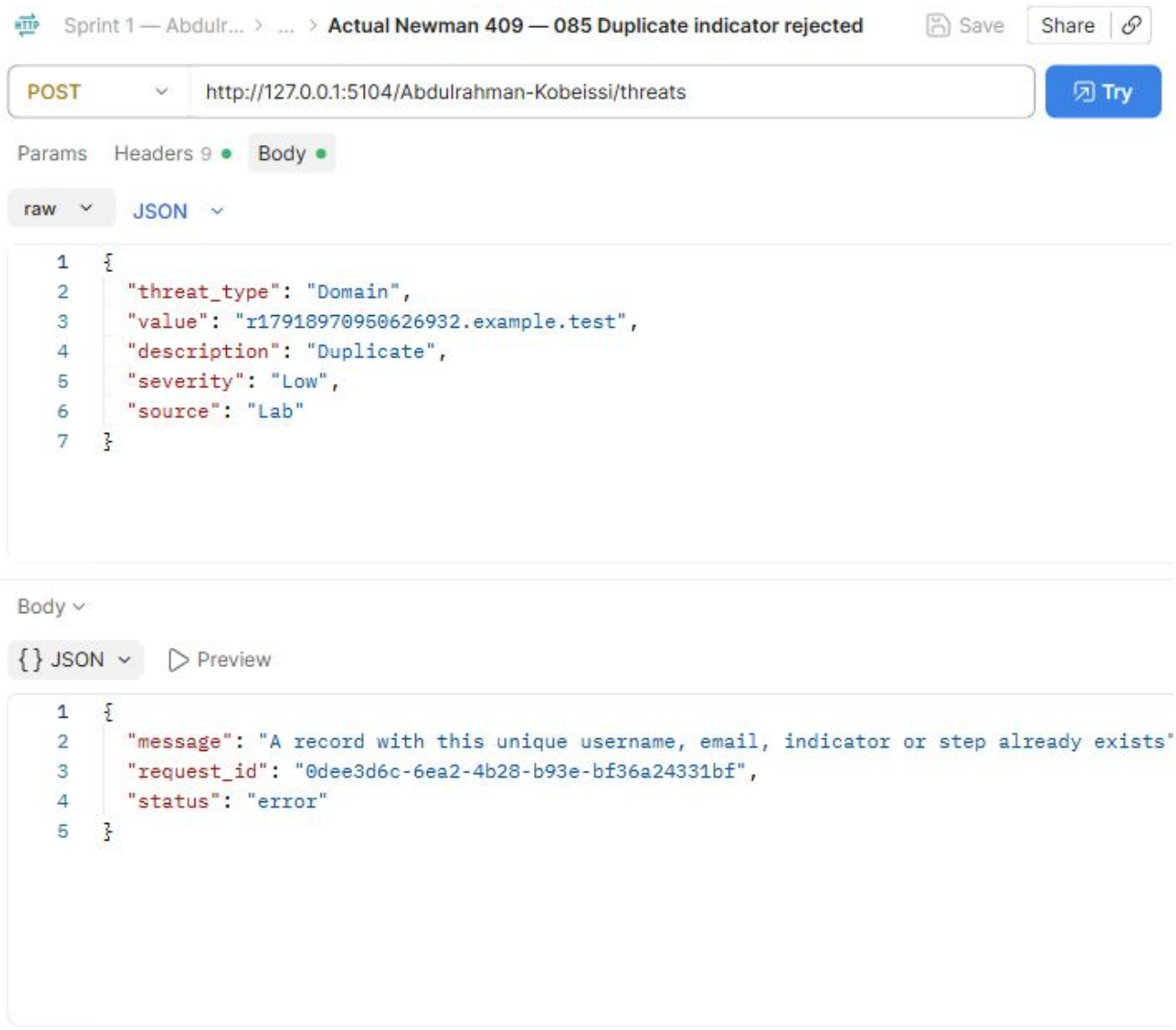


Abdulrahman Kobeissi — 084 Create File Hash indicator. The request and saved actual response demonstrate this operation.

Postman Evidence 85

(Abdulrahman Kobeissi)

Duplicate indicator rejected · POST · observed HTTP 409

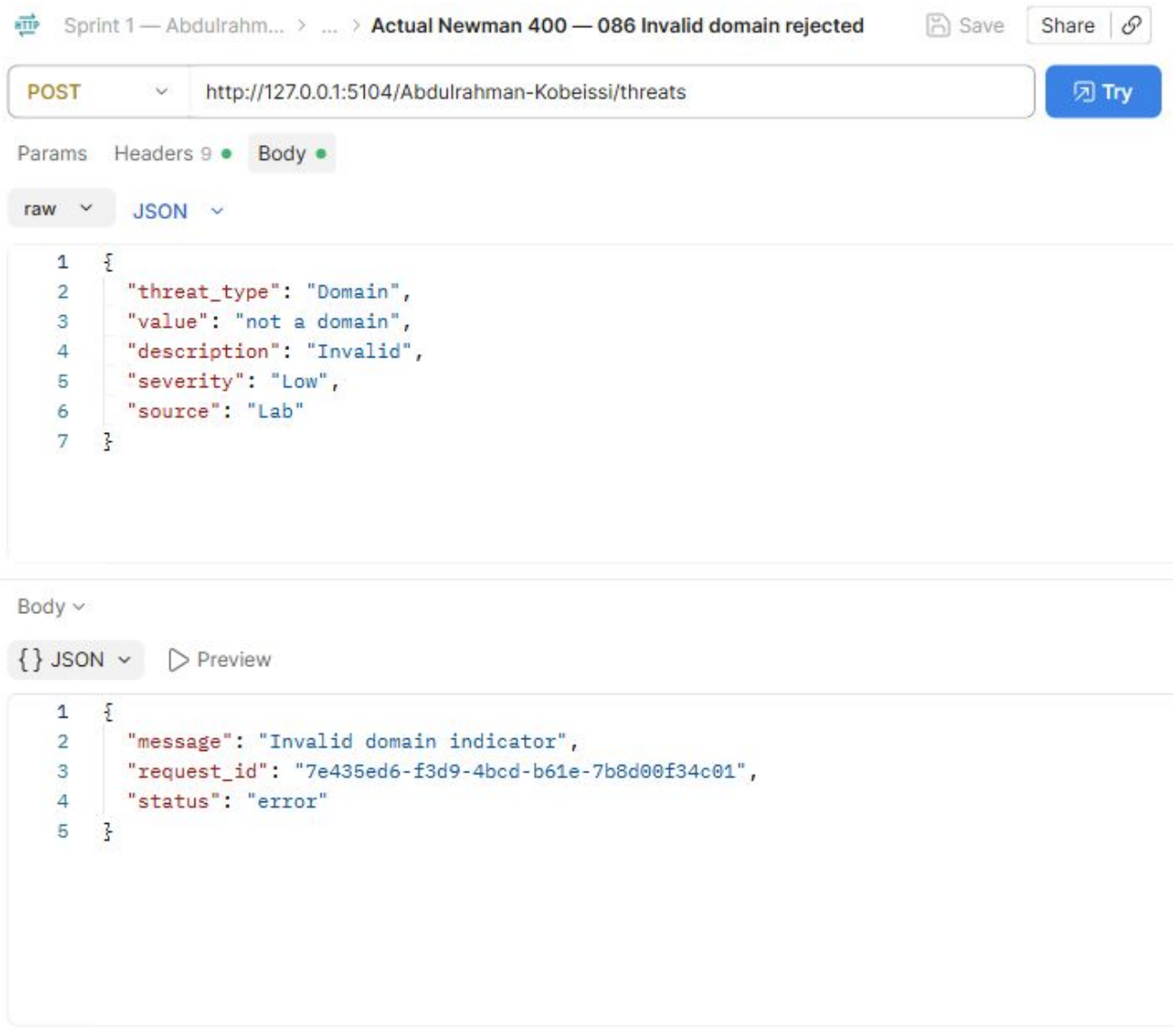


Abdulrahman Kobeissi — 085 Duplicate indicator rejected. The request and saved actual response demonstrate this operation.

Postman Evidence 86

(Abdulrahman Kobeissi)

Invalid domain rejected · POST · observed HTTP 400

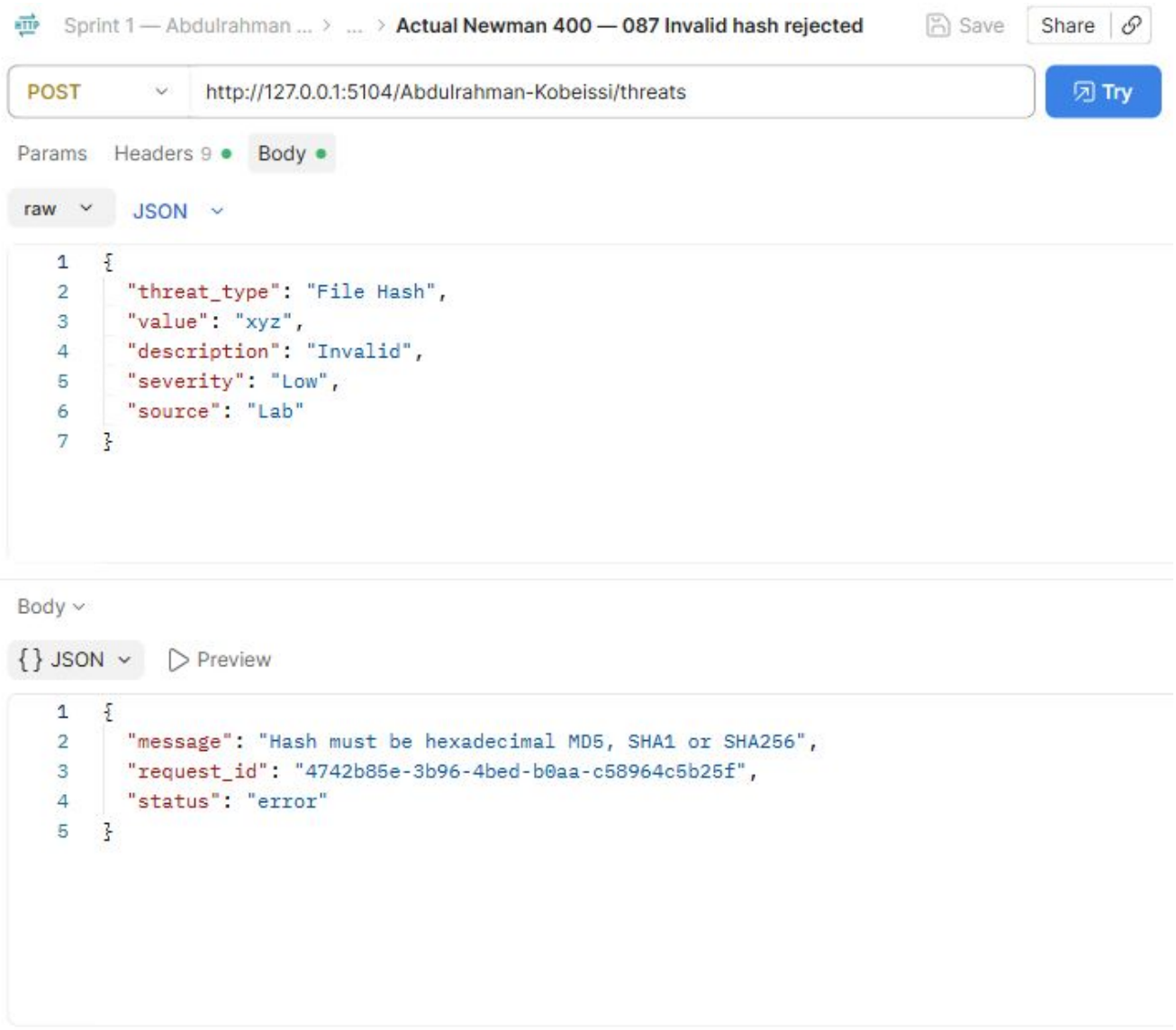


Abdulrahman Kobeissi — 086 Invalid domain rejected. The request and saved actual response demonstrate this operation.

Postman Evidence 87

(Abdulrahman Kobeissi)

Invalid hash rejected · POST · observed HTTP 400

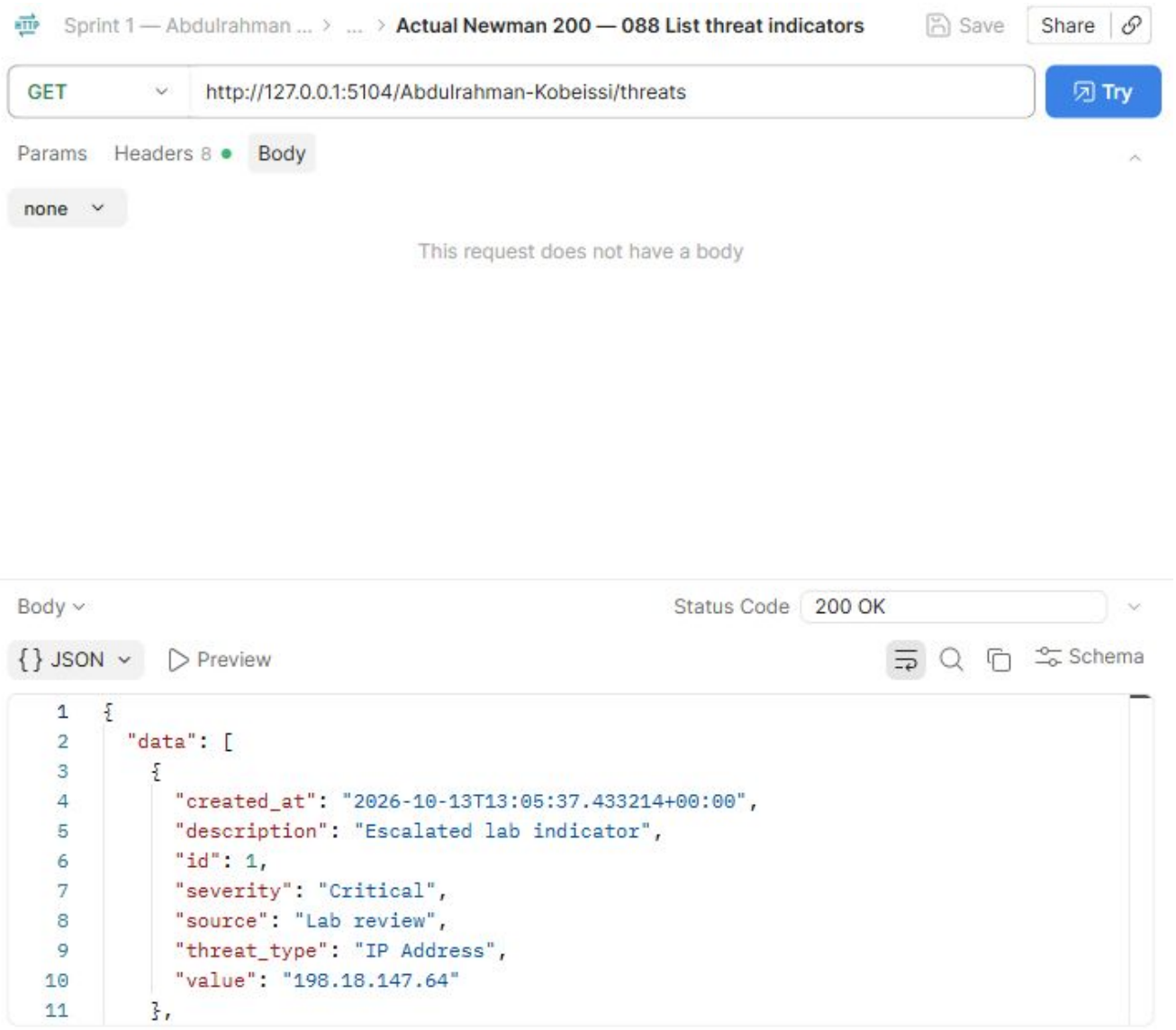


Abdulrahman Kobeissi — 087 Invalid hash rejected. The request and saved actual response demonstrate this operation.

Postman Evidence 88

(Abdulrahman Kobeissi)

List threat indicators · GET · observed HTTP 200

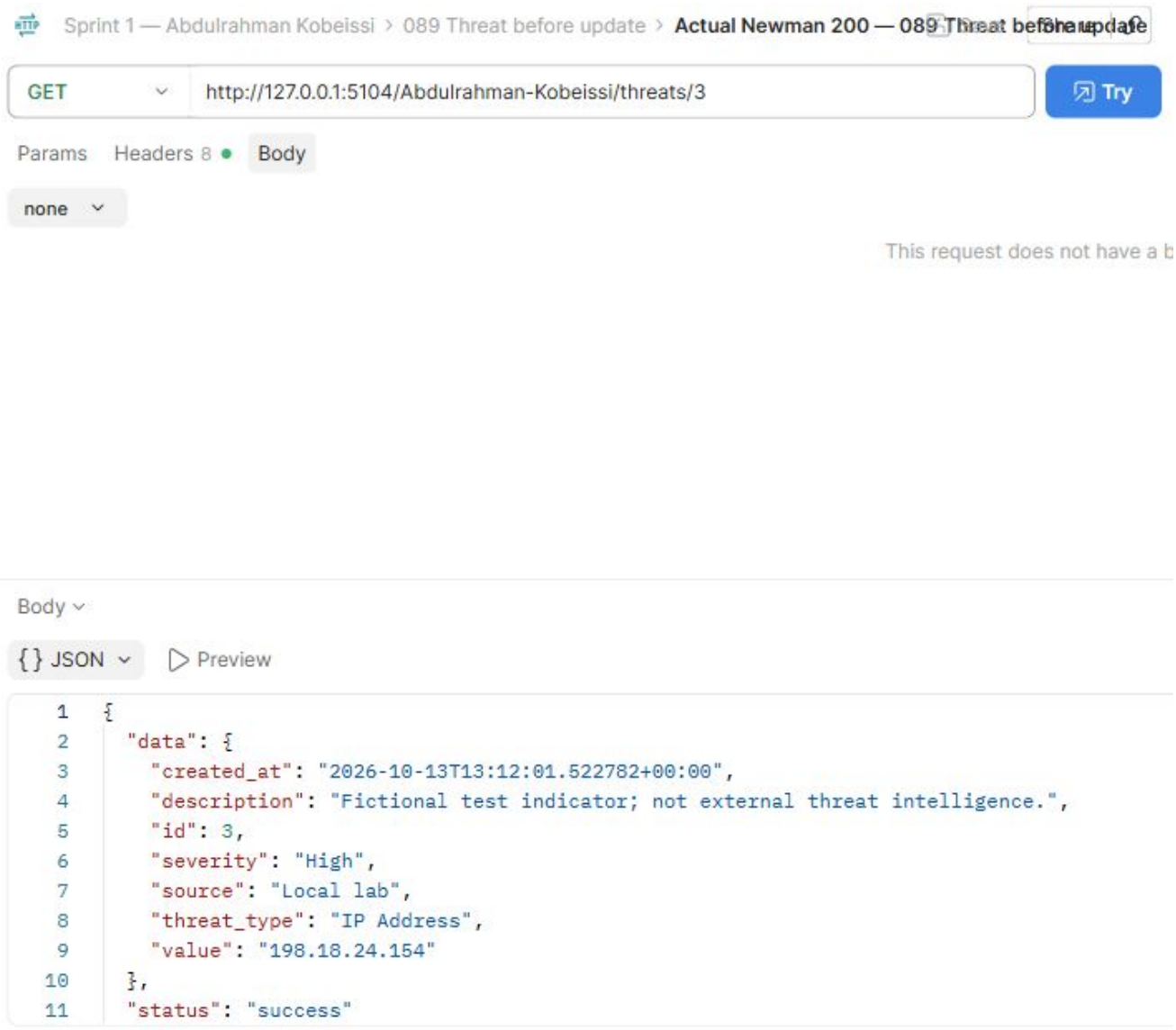


Abdulrahman Kobeissi — 088 List threat indicators. The request and saved actual response demonstrate this operation.

Postman Evidence 89

(Abdulrahman Kobeissi)

Threat before update · GET · observed HTTP 200



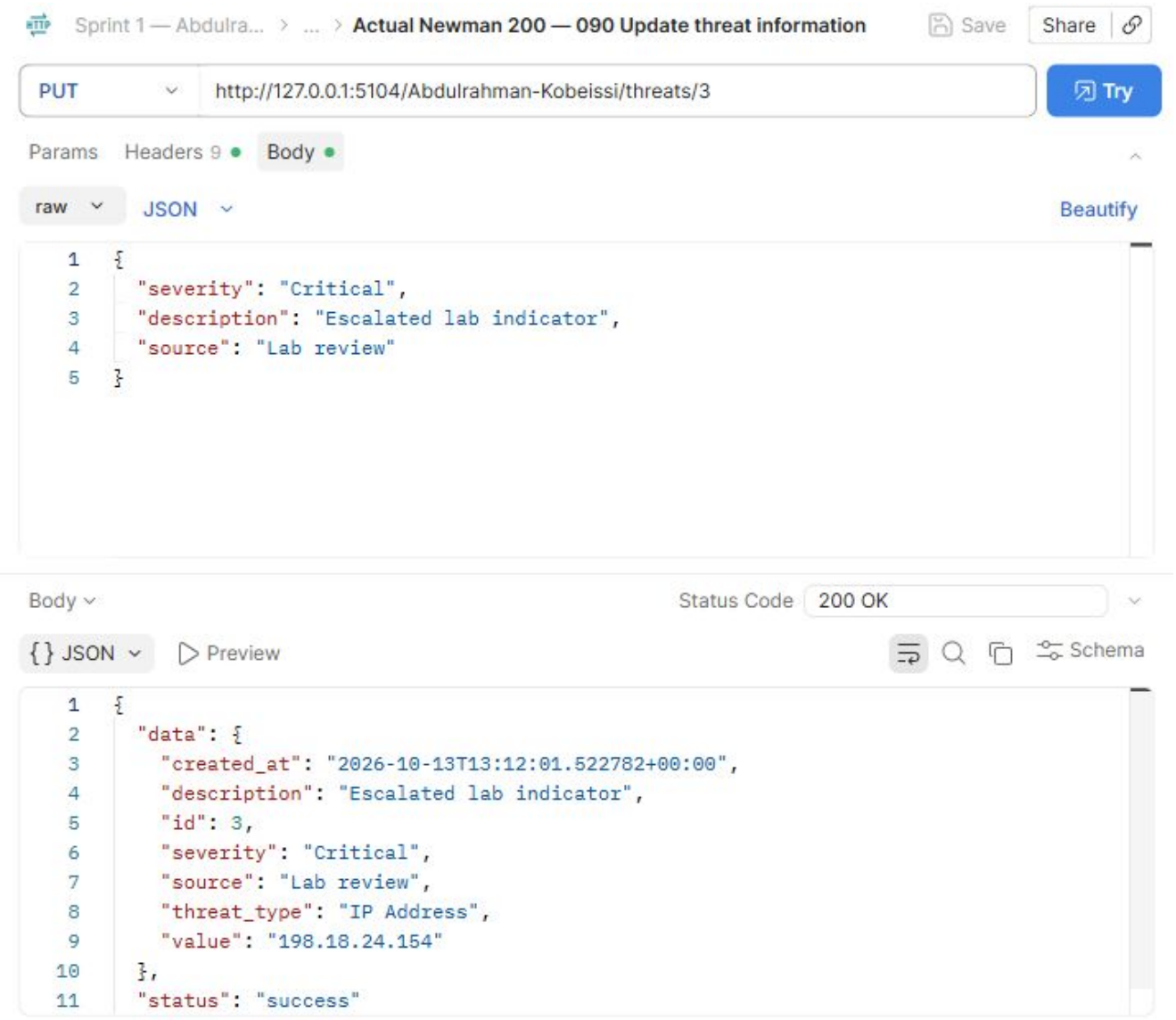
Abdulrahman Kobeissi — 089 Threat before update. The request and saved actual response demonstrate this operation.

Compare this view with the adjacent before/update/after requests. The full response remains available in the collection.

Postman Evidence 90

(Abdulrahman Kobeissi)

Update threat information · PUT · observed HTTP 200

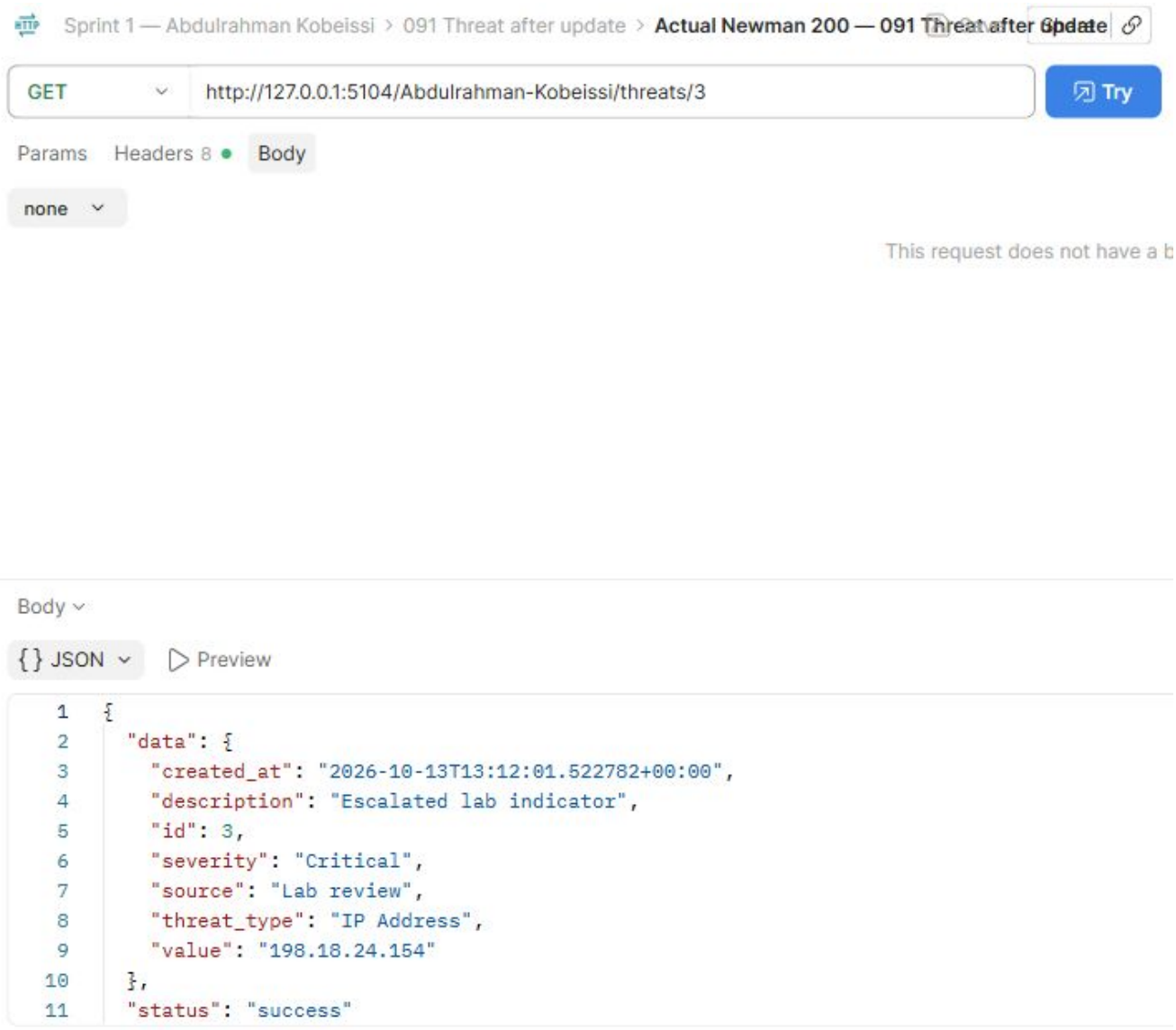


Abdulrahman Kobeissi — 090 Update threat information. The request and saved actual response demonstrate this operation.

Postman Evidence 91

(Abdulrahman Kobeissi)

Threat after update · GET · observed HTTP 200



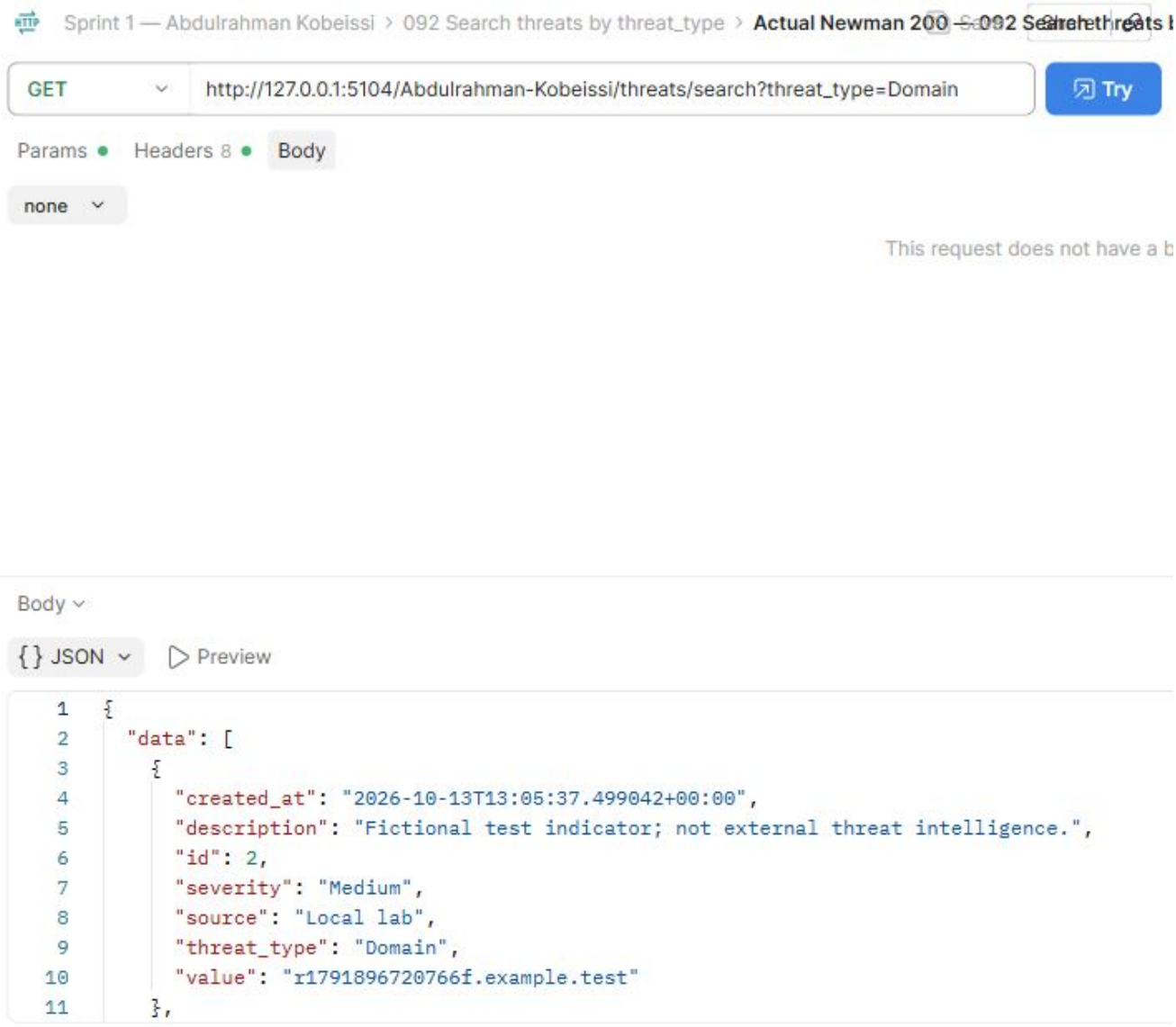
Abdulrahman Kobeissi — 091 Threat after update. The request and saved actual response demonstrate this operation.

Compare this view with the adjacent before/update/after requests. The full response remains available in the collection.

Postman Evidence 92

(Abdulrahman Kobeissi)

Search threats by threat_type · GET · observed HTTP 200

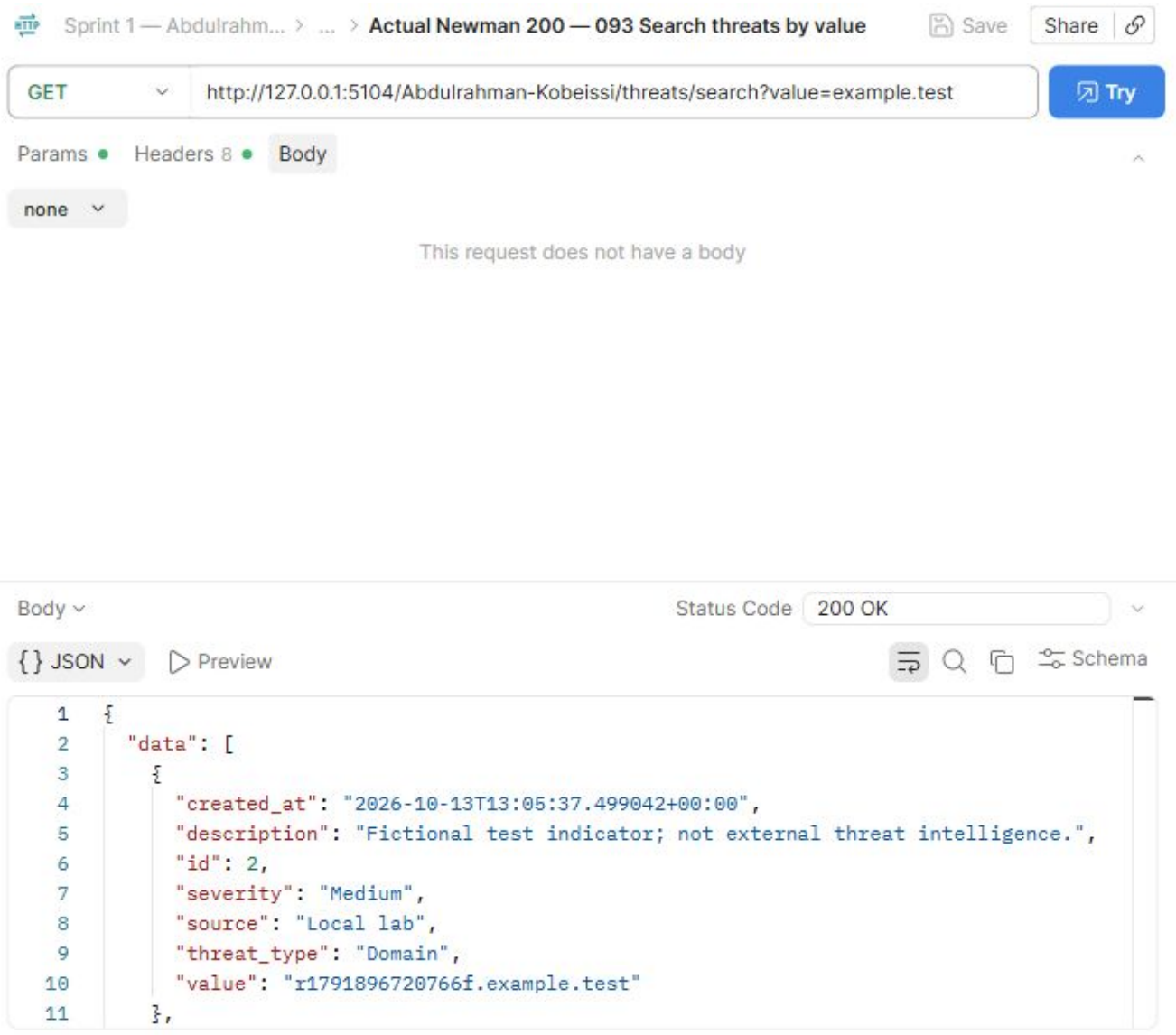


Abdulrahman Kobeissi — 092 Search threats by threat_type. The request and saved actual response demonstrate this operation.

Postman Evidence 93

(Abdulrahman Kobeissi)

Search threats by value · GET · observed HTTP 200

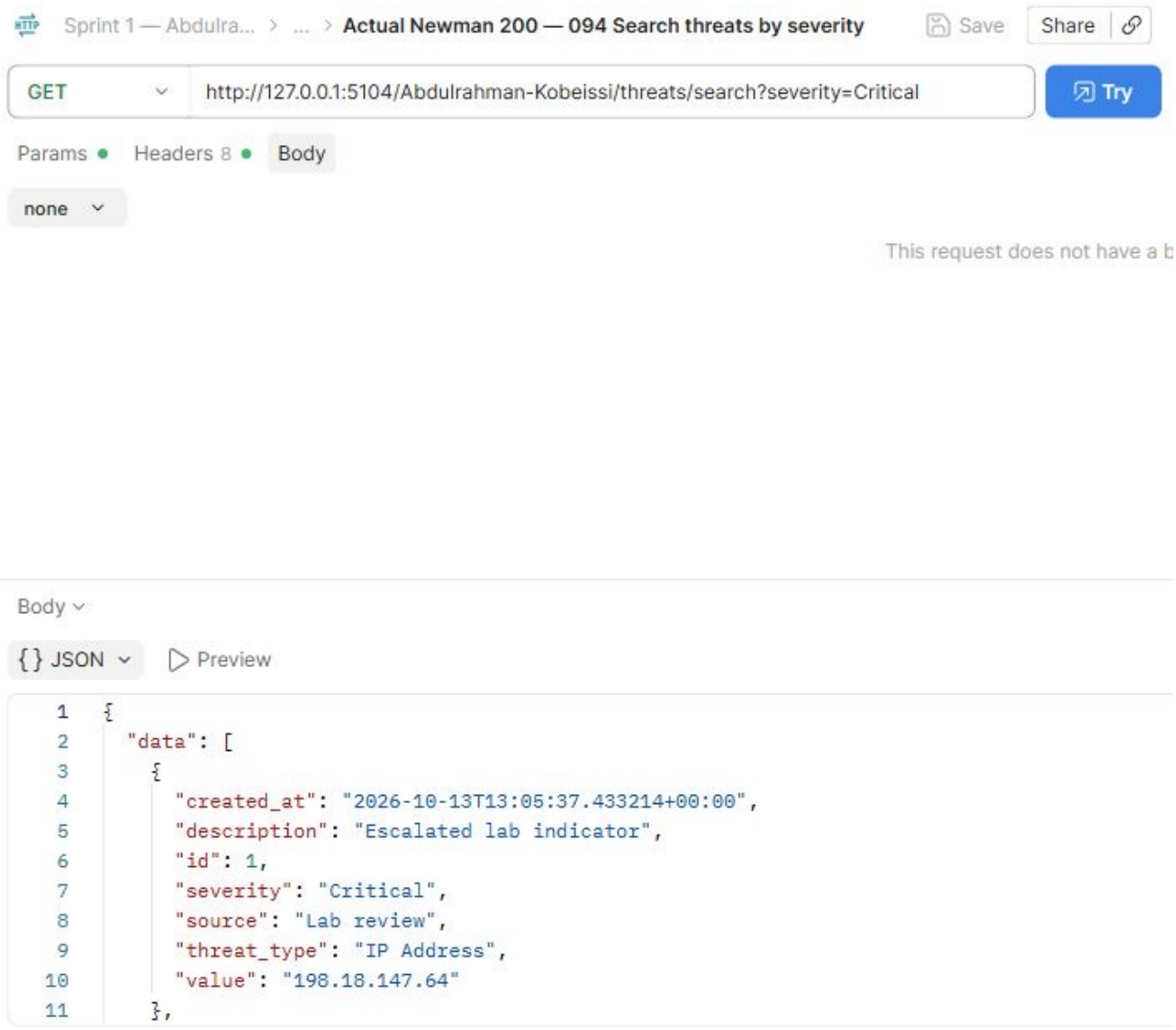


Abdulrahman Kobeissi — 093 Search threats by value. The request and saved actual response demonstrate this operation.

Postman Evidence 94

(Abdulrahman Kobeissi)

Search threats by severity · GET · observed HTTP 200

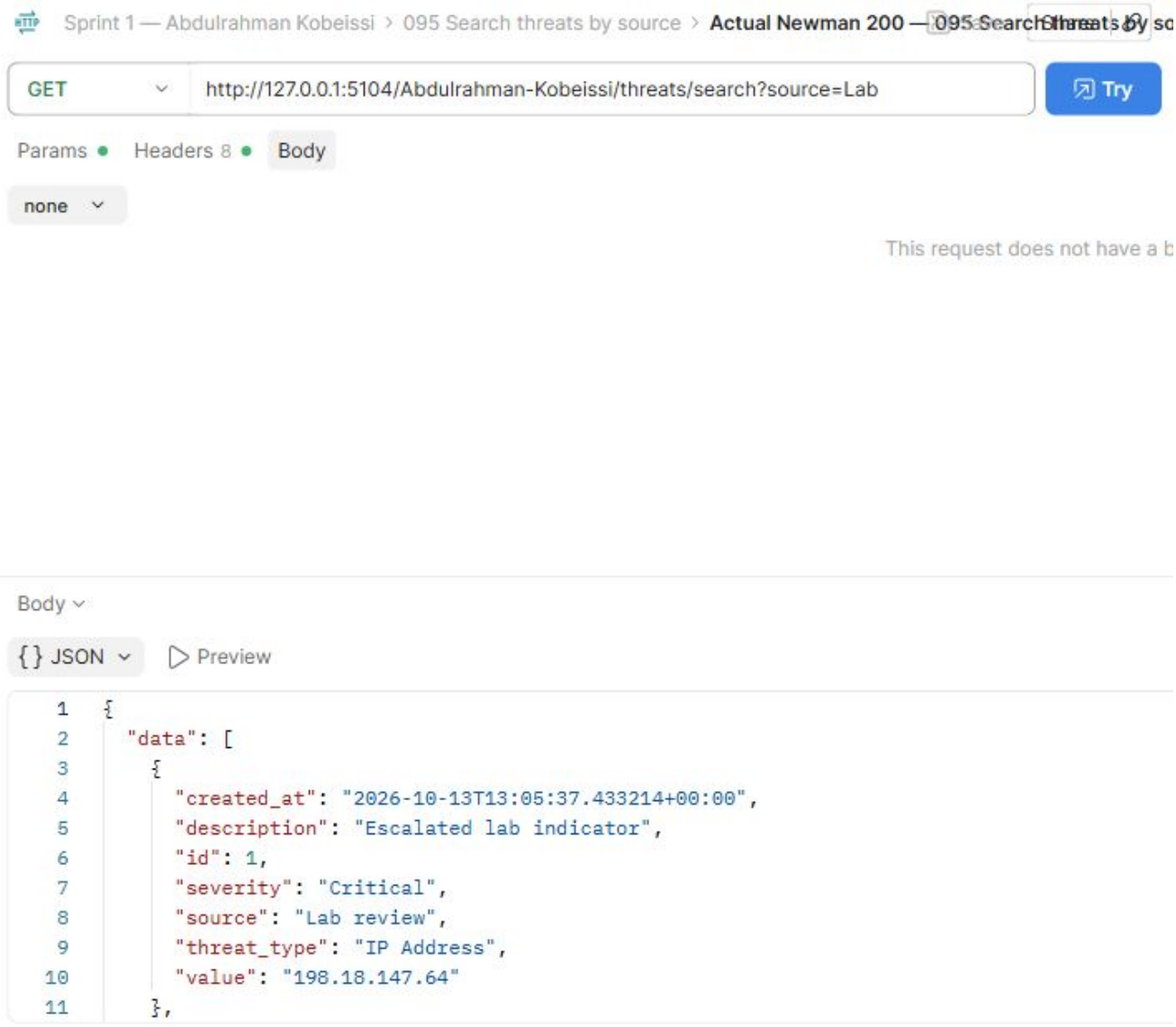


Abdulrahman Kobeissi — 094 Search threats by severity. The request and saved actual response demonstrate this operation.

Postman Evidence 95

(Abdulrahman Kobeissi)

Search threats by source · GET · observed HTTP 200

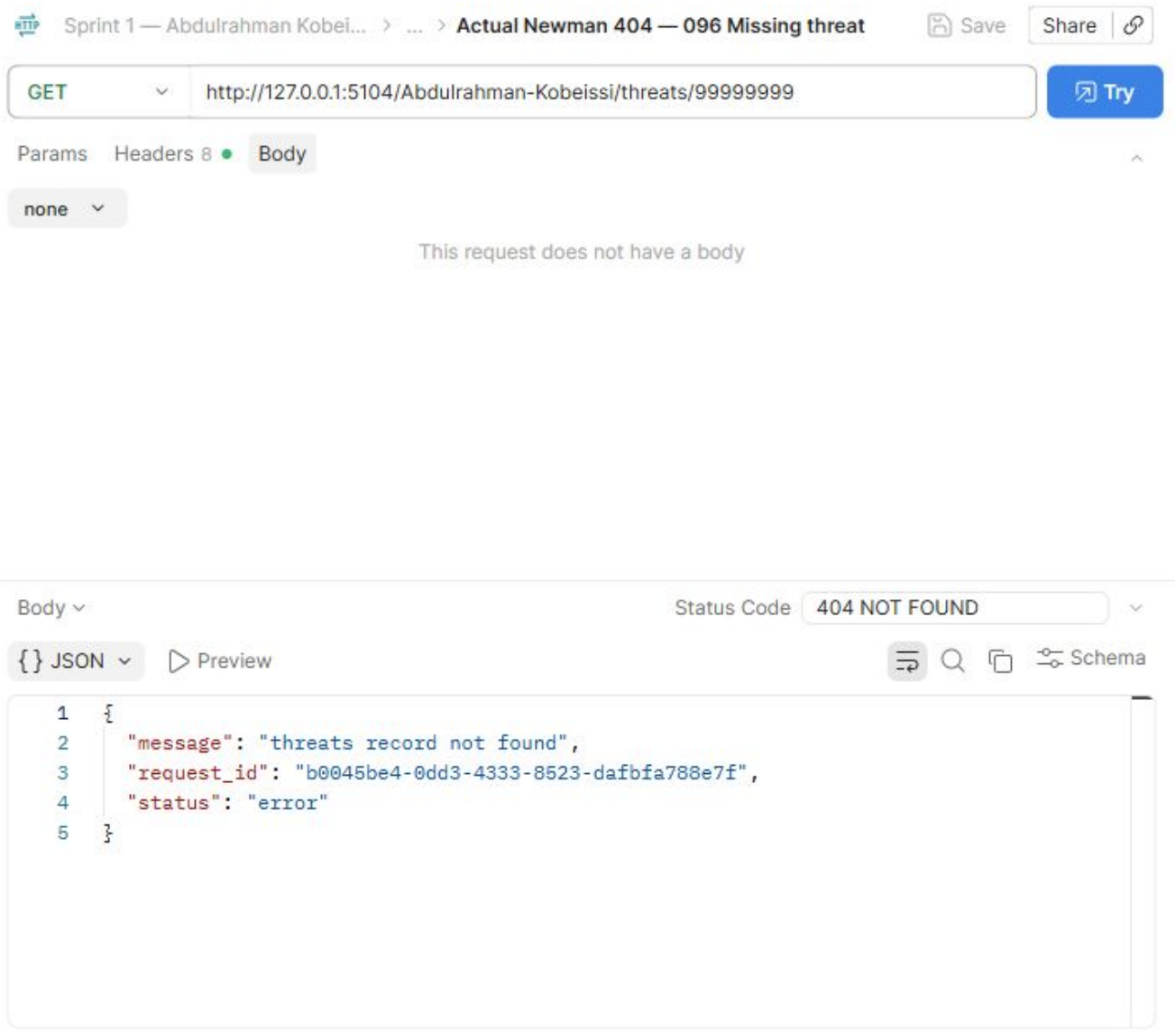


Abdulrahman Kobeissi — 095 Search threats by source. The request and saved actual response demonstrate this operation.

Postman Evidence 96

(Abdulrahman Kobeissi)

Missing threat · GET · observed HTTP 404

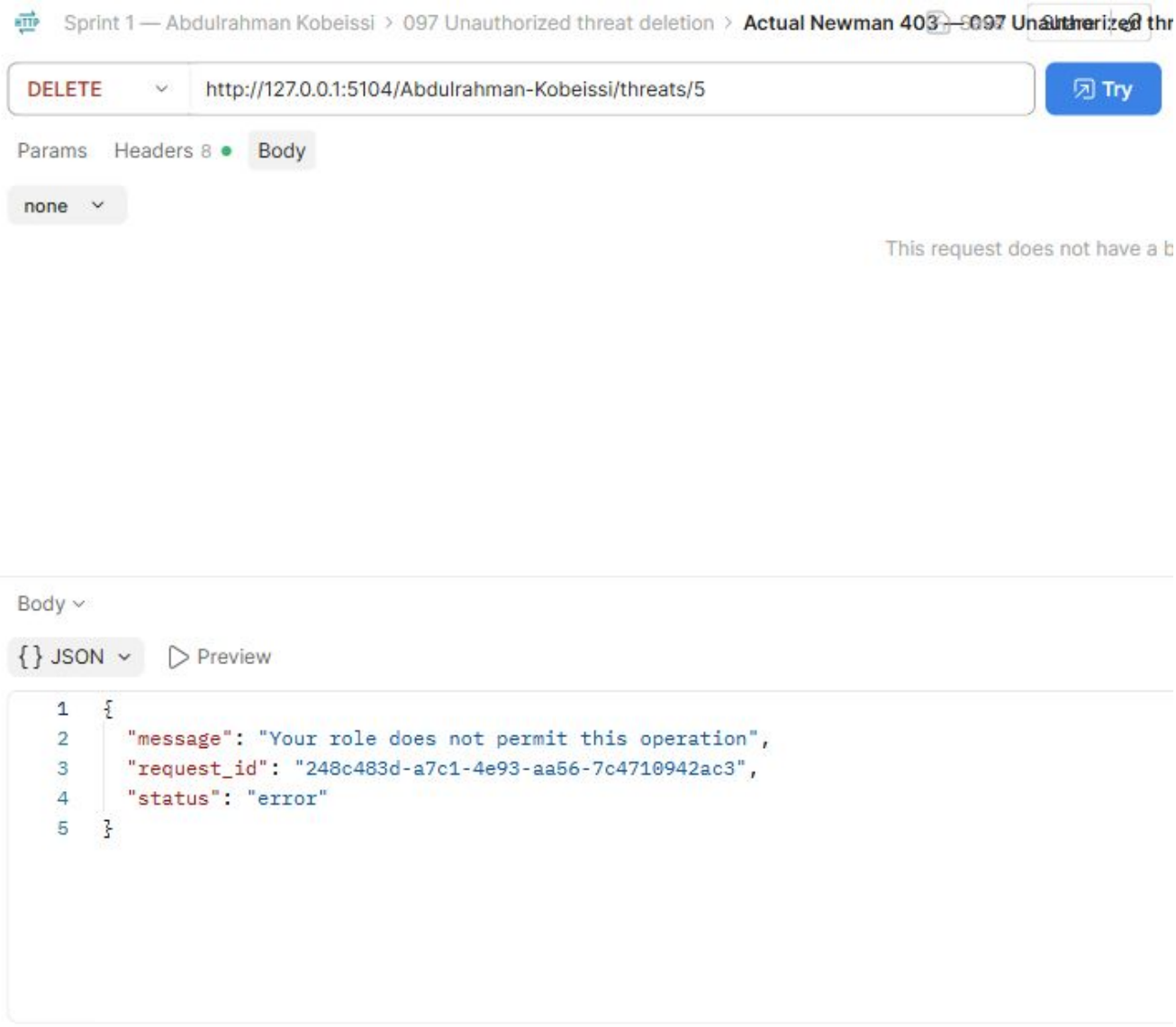


Abdulrahman Kobeissi — 096 Missing threat. The request and saved actual response demonstrate this operation.

Postman Evidence 97

(Abdulrahman Kobeissi)

Unauthorized threat deletion · DELETE · observed HTTP 403

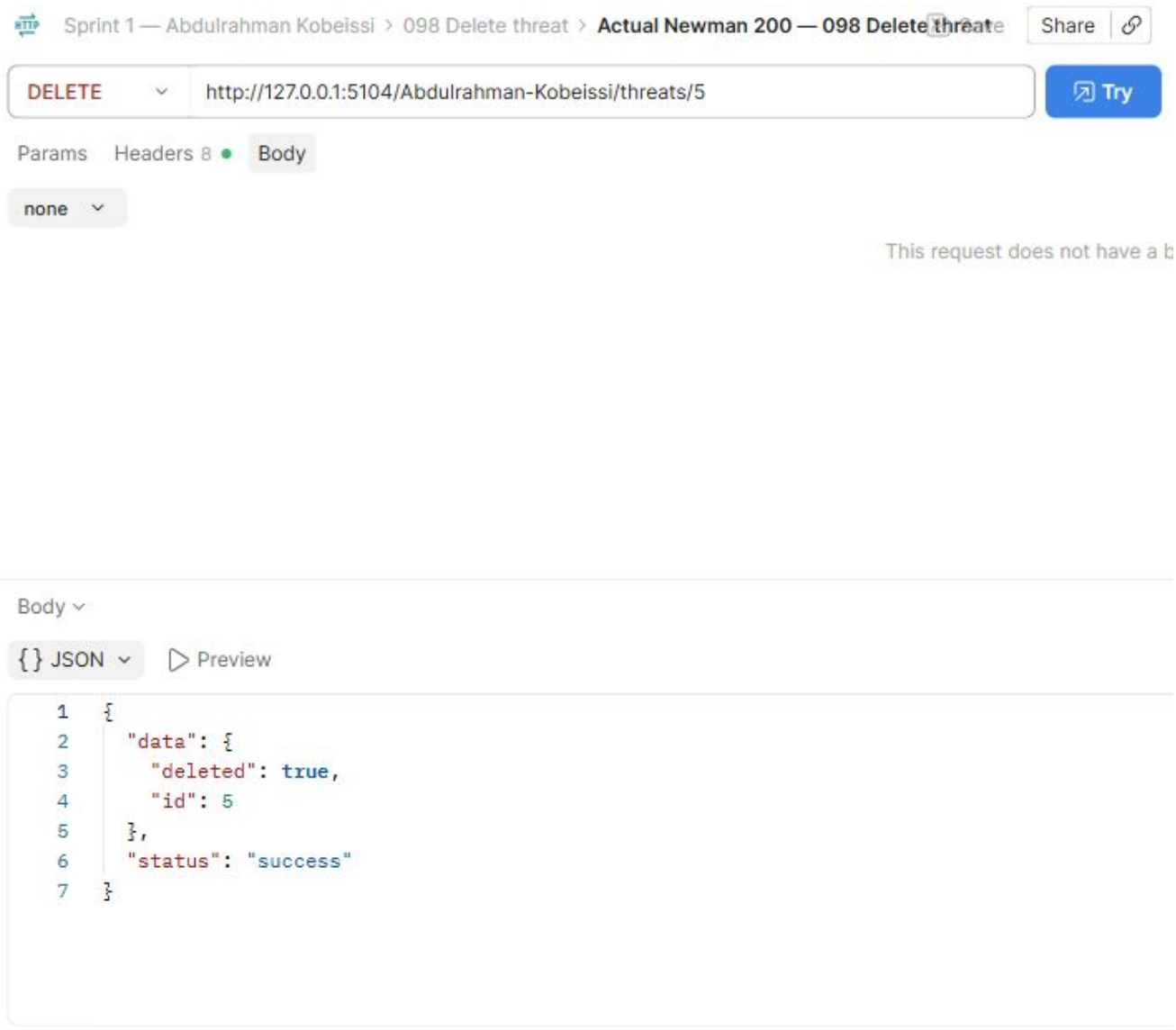


Abdulrahman Kobeissi — 097 Unauthorized threat deletion. The request and saved actual response demonstrate this operation.

Postman Evidence 98

(Abdulrahman Kobeissi)

Delete threat · DELETE · observed HTTP 200

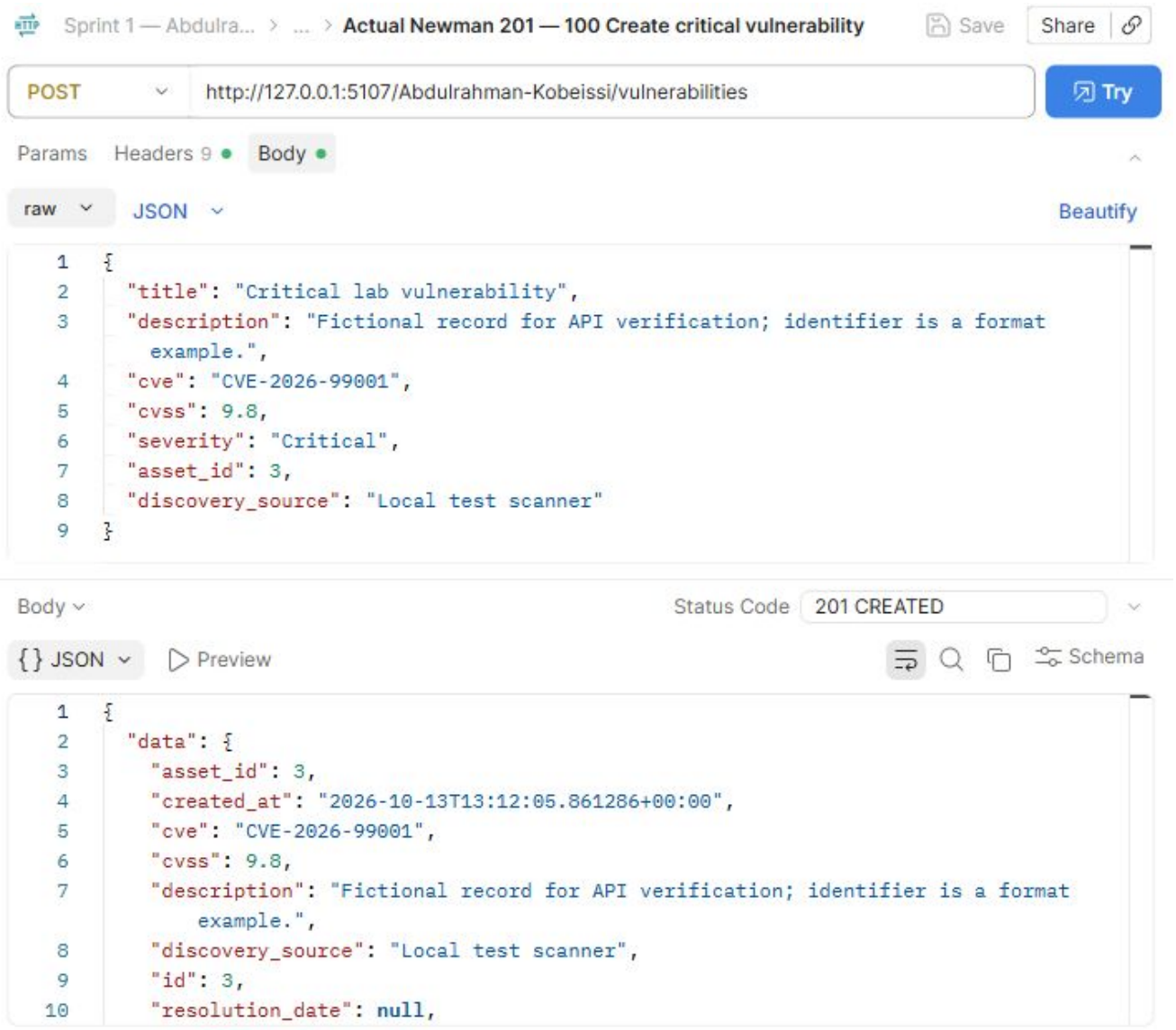


Abdulrahman Kobeissi — 098 Delete threat. The request and saved actual response demonstrate this operation.

Postman Evidence 100

(Abdulrahman Kobeissi)

Create critical vulnerability · POST · observed HTTP 201

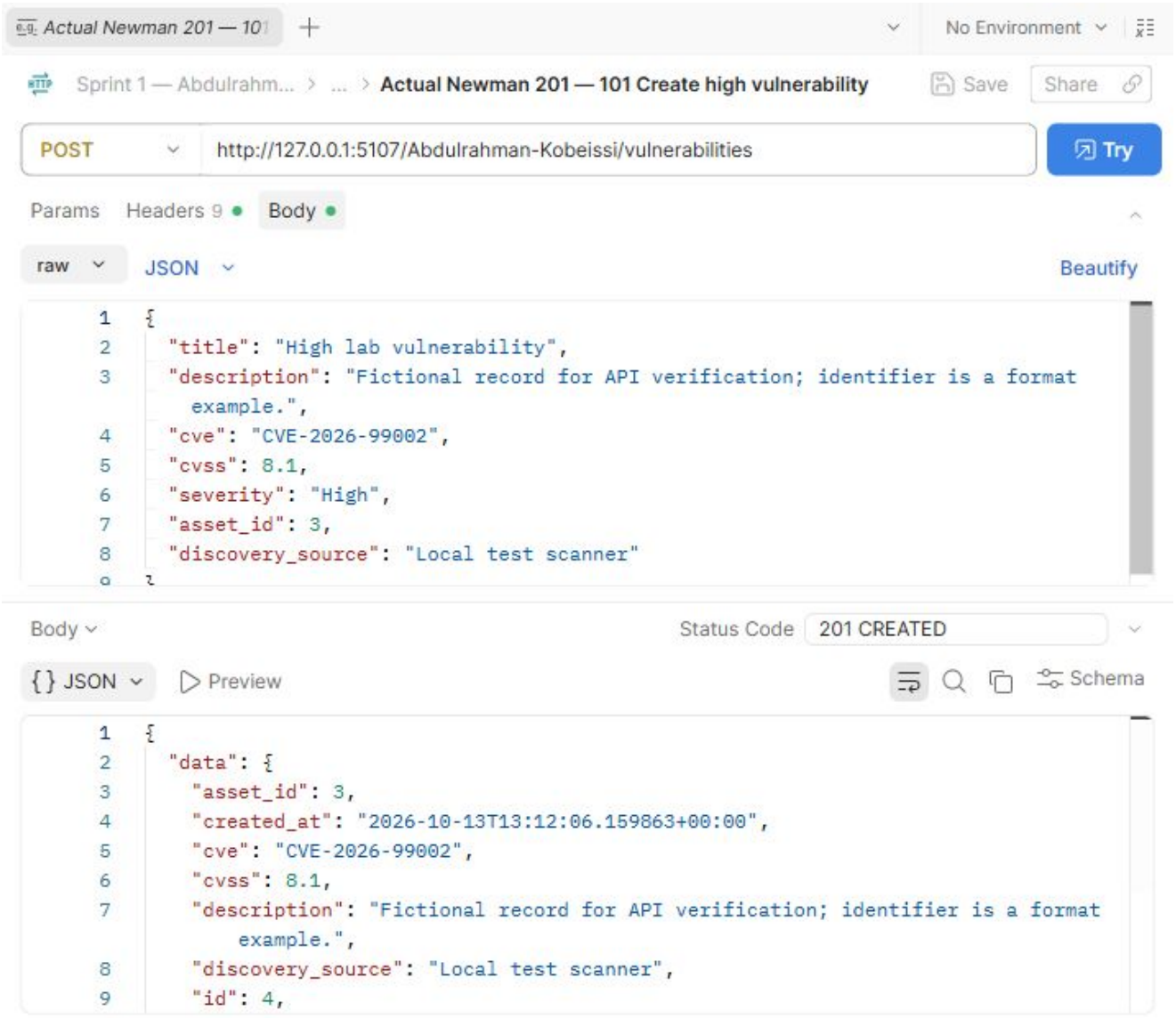


Abdulrahman Kobeissi — 100 Create critical vulnerability. The request and saved actual response demonstrate this operation.

Postman Evidence 101

(Abdulrahman Kobeissi)

Create high vulnerability · POST · observed HTTP 201

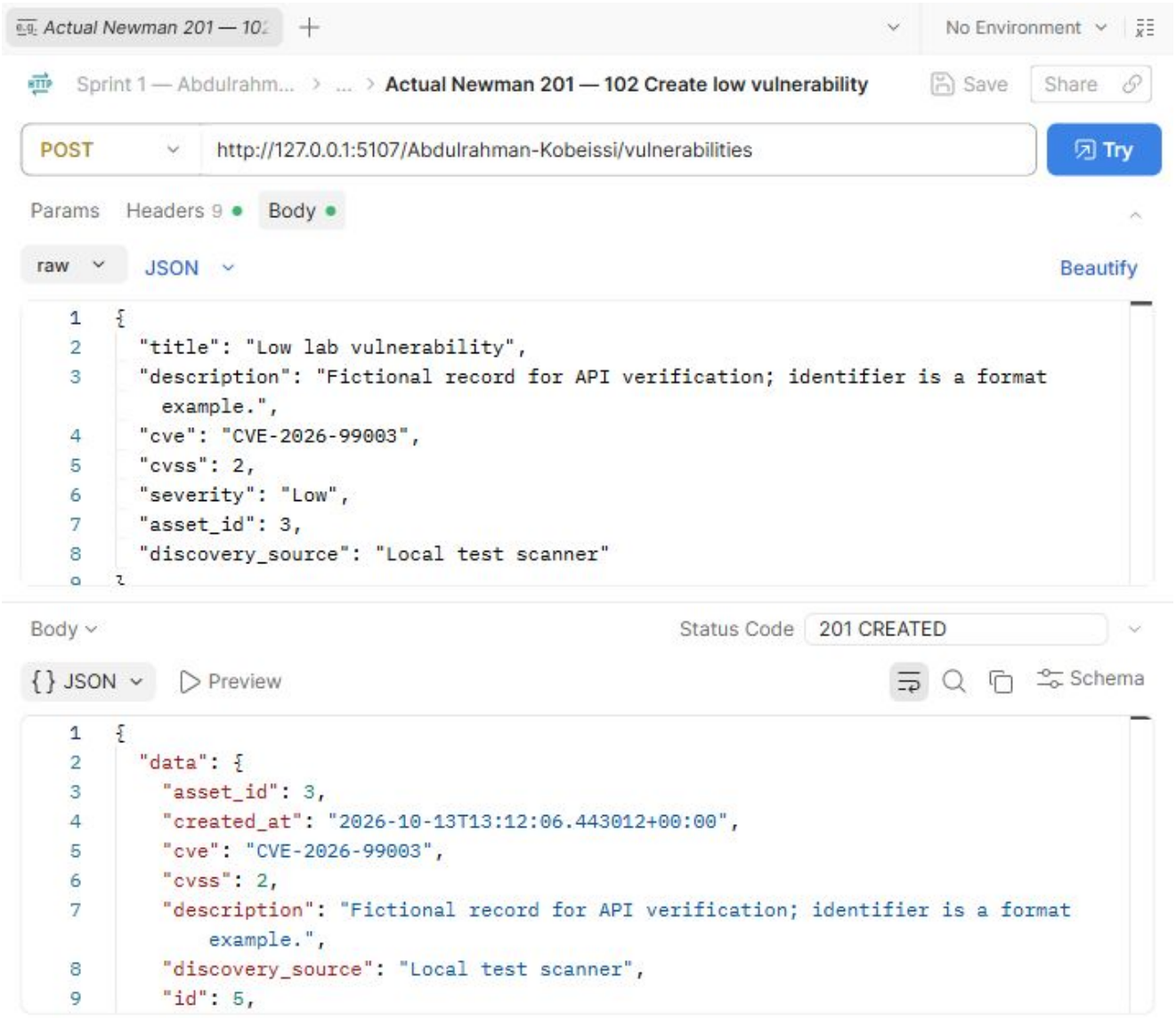


Abdulrahman Kobeissi — 101 Create high vulnerability. The request and saved actual response demonstrate this operation.

Postman Evidence 102

(Abdulrahman Kobeissi)

Create low vulnerability · POST · observed HTTP 201

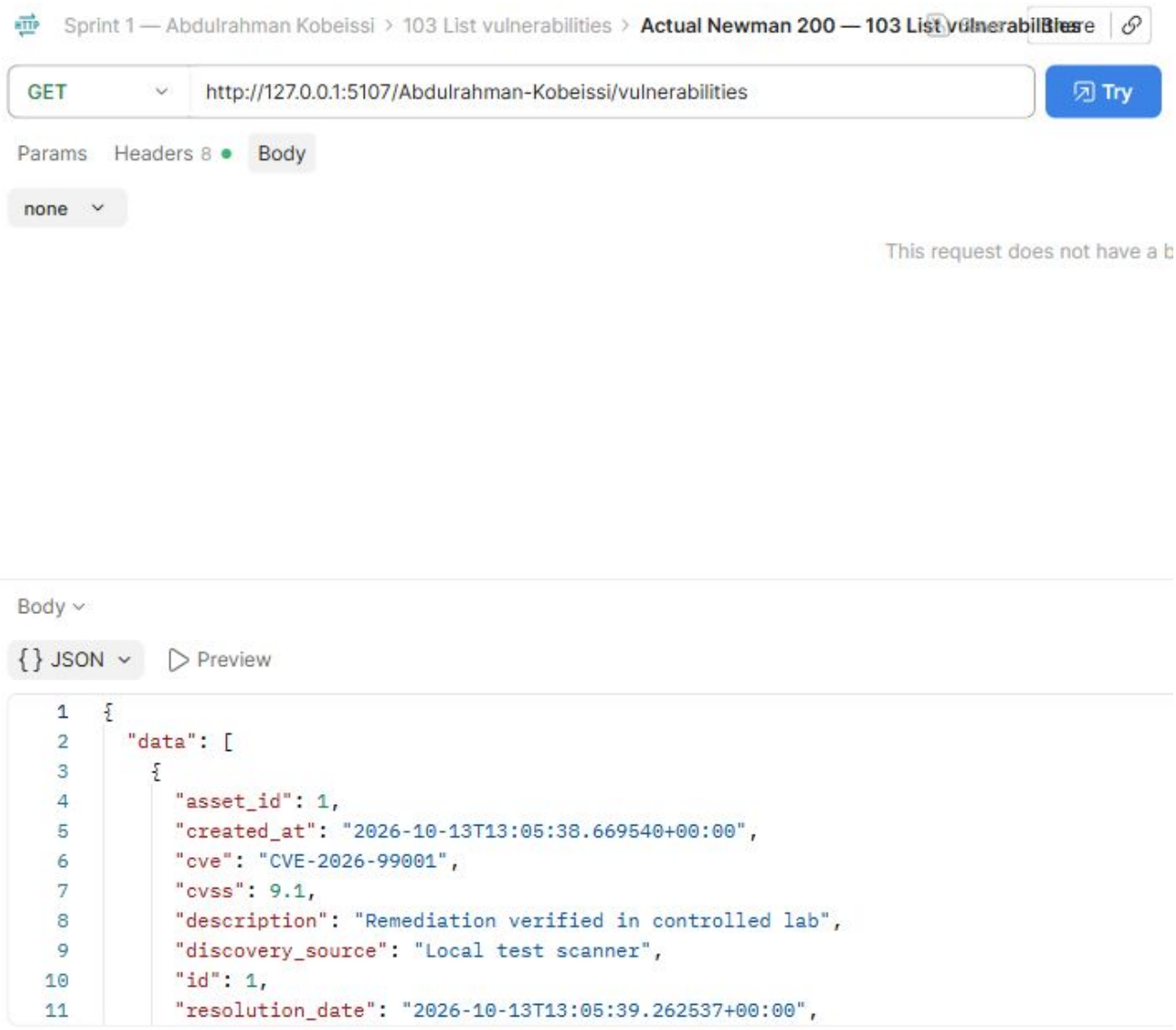


Abdulrahman Kobeissi — 102 Create low vulnerability. The request and saved actual response demonstrate this operation.

Postman Evidence 103

(Abdulrahman Kobeissi)

List vulnerabilities · GET · observed HTTP 200

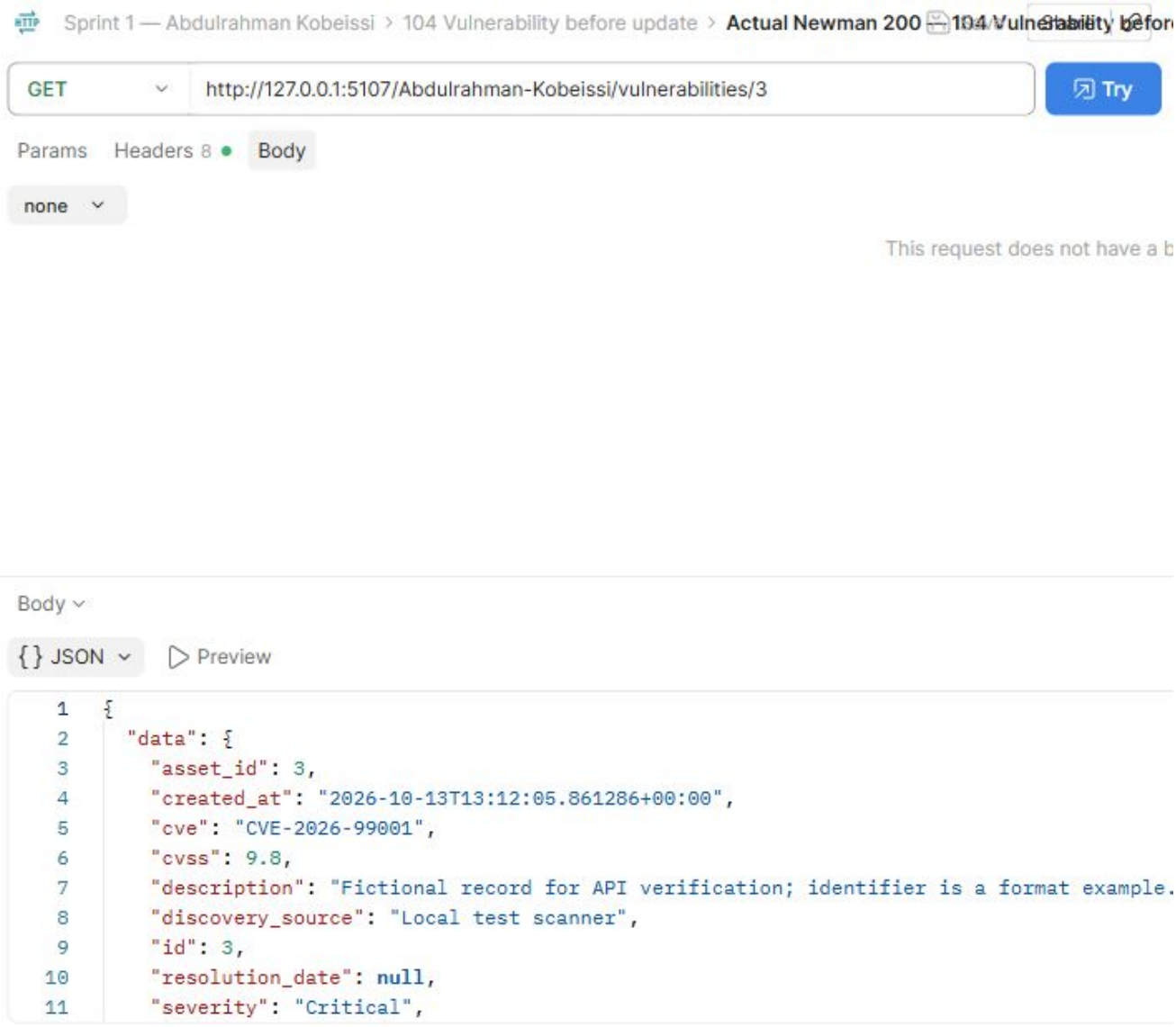


Abdulrahman Kobeissi — 103 List vulnerabilities. The request and saved actual response demonstrate this operation.

Postman Evidence 104

(Abdulrahman Kobeissi)

Vulnerability before update · GET · observed HTTP 200



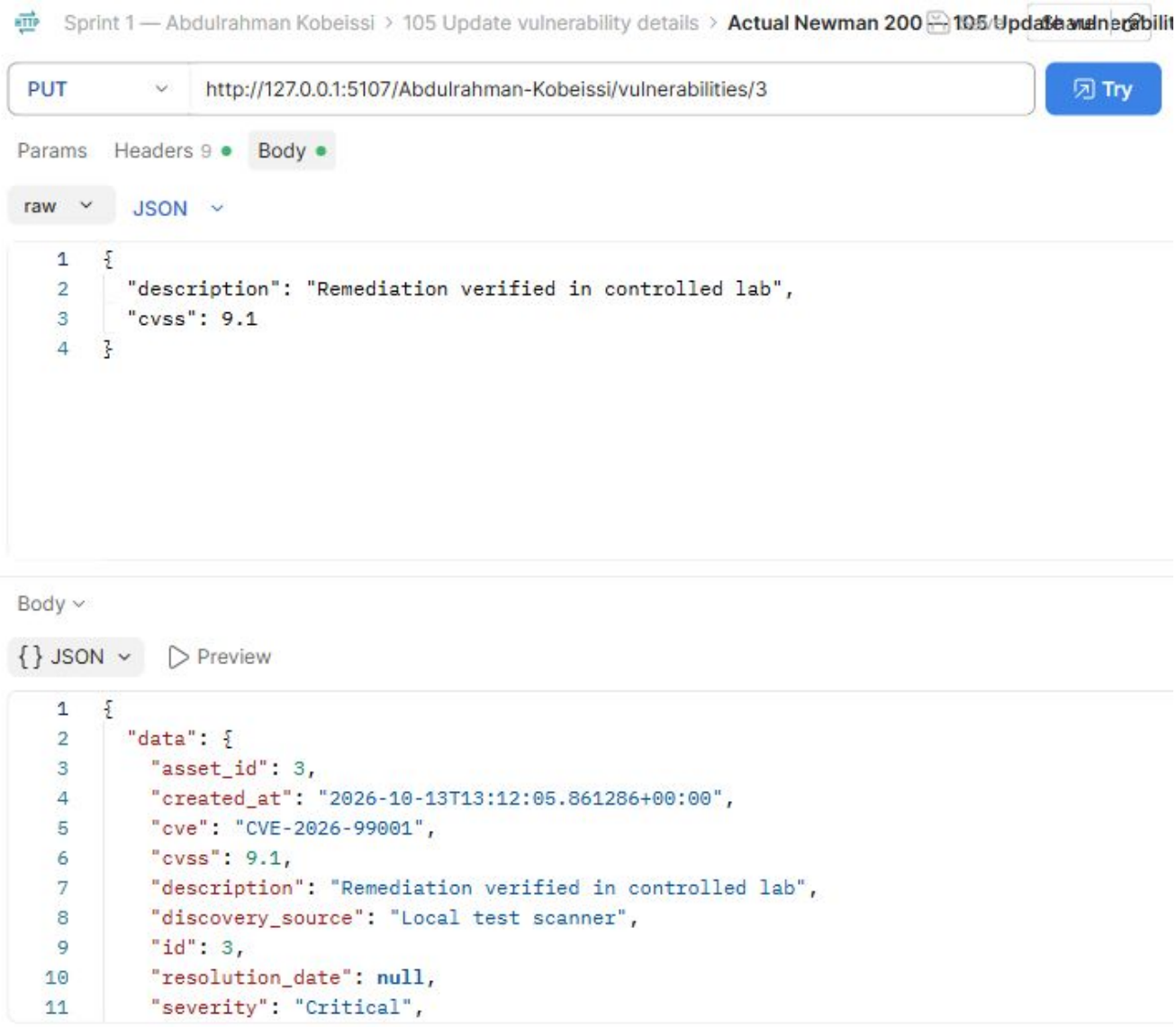
Abdulrahman Kobeissi — 104 Vulnerability before update. The request and saved actual response demonstrate this operation.

Compare this view with the adjacent before/update/after requests. The full response remains available in the collection.

Postman Evidence 105

(Abdulrahman Kobeissi)

Update vulnerability details · PUT · observed HTTP 200

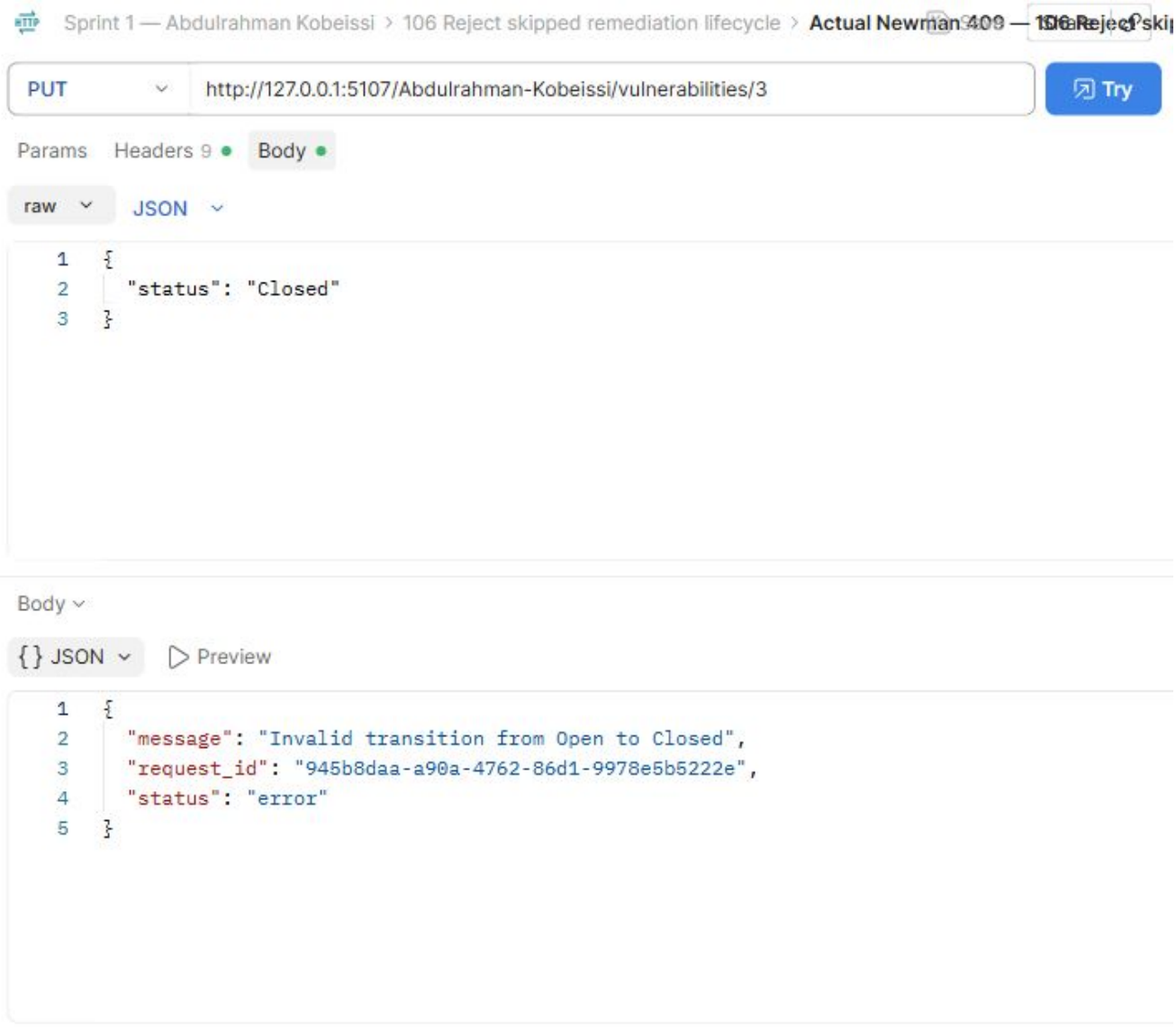


Abdulrahman Kobeissi — 105 Update vulnerability details. The request and saved actual response demonstrate this operation.

Postman Evidence 106

(Abdulrahman Kobeissi)

Reject skipped remediation lifecycle · PUT · observed HTTP 409

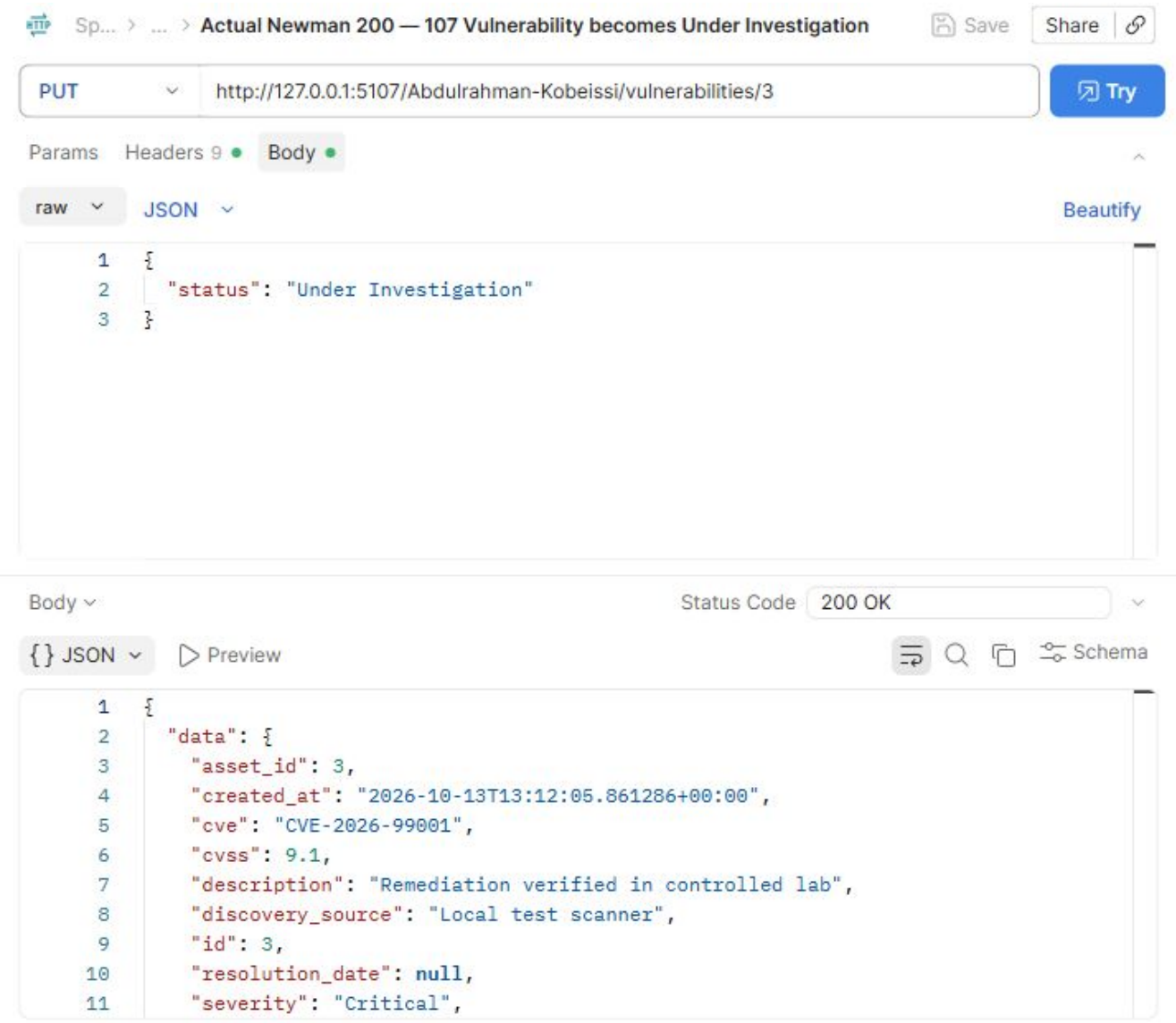


Abdulrahman Kobeissi — 106 Reject skipped remediation lifecycle. The request and saved actual response demonstrate this operation.

Postman Evidence 107

(Abdulrahman Kobeissi)

Vulnerability becomes Under Investigation · PUT · observed HTTP 200

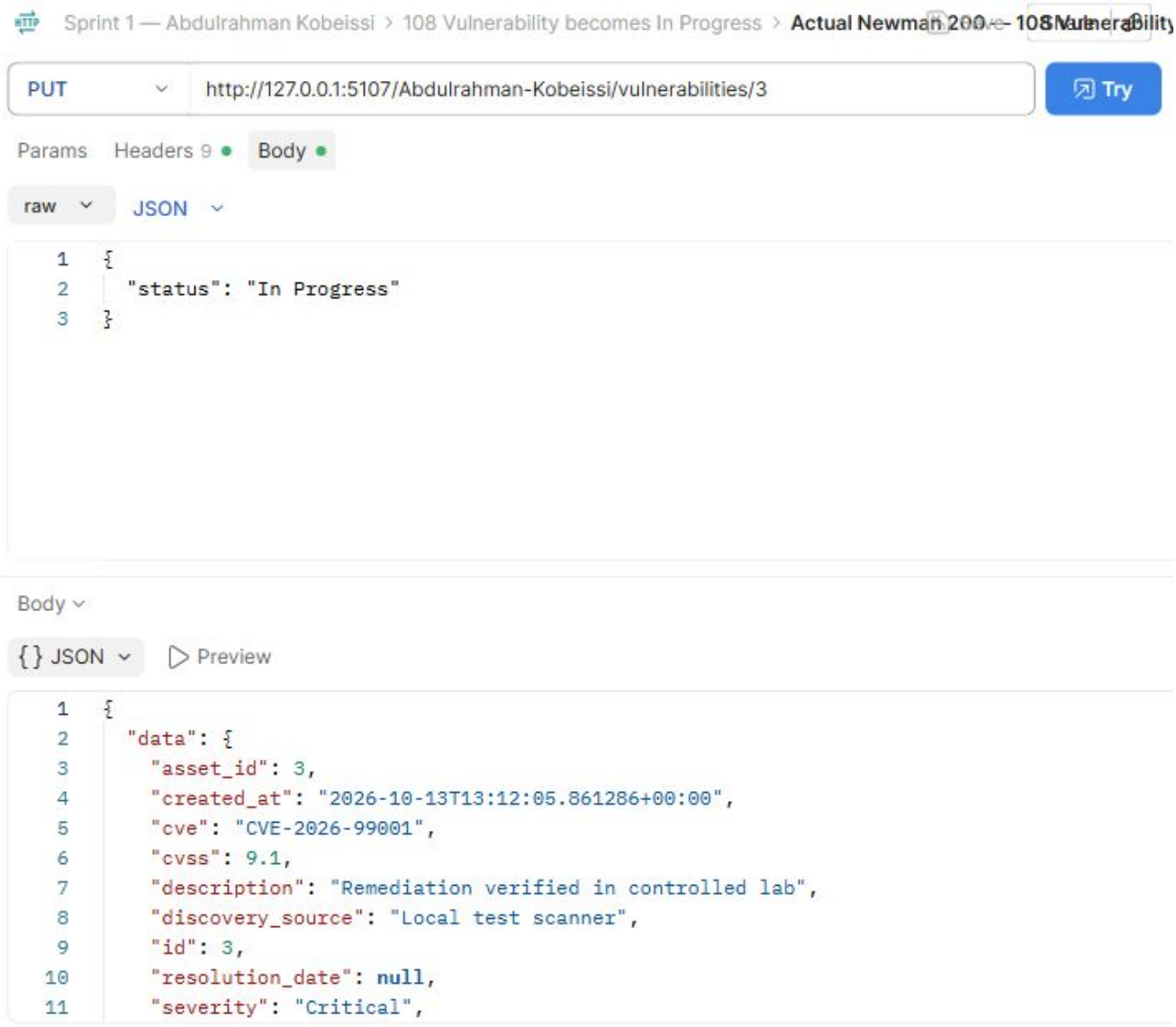


Abdulrahman Kobeissi — 107 Vulnerability becomes Under Investigation. The request and saved actual response demonstrate this operation.

Postman Evidence 108

(Abdulrahman Kobeissi)

Vulnerability becomes In Progress · PUT · observed HTTP 200

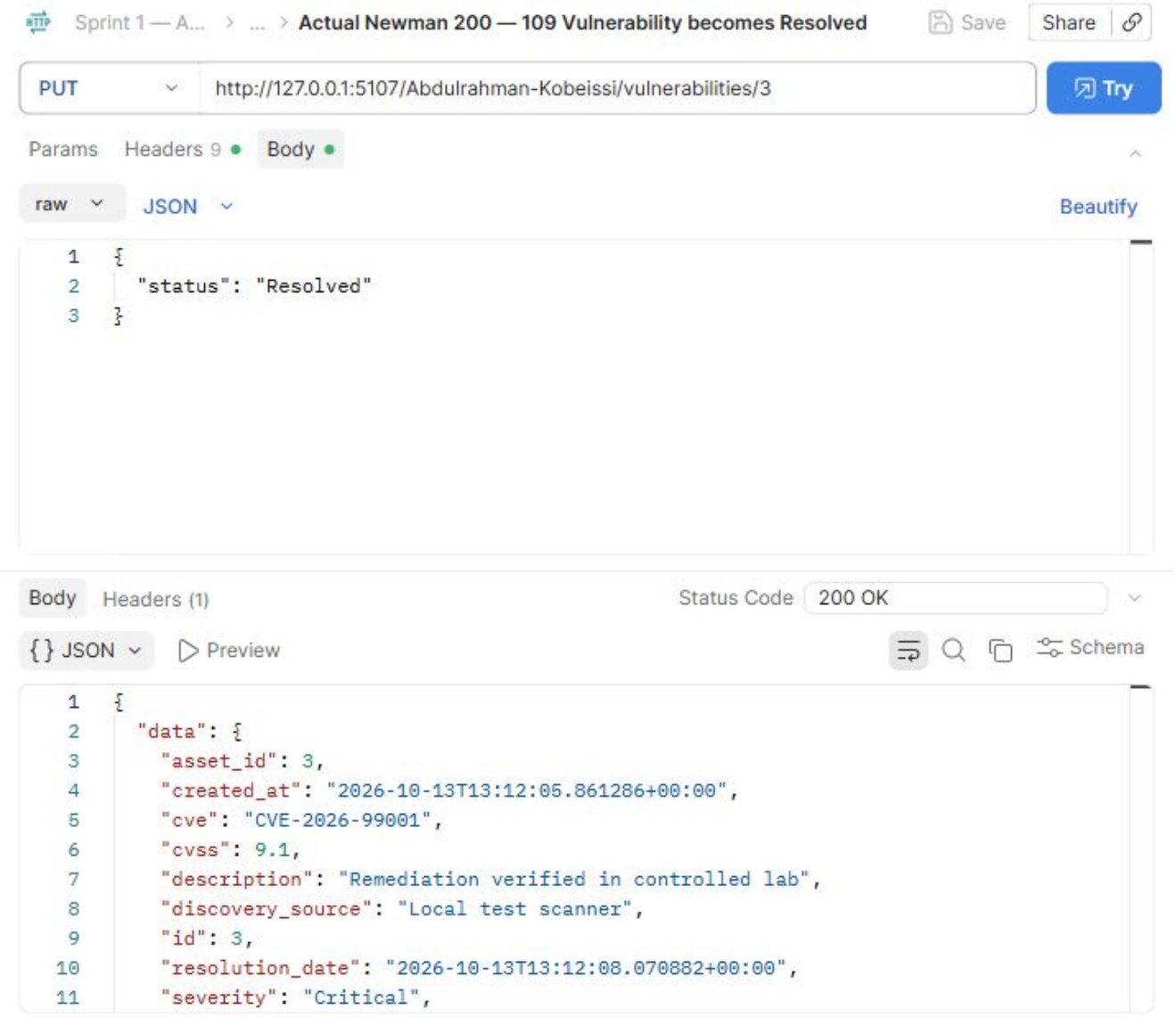


Abdulrahman Kobeissi — 108 Vulnerability becomes In Progress. The request and saved actual response demonstrate this operation.

Postman Evidence 109

(Abdulrahman Kobeissi)

Vulnerability becomes Resolved · PUT · observed HTTP 200

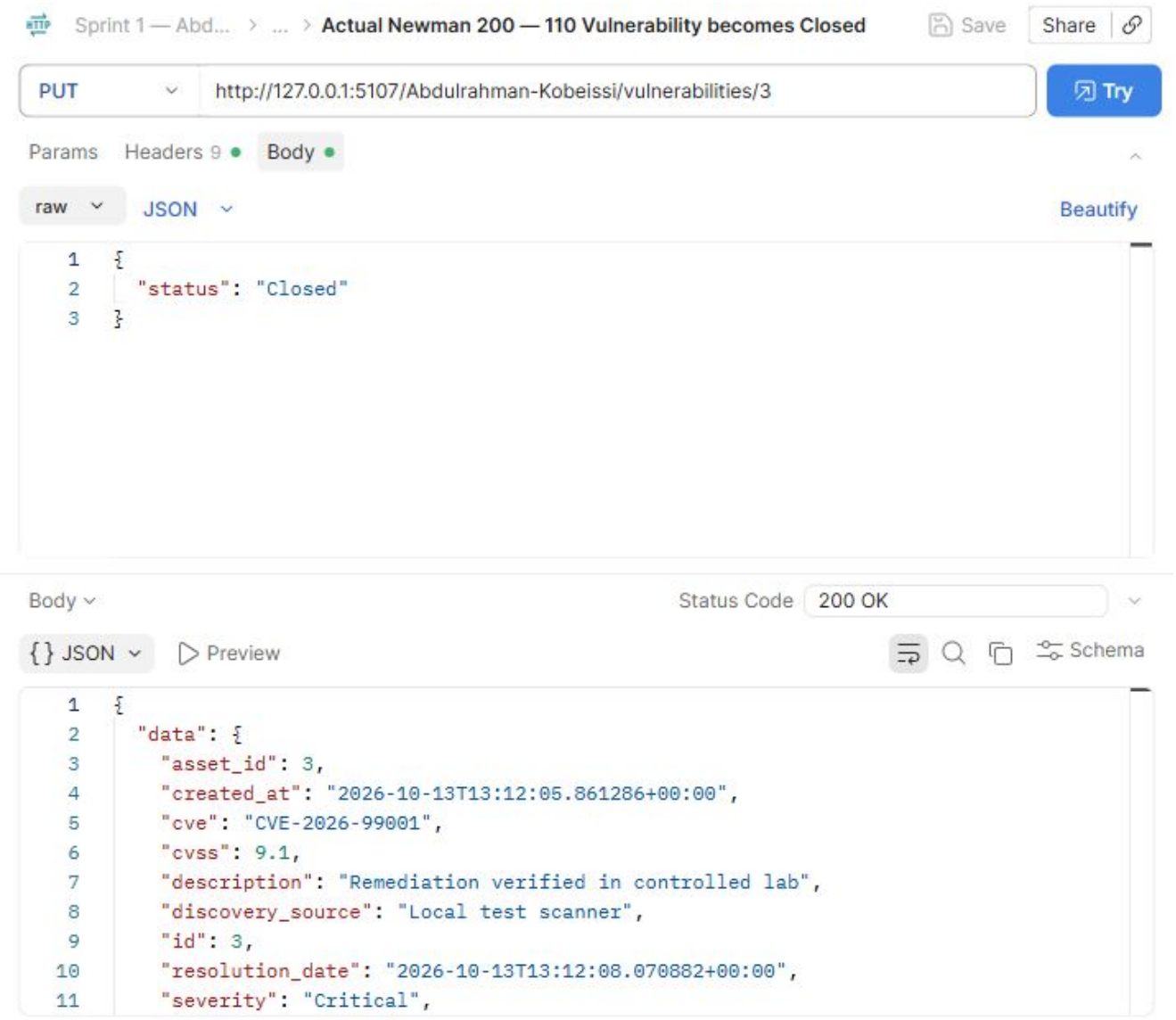


Abdulrahman Kobeissi — 109 Vulnerability becomes Resolved. The request and saved actual response demonstrate this operation.

Postman Evidence 110

(Abdulrahman Kobeissi)

Vulnerability becomes Closed · PUT · observed HTTP 200

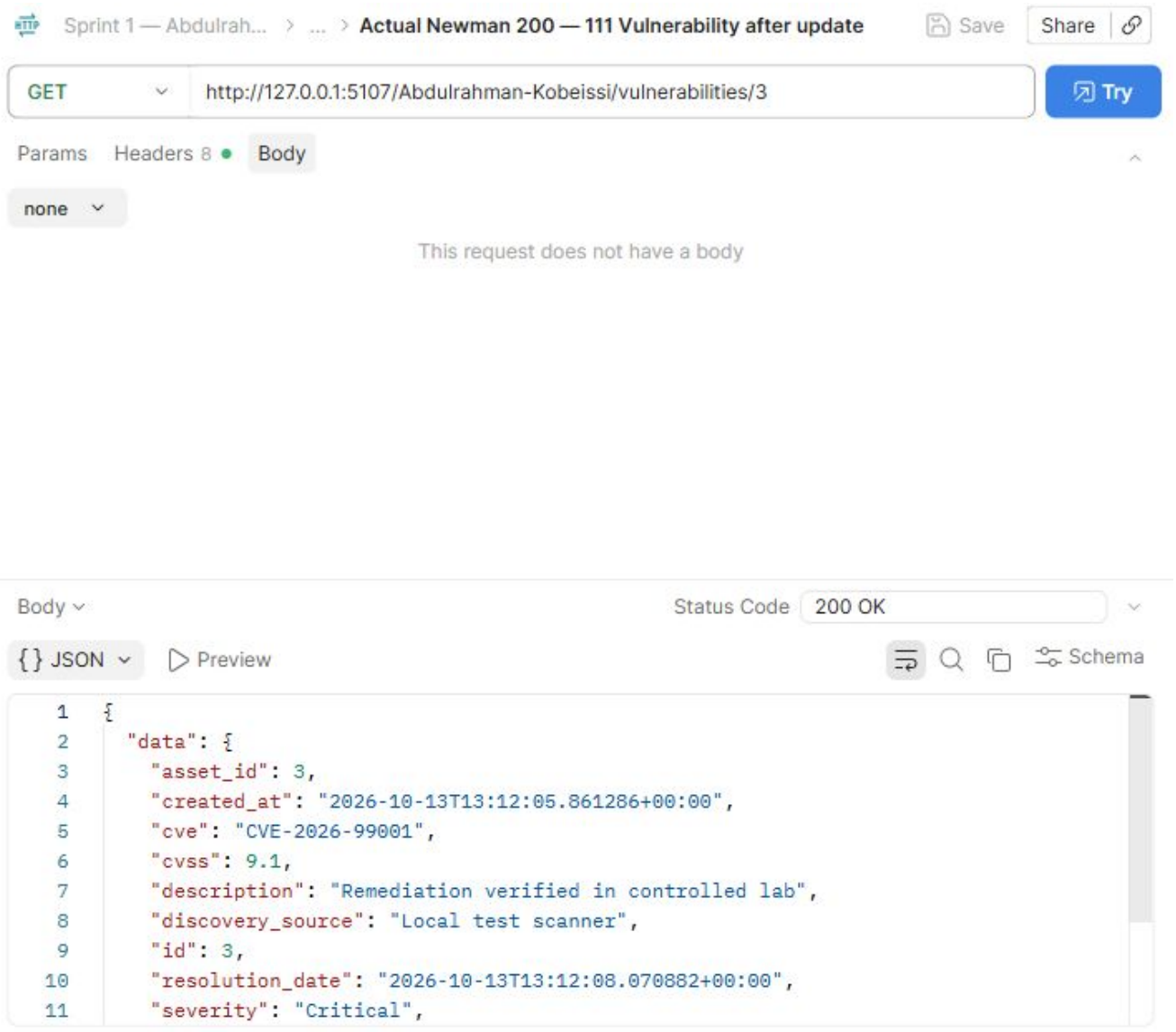


Abdulrahman Kobeissi — 110 Vulnerability becomes Closed. The request and saved actual response demonstrate this operation.

Postman Evidence 111

(Abdulrahman Kobeissi)

Vulnerability after update · GET · observed HTTP 200



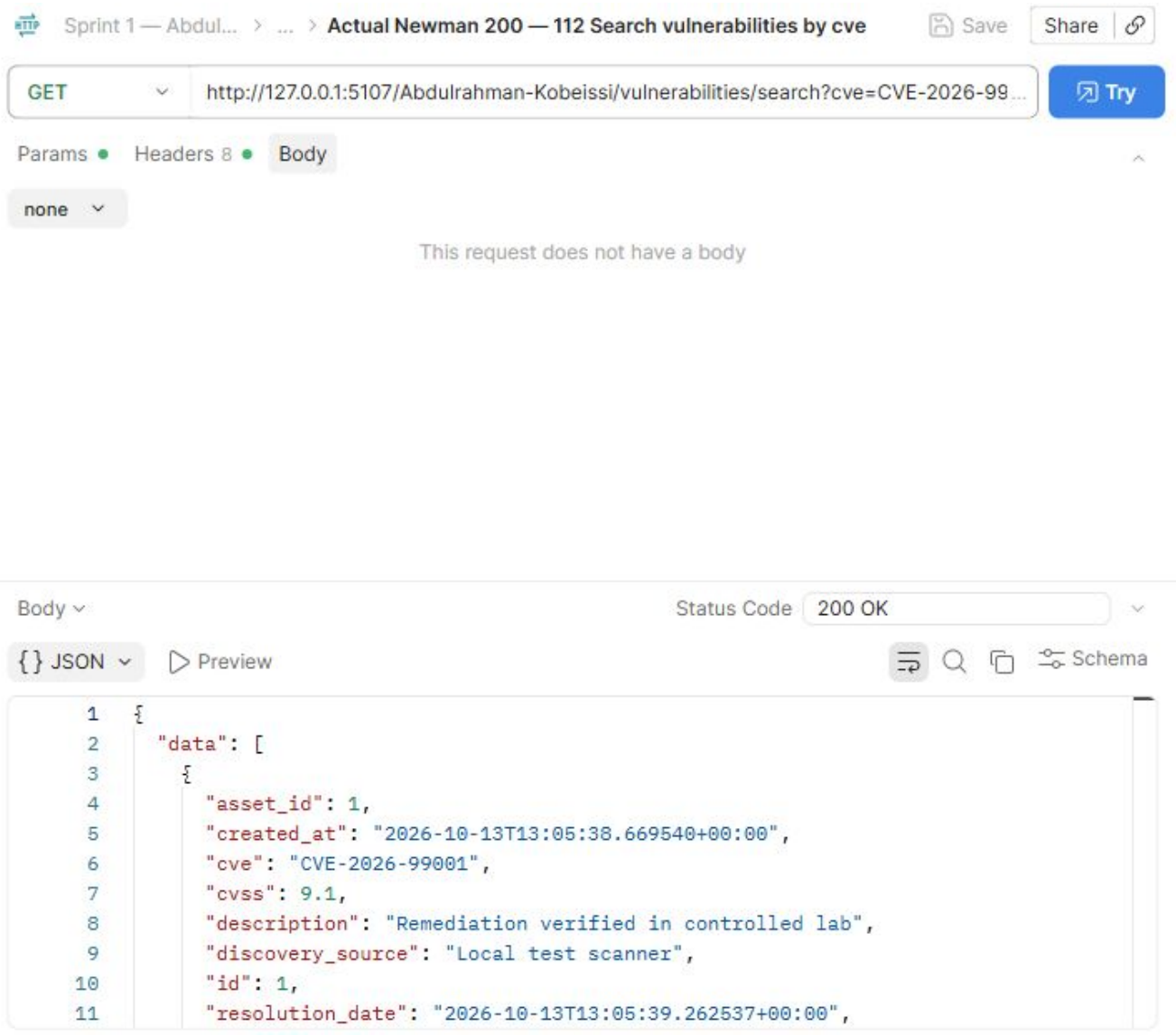
Abdulrahman Kobeissi — 111 Vulnerability after update. The request and saved actual response demonstrate this operation.

Compare this view with the adjacent before/update/after requests. The full response remains available in the collection.

Postman Evidence 112

(Abdulrahman Kobeissi)

Search vulnerabilities by cve · GET · observed HTTP 200

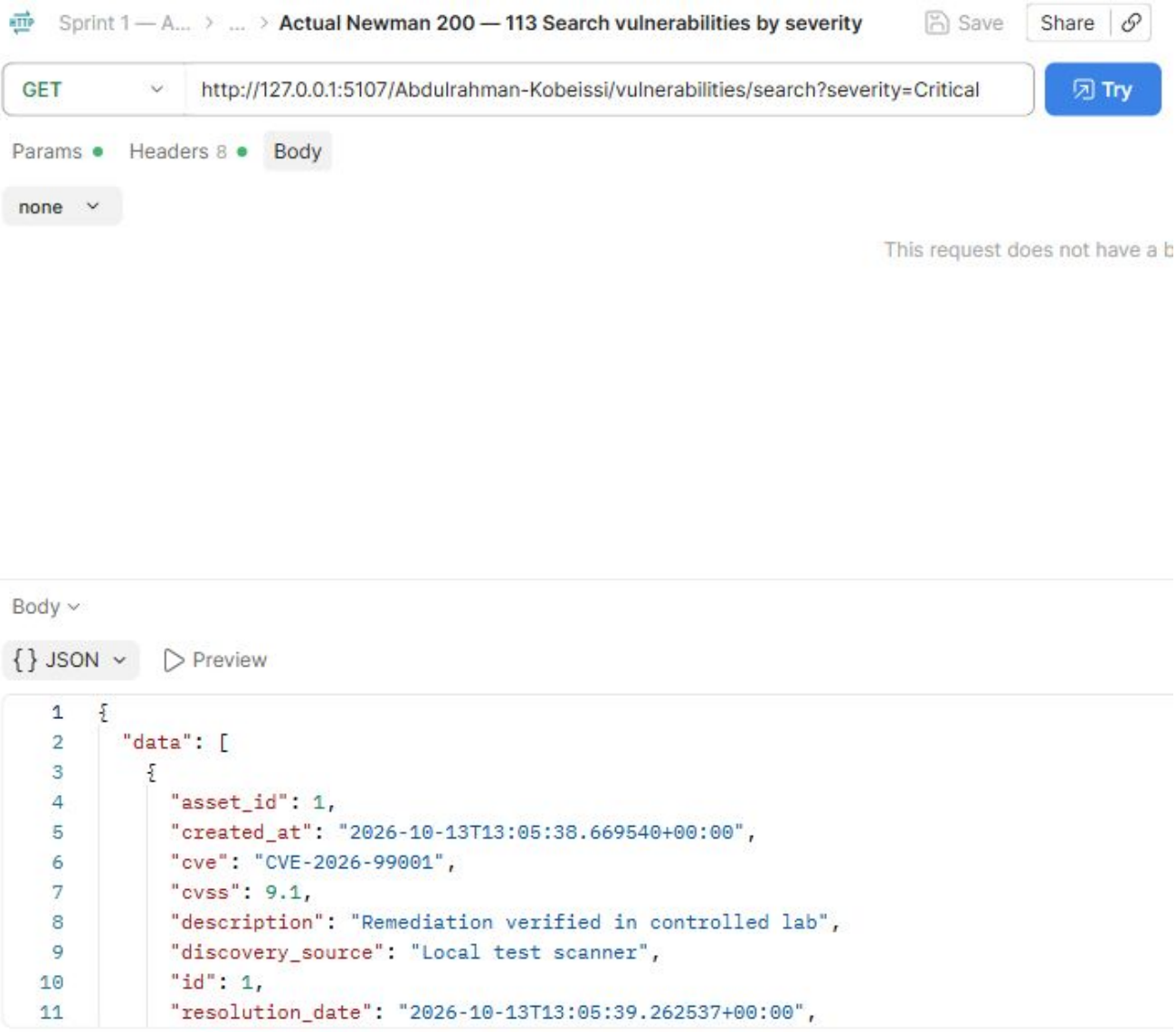


Abdulrahman Kobeissi — 112 Search vulnerabilities by cve. The request and saved actual response demonstrate this operation.

Postman Evidence 113

(Abdulrahman Kobeissi)

Search vulnerabilities by severity · GET · observed HTTP 200

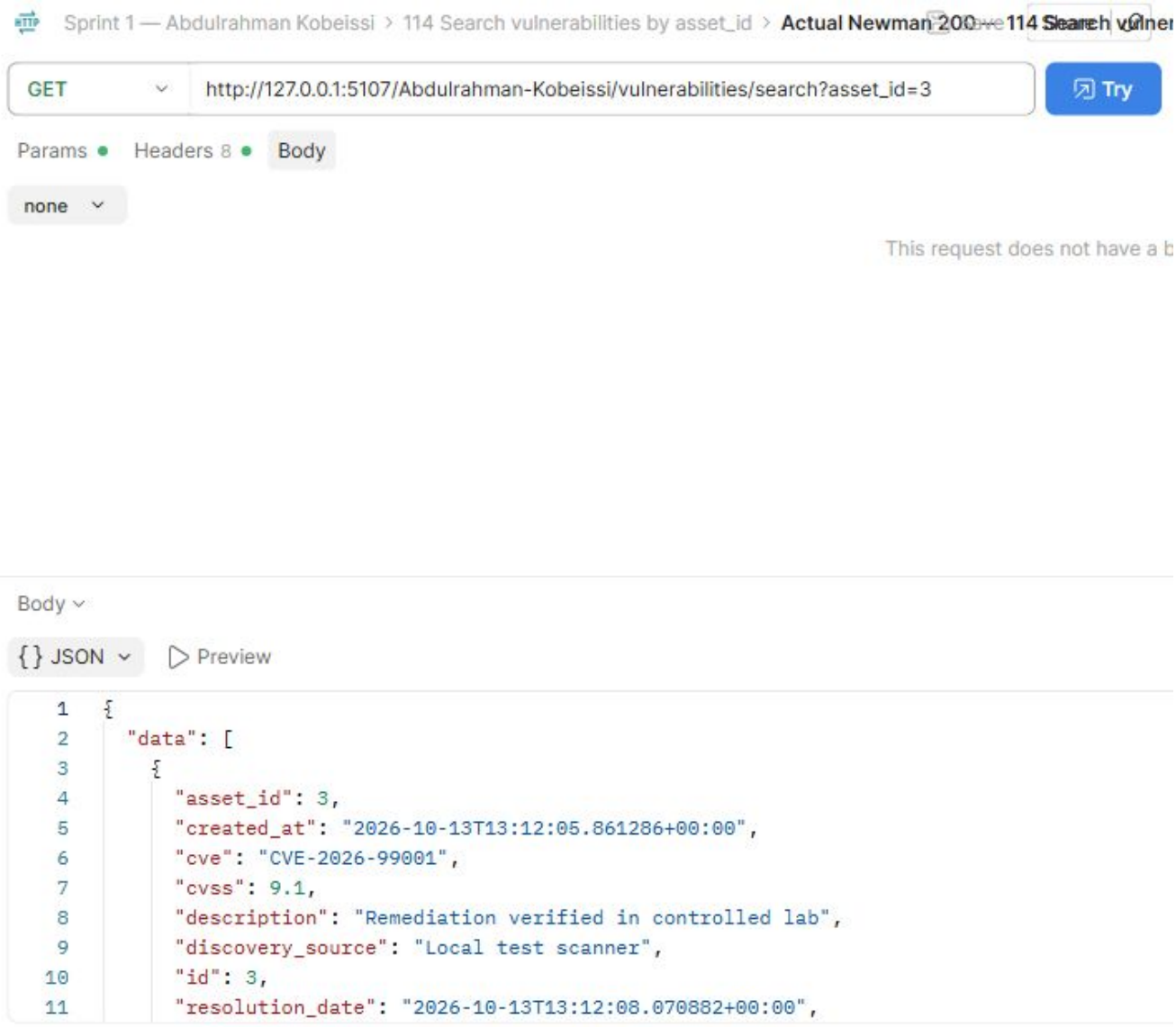


Abdulrahman Kobeissi — 113 Search vulnerabilities by severity. The request and saved actual response demonstrate this operation.

Postman Evidence 114

(Abdulrahman Kobeissi)

Search vulnerabilities by asset_id · GET · observed HTTP 200

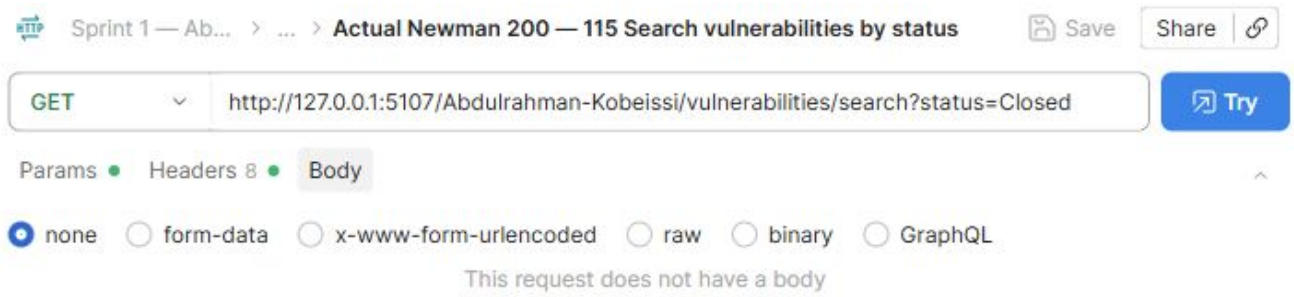


Abdulrahman Kobeissi — 114 Search vulnerabilities by asset_id. The request and saved actual response demonstrate this operation.

Postman Evidence 115

(Abdulrahman Kobeissi)

Search vulnerabilities by status · GET · observed HTTP 200



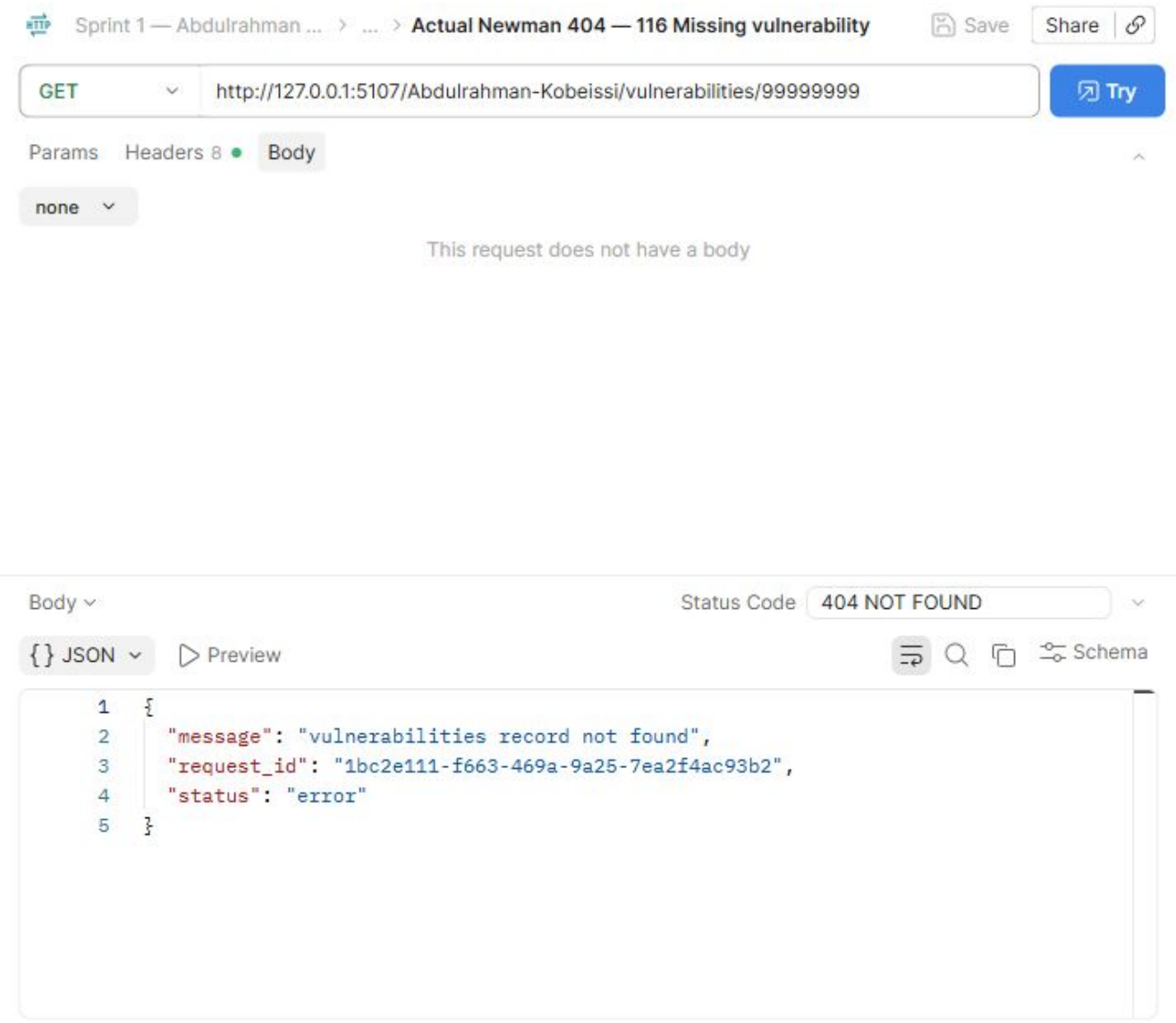


Abdulrahman Kobeissi — 115 Search vulnerabilities by status. The request and saved actual response demonstrate this operation.

Postman Evidence 116

(Abdulrahman Kobeissi)

Missing vulnerability · GET · observed HTTP 404

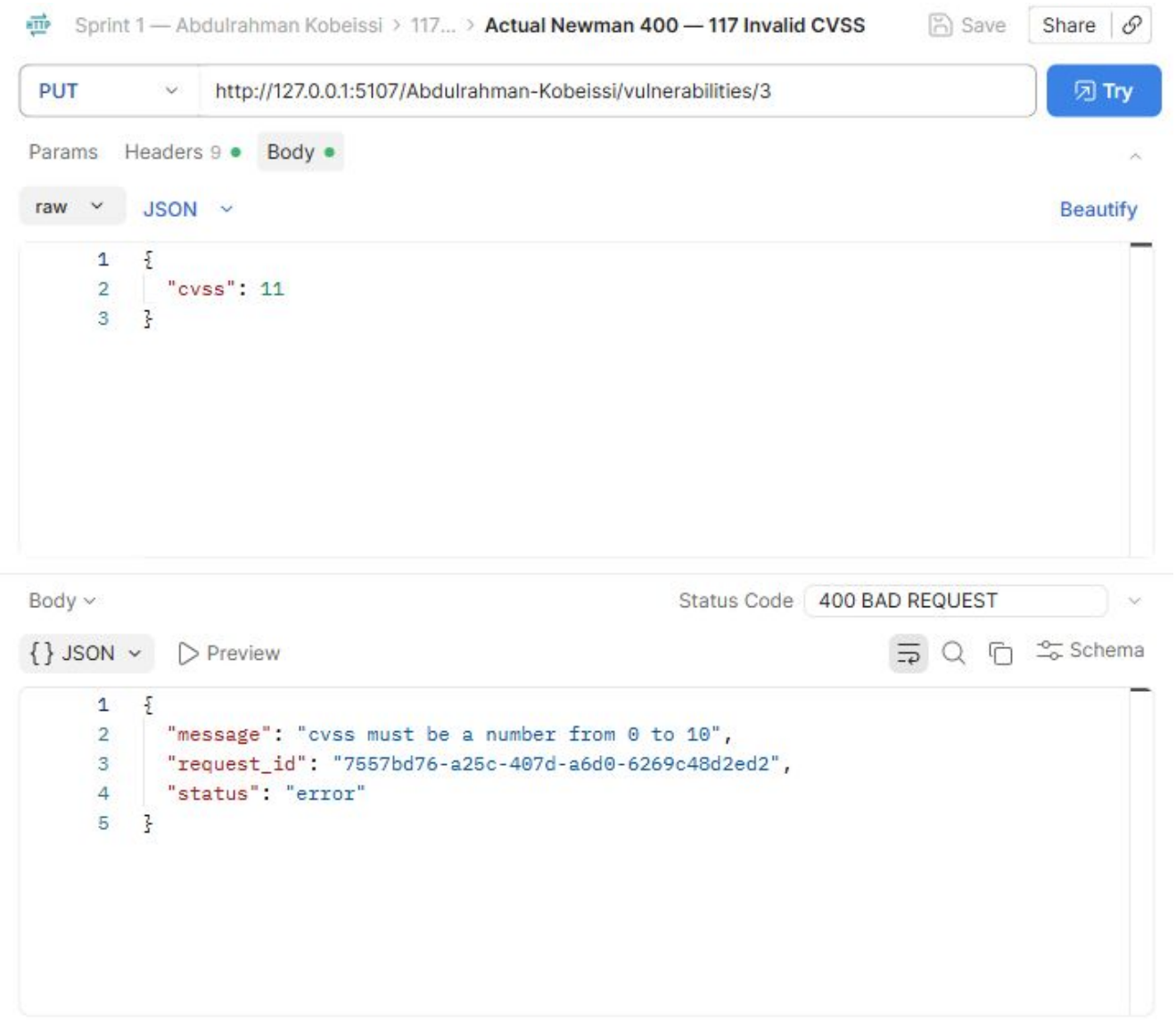


Abdulrahman Kobeissi — 116 Missing vulnerability. The request and saved actual response demonstrate this operation.

Postman Evidence 117

(Abdulrahman Kobeissi)

Invalid CVSS · PUT · observed HTTP 400

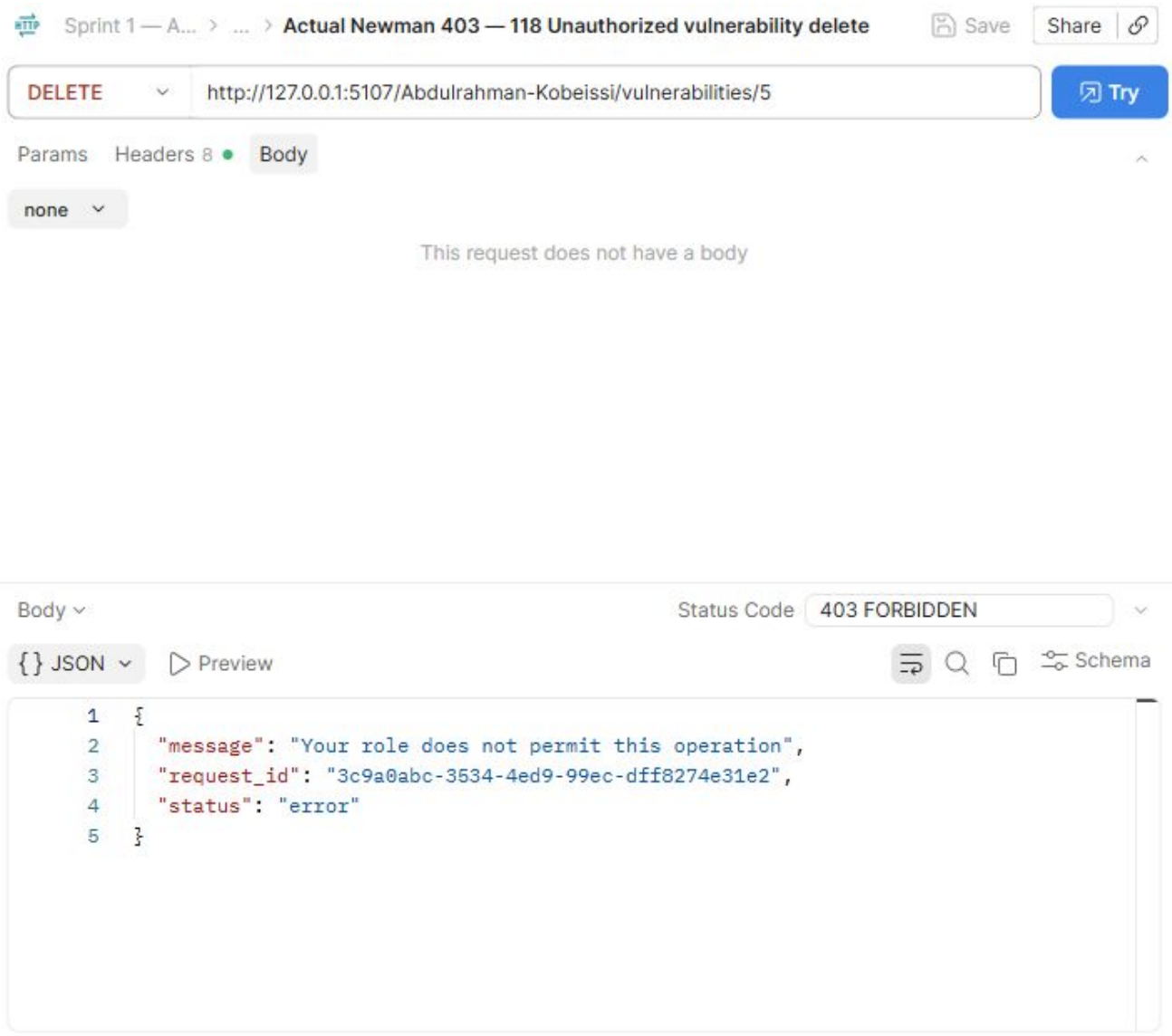


Abdulrahman Kobeissi — 117 Invalid CVSS. The request and saved actual response demonstrate this operation.

Postman Evidence 118

(Abdulrahman Kobeissi)

Unauthorized vulnerability delete · DELETE · observed HTTP 403

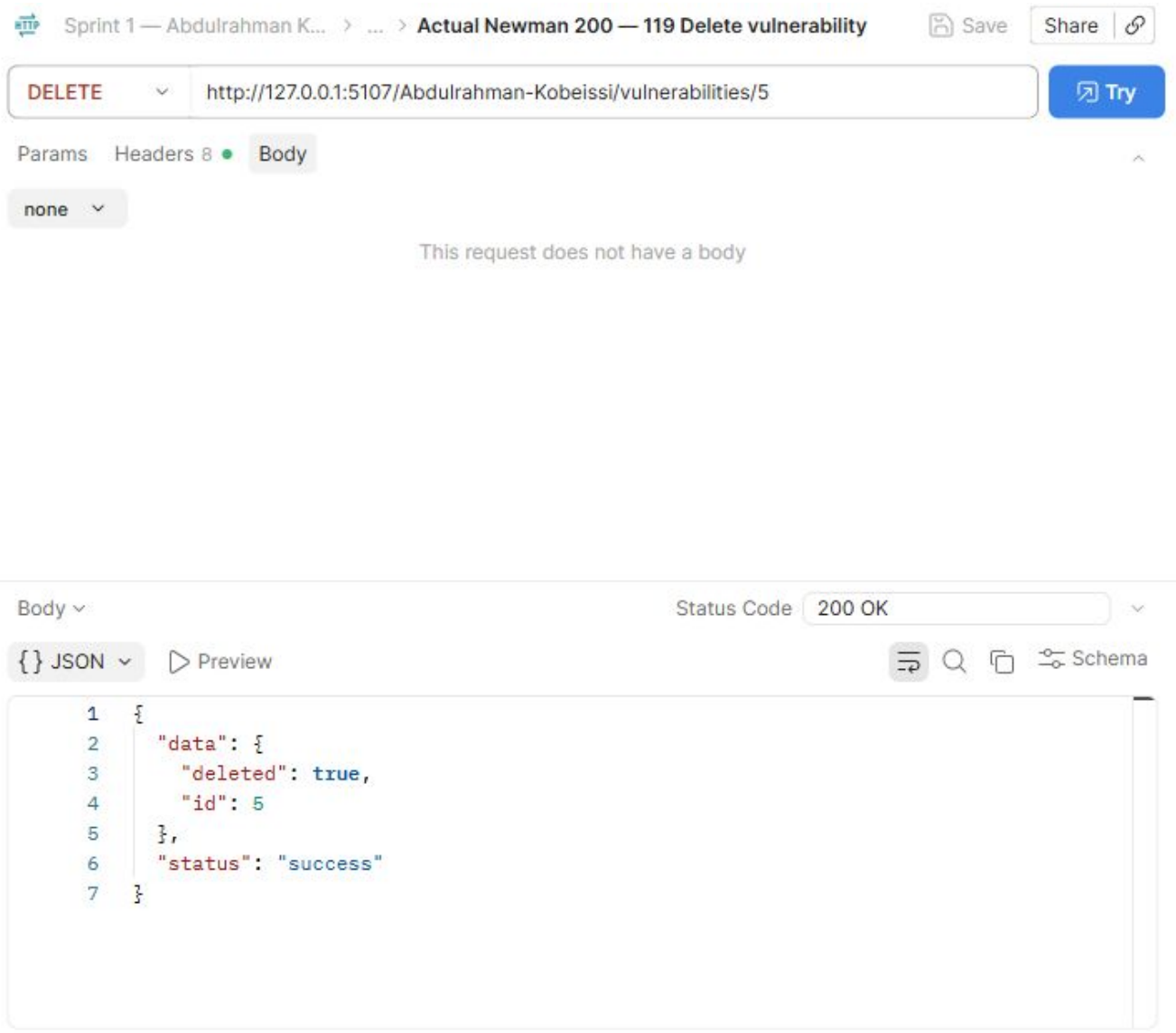


Abdulrahman Kobeissi — 118 Unauthorized vulnerability delete. The request and saved actual response demonstrate this operation.

Postman Evidence 119

(Abdulrahman Kobeissi)

Delete vulnerability · DELETE · observed HTTP 200

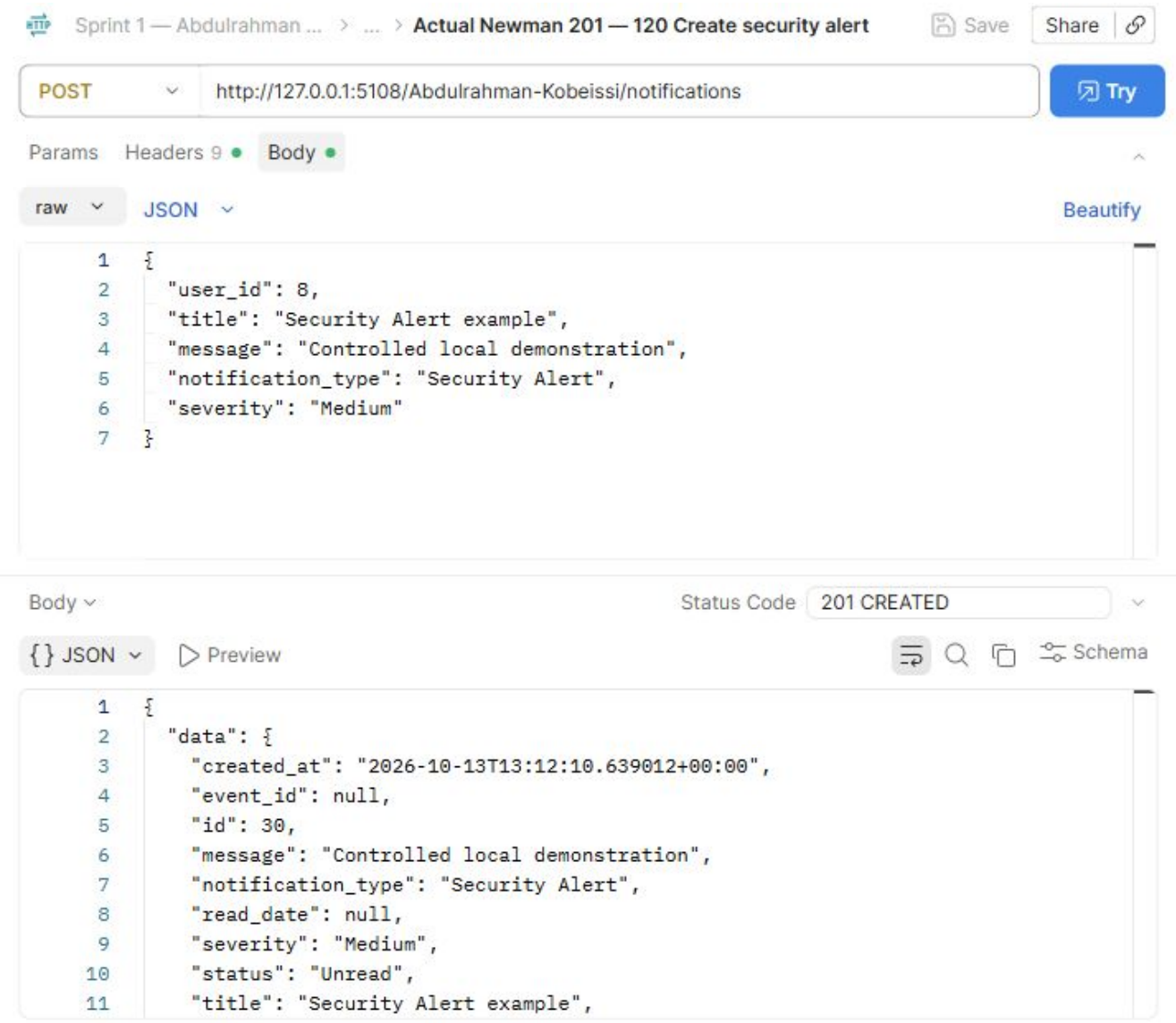


Abdulrahman Kobeissi — 119 Delete vulnerability. The request and saved actual response demonstrate this operation.

Postman Evidence 120

(Abdulrahman Kobeissi)

Create security alert · POST · observed HTTP 201

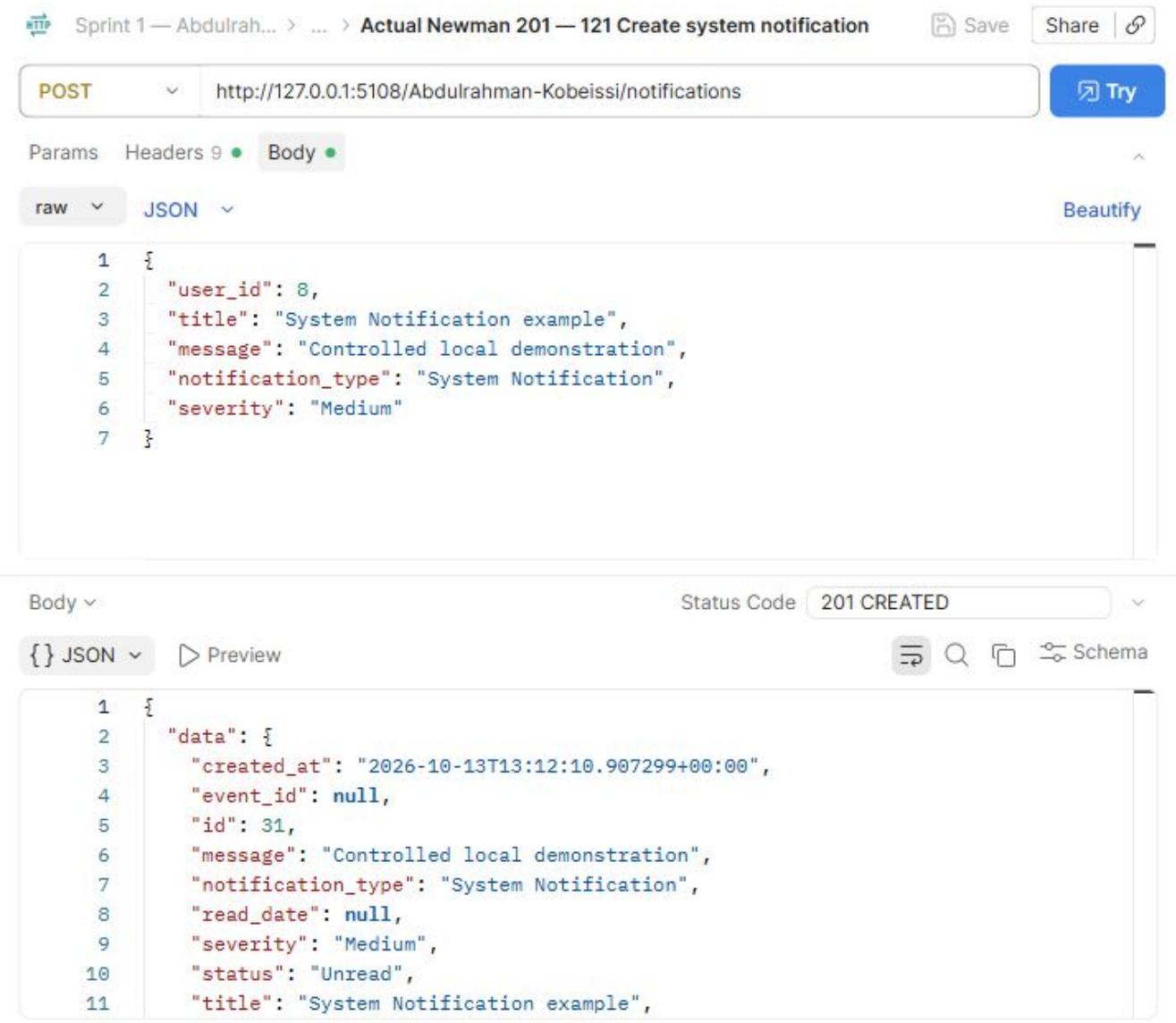


Abdulrahman Kobeissi — 120 Create security alert. The request and saved actual response demonstrate this operation.

Postman Evidence 121

(Abdulrahman Kobeissi)

Create system notification · POST · observed HTTP 201

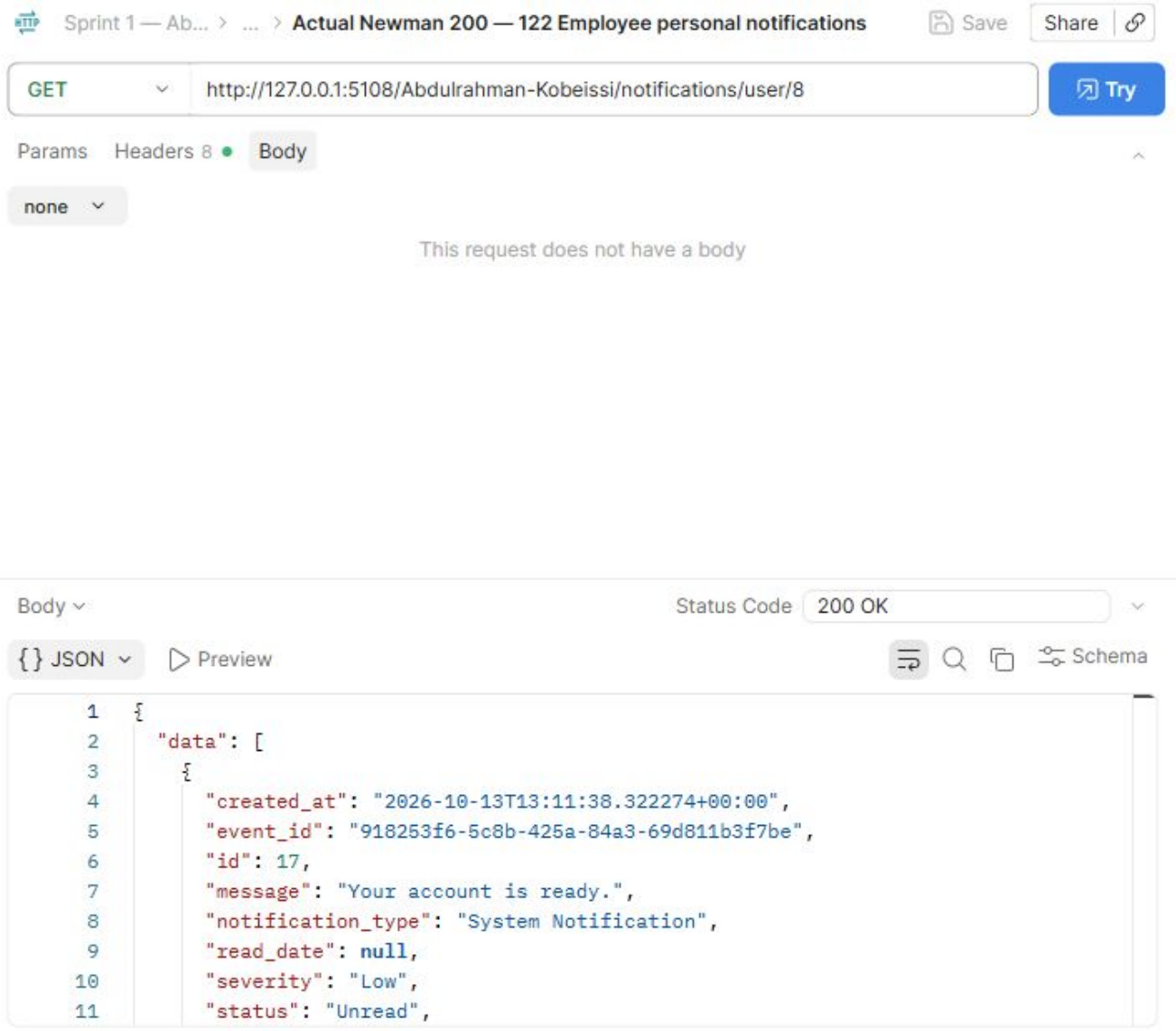


Abdulrahman Kobeissi — 121 Create system notification. The request and saved actual response demonstrate this operation.

Postman Evidence 122

(Abdulrahman Kobeissi)

Employee personal notifications · GET · observed HTTP 200

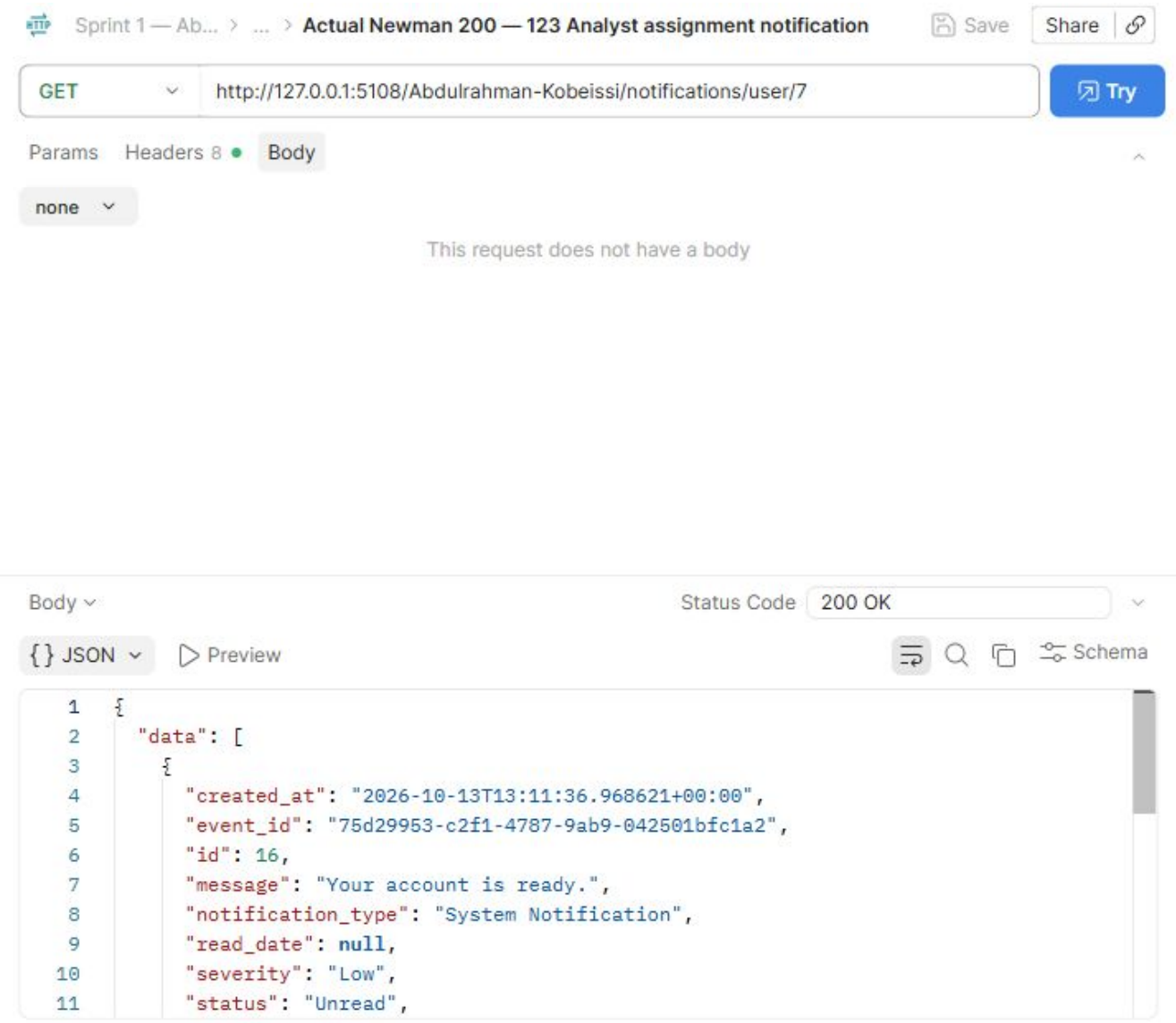


Abdulrahman Kobeissi — 122 Employee personal notifications. The request and saved actual response demonstrate this operation.

Postman Evidence 123

(Abdulrahman Kobeissi)

Analyst assignment notification · GET · observed HTTP 200

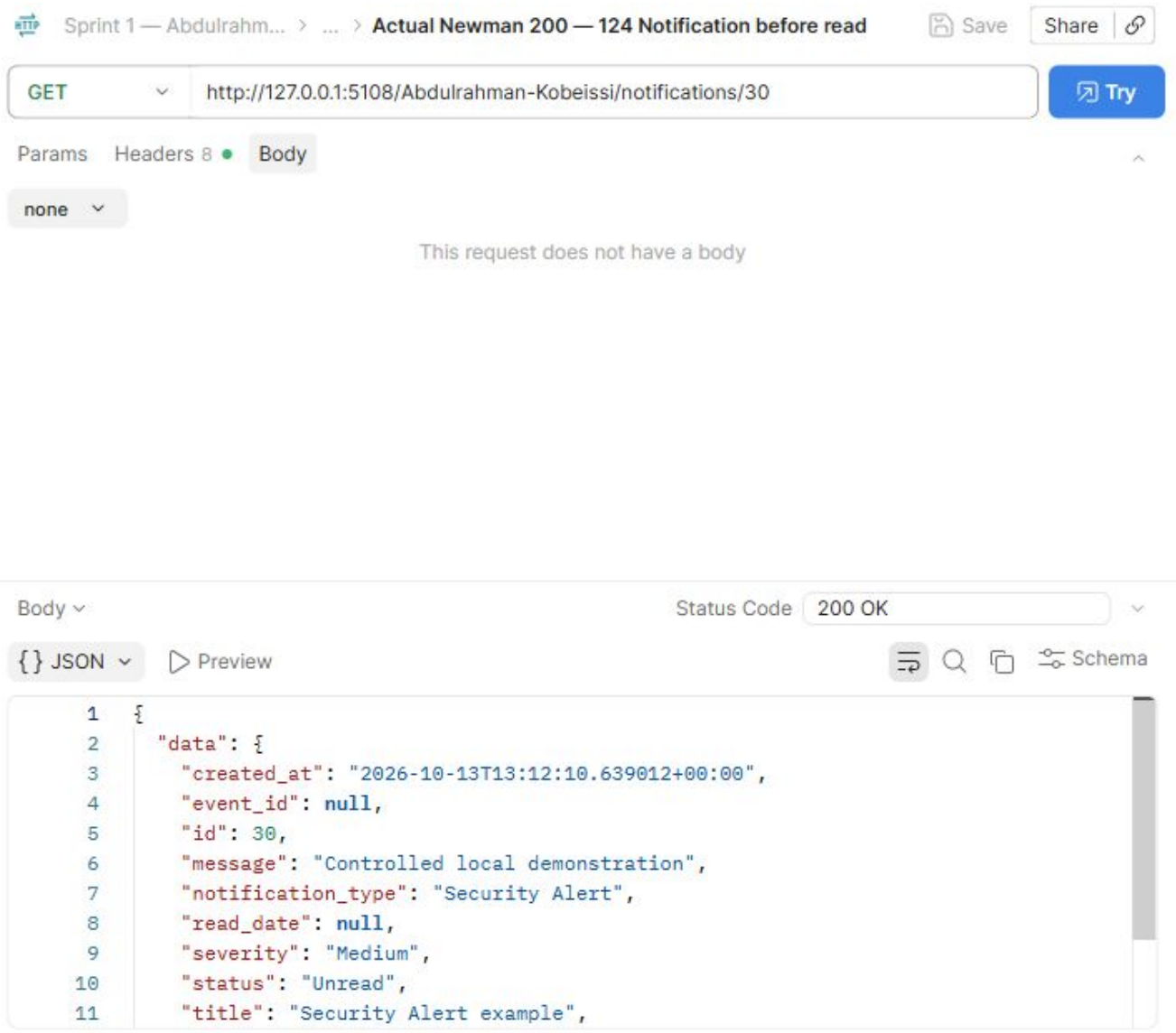


Abdulrahman Kobeissi — 123 Analyst assignment notification. The request and saved actual response demonstrate this operation.

Postman Evidence 124

(Abdulrahman Kobeissi)

Notification before read · GET · observed HTTP 200



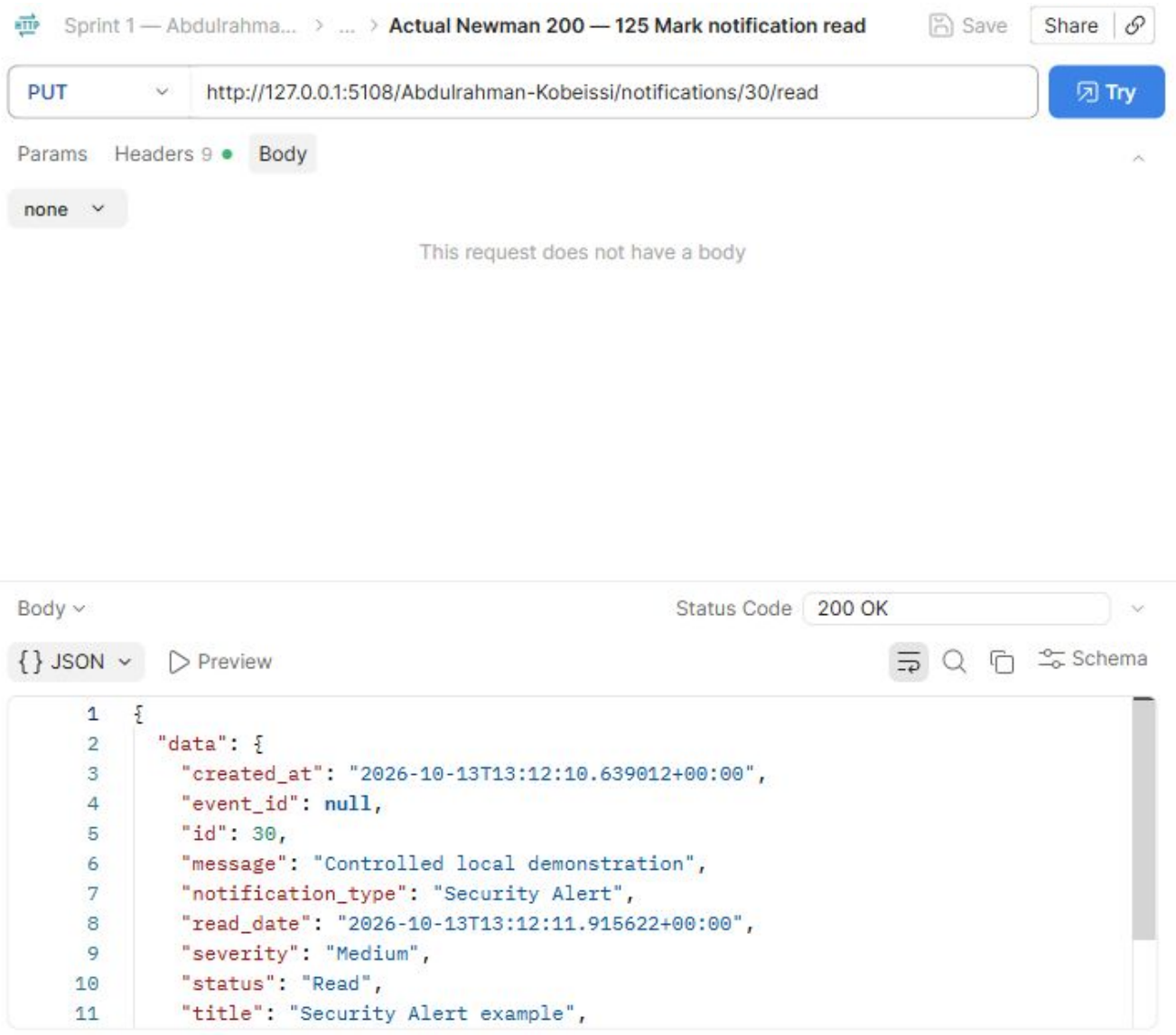
Abdulrahman Kobeissi — 124 Notification before read. The request and saved actual response demonstrate this operation.

Compare this view with the adjacent before/update/after requests. The full response remains available in the collection.

Postman Evidence 125

(Abdulrahman Kobeissi)

Mark notification read · PUT · observed HTTP 200

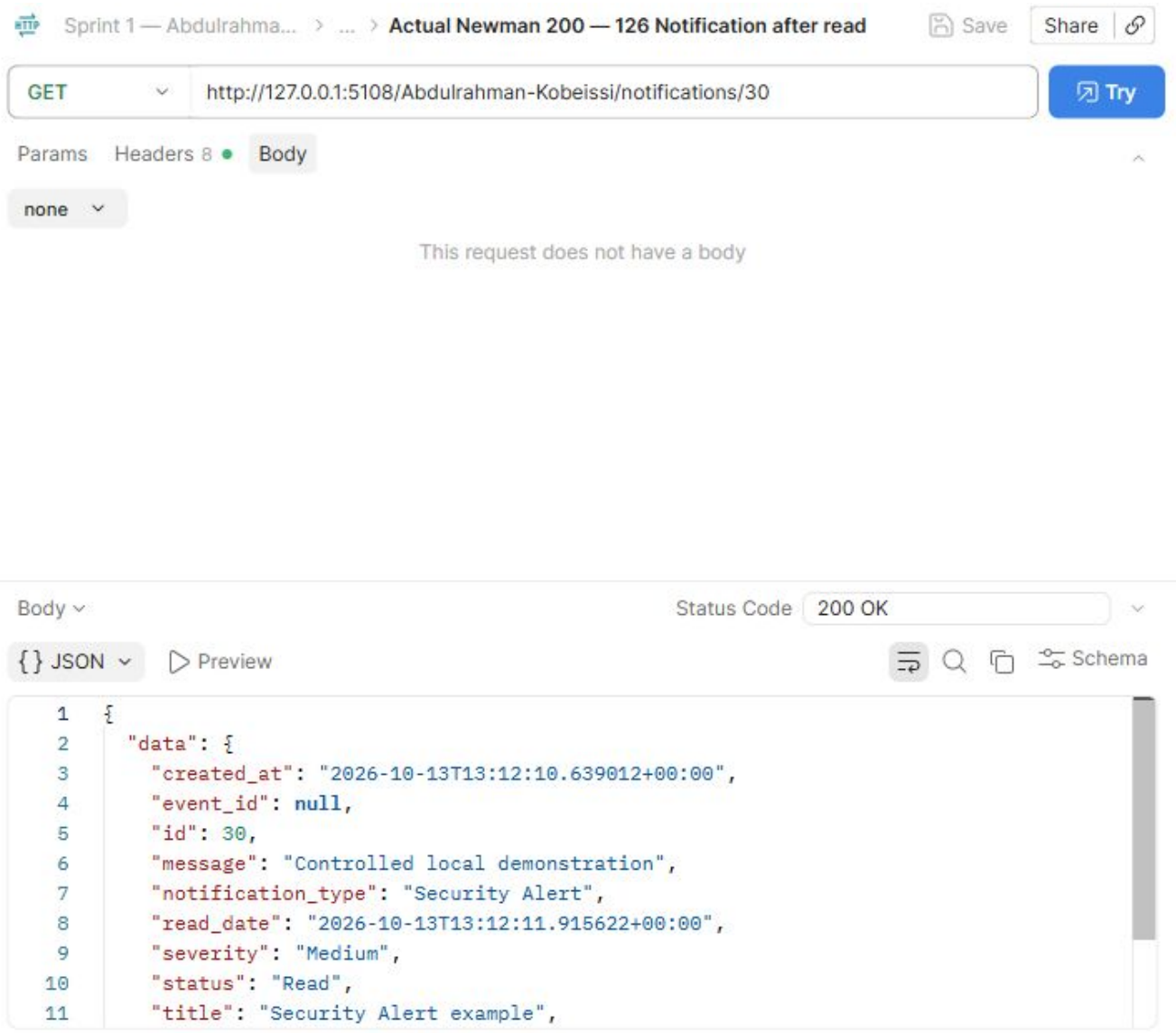


Abdulrahman Kobeissi — 125 Mark notification read. The request and saved actual response demonstrate this operation.

Postman Evidence 126

(Abdulrahman Kobeissi)

Notification after read · GET · observed HTTP 200



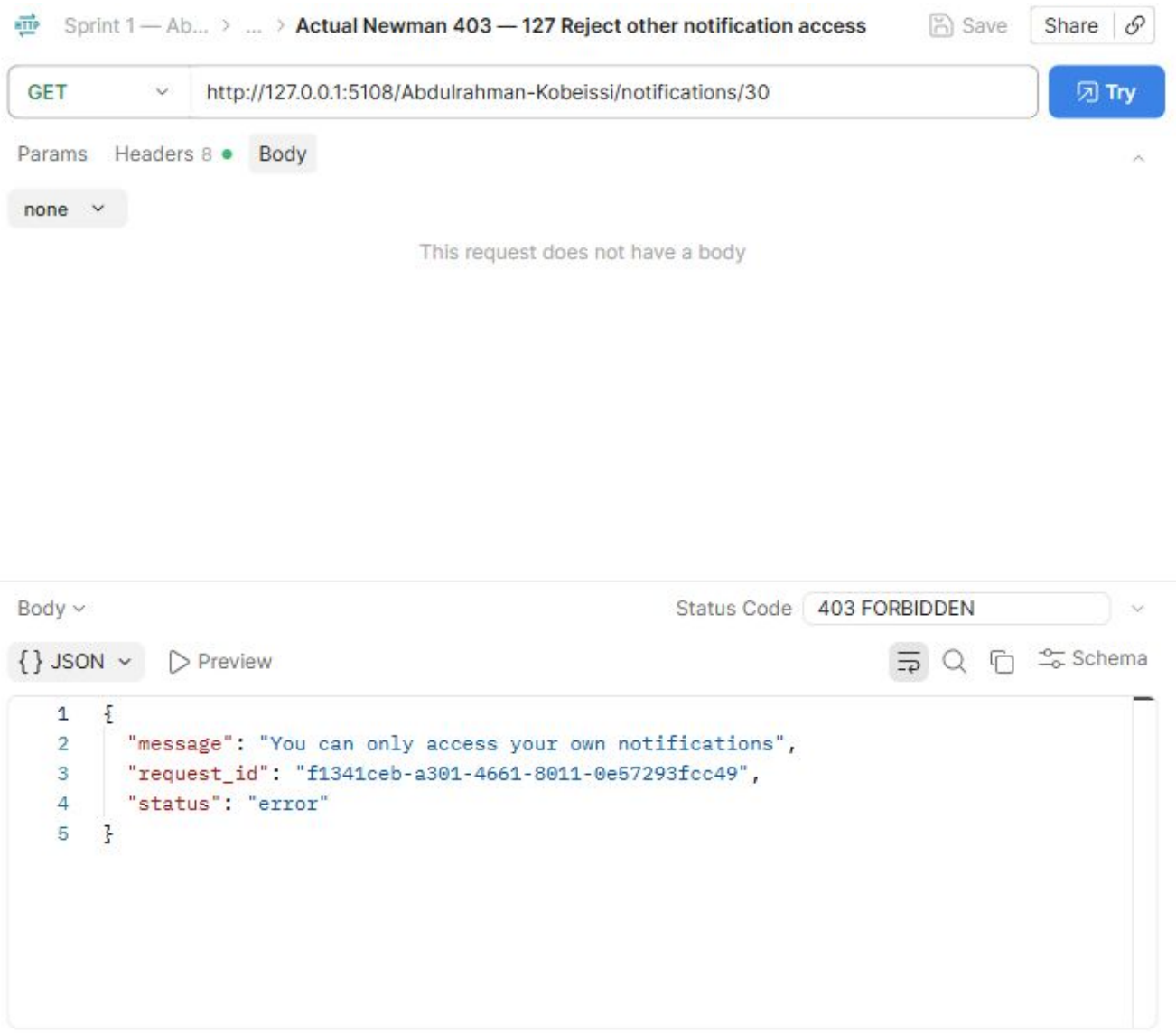
Abdulrahman Kobeissi — 126 Notification after read. The request and saved actual response demonstrate this operation.

Compare this view with the adjacent before/update/after requests. The full response remains available in the collection.

Postman Evidence 127

(Abdulrahman Kobeissi)

Reject other notification access · GET · observed HTTP 403

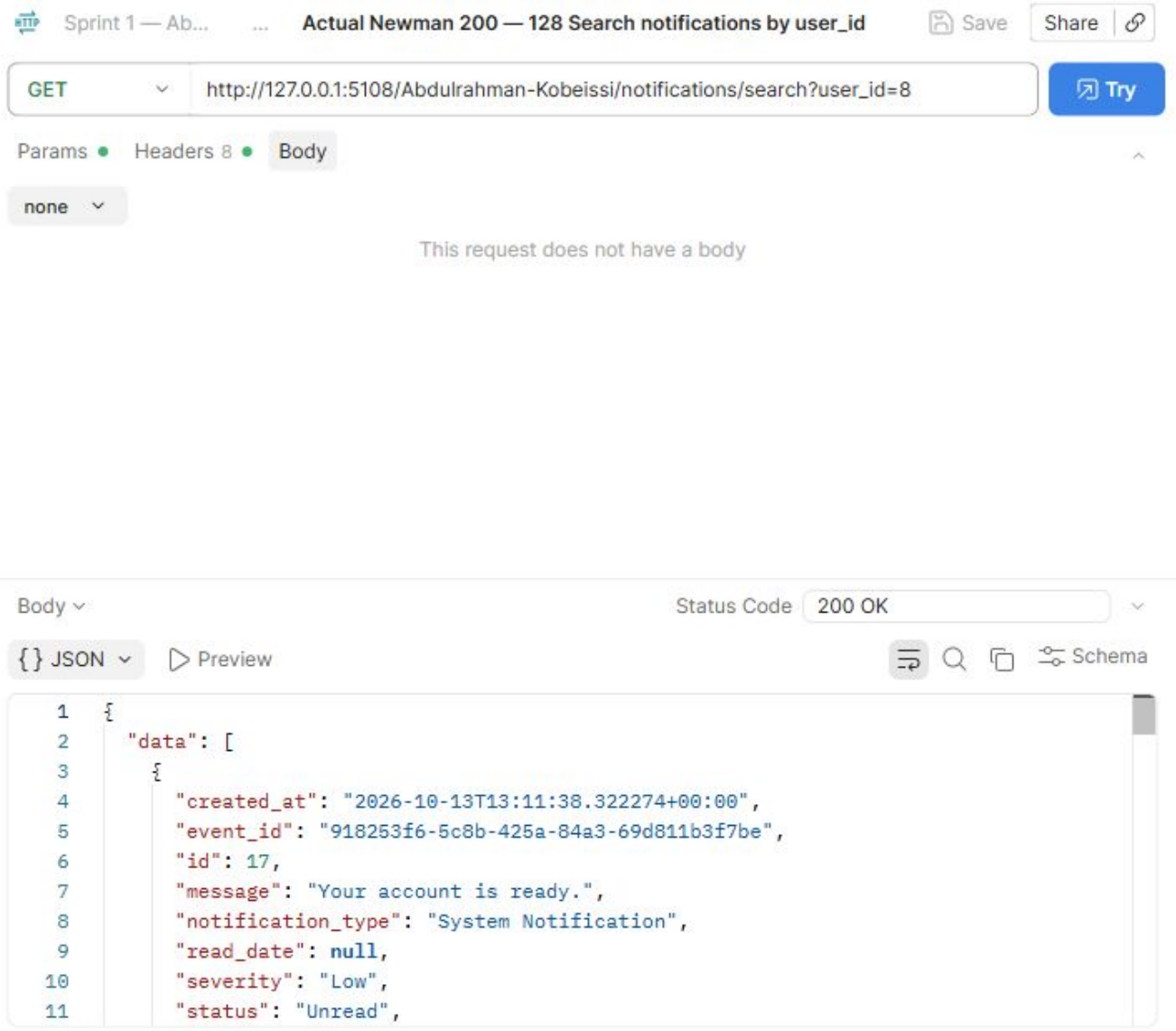


Abdulrahman Kobeissi — 127 Reject other notification access. The request and saved actual response demonstrate this operation.

Postman Evidence 128

(Abdulrahman Kobeissi)

Search notifications by user_id · GET · observed HTTP 200

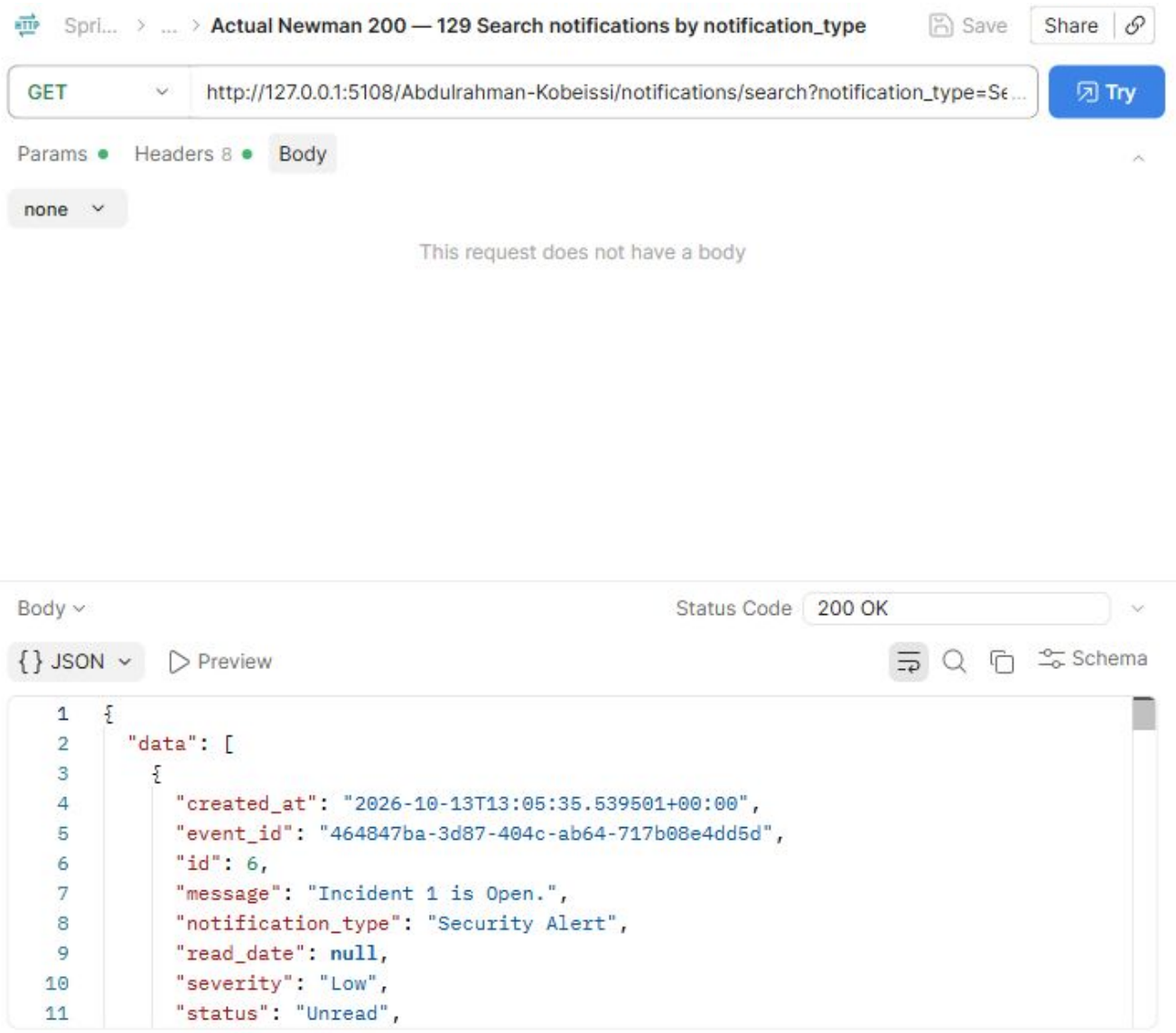


Abdulrahman Kobeissi — 128 Search notifications by user_id. The request and saved actual response demonstrate this operation.

Postman Evidence 129

(Abdulrahman Kobeissi)

Search notifications by notification_type · GET · observed HTTP 200

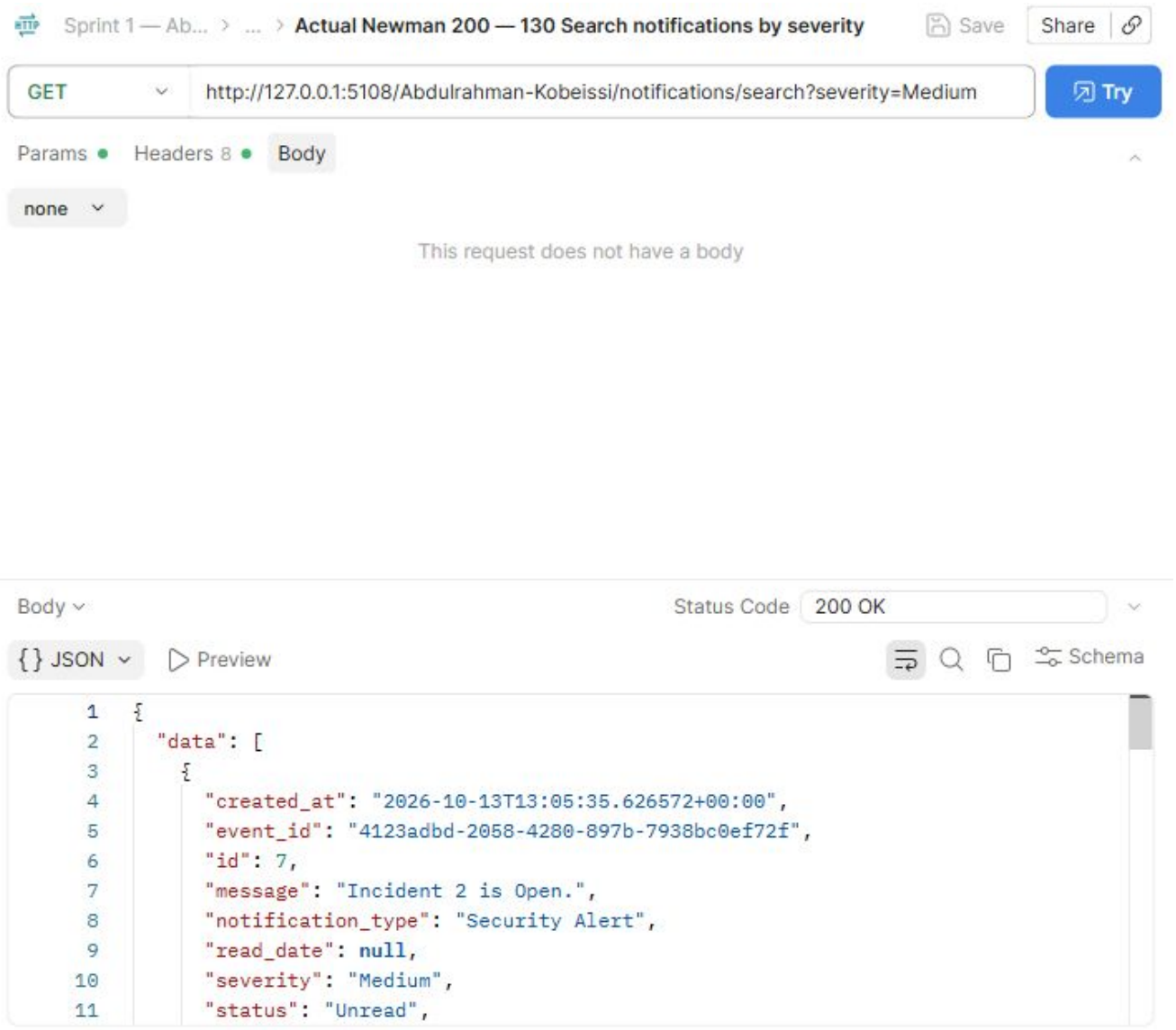


Abdulrahman Kobeissi — 129 Search notifications by notification_type. The request and saved actual response demonstrate this operation.

Postman Evidence 130

(Abdulrahman Kobeissi)

Search notifications by severity · GET · observed HTTP 200

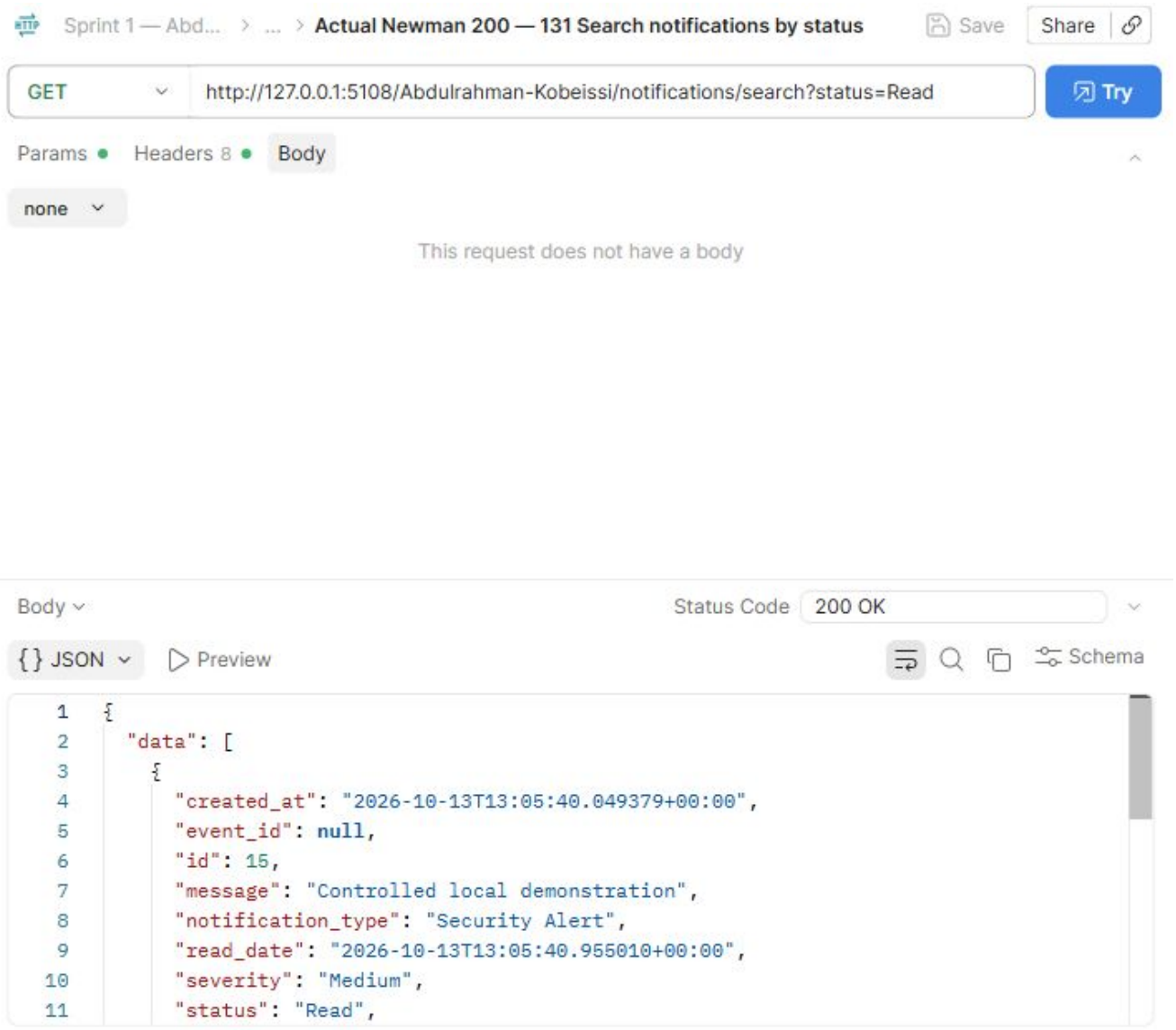


Abdulrahman Kobeissi — 130 Search notifications by severity. The request and saved actual response demonstrate this operation.

Postman Evidence 131

(Abdulrahman Kobeissi)

Search notifications by status · GET · observed HTTP 200

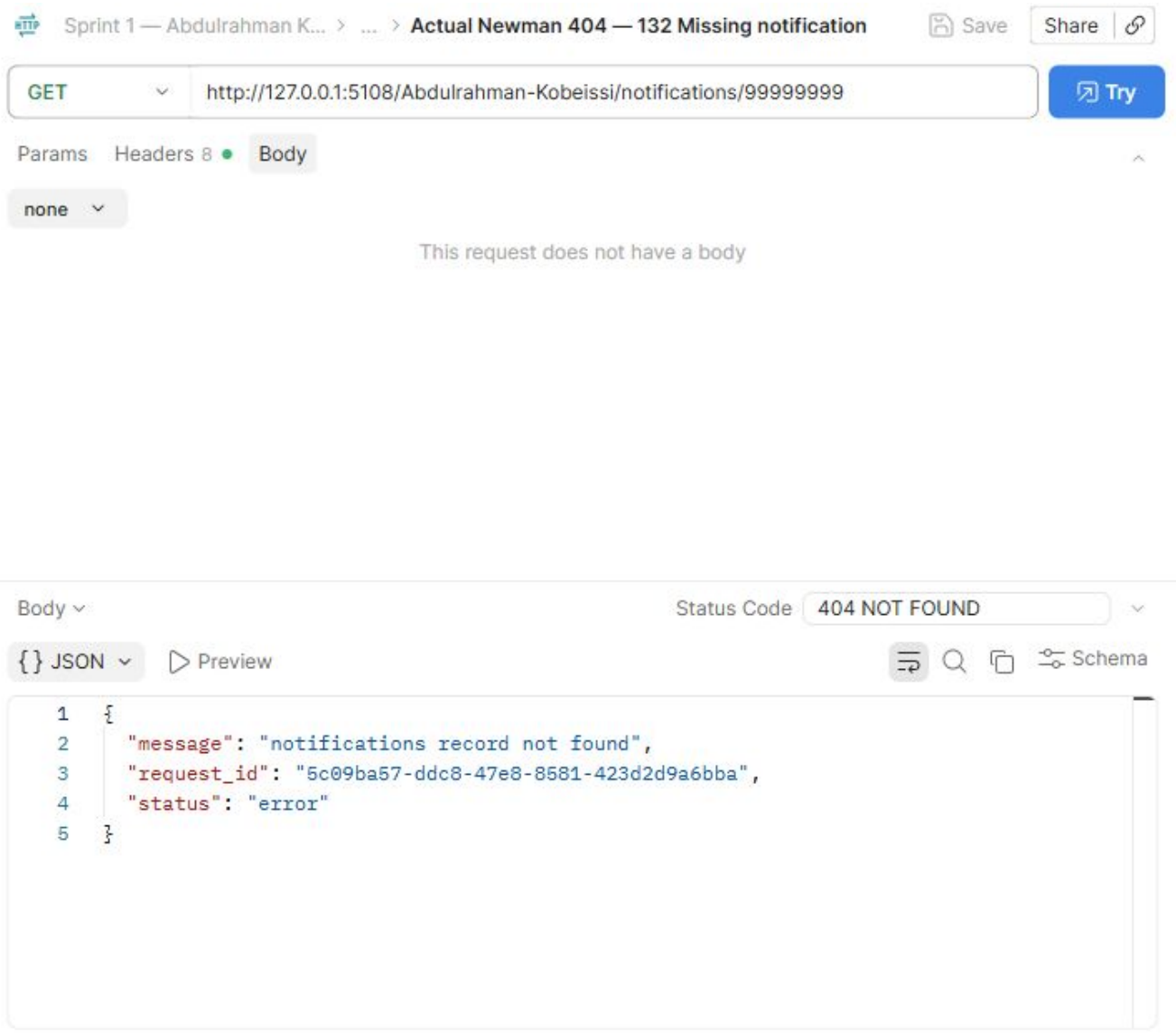


Abdulrahman Kobeissi — 131 Search notifications by status. The request and saved actual response demonstrate this operation.

Postman Evidence 132

(Abdulrahman Kobeissi)

Missing notification · GET · observed HTTP 404

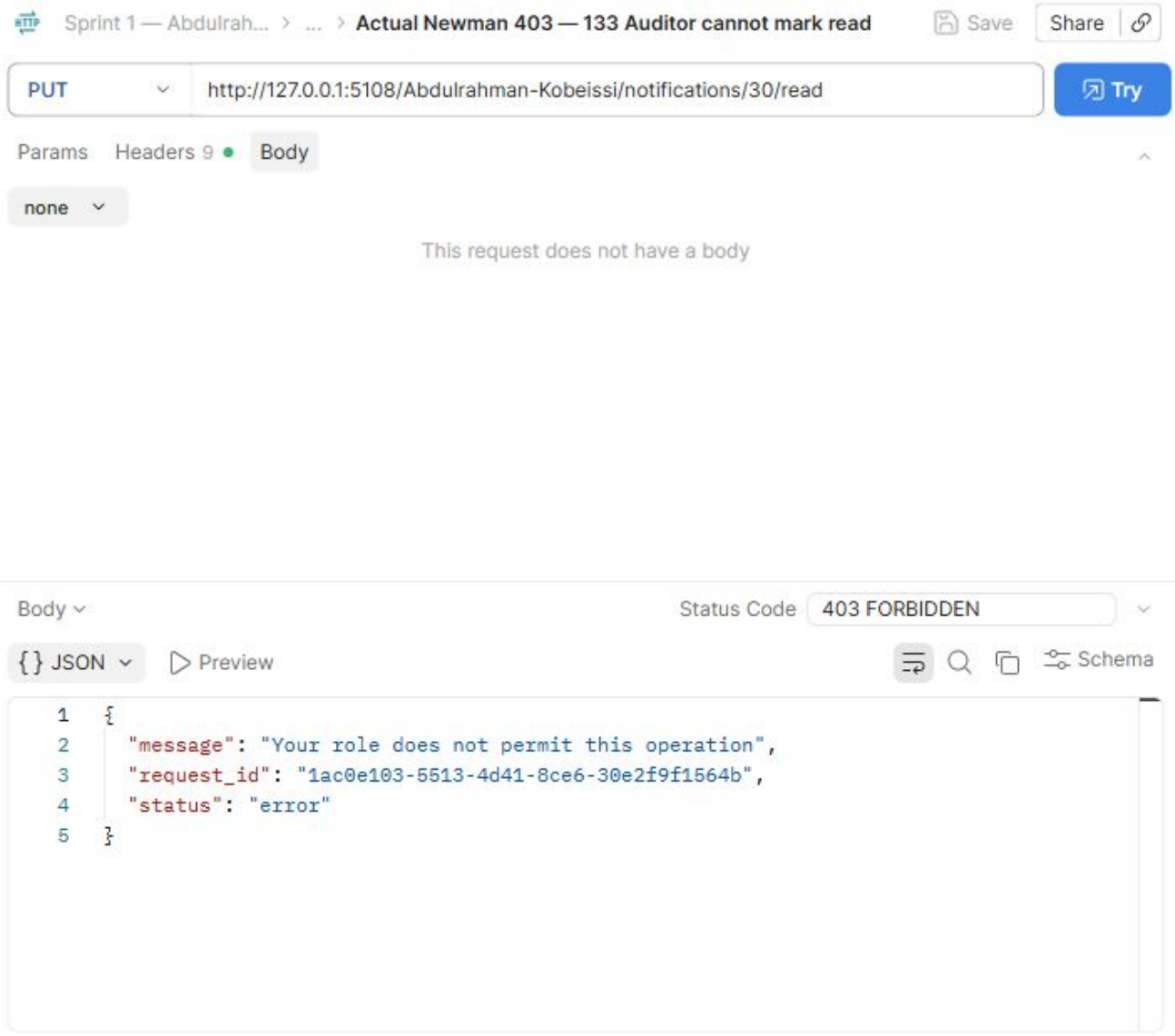


Abdulrahman Kobeissi — 132 Missing notification. The request and saved actual response demonstrate this operation.

Postman Evidence 133

(Abdulrahman Kobeissi)

Auditor cannot mark read · PUT · observed HTTP 403

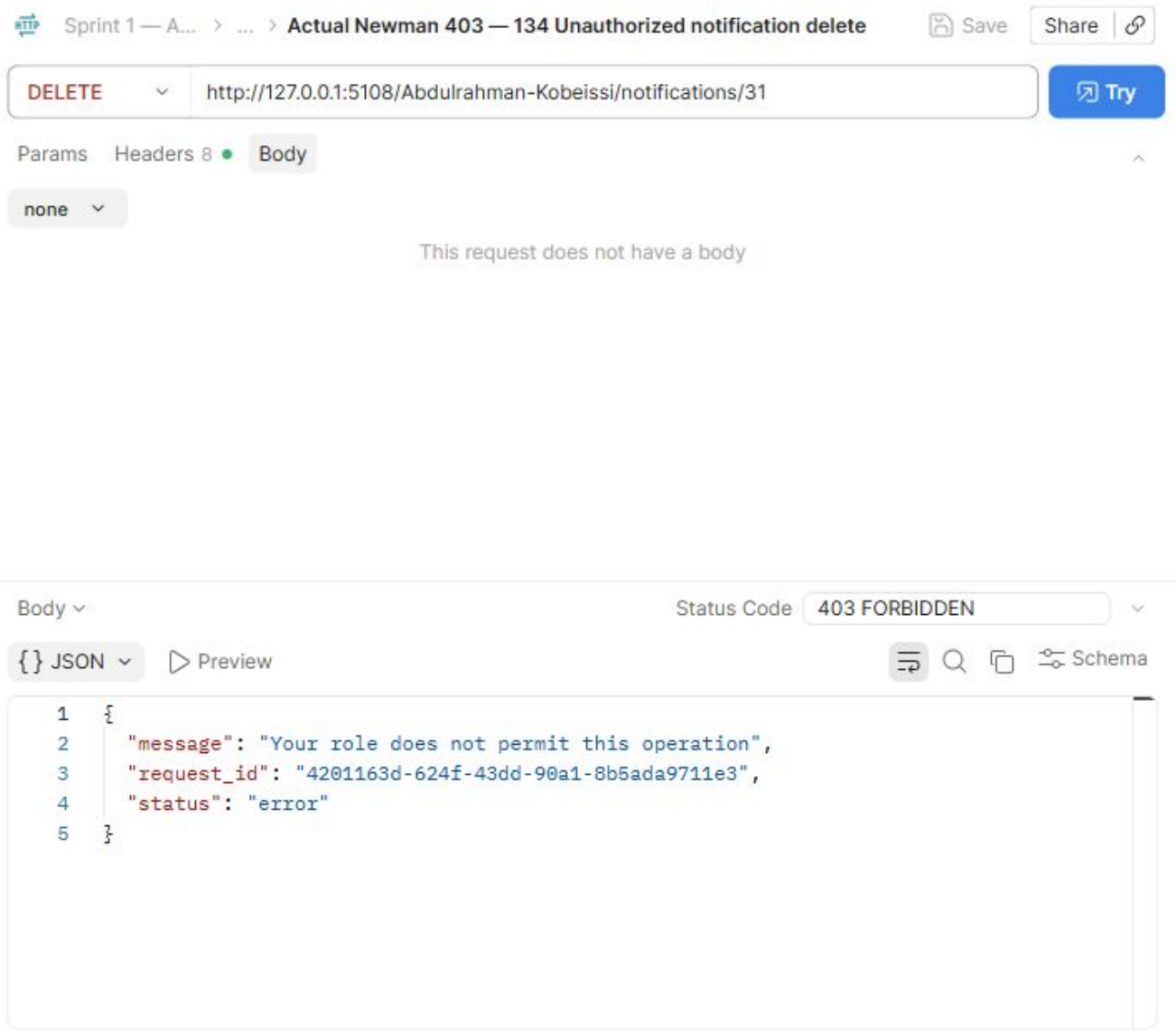


Abdulrahman Kobeissi — 133 Auditor cannot mark read. The request and saved actual response demonstrate this operation.

Postman Evidence 134

(Abdulrahman Kobeissi)

Unauthorized notification delete · DELETE · observed HTTP 403

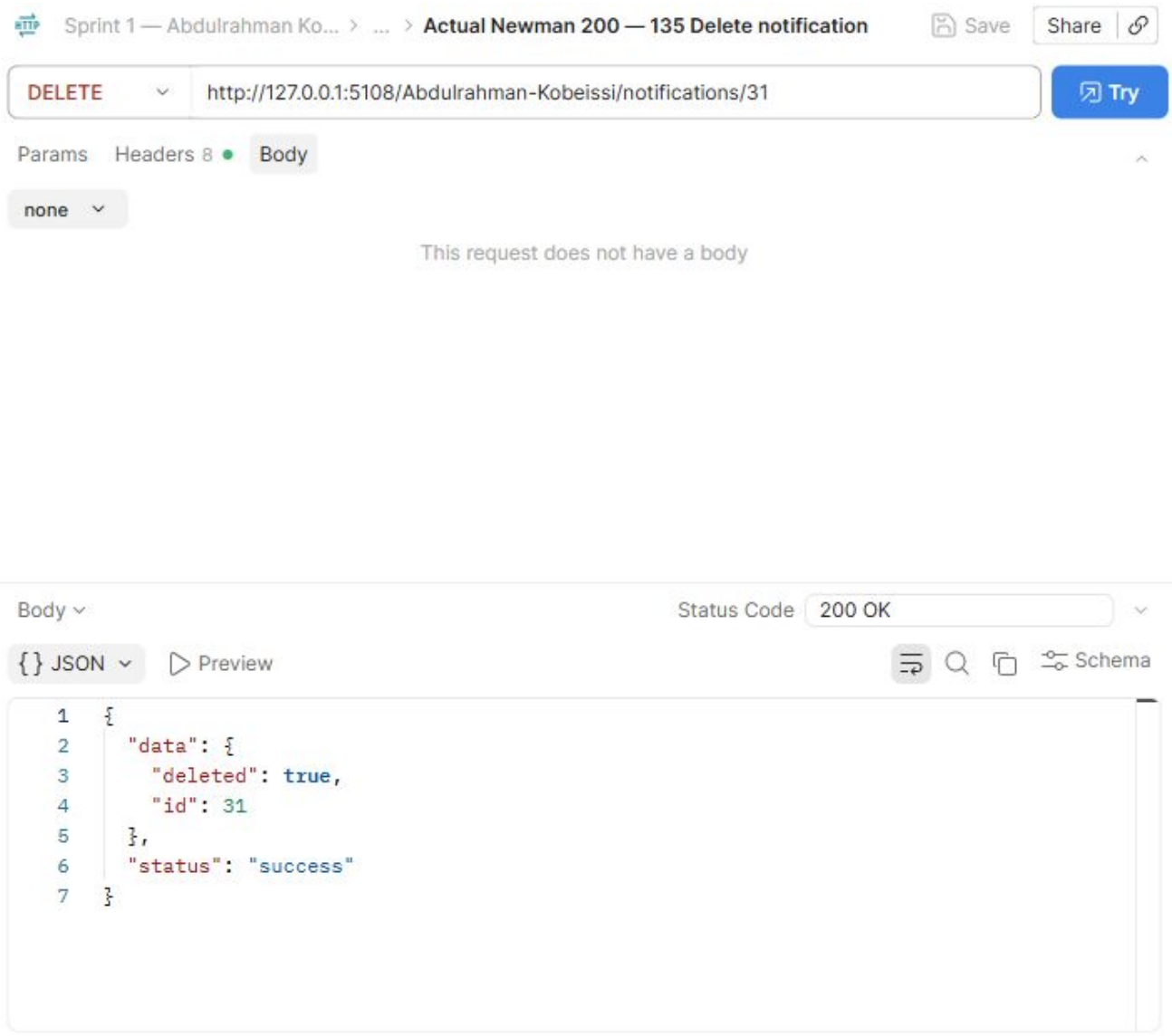


Abdulrahman Kobeissi — 134 Unauthorized notification delete. The request and saved actual response demonstrate this operation.

Postman Evidence 135

(Abdulrahman Kobeissi)

Delete notification · DELETE · observed HTTP 200

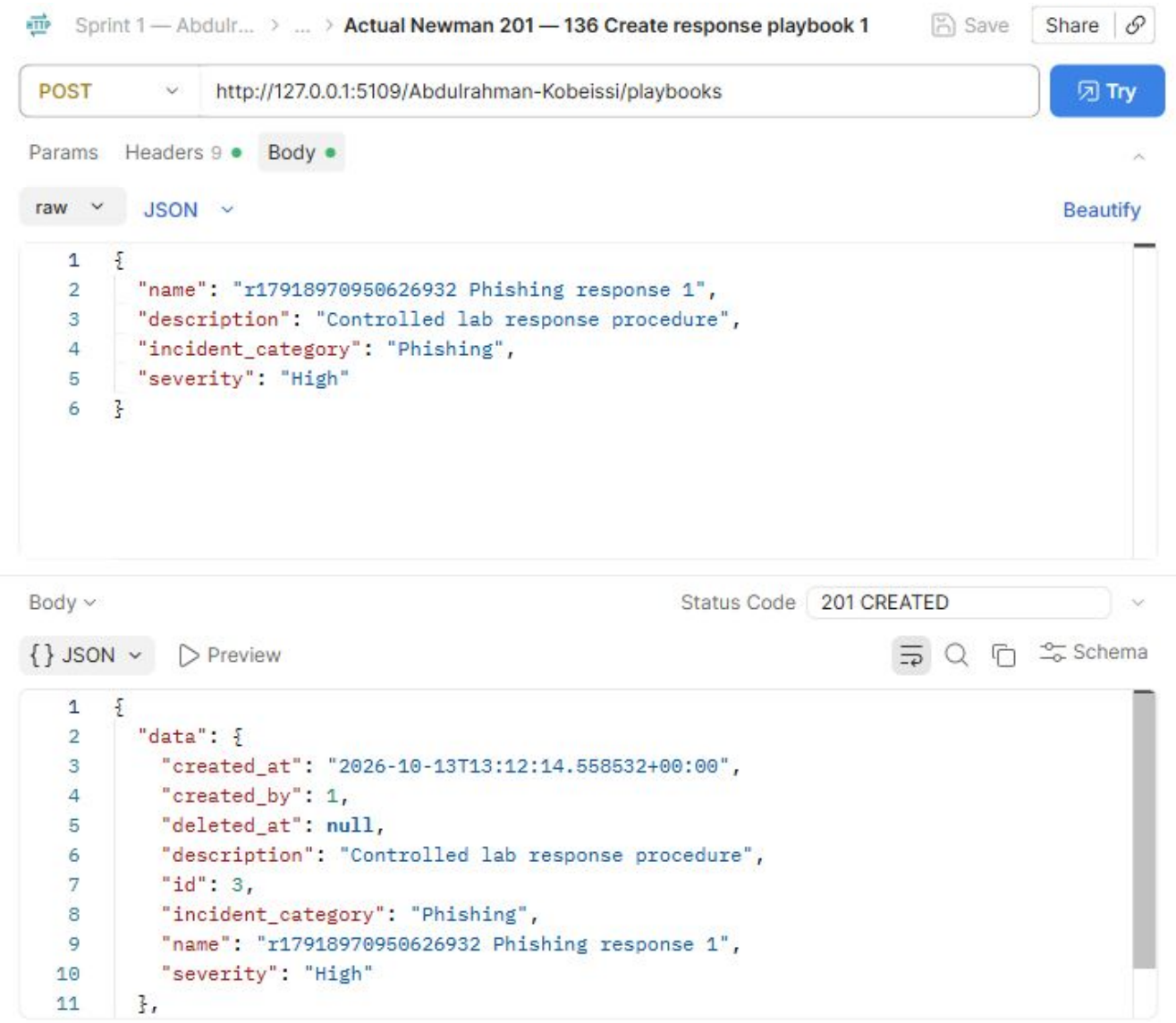


Abdulrahman Kobeissi — 135 Delete notification. The request and saved actual response demonstrate this operation.

Postman Evidence 136

(Abdulrahman Kobeissi)

Create response playbook 1 · POST · observed HTTP 201

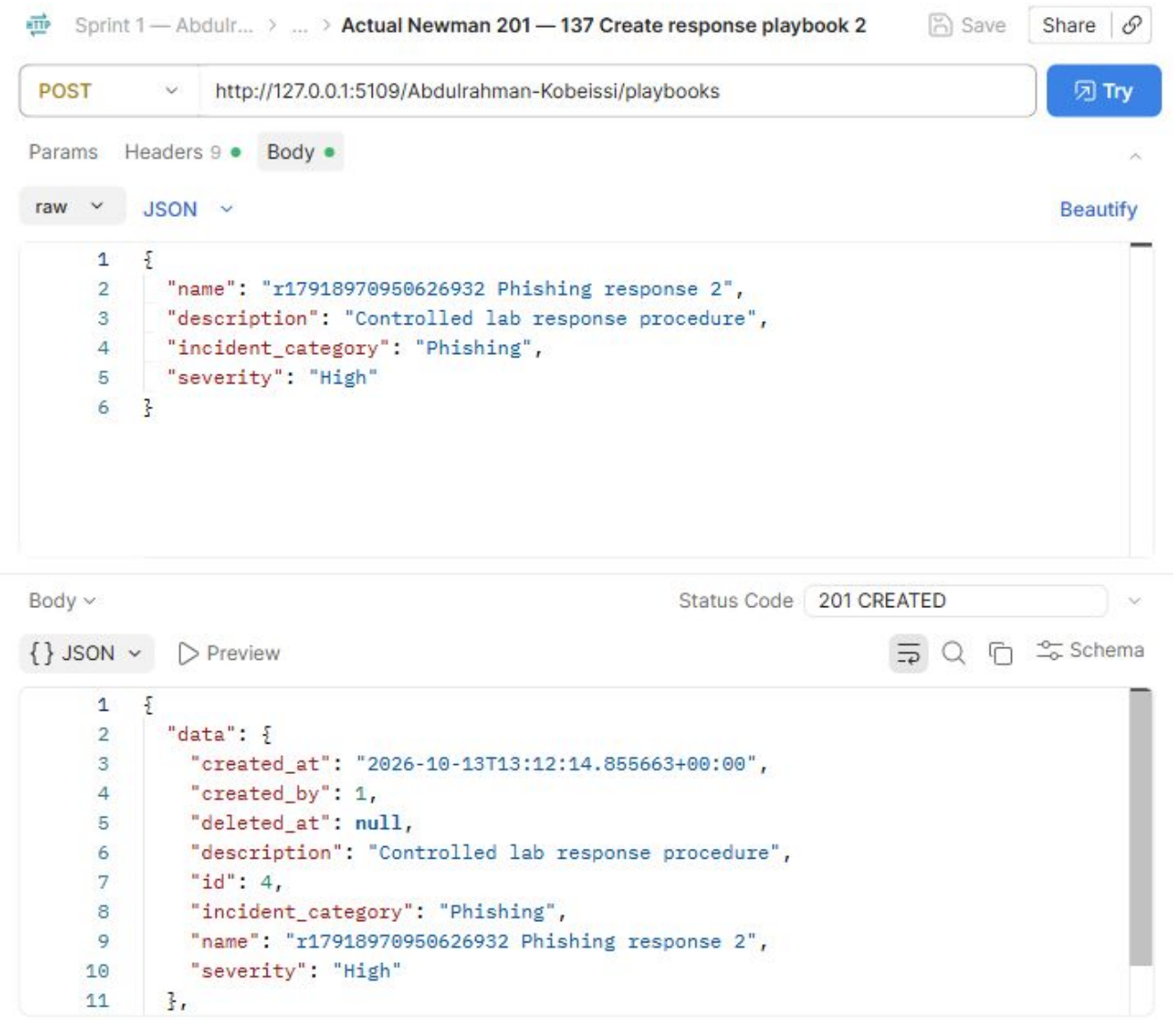


Abdulrahman Kobeissi — 136 Create response playbook 1. The request and saved actual response demonstrate this operation.

Postman Evidence 137

(Abdulrahman Kobeissi)

Create response playbook 2 · POST · observed HTTP 201

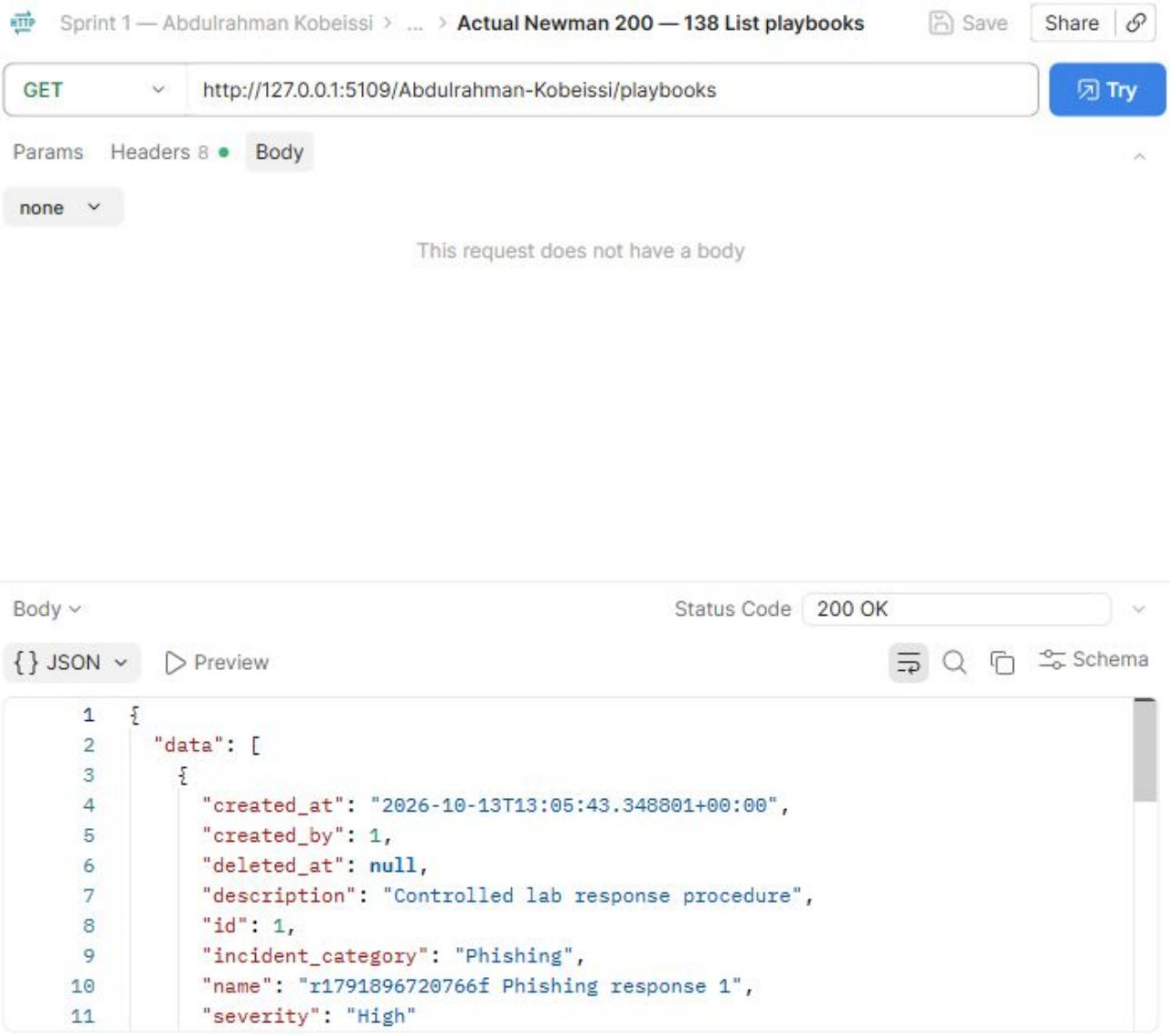


Abdulrahman Kobeissi — 137 Create response playbook 2. The request and saved actual response demonstrate this operation.

Postman Evidence 138

(Abdulrahman Kobeissi)

List playbooks · GET · observed HTTP 200

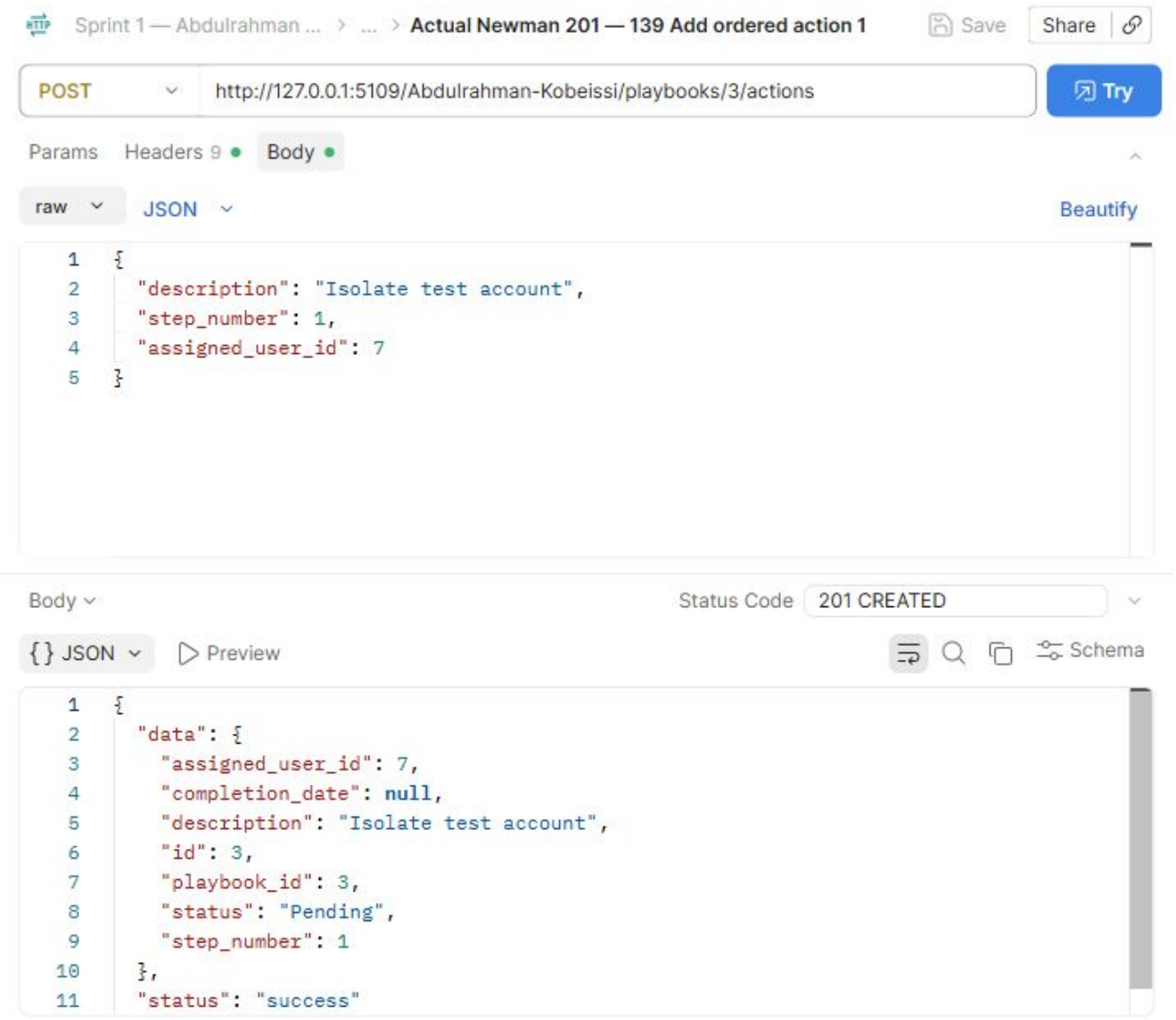


Abdulrahman Kobeissi — 138 List playbooks. The request and saved actual response demonstrate this operation.

Postman Evidence 139

(Abdulrahman Kobeissi)

Add ordered action 1 · POST · observed HTTP 201

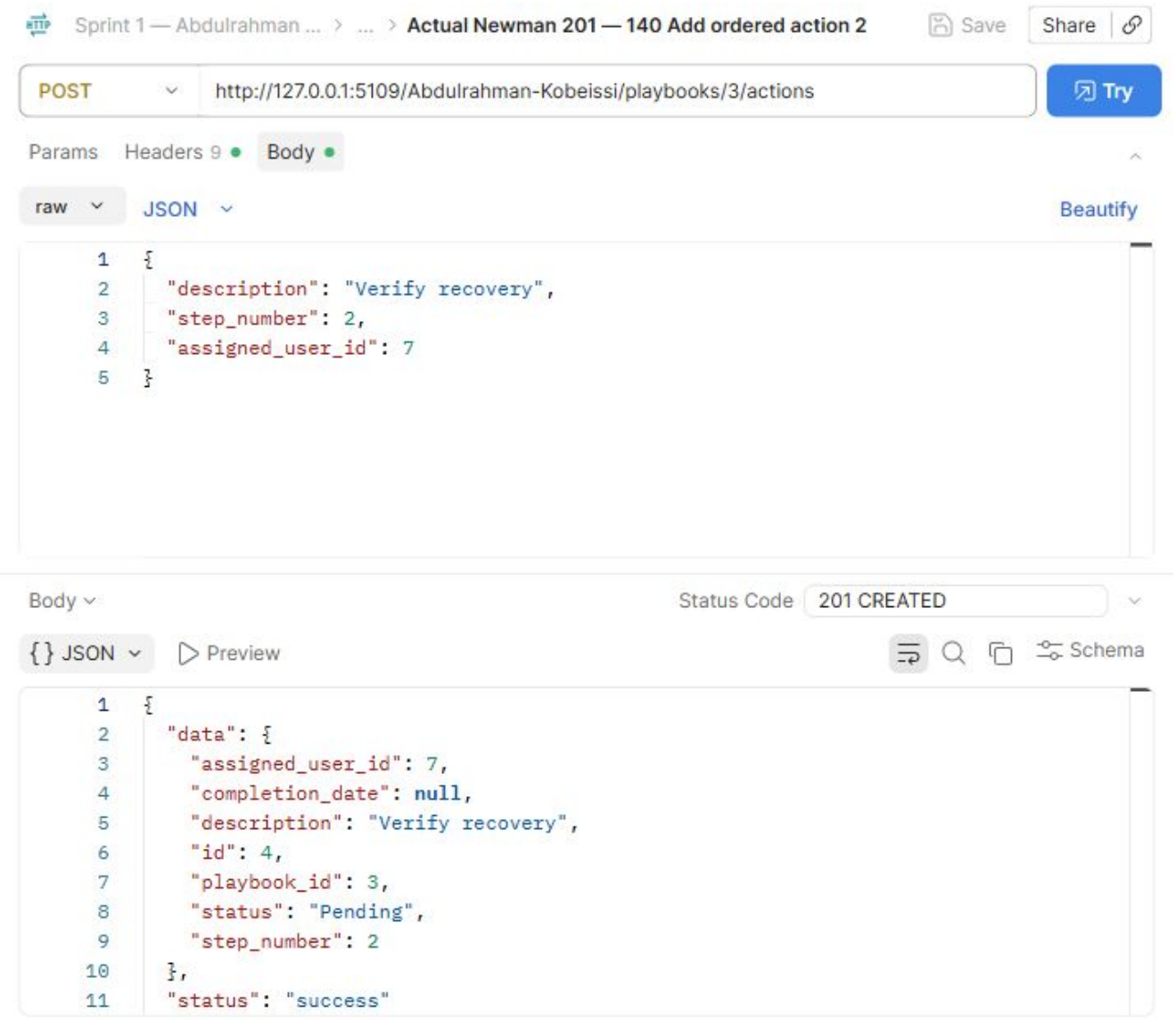


Abdulrahman Kobeissi — 139 Add ordered action 1. The request and saved actual response demonstrate this operation.

Postman Evidence 140

(Abdulrahman Kobeissi)

Add ordered action 2 · POST · observed HTTP 201

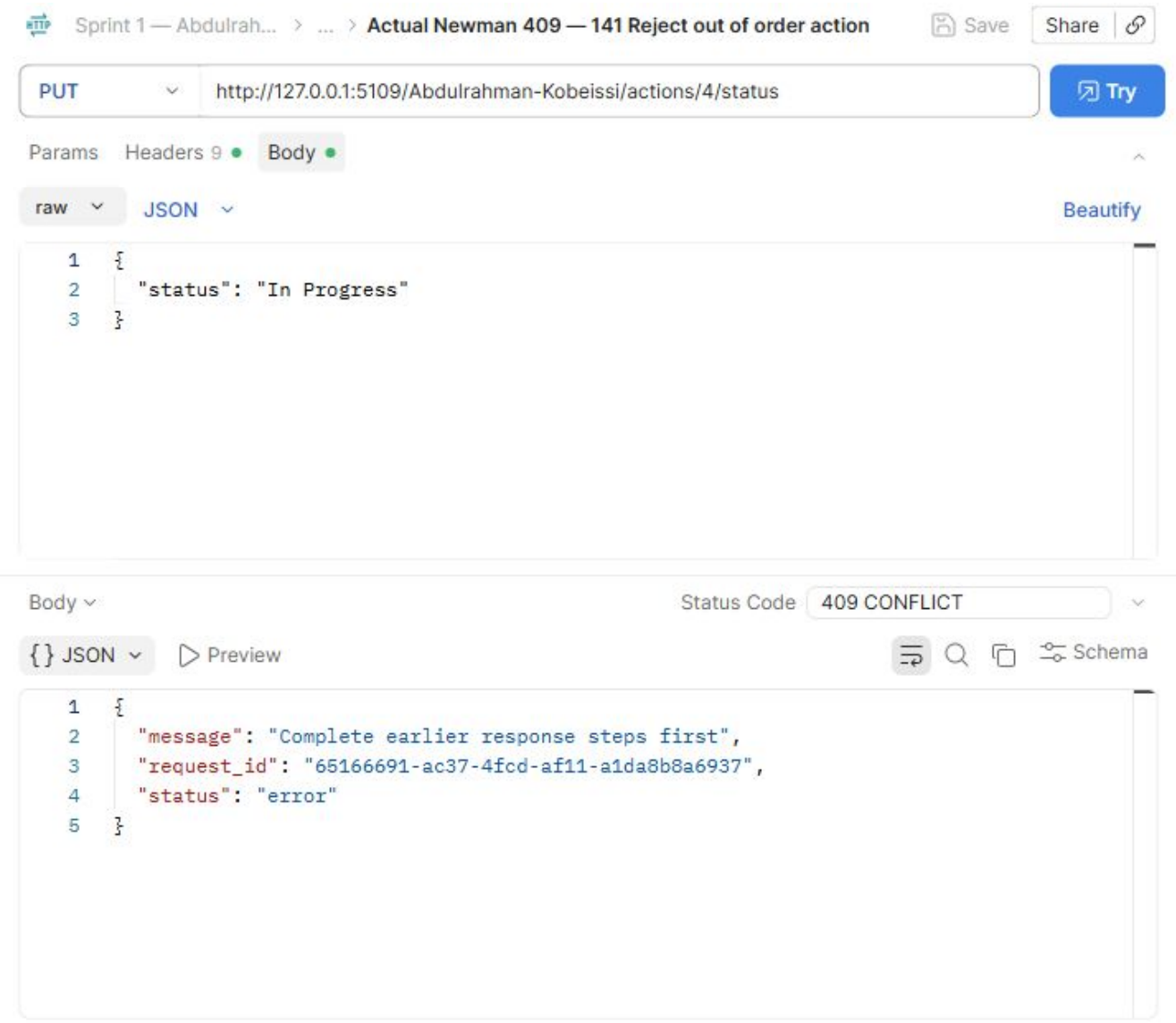


Abdulrahman Kobeissi — 140 Add ordered action 2. The request and saved actual response demonstrate this operation.

Postman Evidence 141

(Abdulrahman Kobeissi)

Reject out of order action · PUT · observed HTTP 409

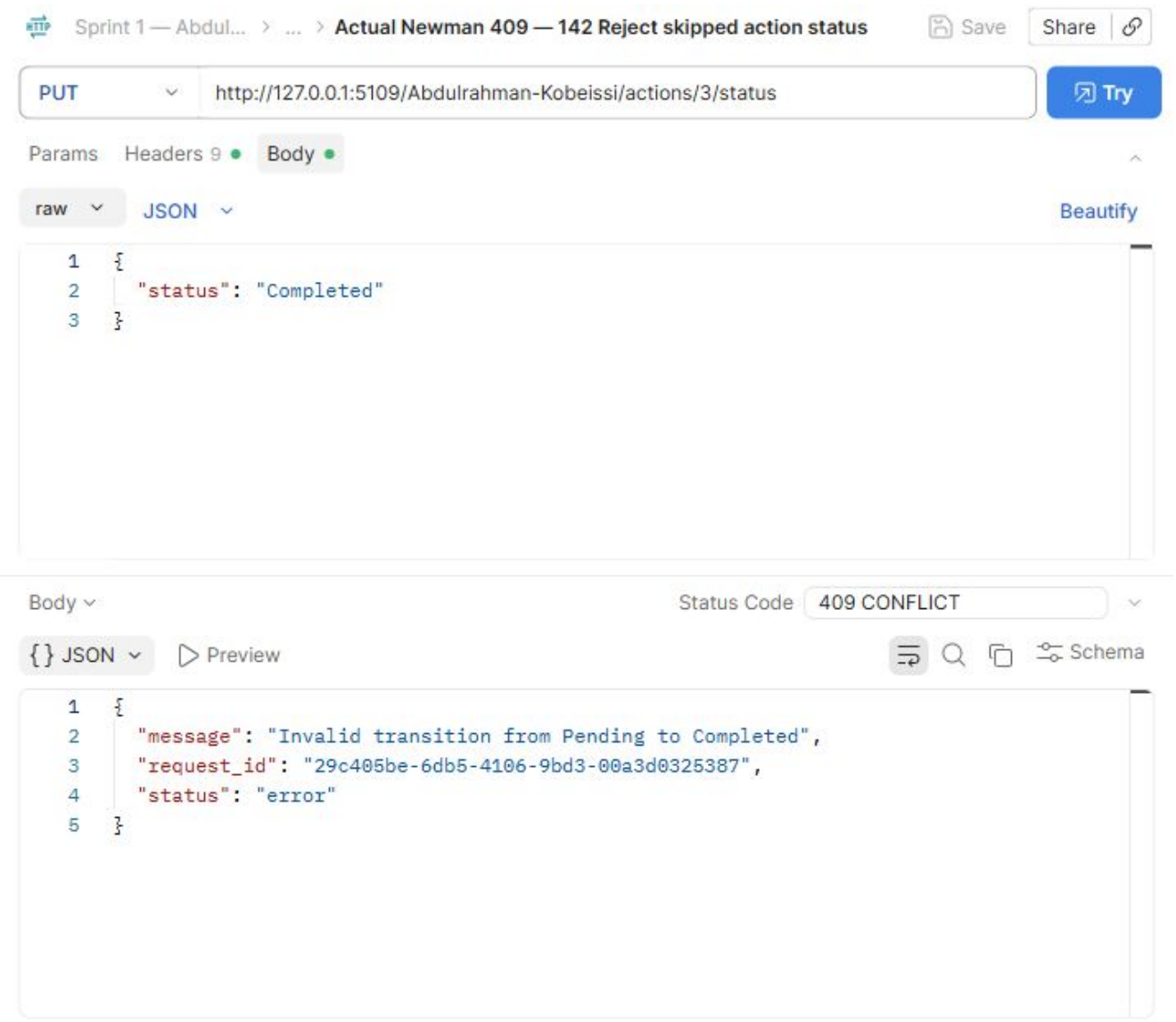


Abdulrahman Kobeissi — 141 Reject out of order action. The request and saved actual response demonstrate this operation.

Postman Evidence 142

(Abdulrahman Kobeissi)

Reject skipped action status · PUT · observed HTTP 409

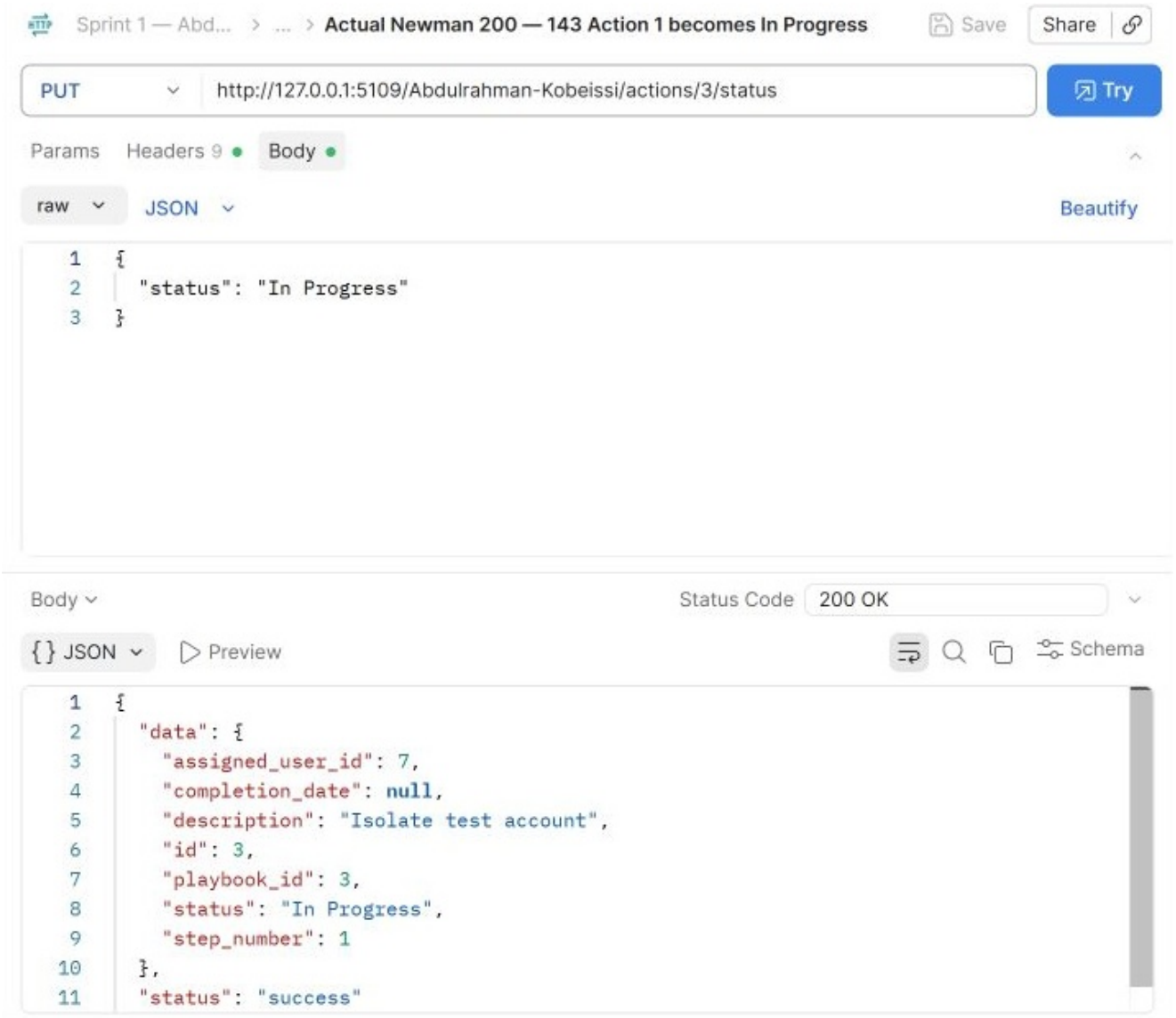


Abdulrahman Kobeissi — 142 Reject skipped action status. The request and saved actual response demonstrate this operation.

Postman Evidence 143

(Abdulrahman Kobeissi)

Action 1 becomes In Progress · PUT · observed HTTP 200

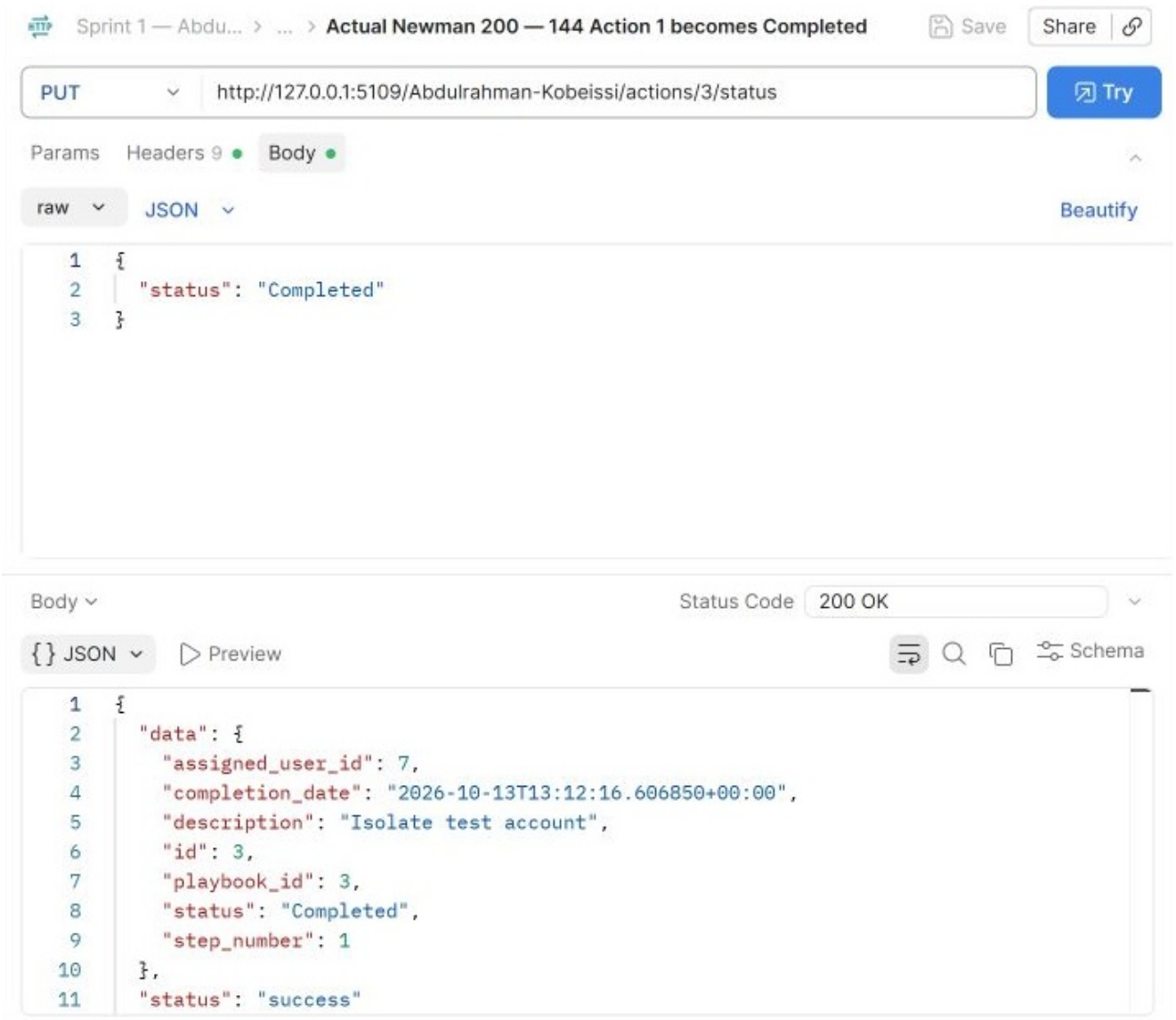


Abdulrahman Kobeissi — 143 Action 1 becomes In Progress. The request and saved actual response demonstrate this operation.

Postman Evidence 144

(Abdulrahman Kobeissi)

Action 1 becomes Completed · PUT · observed HTTP 200

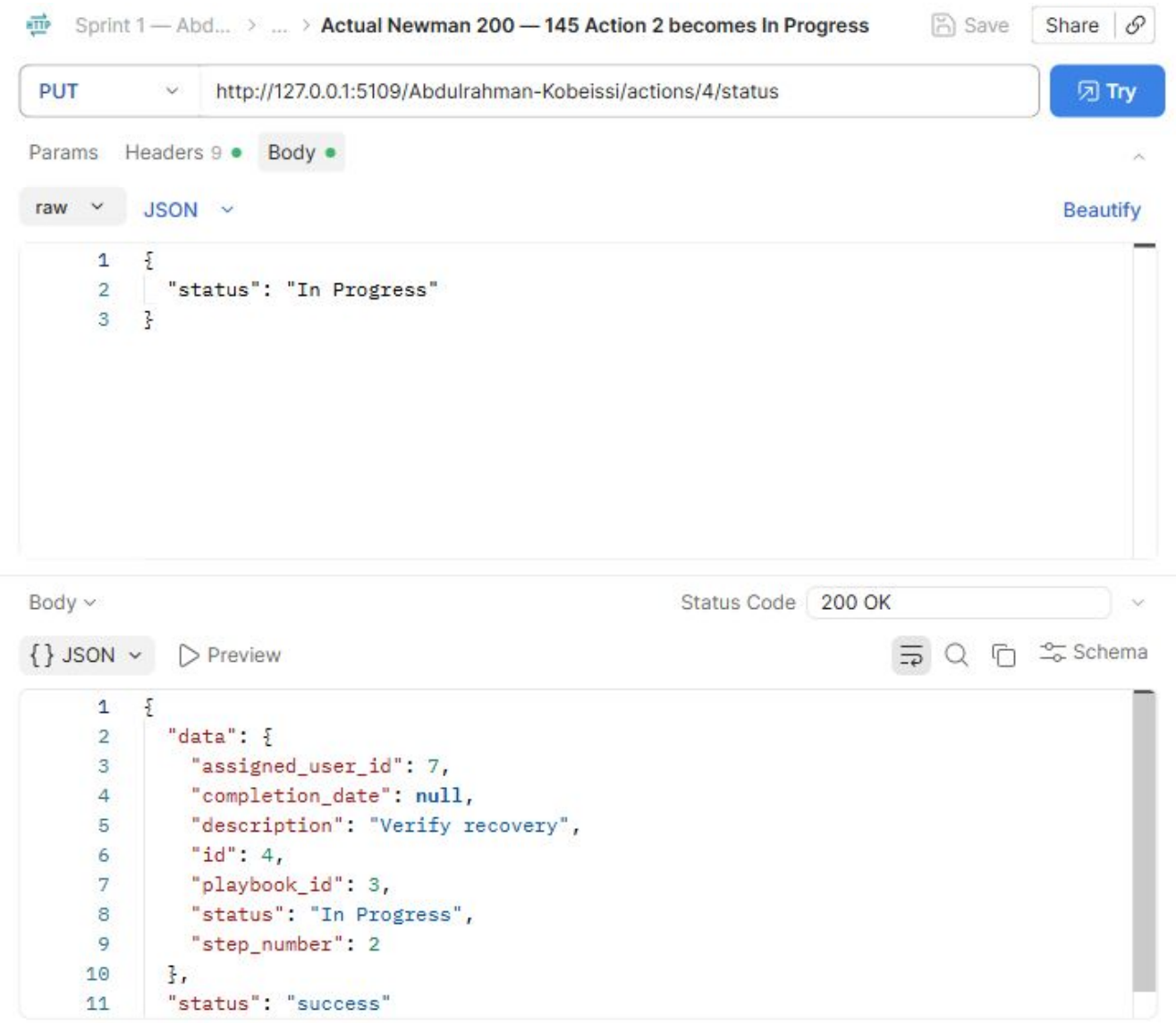


Abdulrahman Kobeissi — 144 Action 1 becomes Completed. The request and saved actual response demonstrate this operation.

Postman Evidence 145

(Abdulrahman Kobeissi)

Action 2 becomes In Progress · PUT · observed HTTP 200

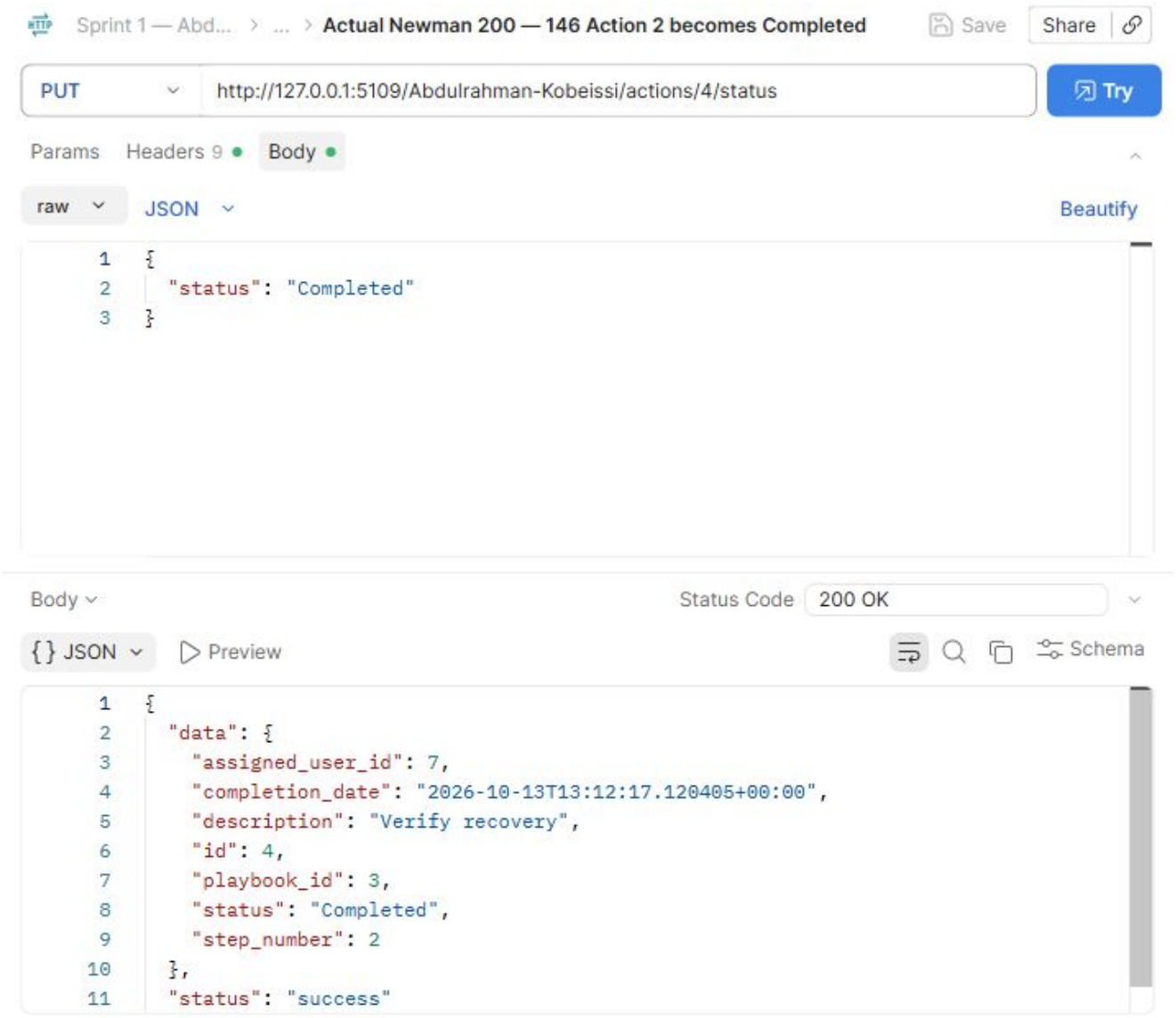


Abdulrahman Kobeissi — 145 Action 2 becomes In Progress. The request and saved actual response demonstrate this operation.

Postman Evidence 146

(Abdulrahman Kobeissi)

Action 2 becomes Completed · PUT · observed HTTP 200

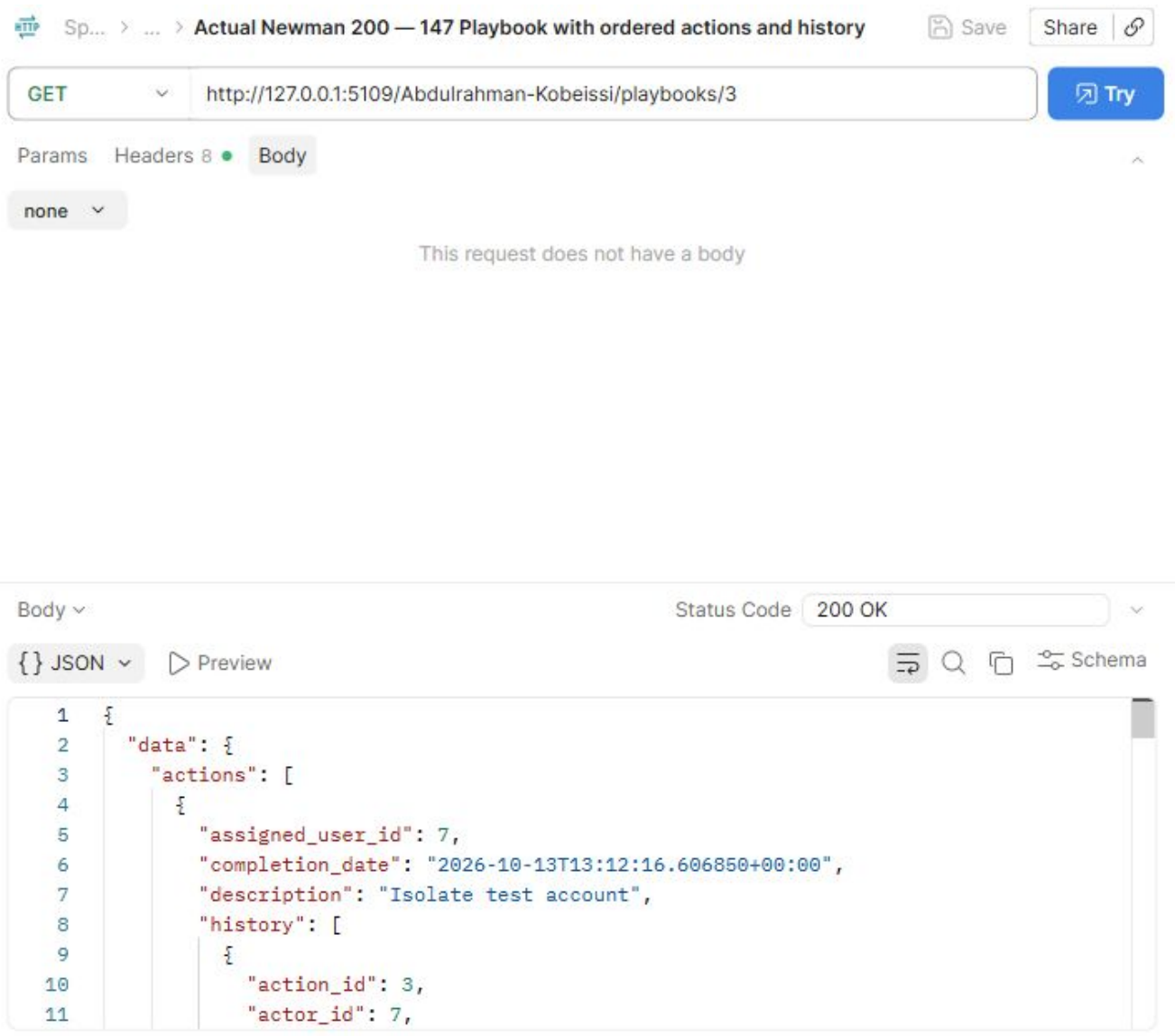


Abdulrahman Kobeissi — 146 Action 2 becomes Completed. The request and saved actual response demonstrate this operation.

Postman Evidence 147

(Abdulrahman Kobeissi)

Playbook with ordered actions and history · GET · observed HTTP 200

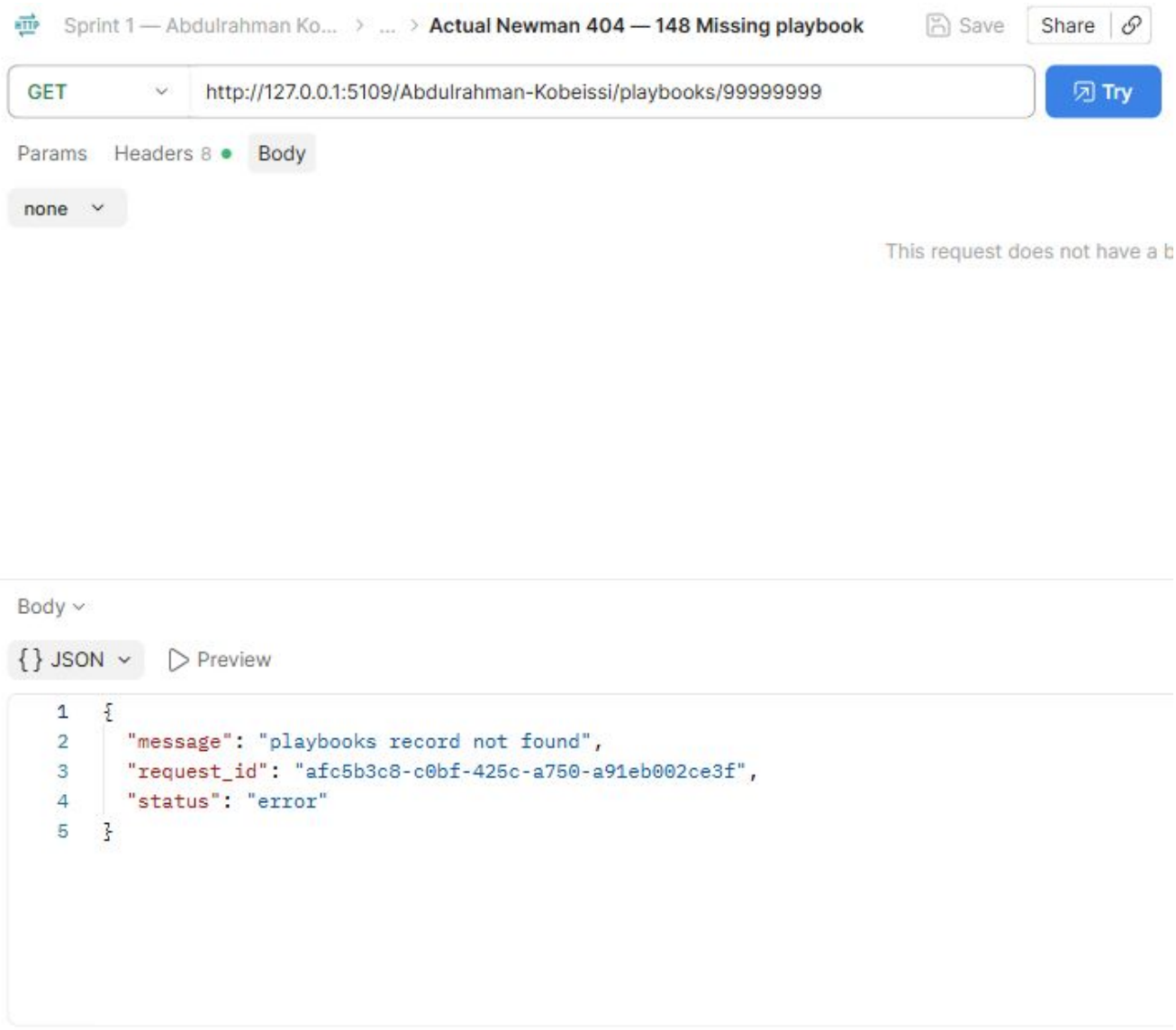


Abdulrahman Kobeissi — 147 Playbook with ordered actions and history. The request and saved actual response demonstrate this operation.

Postman Evidence 148

(Abdulrahman Kobeissi)

Missing playbook · GET · observed HTTP 404

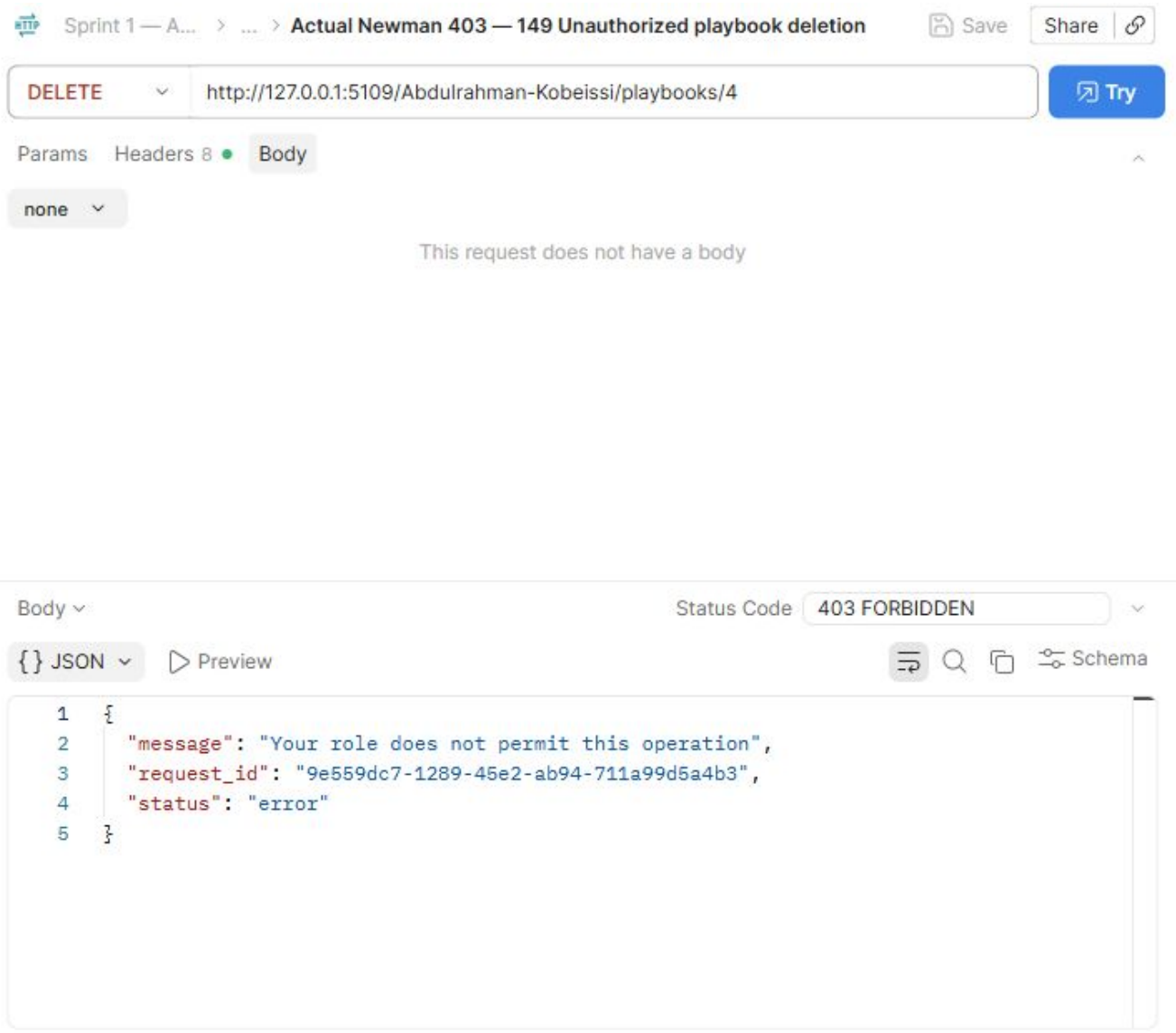


Abdulrahman Kobeissi — 148 Missing playbook. The request and saved actual response demonstrate this operation.

Postman Evidence 149

(Abdulrahman Kobeissi)

Unauthorized playbook deletion · DELETE · observed HTTP 403

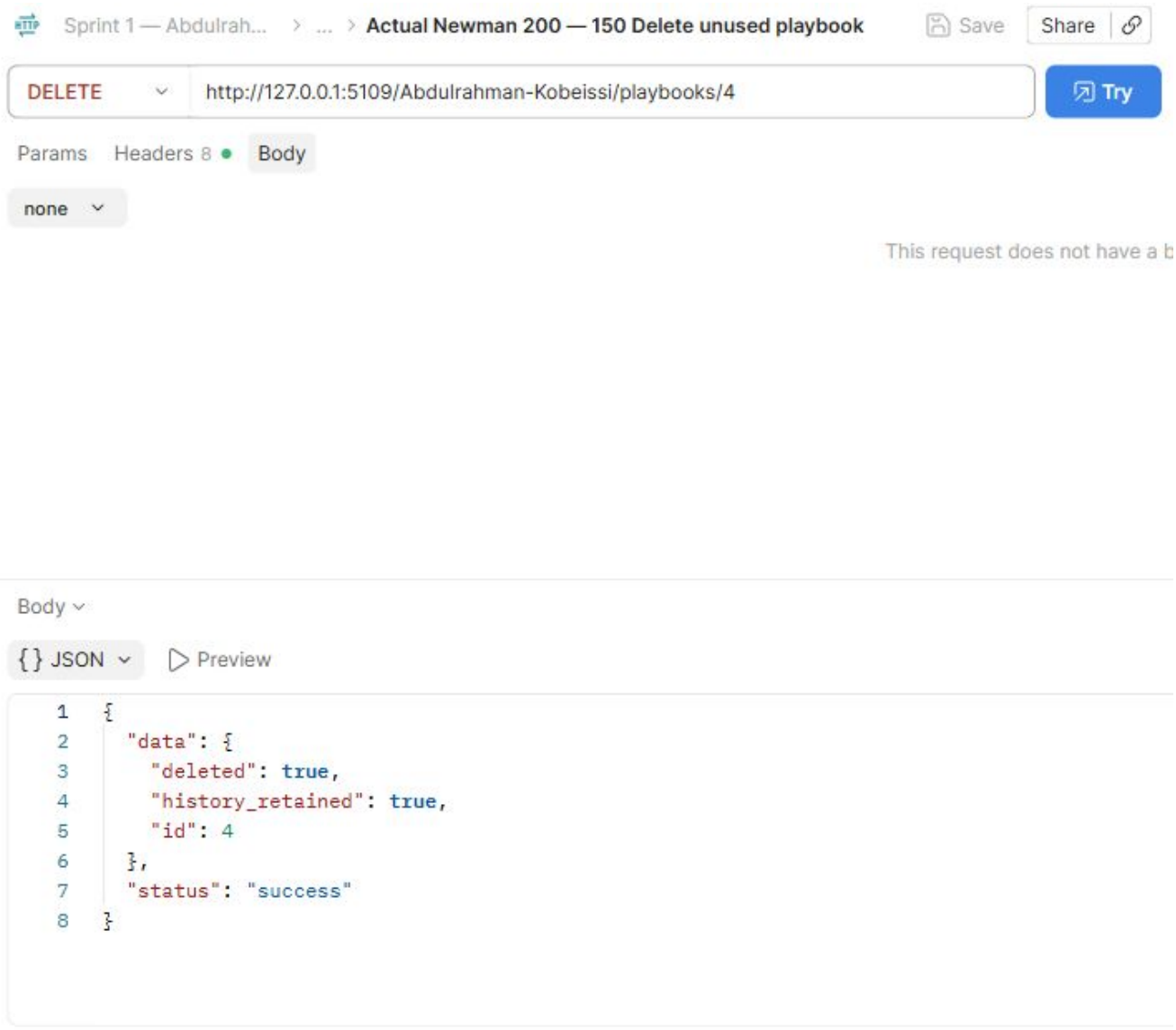


Abdulrahman Kobeissi — 149 Unauthorized playbook deletion. The request and saved actual response demonstrate this operation.

Postman Evidence 150

(Abdulrahman Kobeissi)

Delete unused playbook · DELETE · observed HTTP 200

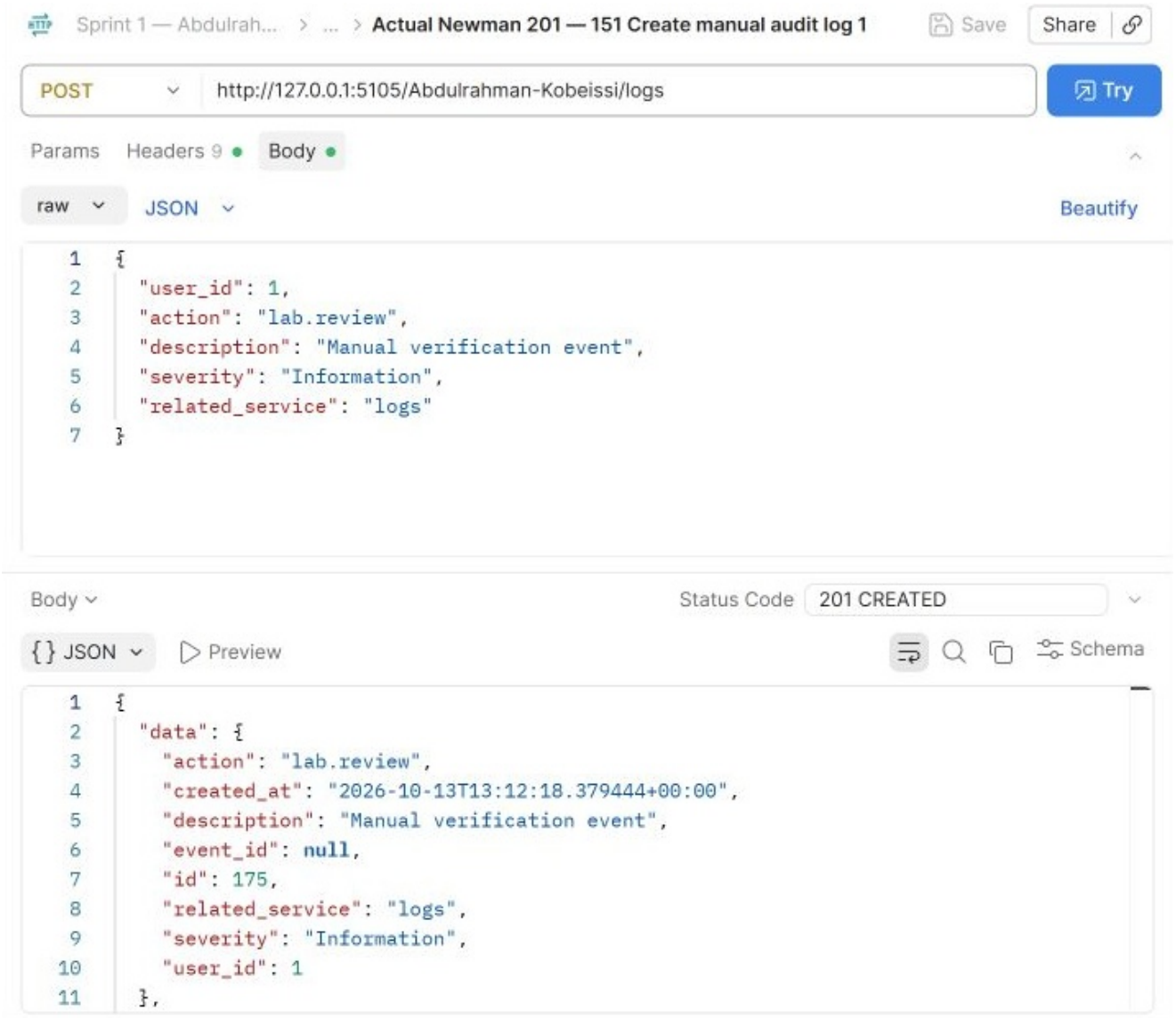


Abdulrahman Kobeissi — 150 Delete unused playbook. The request and saved actual response demonstrate this operation.

Postman Evidence 151

(Abdulrahman Kobeissi)

Create manual audit log 1 · POST · observed HTTP 201

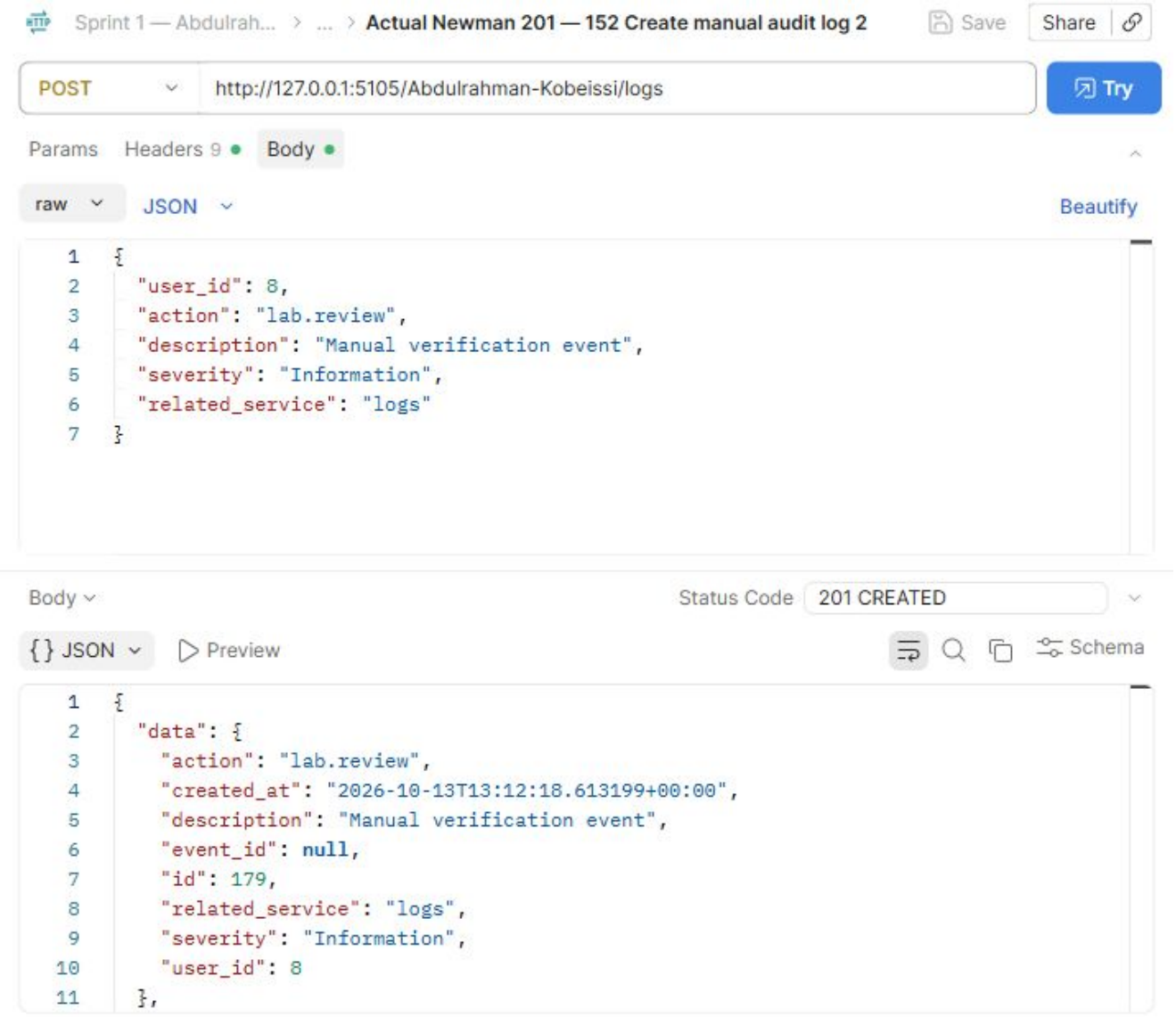


Abdulrahman Kobeissi — 151 Create manual audit log 1. The request and saved actual response demonstrate this operation.

Postman Evidence 152

(Abdulrahman Kobeissi)

Create manual audit log 2 · POST · observed HTTP 201

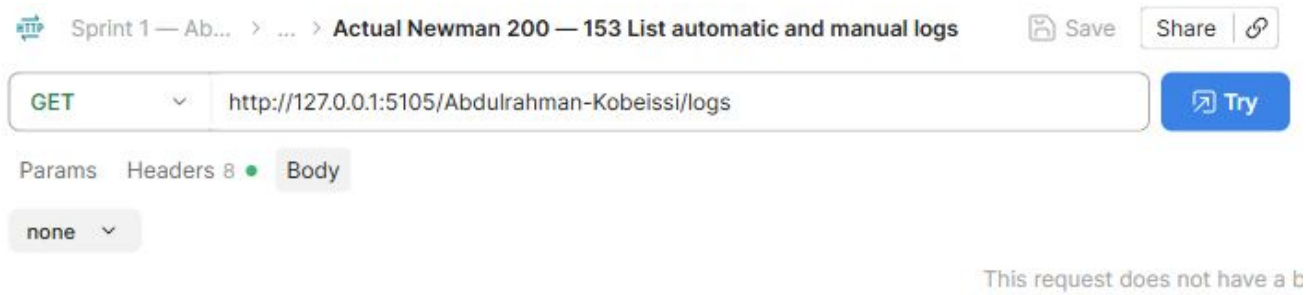


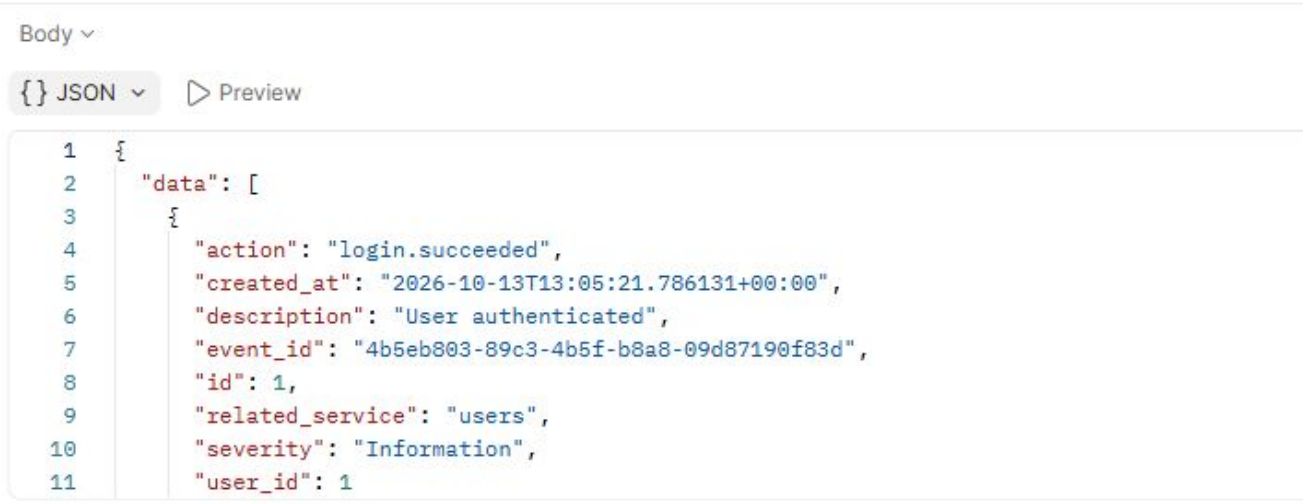
Abdulrahman Kobeissi — 152 Create manual audit log 2. The request and saved actual response demonstrate this operation.

Postman Evidence 153

(Abdulrahman Kobeissi)

List automatic and manual logs · GET · observed HTTP 200



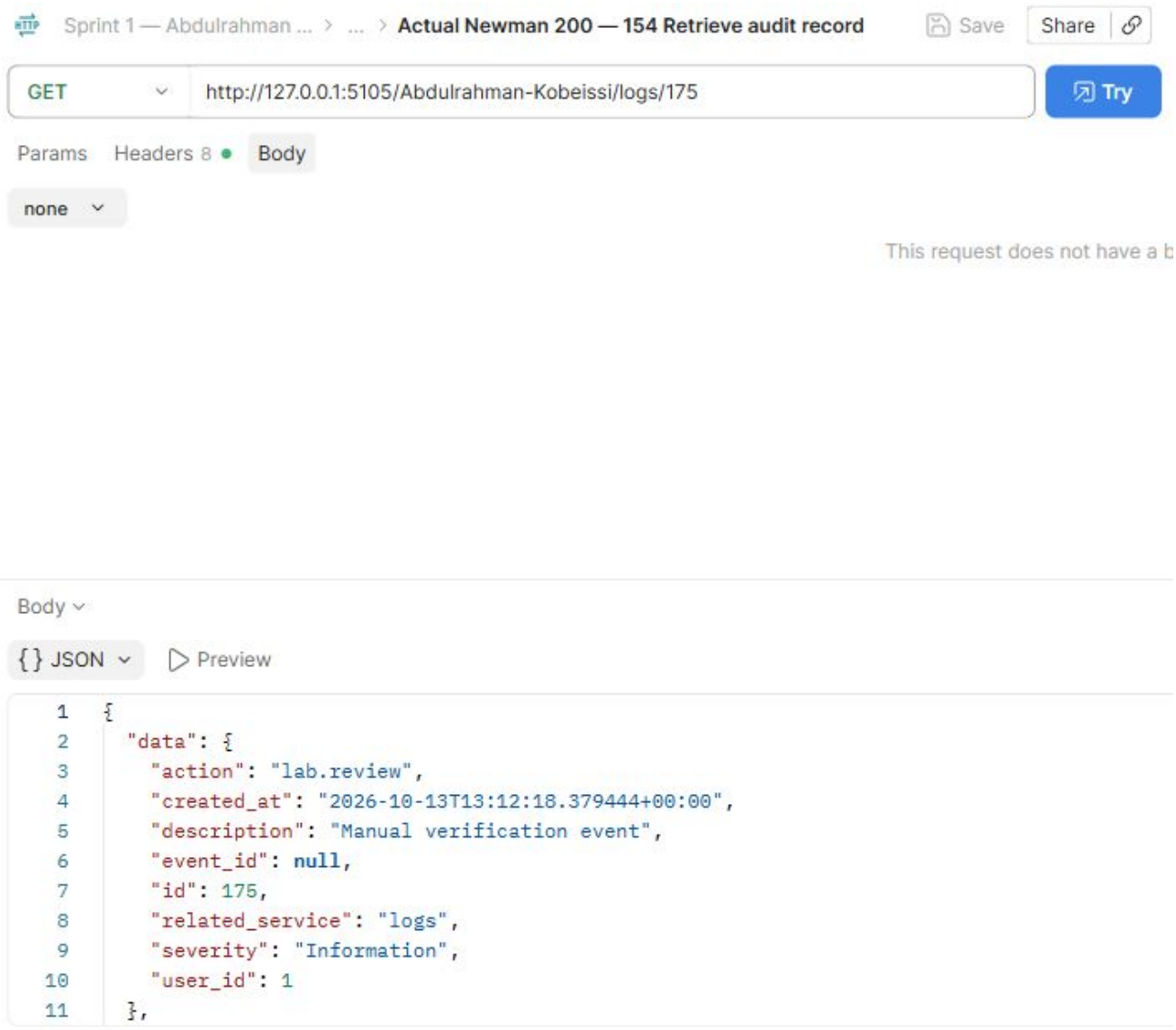


Abdulrahman Kobeissi — 153 List automatic and manual logs. The request and saved actual response demonstrate this operation.

Postman Evidence 154

(Abdulrahman Kobeissi)

Retrieve audit record · GET · observed HTTP 200

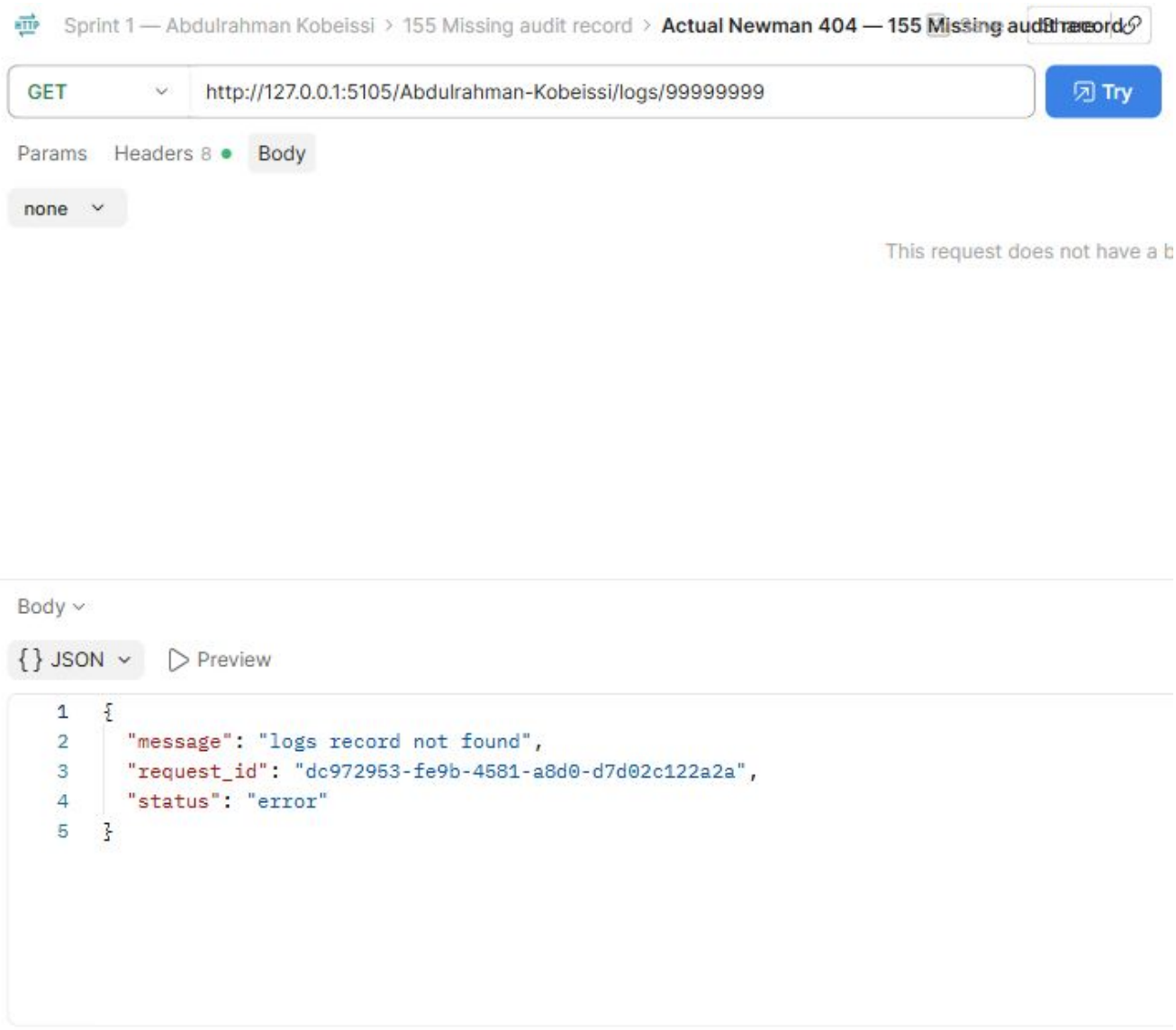


Abdulrahman Kobeissi — 154 Retrieve audit record. The request and saved actual response demonstrate this operation.

Postman Evidence 155

(Abdulrahman Kobeissi)

Missing audit record · GET · observed HTTP 404

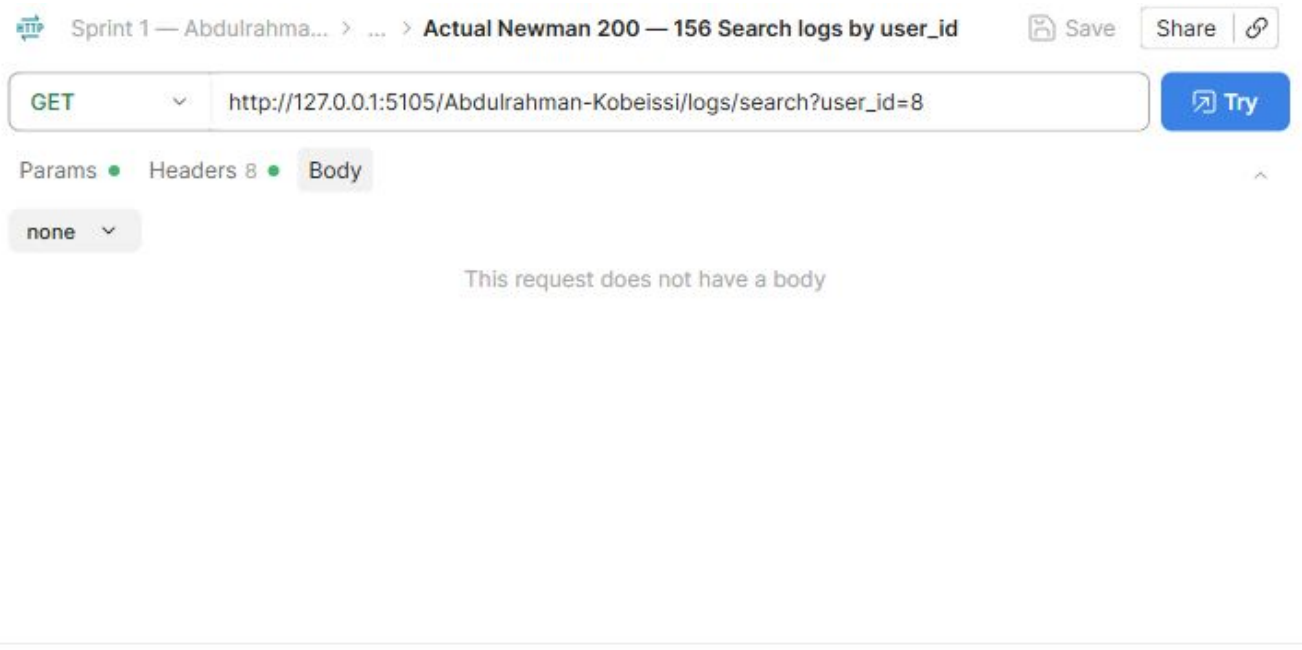


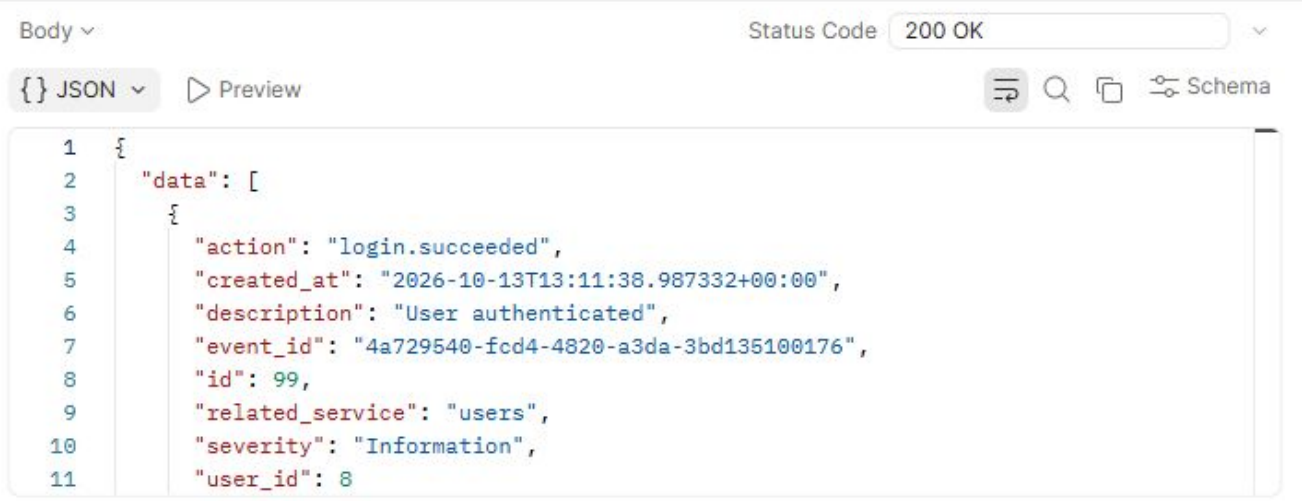
Abdulrahman Kobeissi — 155 Missing audit record. The request and saved actual response demonstrate this operation.

Postman Evidence 156

(Abdulrahman Kobeissi)

Search logs by user_id · GET · observed HTTP 200



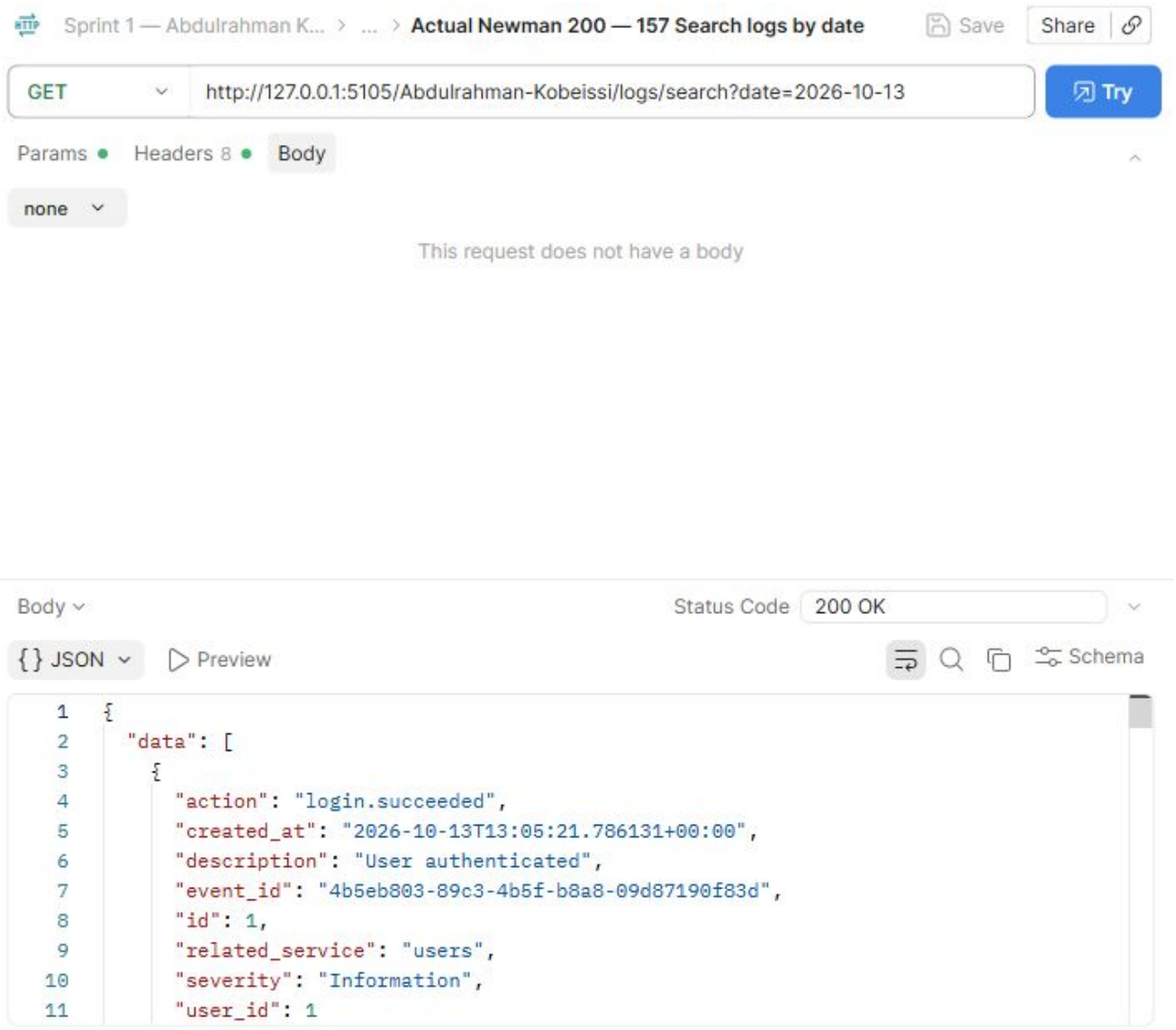


Abdulrahman Kobeissi — 156 Search logs by user_id. The request and saved actual response demonstrate this operation.

Postman Evidence 157

(Abdulrahman Kobeissi)

Search logs by date · GET · observed HTTP 200

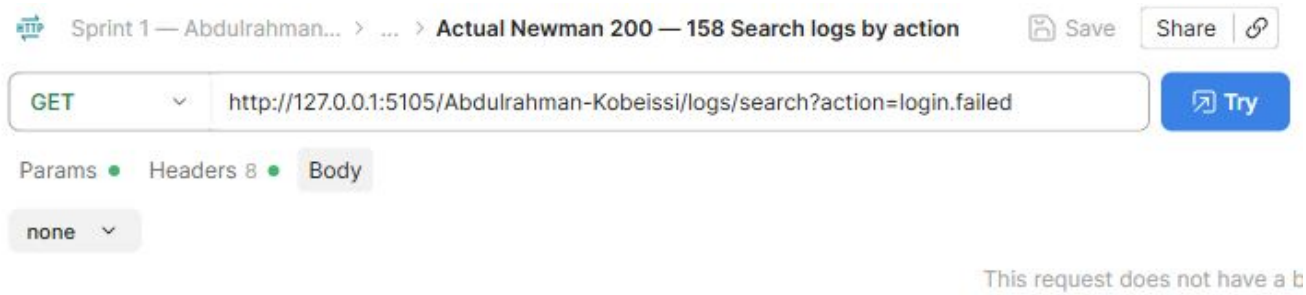


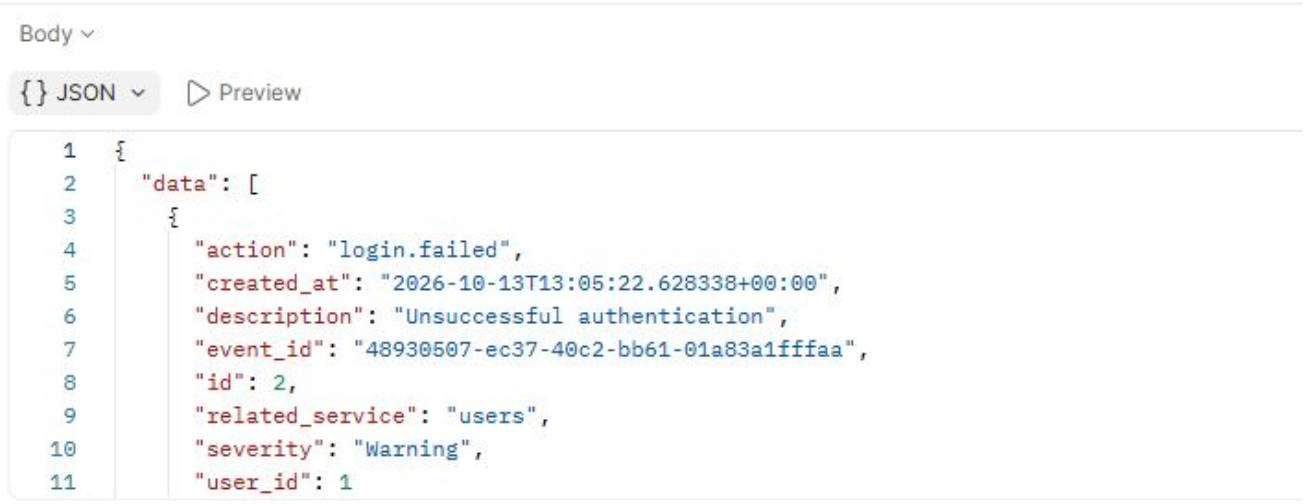
Abdulrahman Kobeissi — 157 Search logs by date. The request and saved actual response demonstrate this operation.

Postman Evidence 158

(Abdulrahman Kobeissi)

Search logs by action · GET · observed HTTP 200



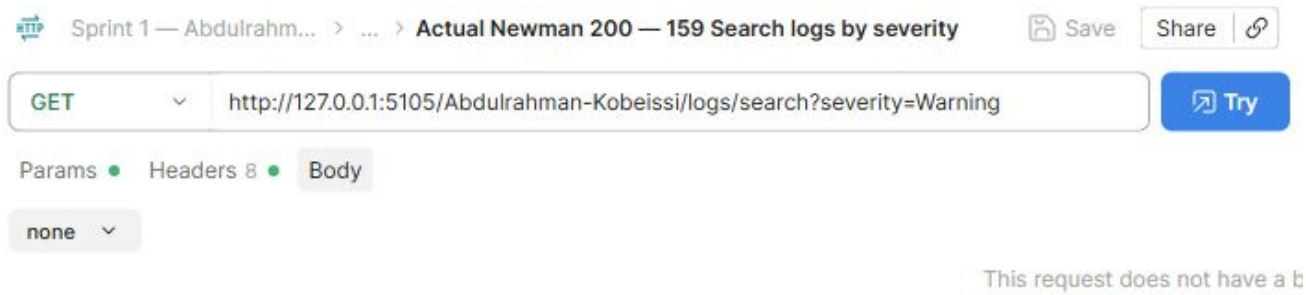


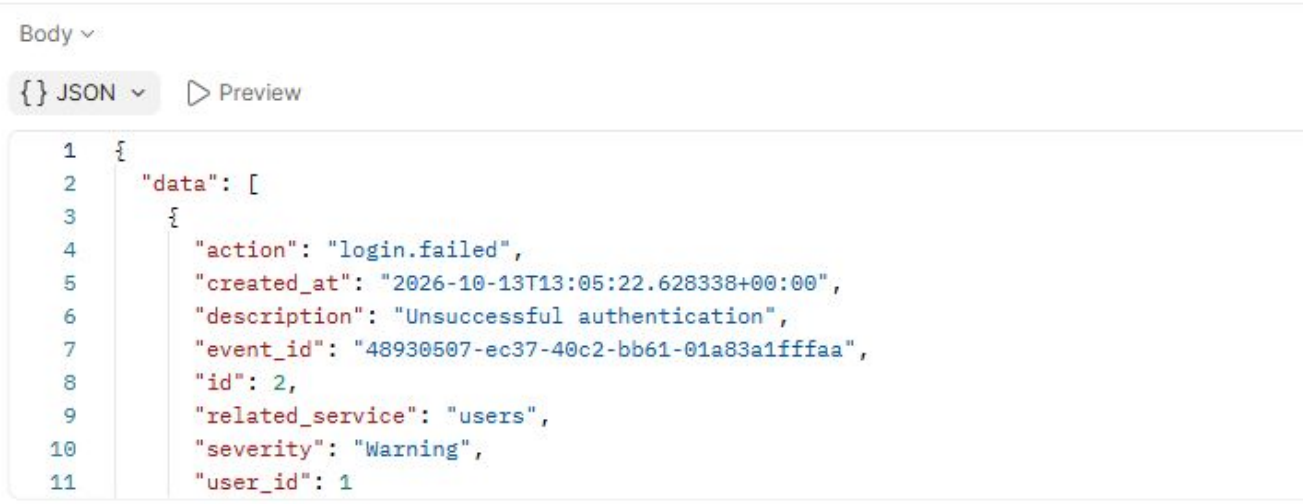
Abdulrahman Kobeissi — 158 Search logs by action. The request and saved actual response demonstrate this operation.

Postman Evidence 159

(Abdulrahman Kobeissi)

Search logs by severity · GET · observed HTTP 200



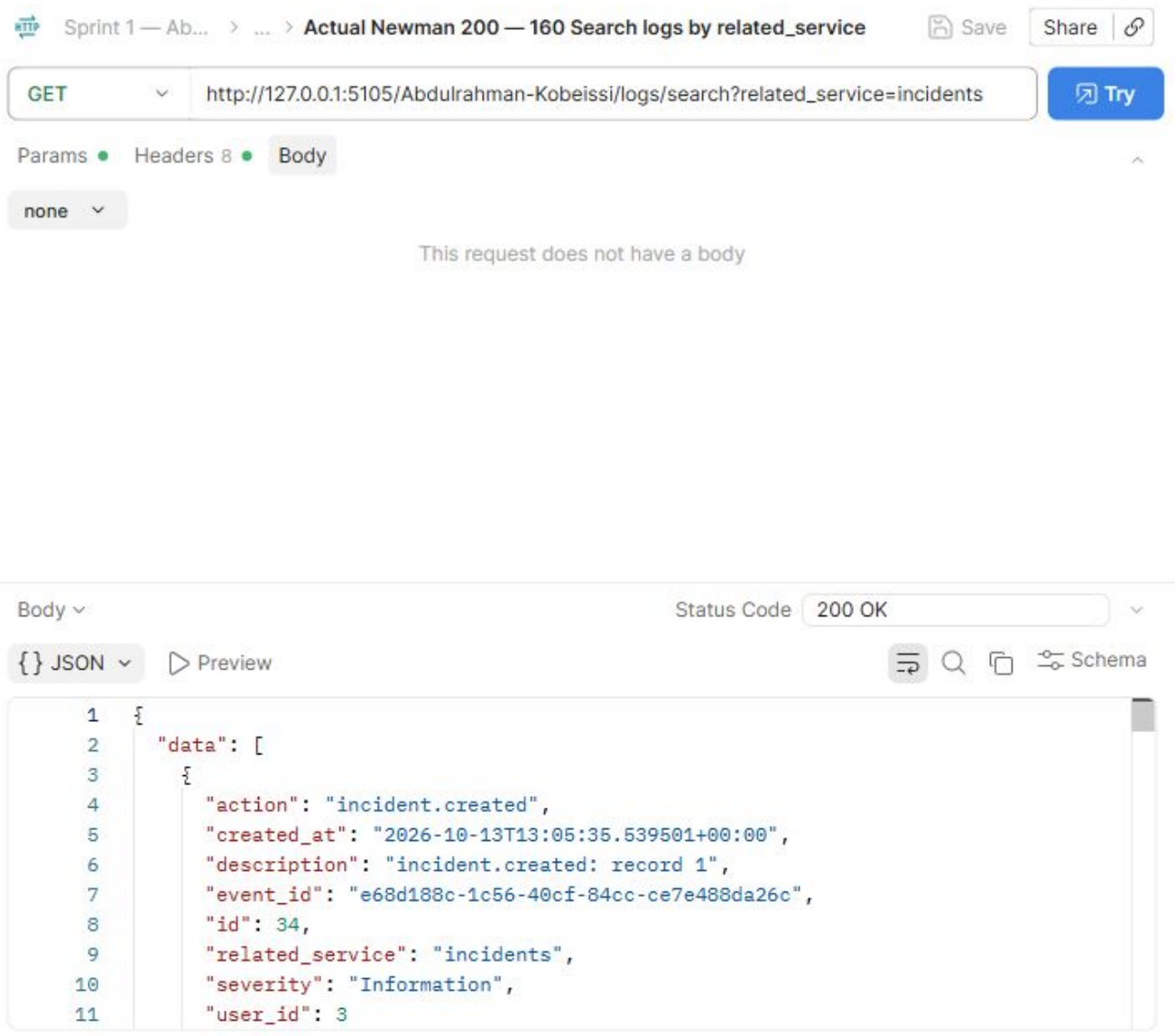


Abdulrahman Kobeissi — 159 Search logs by severity. The request and saved actual response demonstrate this operation.

Postman Evidence 160

(Abdulrahman Kobeissi)

Search logs by related_service · GET · observed HTTP 200

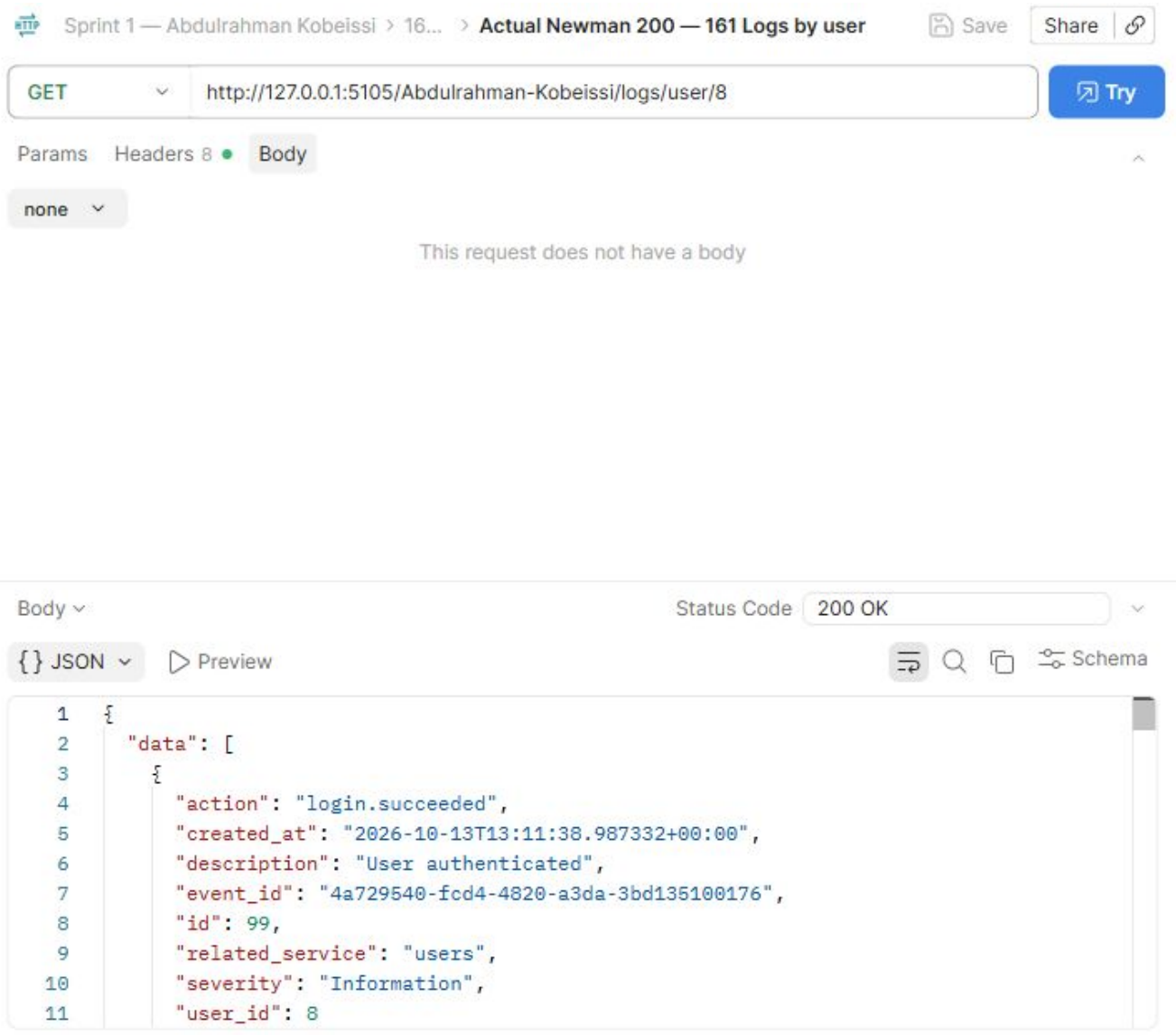


Abdulrahman Kobeissi — 160 Search logs by related_service. The request and saved actual response demonstrate this operation.

Postman Evidence 161

(Abdulrahman Kobeissi)

Logs by user · GET · observed HTTP 200

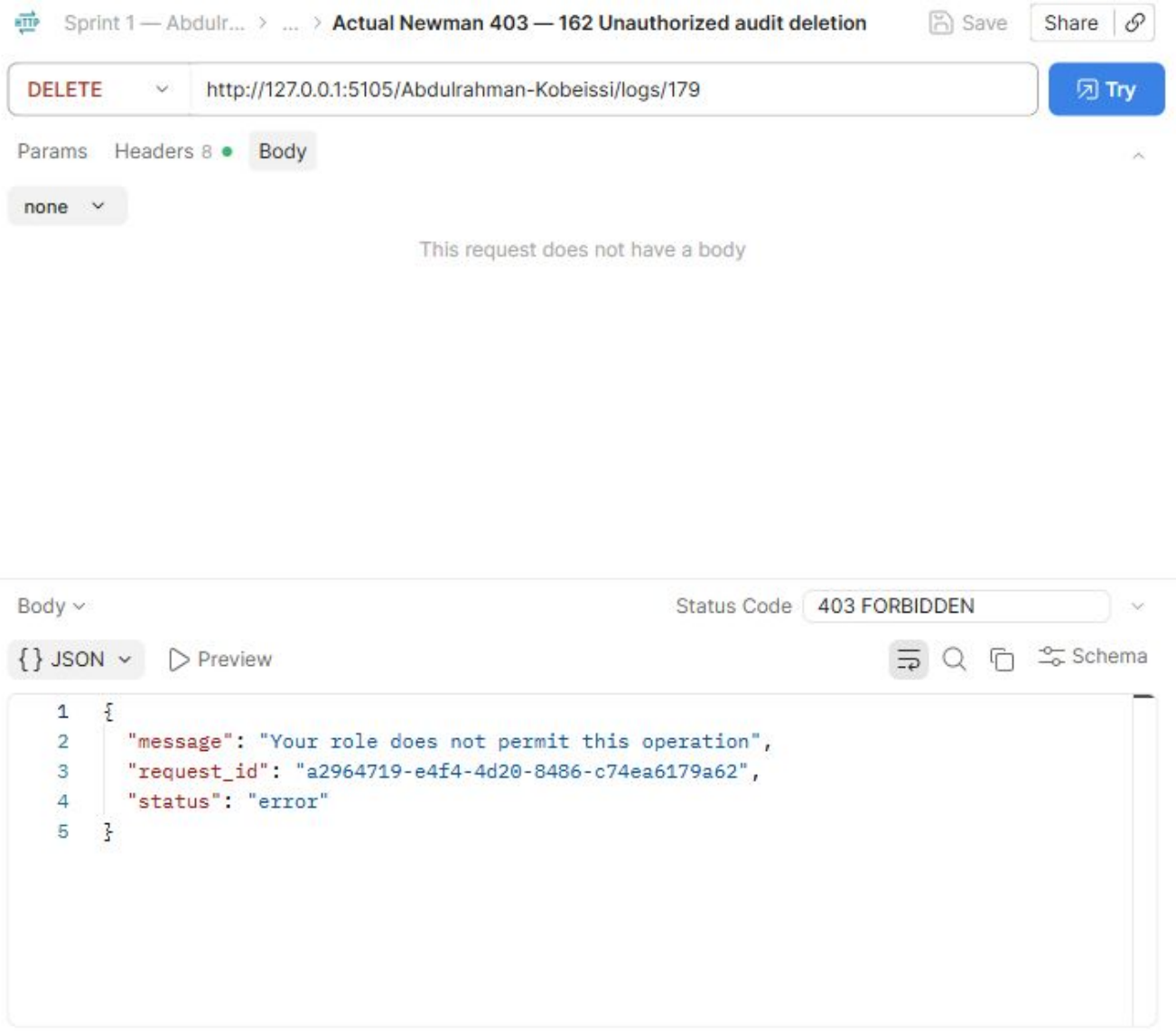


Abdulrahman Kobeissi — 161 Logs by user. The request and saved actual response demonstrate this operation.

Postman Evidence 162

(Abdulrahman Kobeissi)

Unauthorized audit deletion · DELETE · observed HTTP 403

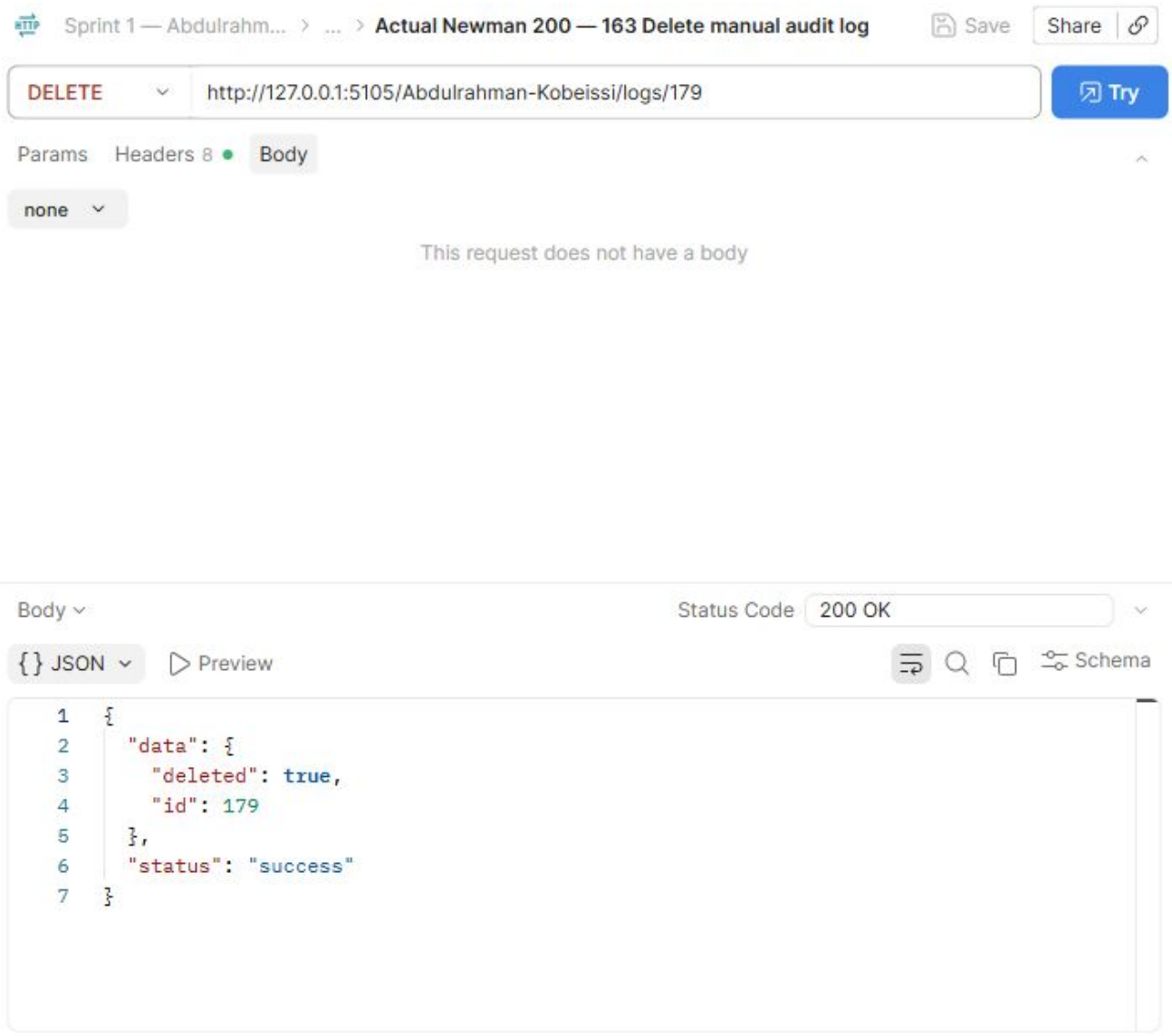


Abdulrahman Kobeissi — 162 Unauthorized audit deletion. The request and saved actual response demonstrate this operation.

Postman Evidence 163

(Abdulrahman Kobeissi)

Delete manual audit log · DELETE · observed HTTP 200

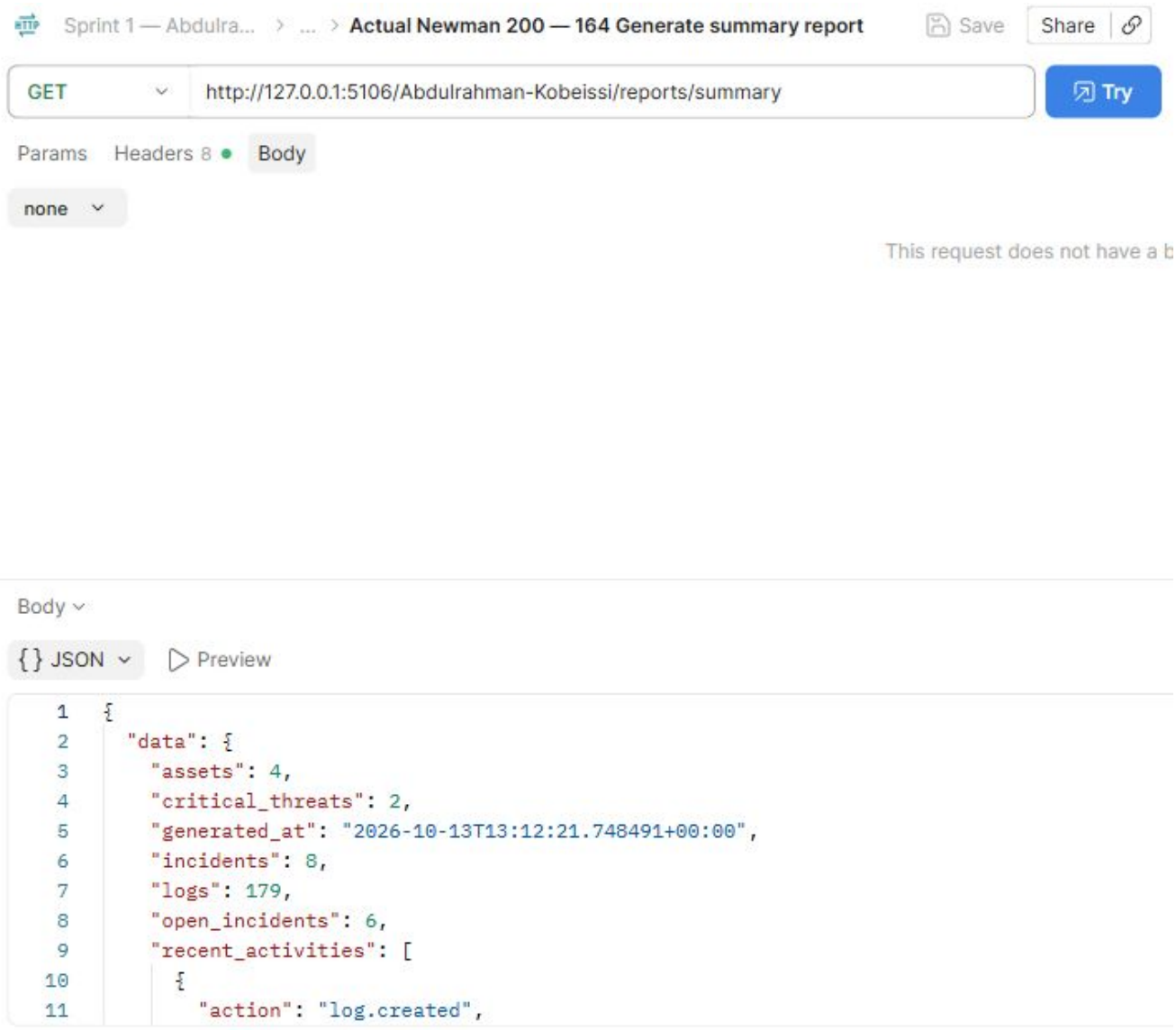


Abdulrahman Kobeissi — 163 Delete manual audit log. The request and saved actual response demonstrate this operation.

Postman Evidence 164

(Abdulrahman Kobeissi)

Generate summary report · GET · observed HTTP 200

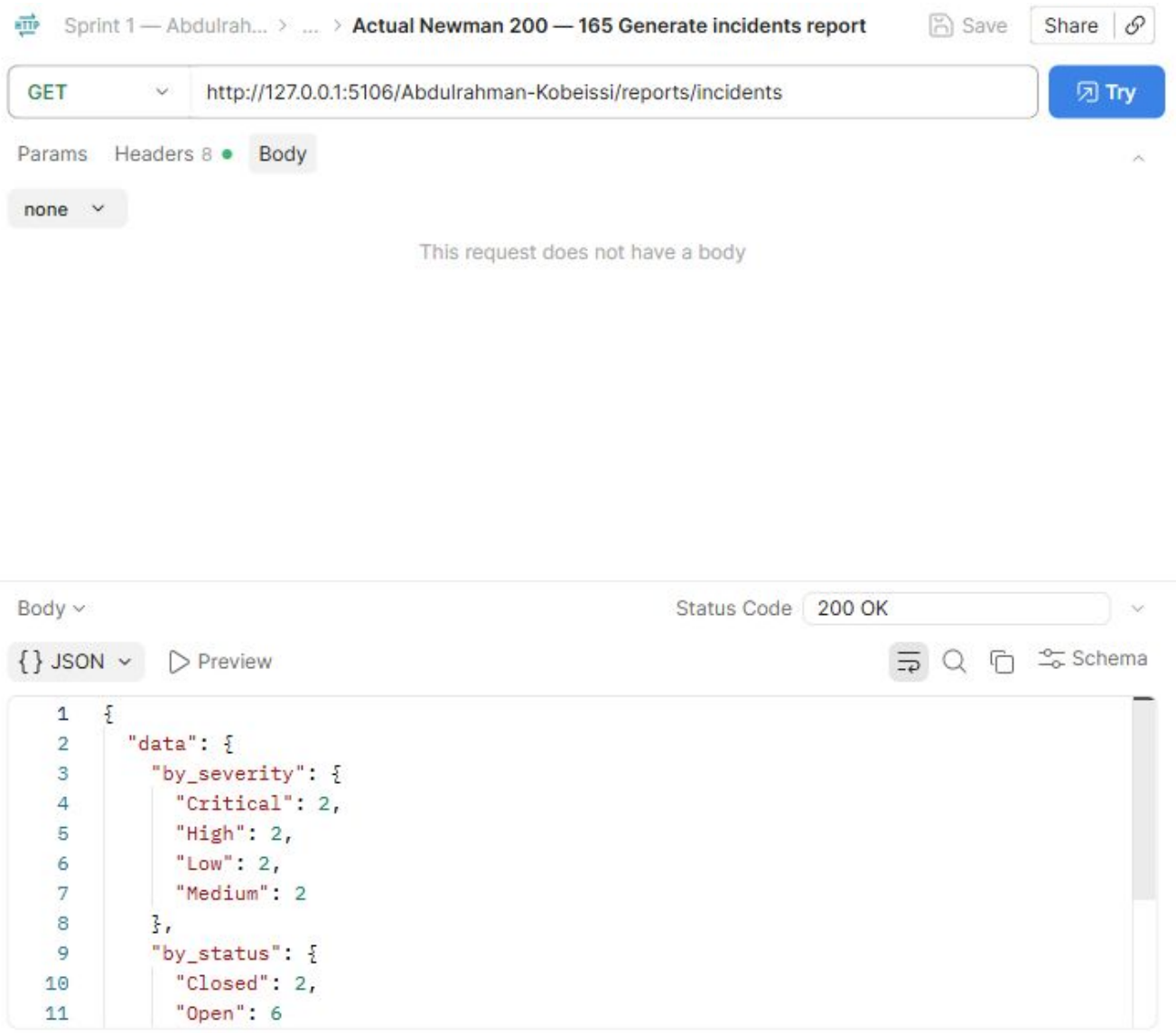


Abdulrahman Kobeissi — 164 Generate summary report. The request and saved actual response demonstrate this operation.

Postman Evidence 165

(Abdulrahman Kobeissi)

Generate incidents report · GET · observed HTTP 200

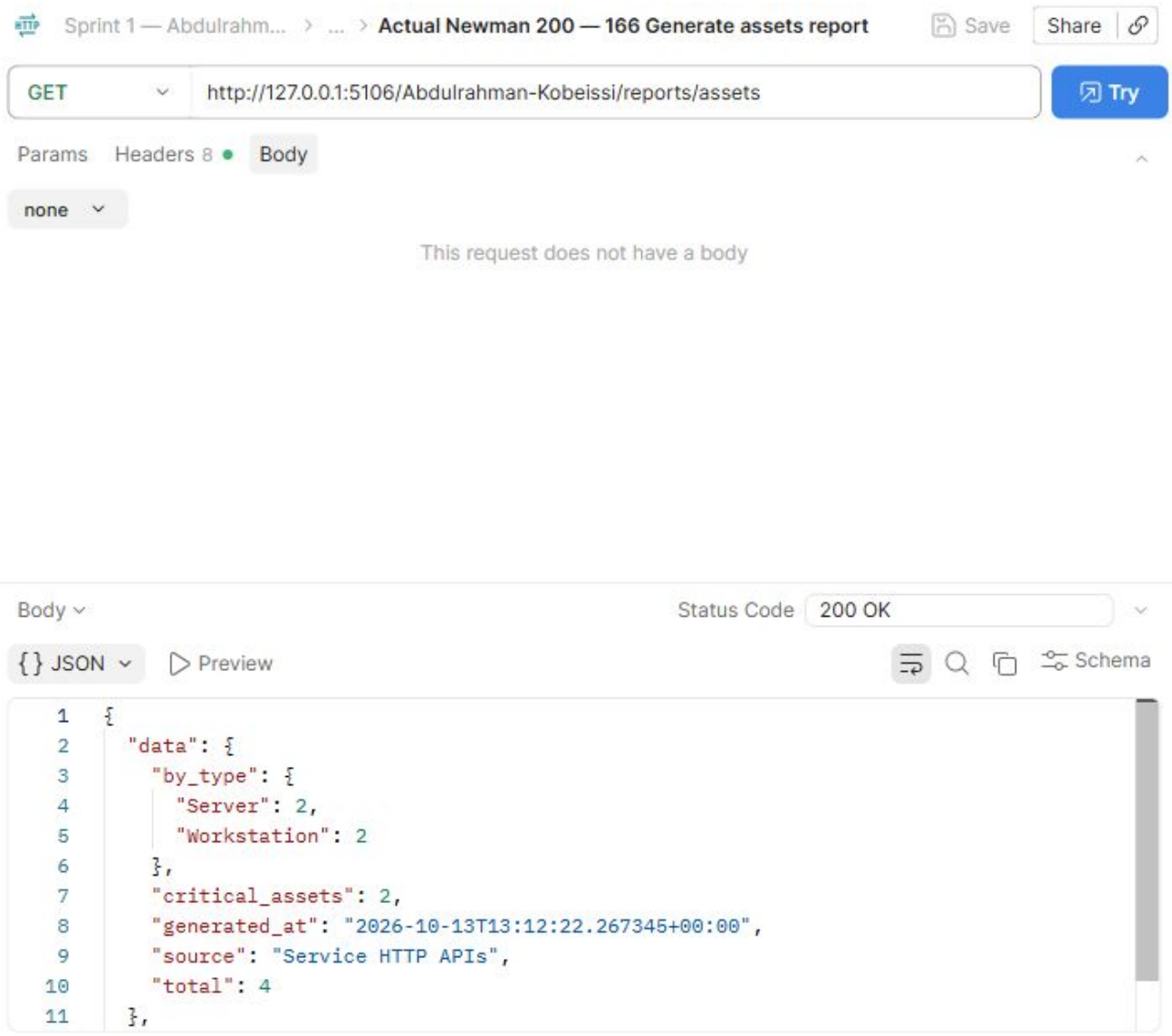


Abdulrahman Kobeissi — 165 Generate incidents report. The request and saved actual response demonstrate this operation.

Postman Evidence 166

(Abdulrahman Kobeissi)

Generate assets report · GET · observed HTTP 200

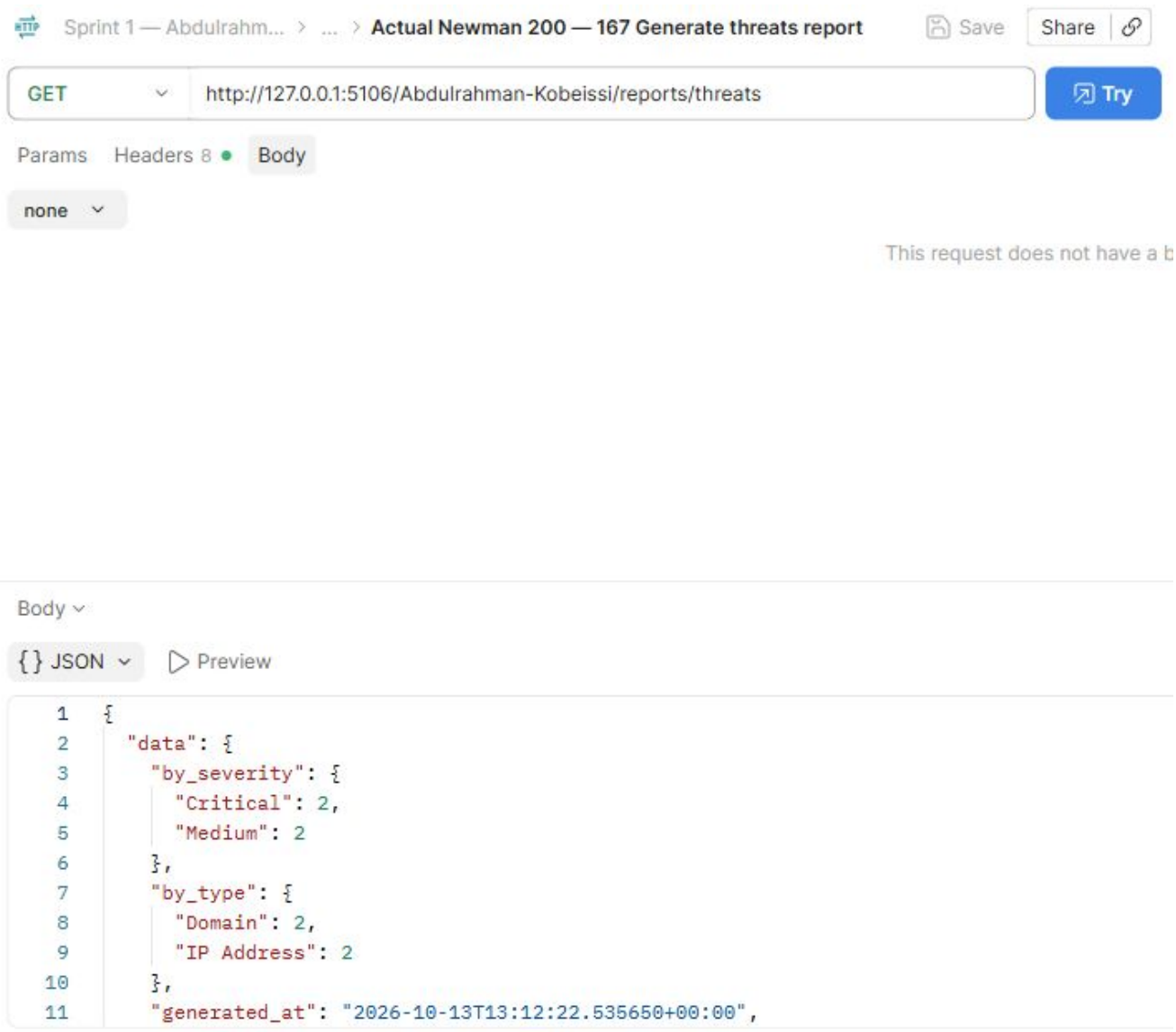


Abdulrahman Kobeissi — 166 Generate assets report. The request and saved actual response demonstrate this operation.

Postman Evidence 167

(Abdulrahman Kobeissi)

Generate threats report · GET · observed HTTP 200

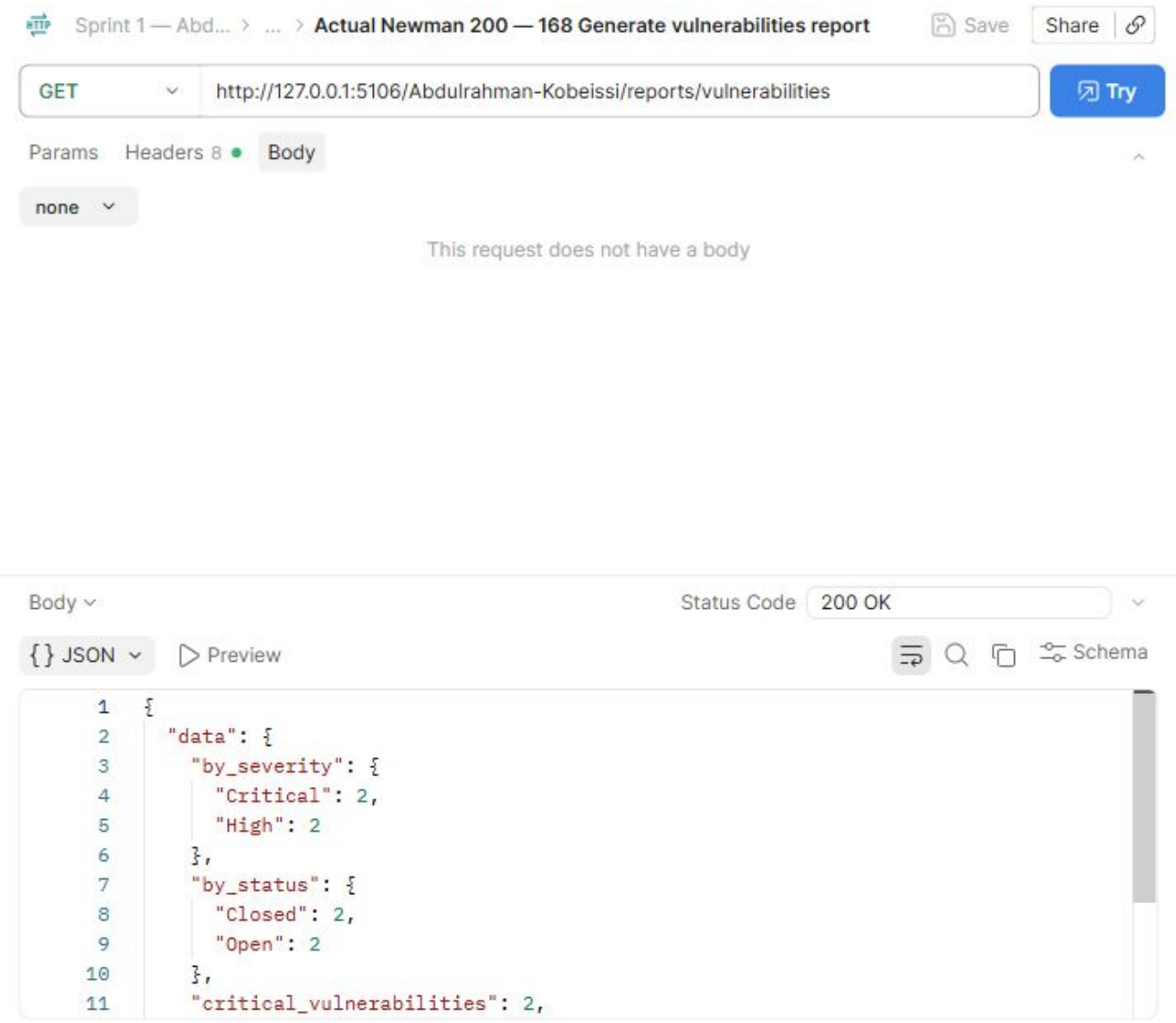


Abdulrahman Kobeissi — 167 Generate threats report. The request and saved actual response demonstrate this operation.

Postman Evidence 168

(Abdulrahman Kobeissi)

Generate vulnerabilities report · GET · observed HTTP 200

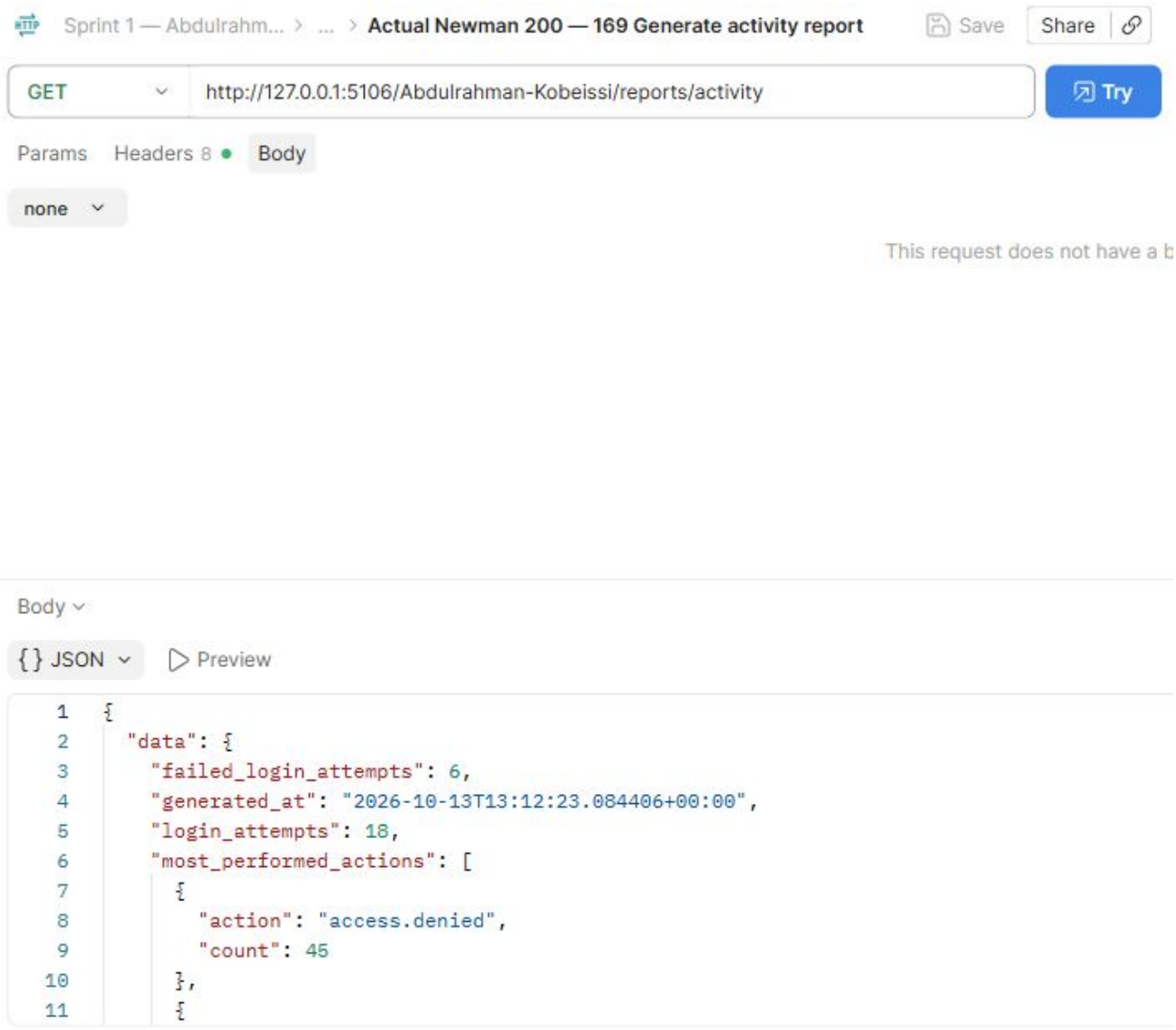


Abdulrahman Kobeissi — 168 Generate vulnerabilities report. The request and saved actual response demonstrate this operation.

Postman Evidence 169

(Abdulrahman Kobeissi)

Generate activity report · GET · observed HTTP 200

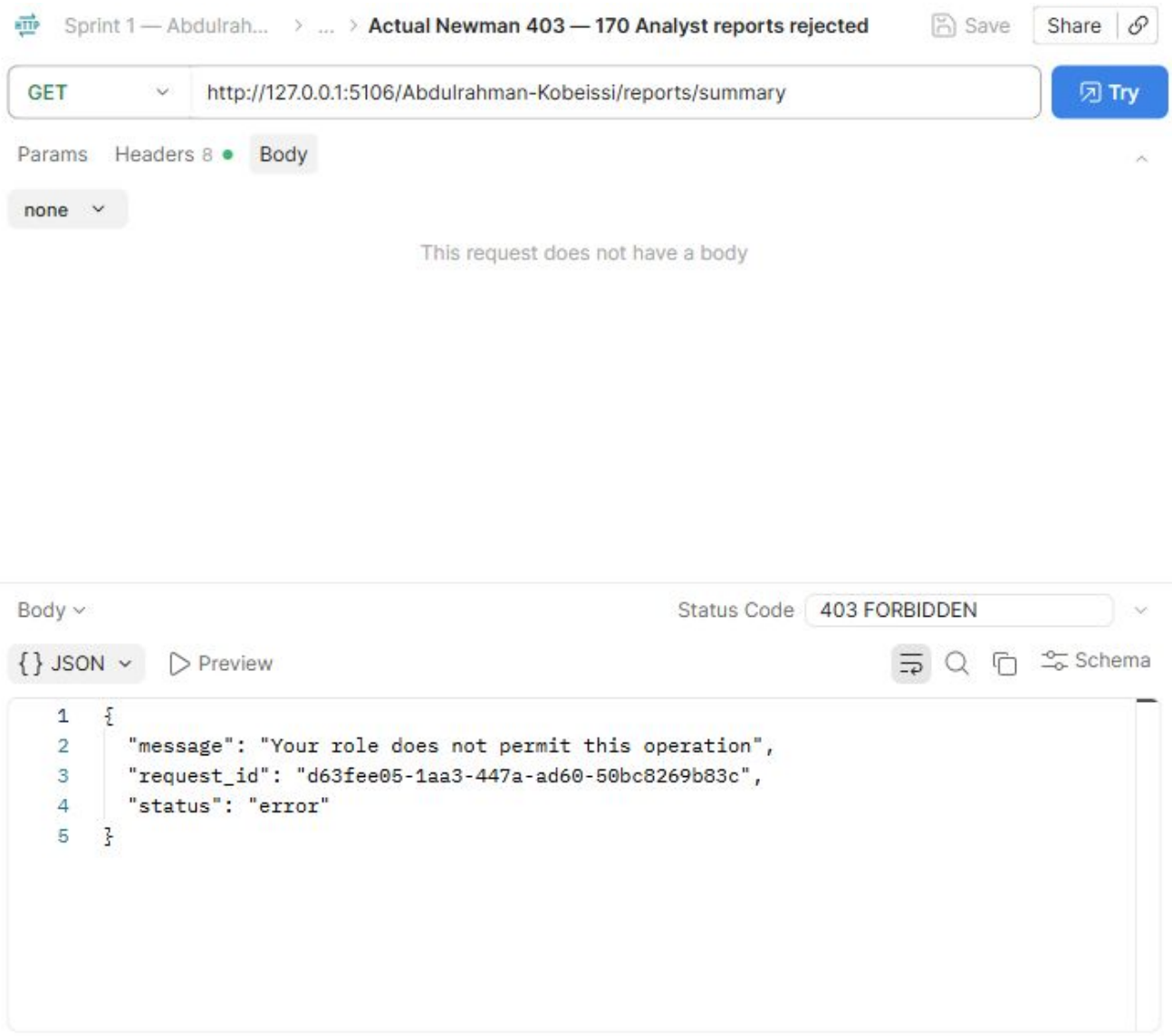


Abdulrahman Kobeissi — 169 Generate activity report. The request and saved actual response demonstrate this operation.

Postman Evidence 170

(Abdulrahman Kobeissi)

Analyst reports rejected · GET · observed HTTP 403

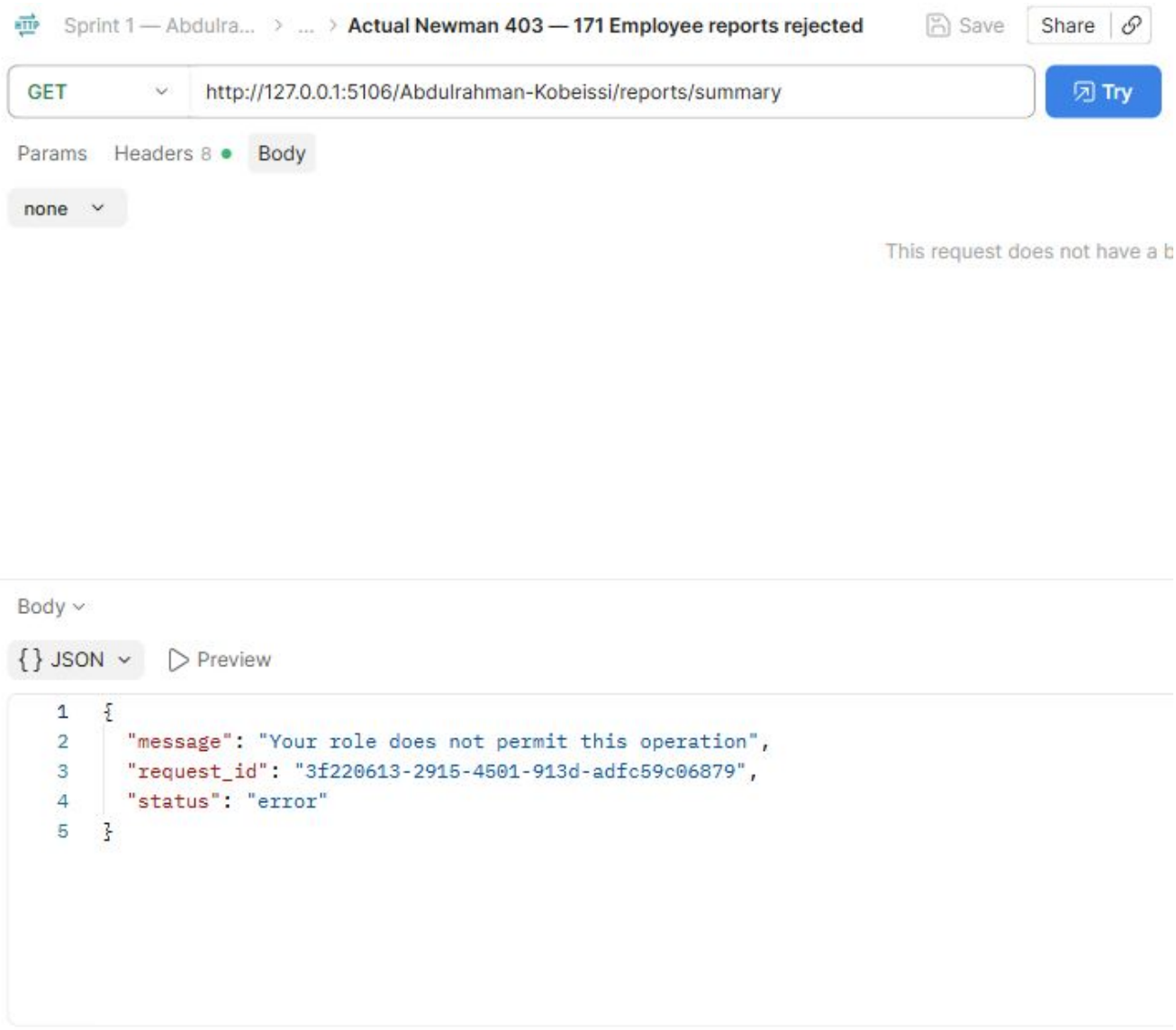


Abdulrahman Kobeissi — 170 Analyst reports rejected. The request and saved actual response demonstrate this operation.

Postman Evidence 171

(Abdulrahman Kobeissi)

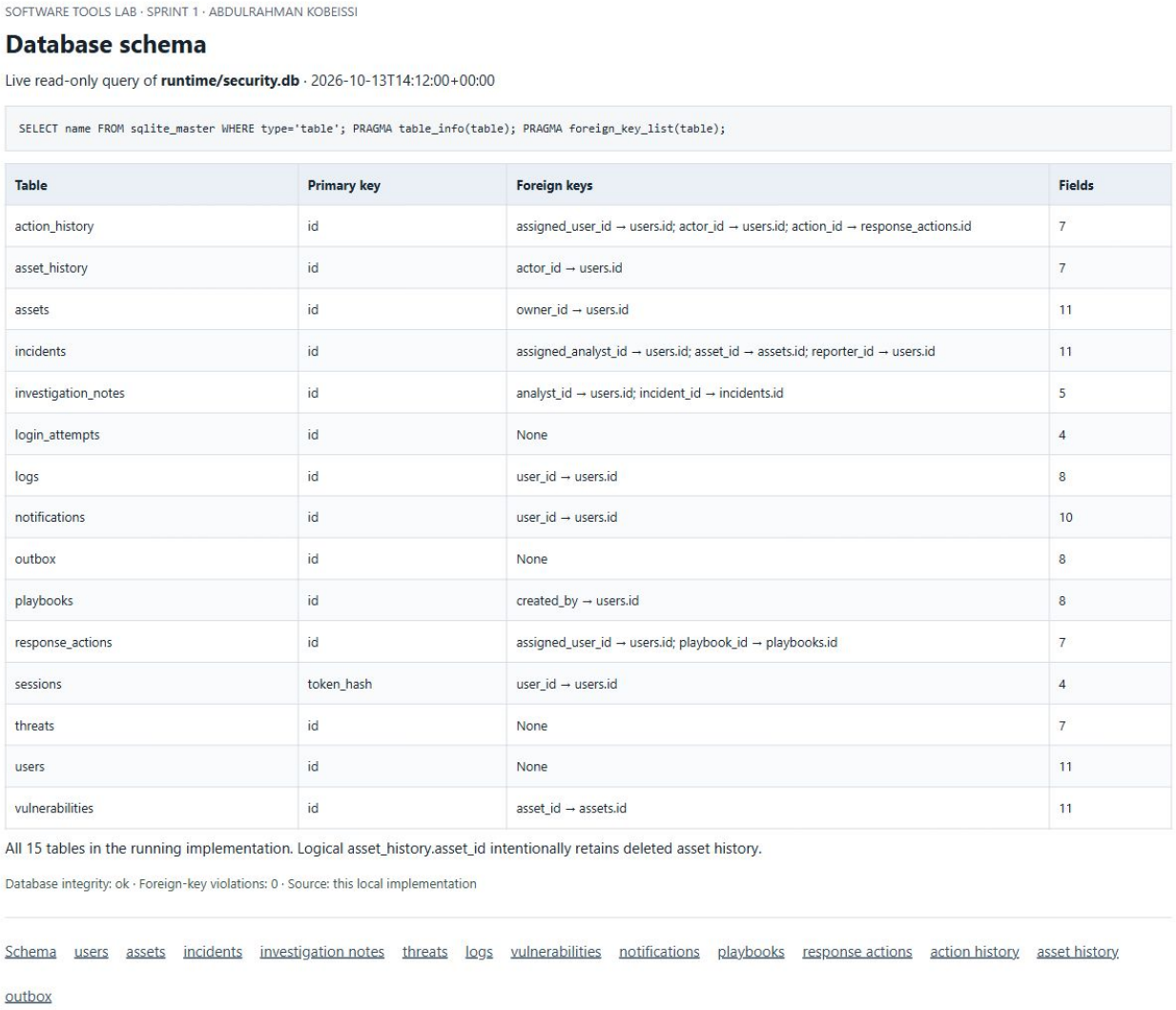
Employee reports rejected · GET · observed HTTP 403



Abdulrahman Kobeissi — 171 Employee reports rejected. The request and saved actual response demonstrate this operation.

Live Database Schema

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Database schema

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:00+00:00

```
SELECT name FROM sqlite_master WHERE type='table'; PRAGMA table_info(table); PRAGMA foreign_key_list(table);
```

Table	Primary key	Foreign keys	Fields
action_history	id	assigned_user_id → users.id; actor_id → users.id; action_id → response_actions.id	7
asset_history	id	actor_id → users.id	7
assets	id	owner_id → users.id	11
incidents	id	assigned_analyst_id → users.id; asset_id → assets.id; reporter_id → users.id	11
investigation_notes	id	analyst_id → users.id; incident_id → incidents.id	5
login_attempts	id	None	4
logs	id	user_id → users.id	8
notifications	id	user_id → users.id	10
outbox	id	None	8
playbooks	id	created_by → users.id	8
response_actions	id	assigned_user_id → users.id; playbook_id → playbooks.id	7
sessions	token_hash	user_id → users.id	4
threats	id	None	7
users	id	None	11
vulnerabilities	id	asset_id → assets.id	11

All 15 tables in the running implementation. Logical asset_history.asset_id intentionally retains deleted asset history.

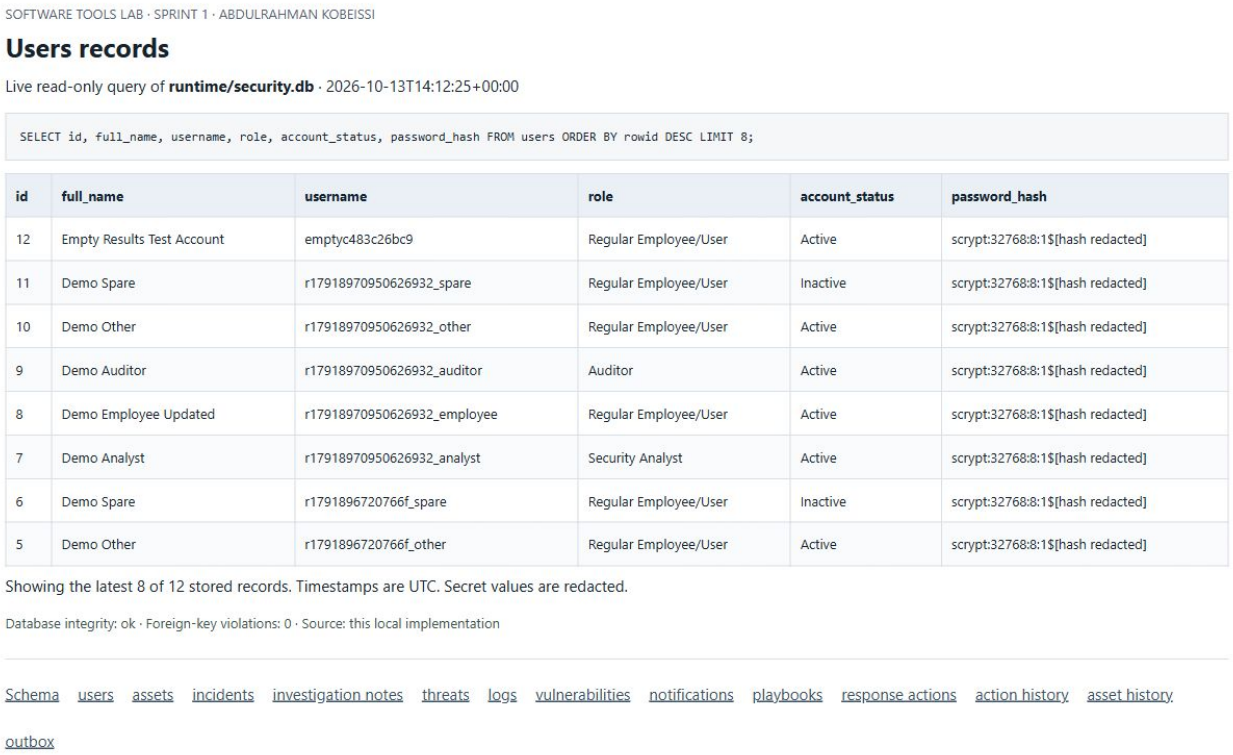
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual SQLite schema query showing all tables, primary keys and foreign keys.

Stored users Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Users records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:25+00:00

```
SELECT id, full_name, username, role, account_status, password_hash FROM users ORDER BY rowid DESC LIMIT 8;
```

id	full_name	username	role	account_status	password_hash
12	Empty Results Test Account	emptyc483c26bc9	Regular Employee/User	Active	scrypt:32768:8:1$[hash redacted]
11	Demo Spare	r17918970950626932_spare	Regular Employee/User	Inactive	scrypt:32768:8:1$[hash redacted]
10	Demo Other	r17918970950626932_other	Regular Employee/User	Active	scrypt:32768:8:1$[hash redacted]
9	Demo Auditor	r17918970950626932_auditor	Auditor	Active	scrypt:32768:8:1$[hash redacted]
8	Demo Employee Updated	r17918970950626932_employee	Regular Employee/User	Active	scrypt:32768:8:1$[hash redacted]
7	Demo Analyst	r17918970950626932_analyst	Security Analyst	Active	scrypt:32768:8:1$[hash redacted]
6	Demo Spare	r1791896720766f_spare	Regular Employee/User	Inactive	scrypt:32768:8:1$[hash redacted]
5	Demo Other	r1791896720766f_other	Regular Employee/User	Active	scrypt:32768:8:1$[hash redacted]

Showing the latest 8 of 12 stored records. Timestamps are UTC. Secret values are redacted.

Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored users records. Credential hashes are redacted where present.

Stored assets Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Assets records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:27+00:00

```
SELECT id, name, ip_address, owner_id, criticality, status FROM assets ORDER BY rowid DESC LIMIT 8;
```

id	name	ip_address	owner_id	criticality	status
4	r17918970950626932 Workstation	192.0.2.12	8	Medium	Active
3	r17918970950626932 Protected Server	192.0.2.11	8	Critical	Under Maintenance
2	r1791896720766f Workstation	192.0.2.12	3	Medium	Active
1	r1791896720766f Protected Server	192.0.2.11	3	Critical	Under Maintenance

Showing the latest 4 of 4 stored records. Timestamps are UTC. Secret values are redacted.

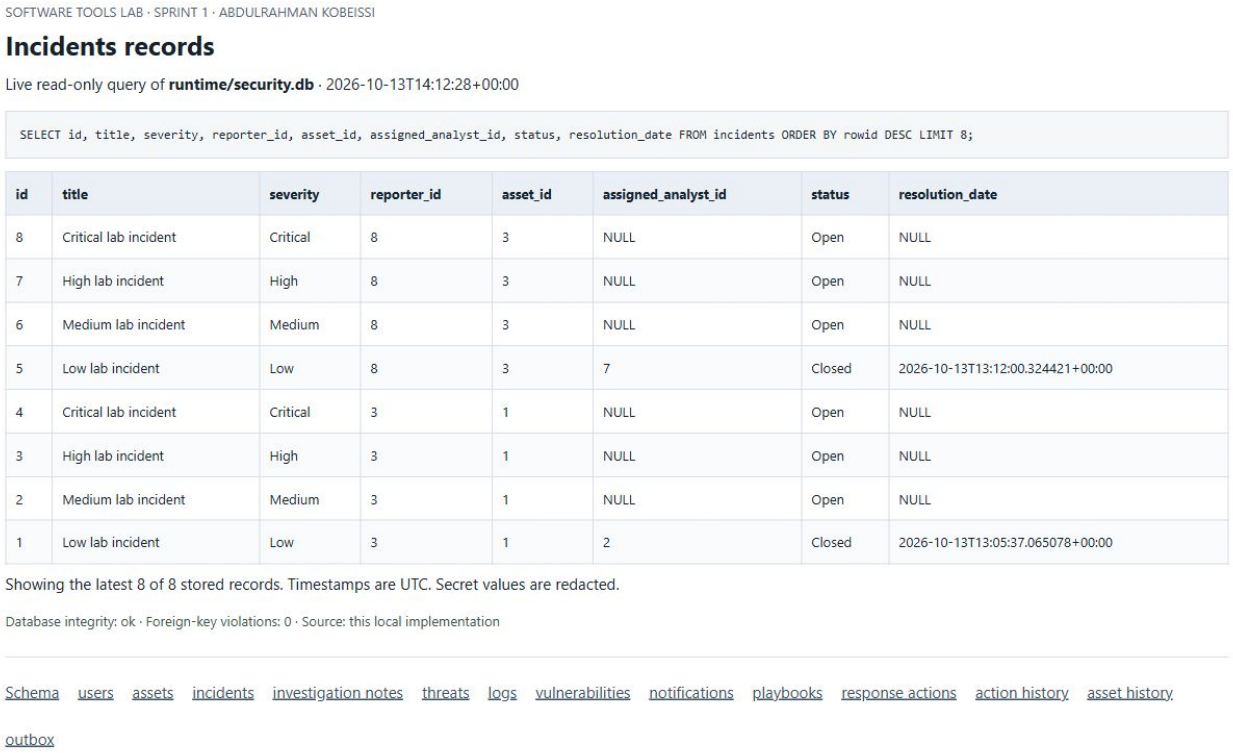
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored assets records. Credential hashes are redacted where present.

Stored incidents Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Incidents records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:28+00:00

```
SELECT id, title, severity, reporter_id, asset_id, assigned_analyst_id, status, resolution_date FROM incidents ORDER BY rowid DESC LIMIT 8;
```

id	title	severity	reporter_id	asset_id	assigned_analyst_id	status	resolution_date
8	Critical lab incident	Critical	8	3	NULL	Open	NULL
7	High lab incident	High	8	3	NULL	Open	NULL
6	Medium lab incident	Medium	8	3	NULL	Open	NULL
5	Low lab incident	Low	8	3	7	Closed	2026-10-13T13:12:00.324421+00:00
4	Critical lab incident	Critical	3	1	NULL	Open	NULL
3	High lab incident	High	3	1	NULL	Open	NULL
2	Medium lab incident	Medium	3	1	NULL	Open	NULL
1	Low lab incident	Low	3	1	2	Closed	2026-10-13T13:05:37.065078+00:00

Showing the latest 8 of 8 stored records. Timestamps are UTC. Secret values are redacted.

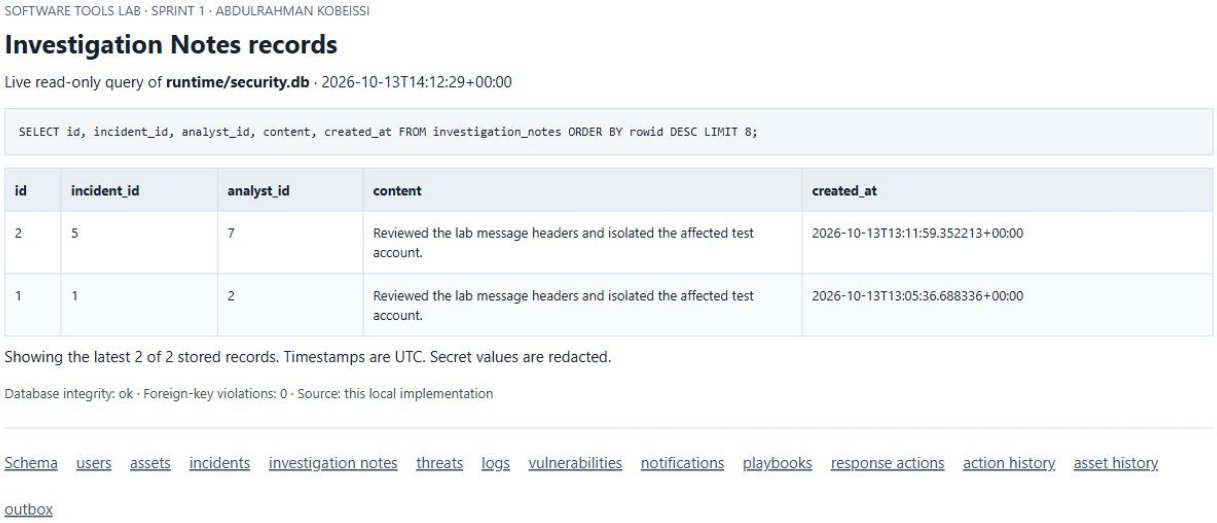
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored incidents records. Credential hashes are redacted where present.

Stored investigation notes Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Investigation Notes records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:29+00:00

```
SELECT id, incident_id, analyst_id, content, created_at FROM investigation_notes ORDER BY rowid DESC LIMIT 8;
```

id	incident_id	analyst_id	content	created_at
2	5	7	Reviewed the lab message headers and isolated the affected test account.	2026-10-13T13:11:59.352213+00:00
1	1	2	Reviewed the lab message headers and isolated the affected test account.	2026-10-13T13:05:36.688336+00:00

Showing the latest 2 of 2 stored records. Timestamps are UTC. Secret values are redacted.

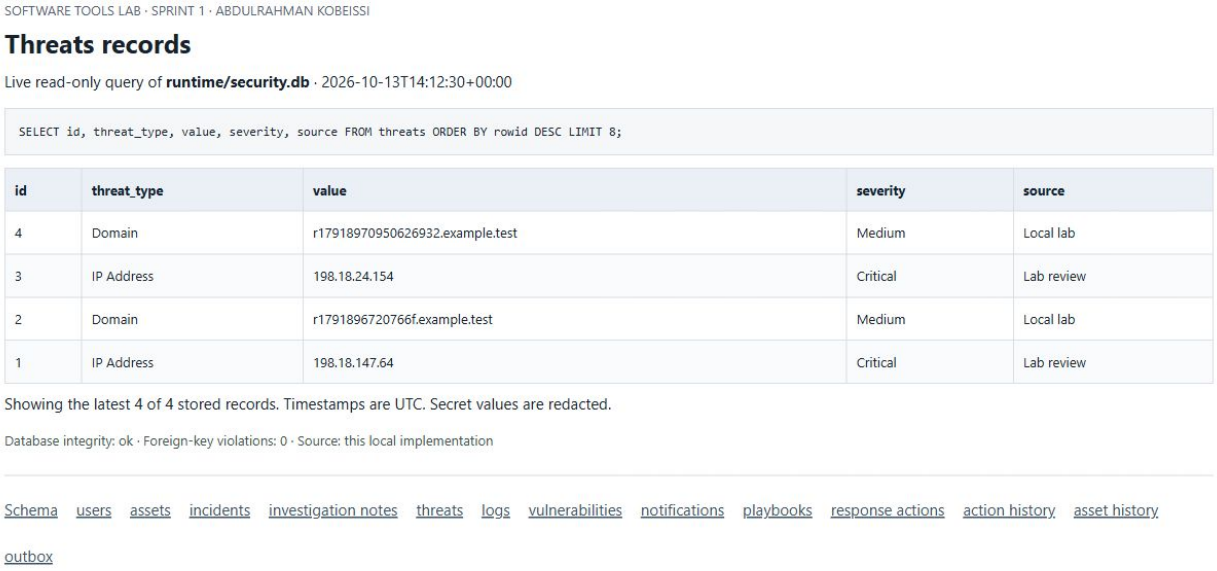
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored investigation notes records. Credential hashes are redacted where present.

Stored threats Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Threats records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:30+00:00

```
SELECT id, threat_type, value, severity, source FROM threats ORDER BY rowid DESC LIMIT 8;
```

id	threat_type	value	severity	source
4	Domain	r1791897095062693 2.example.test	Medium	Local lab
3	IP Address	198.18.24.154	Critical	Lab review
2	Domain	r1791896720766f.example.test	Medium	Local lab
1	IP Address	198.18.147.64	Critical	Lab review

Showing the latest 4 of 4 stored records. Timestamps are UTC. Secret values are redacted.

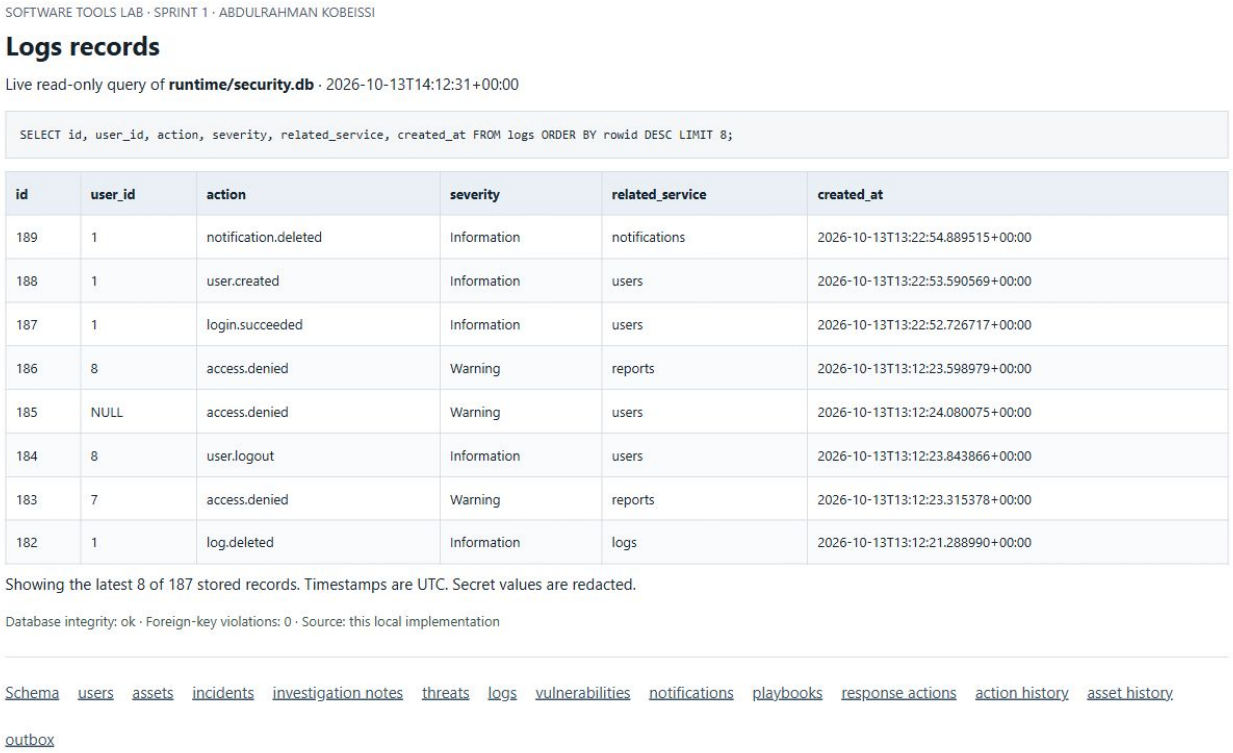
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored threats records. Credential hashes are redacted where present.

Stored logs Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Logs records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:31+00:00

```
SELECT id, user_id, action, severity, related_service, created_at FROM logs ORDER BY rowid DESC LIMIT 8;
```

id	user_id	action	severity	related_service	created_at
189	1	notification.deleted	Information	notifications	2026-10-13T13:22:54.889515+00:00
188	1	user.created	Information	users	2026-10-13T13:22:53.590569+00:00
187	1	login.succeeded	Information	users	2026-10-13T13:22:52.726717+00:00
186	8	access.denied	Warning	reports	2026-10-13T13:12:23.598979+00:00
185	NULL	access.denied	Warning	users	2026-10-13T13:12:24.080075+00:00
184	8	user.logout	Information	users	2026-10-13T13:12:23.843866+00:00
183	7	access.denied	Warning	reports	2026-10-13T13:12:23.315378+00:00
182	1	log.deleted	Information	logs	2026-10-13T13:12:21.288990+00:00

Showing the latest 8 of 187 stored records. Timestamps are UTC. Secret values are redacted.

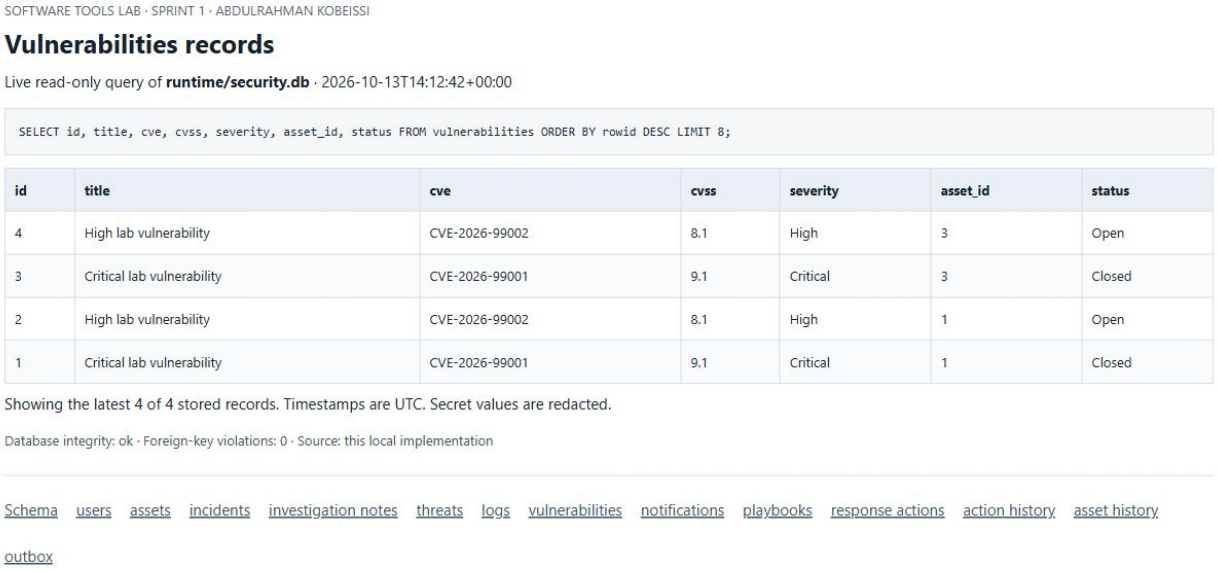
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored logs records. Credential hashes are redacted where present.

Stored vulnerabilities Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Vulnerabilities records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:42+00:00

```
SELECT id, title, cve, cvss, severity, asset_id, status FROM vulnerabilities ORDER BY rowid DESC LIMIT 8;
```

id	title	cve	cvss	severity	asset_id	status
4	High lab vulnerability	CVE-2026-99002	8.1	High	3	Open
3	Critical lab vulnerability	CVE-2026-99001	9.1	Critical	3	Closed
2	High lab vulnerability	CVE-2026-99002	8.1	High	1	Open
1	Critical lab vulnerability	CVE-2026-99001	9.1	Critical	1	Closed

Showing the latest 4 of 4 stored records. Timestamps are UTC. Secret values are redacted.

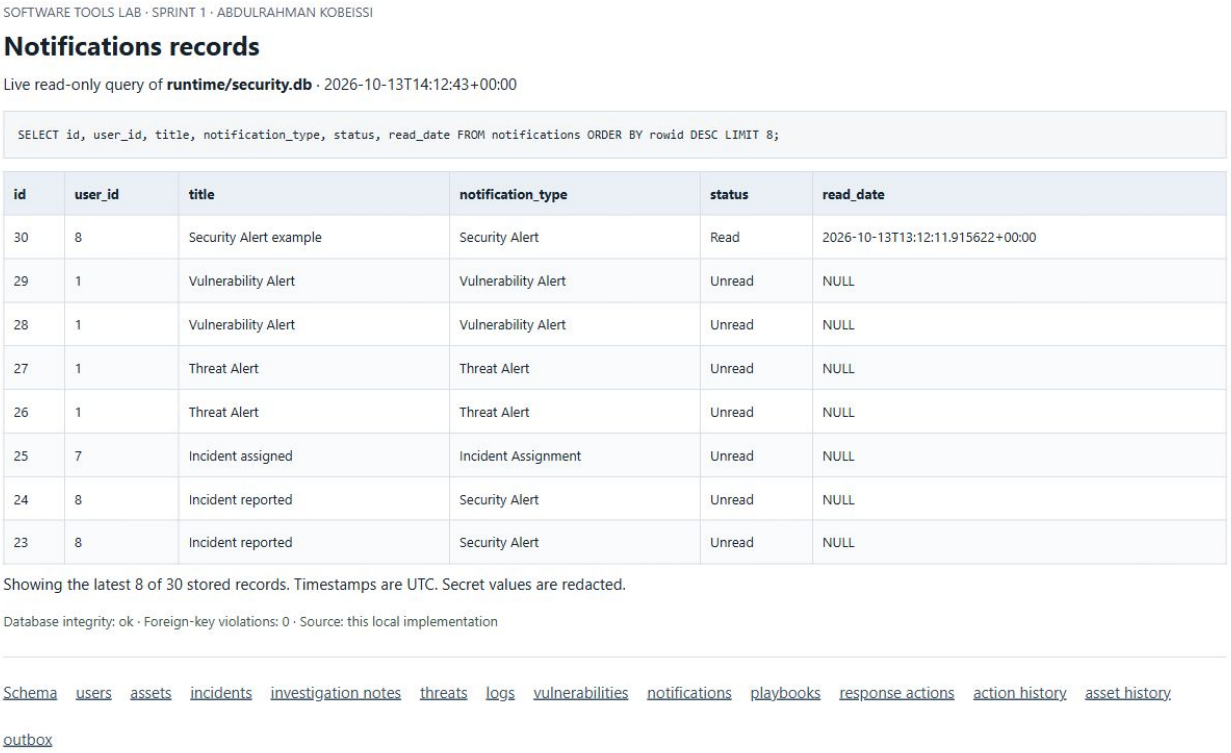
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored vulnerabilities records.

Stored notifications Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Notifications records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:43+00:00

```
SELECT id, user_id, title, notification_type, status, read_date FROM notifications ORDER BY rowid DESC LIMIT 8;
```

id	user_id	title	notification_type	status	read_date
30	8	Security Alert example	Security Alert	Read	2026-10-13T13:12:11.915622+00:00
29	1	Vulnerability Alert	Vulnerability Alert	Unread	NULL
28	1	Vulnerability Alert	Vulnerability Alert	Unread	NULL
27	1	Threat Alert	Threat Alert	Unread	NULL
26	1	Threat Alert	Threat Alert	Unread	NULL
25	7	Incident assigned	Incident Assignment	Unread	NULL
24	8	Incident reported	Security Alert	Unread	NULL
23	8	Incident reported	Security Alert	Unread	NULL

Showing the latest 8 of 30 stored records. Timestamps are UTC. Secret values are redacted.

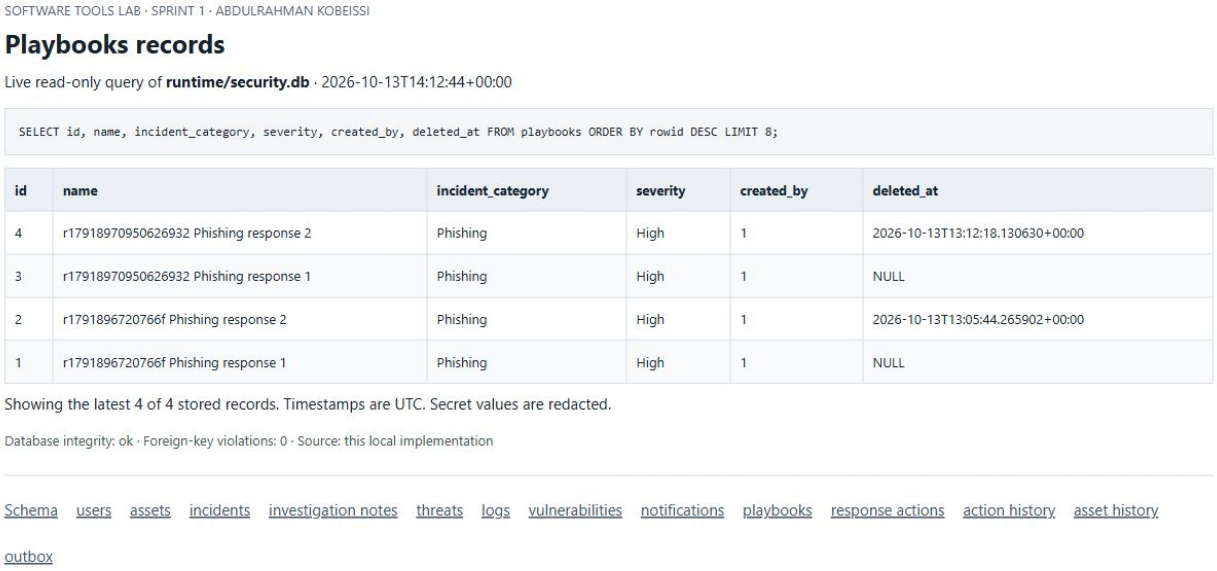
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored notifications records.

Stored playbooks Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Playbooks records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:44+00:00

```
SELECT id, name, incident_category, severity, created_by, deleted_at FROM playbooks ORDER BY rowid DESC LIMIT 8;
```

id	name	incident_category	severity	created_by	deleted_at
4	r1791897095062693 2 Phishing response 2	Phishing	High	1	2026-10-13T13:12:18.130630+00:00
3	r1791897095062693 2 Phishing response 1	Phishing	High	1	NULL
2	r1791896720766f Phishing response 2	Phishing	High	1	2026-10-13T13:05:44.265902+00:00
1	r1791896720766f Phishing response 1	Phishing	High	1	NULL

Showing the latest 4 of 4 stored records. Timestamps are UTC. Secret values are redacted.

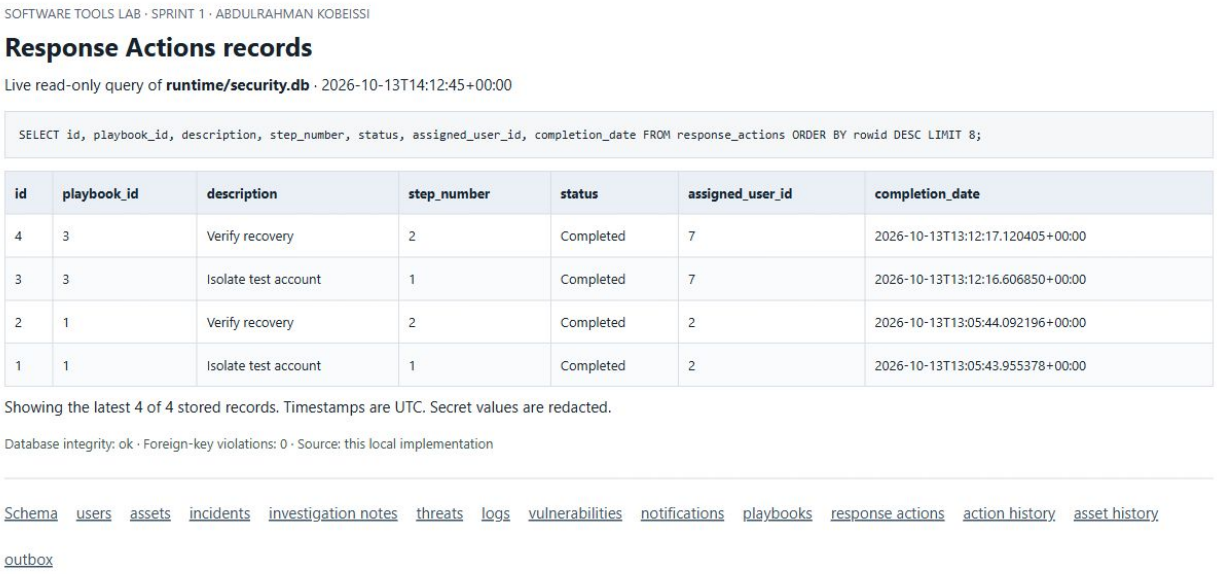
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored playbooks records.

Stored response actions Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Response Actions records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:45+00:00

```
SELECT id, playbook_id, description, step_number, status, assigned_user_id, completion_date FROM response_actions ORDER BY rowid DESC LIMIT 8;
```

id	playbook_id	description	step_number	status	assigned_user_id	completion_date
4	3	Verify recovery	2	Completed	7	2026-10-13T13:12:17.120405+00:00
3	3	Isolate test account	1	Completed	7	2026-10-13T13:12:16.606850+00:00
2	1	Verify recovery	2	Completed	2	2026-10-13T13:05:44.092196+00:00
1	1	Isolate test account	1	Completed	2	2026-10-13T13:05:43.955378+00:00

Showing the latest 4 of 4 stored records. Timestamps are UTC. Secret values are redacted.

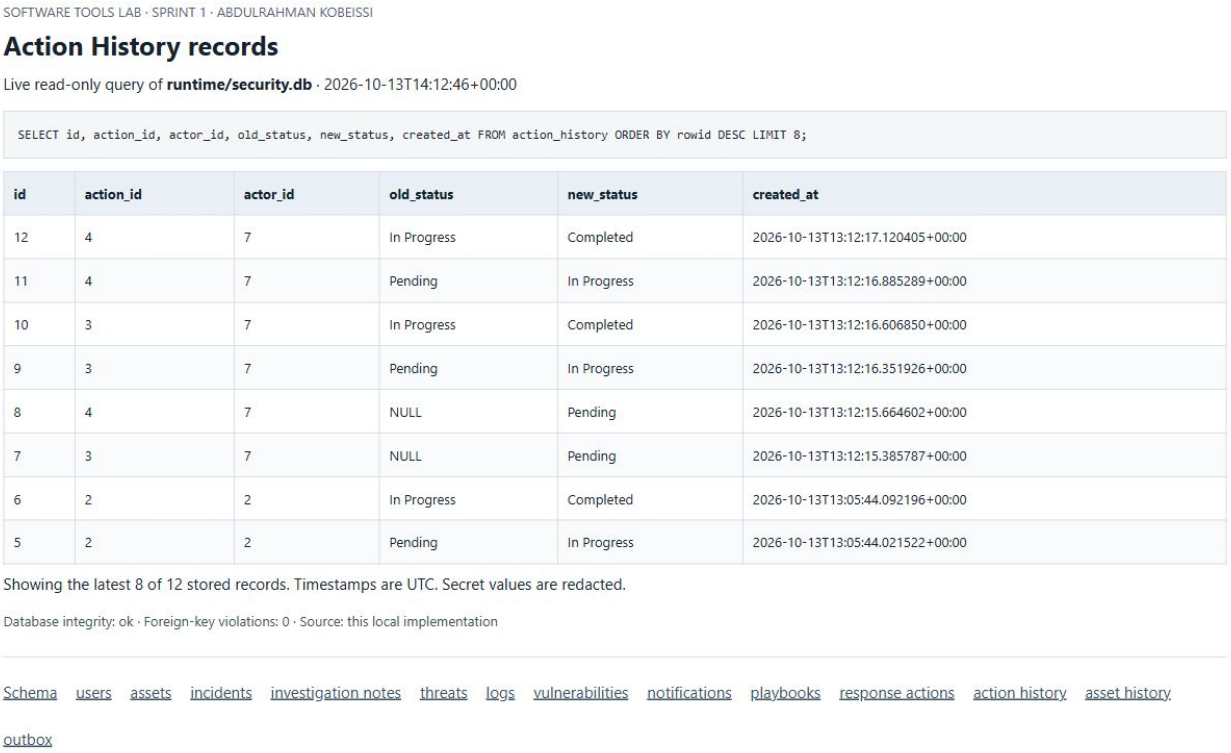
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored response actions records.

Stored action history Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Action History records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:46+00:00

```
SELECT id, action_id, actor_id, old_status, new_status, created_at FROM action_history ORDER BY rowid DESC LIMIT 8;
```

id	action_id	actor_id	old_status	new_status	created_at
12	4	7	In Progress	Completed	2026-10-13T13:12:17.120405+00:00
11	4	7	Pending	In Progress	2026-10-13T13:12:16.885289+00:00
10	3	7	In Progress	Completed	2026-10-13T13:12:16.606850+00:00
9	3	7	Pending	In Progress	2026-10-13T13:12:16.351926+00:00
8	4	7	NULL	Pending	2026-10-13T13:12:15.664602+00:00
7	3	7	NULL	Pending	2026-10-13T13:12:15.385787+00:00
6	2	2	In Progress	Completed	2026-10-13T13:05:44.092196+00:00
5	2	2	Pending	In Progress	2026-10-13T13:05:44.021522+00:00

Showing the latest 8 of 12 stored records. Timestamps are UTC. Secret values are redacted.

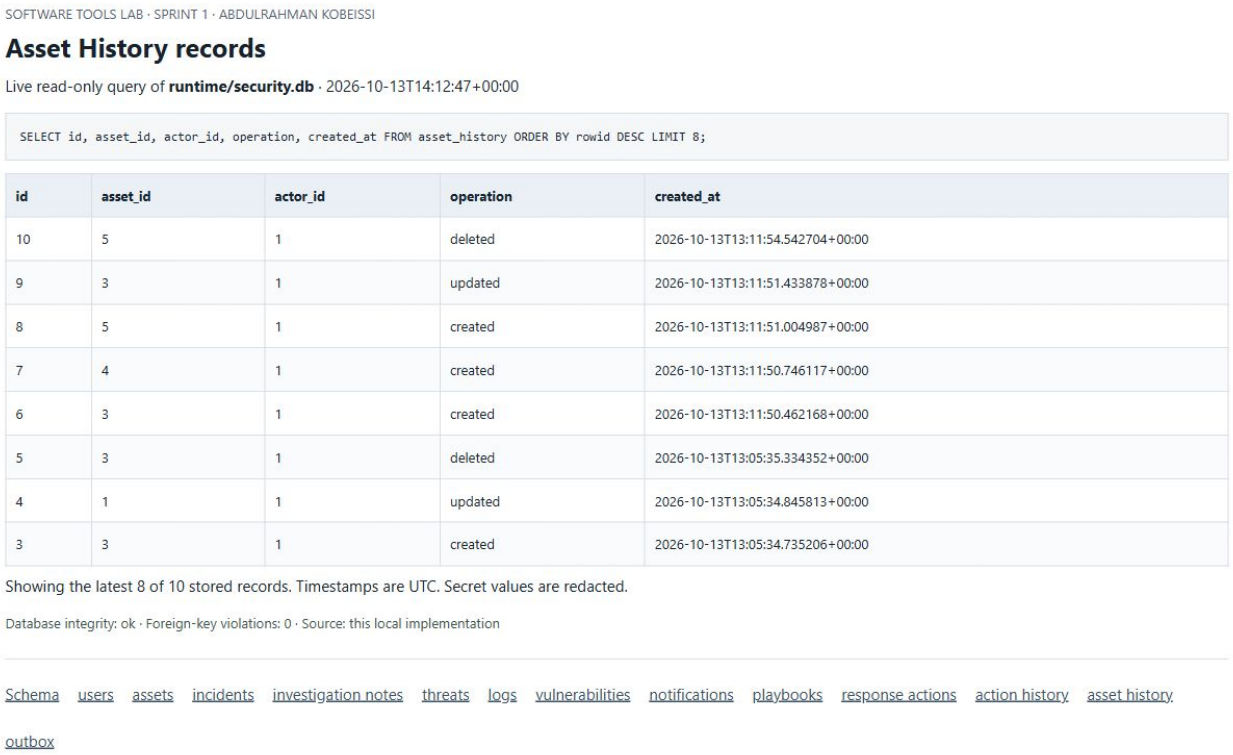
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored action history records.

Stored asset history Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Asset History records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:47+00:00

```
SELECT id, asset_id, actor_id, operation, created_at FROM asset_history ORDER BY rowid DESC LIMIT 8;
```

id	asset_id	actor_id	operation	created_at
10	5	1	deleted	2026-10-13T13:11:54.542704+00:00
9	3	1	updated	2026-10-13T13:11:51.433878+00:00
8	5	1	created	2026-10-13T13:11:51.004987+00:00
7	4	1	created	2026-10-13T13:11:50.746117+00:00
6	3	1	created	2026-10-13T13:11:50.462168+00:00
5	3	1	deleted	2026-10-13T13:05:35.334352+00:00
4	1	1	updated	2026-10-13T13:05:34.845813+00:00
3	3	1	created	2026-10-13T13:05:34.735206+00:00

Showing the latest 8 of 10 stored records. Timestamps are UTC. Secret values are redacted.

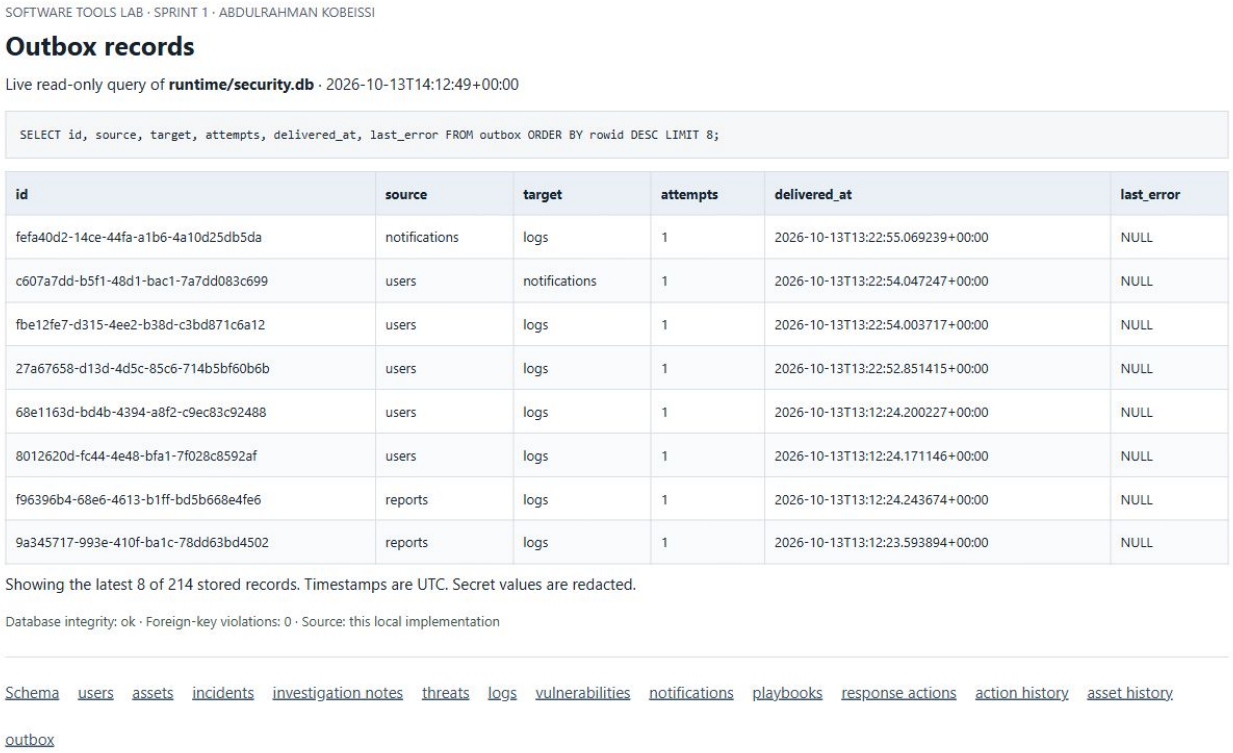
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored asset history records.

Stored outbox Records

(Abdulrahman Kobeissi)



SOFTWARE TOOLS LAB · SPRINT 1 · ABDULRAHMAN KOBEISSI

Outbox records

Live read-only query of **runtime/security.db** · 2026-10-13T14:12:49+00:00

```
SELECT id, source, target, attempts, delivered_at, last_error FROM outbox ORDER BY rowid DESC LIMIT 8;
```

id	source	target	attempts	delivered_at	last_error
fefa40d2-14ce-44fa-a1b6-4a10d25db5da	notifications	logs	1	2026-10-13T13:22:55.069239+00:00	NULL
c607a7dd-b5f1-48d1-bac1-7a7dd083c699	users	notifications	1	2026-10-13T13:22:54.047247+00:00	NULL
fbe12fe7-d315-4ee2-b38d-c3bd871c6a12	users	logs	1	2026-10-13T13:22:54.003717+00:00	NULL
27a67658-d13d-4d5c-85c6-714b5bf60b6b	users	logs	1	2026-10-13T13:22:52.851415+00:00	NULL
68e1163d-bd4b-4394-a8f2-c9ec83c92488	users	logs	1	2026-10-13T13:12:24.200227+00:00	NULL
8012620d-fc44-4e48-bfa1-7f028c8592af	users	logs	1	2026-10-13T13:12:24.171146+00:00	NULL
f96396b4-68e6-4613-b1ff-bd5b668e4fe6	reports	logs	1	2026-10-13T13:12:24.243674+00:00	NULL
9a345717-993e-410f-ba1c-78dd63bd4502	reports	logs	1	2026-10-13T13:12:23.593894+00:00	NULL

Showing the latest 8 of 214 stored records. Timestamps are UTC. Secret values are redacted.

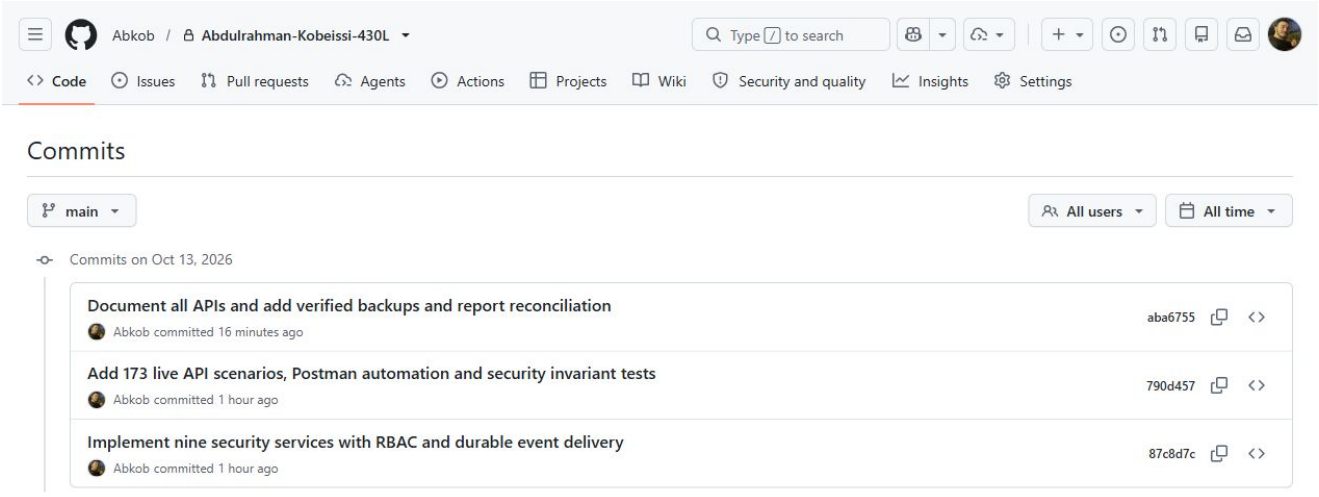
Database integrity: ok · Foreign-key violations: 0 · Source: this local implementation

Schema users assets incidents investigation notes threats logs vulnerabilities notifications playbooks response actions action history asset history outbox

Abdulrahman Kobeissi — Actual read-only SQLite query showing stored outbox records.

GitHub Commit History

(Abdulrahman Kobeissi)



Abdulrahman Kobeissi — Actual private repository history showing the implementation, testing and documentation commits by Abkob on 13 October 2026.

Later report-packaging commits may follow this screenshot.